DARA Studio — Admin Full Export

All admin source code + database data (CSV)

Generated for Claude analysis

SOURCE CODE FILES

- src/App.tsx
- src/components/LanguageSelector.tsx
- src/components/admin/AddressAutocomplete.tsx
- src/components/admin/AdminLayout.tsx
- src/components/admin/AdminSidebar.tsx
- src/components/admin/CompanyEmployees.tsx
- src/components/admin/ProjectDocuments.tsx
- src/components/admin/ProjectKanbanBoard.tsx
- src/components/admin/ProjectTimeline.tsx
- src/components/admin/dashboard/ActivityFeed.tsx
- src/components/admin/dashboard/ClientActivityFeed.tsx
- src/components/admin/dashboard/ClientTicketsWidget.tsx
- src/components/admin/dashboard/FinancialChart.tsx
- src/components/admin/dashboard/KPICards.tsx
- src/components/admin/dashboard/NewLeadsTable.tsx
- src/components/admin/dashboard/ProjectPipeline.tsx
- src/components/admin/dashboard/QuickActions.tsx
- src/components/admin/dashboard/TodaysTasks.tsx
- src/components/admin/timeline/timelineUtils.ts
- src/components/dashboard/ClientProjectDocuments.tsx
- src/components/dashboard/ClientTimeline.tsx
- src/components/dashboard/DashboardLayout.tsx
- src/components/dashboard/DashboardSidebar.tsx
- src/components/dashboard/ProjectCard.tsx
- src/components/layout/Footer.tsx
- src/components/layout/Header.tsx
- src/components/layout/MainLayout.tsx
- src/components/layout/PublicLayout.tsx
- src/components/ui/sidebar.tsx
- src/constants/projectStages.ts
- src/hooks/useAdminRole.tsx
- src/hooks/useAuth.tsx
- src/hooks/useFreelancerRole.tsx
- src/hooks/useLanguage.tsx
- src/i18n/translations.ts
- src/index.css
- src/integrations/supabase/client.ts
- src/integrations/supabase/types.ts
- src/main.tsx
- src/pages/admin/AdminAICodeCheck.tsx
- src/pages/admin/AdminActivity.tsx
- src/pages/admin/AdminAnalytics.tsx
- src/pages/admin/AdminClientPortal.tsx
- src/pages/admin/AdminClientProfile.tsx
- src/pages/admin/AdminCompanies.tsx
- src/pages/admin/AdminContracts.tsx
- src/pages/admin/AdminDashboard.tsx
- src/pages/admin/AdminDrawings.tsx
- src/pages/admin/AdminFiles.tsx
- src/pages/admin/AdminFinanceOverview.tsx
- src/pages/admin/AdminFreelancers.tsx
- src/pages/admin/AdminGantt.tsx
- src/pages/admin/AdminLeads.tsx
- src/pages/admin/AdminLogin.tsx
- src/pages/admin/AdminMap.tsx
- src/pages/admin/AdminMilestones.tsx
- src/pages/admin/AdminPayments.tsx
- src/pages/admin/AdminPermits.tsx
- src/pages/admin/AdminProjectDetail.tsx
- src/pages/admin/AdminProjects.tsx
- src/pages/admin/AdminProposals.tsx
- src/pages/admin/AdminSettings.tsx
- src/pages/admin/AdminSiteUpdates.tsx
- src/pages/admin/AdminTickets.tsx
- src/pages/admin/Dashboard.tsx
- src/services/estimateService.ts
- src/services/index.ts
- src/services/paymentService.ts
- src/services/projectService.ts
- src/services/supabase/authService.ts
- src/services/supabase/client.ts
- src/services/supabase/estimateService.ts
- src/services/supabase/freelancerService.ts

- src/services/supabase/paymentService.ts
- src/services/supabase/projectService.ts
- tailwind.config.ts

src/App.tsx

```
import { Toaster } from "@/components/ui/toaster";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import { AuthProvider } from "@/hooks/useAuth";
import { LanguageProvider } from "@/hooks/useLanguage";

/* LAYOUT */
import MainLayout from "./components/layout/MainLayout";

/* PUBLIC PAGES */
import Home from "./pages/public/Home";
import HowWeWork from "./pages/public/HowWeWork";
import EstimateRequest from "./pages/public/EstimateRequest";
import ThankYou from "./pages/public/ThankYou";
import NotFound from "./pages/public/NotFound";

/* AUTH */
import Login from "./pages/auth/Login";
import FreelancerLogin from "./pages/auth/FreelancerLogin";
import AdminLogin from "./pages/admin/AdminLogin";

/* DASHBOARD (Client) */
import DashboardLayout from "./components/dashboard/DashboardLayout";
import DashboardHome from "./pages/dashboard/DashboardHome";
import Projects from "./pages/dashboard/Projects";
import ClientProjectDetail from "./pages/dashboard/ClientProjectDetail";
import Invoices from "./pages/dashboard/Invoices";
import Files from "./pages/dashboard/Files";
import Messages from "./pages/dashboard/Messages";
import Profile from "./pages/dashboard/Profile";
import Company from "./pages/dashboard/Company";
import NewQuote from "./pages/dashboard/NewQuote";

/* FREELANCER WORKSPACE */
import FreelancerLayout from "./components/freelancer/FreelancerLayout";
import FreelancerHome from "./pages/freelancer/FreelancerHome";
import FreelancerProjects from "./pages/freelancer/FreelancerProjects";
import FreelancerPayments from "./pages/freelancer/FreelancerPayments";
import FreelancerContracts from "./pages/freelancer/FreelancerContracts";
import FreelancerCalendar from "./pages/freelancer/FreelancerCalendar";
import FreelancerProfile from "./pages/freelancer/FreelancerProfile";

/* ADMIN PANEL */
import AdminLayout from "./components/admin/AdminLayout";
import AdminDashboard from "./pages/admin/AdminDashboard";
import AdminCompanies from "./pages/admin/AdminCompanies";
import AdminProjects from "./pages/admin/AdminProjects";
import AdminProjectDetail from "./pages/admin/AdminProjectDetail";
import AdminPayments from "./pages/admin/AdminPayments";
import AdminMilestones from "./pages/admin/AdminMilestones";
import AdminActivity from "./pages/admin/AdminActivity";
import AdminMap from "./pages/admin/AdminMap";
import AdminGantt from "./pages/admin/AdminGantt";
import AdminFreelancers from "./pages/admin/AdminFreelancers";
import AdminTickets from "./pages/admin/AdminTickets";
import AdminDrawings from "./pages/admin/AdminDrawings";
import AdminLeads from "./pages/admin/AdminLeads";
import AdminProposals from "./pages/admin/AdminProposals";
import AdminContracts from "./pages/admin/AdminContracts";
import AdminFiles from "./pages/admin/AdminFiles";
import AdminPermits from "./pages/admin/AdminPermits";
import AdminAICodeCheck from "./pages/admin/AdminAICodeCheck";
import AdminClientPortal from "./pages/admin/AdminClientPortal";
import AdminAnalytics from "./pages/admin/AdminAnalytics";
import AdminSettings from "./pages/admin/AdminSettings";
import AdminClientProfile from "./pages/admin/AdminClientProfile";
import AdminFinanceOverview from "./pages/admin/AdminFinanceOverview";
import AdminSiteUpdates from "./pages/admin/AdminSiteUpdates";

const queryClient = new QueryClient();

const App = () =&gt; (
  &lt;QueryClientProvider client={queryClient}&gt;
    &lt;LanguageProvider&gt;
      &lt;TooltipProvider&gt;
        &lt;Toaster /&gt;
        &lt;Sonner /&gt;
        &lt;BrowserRouter&gt;
          &lt;AuthProvider&gt;
            &lt;MainLayout&gt;
              &lt;Routes&gt;
                {/* PUBLIC WEBSITE */}
```

```
            &lt;Route path="/" element={&lt;Home /&gt;} /&gt;
            &lt;Route path="/how-we-work" element={&lt;HowWeWork /&gt;} /&gt;
            &lt;Route path="/estimate-request" element={&lt;EstimateRequest /&gt;} /&gt;
            &lt;Route path="/orcamento" element={&lt;EstimateRequest /&gt;} /&gt;
            &lt;Route path="/thank-you" element={&lt;ThankYou /&gt;} /&gt;

            {/* AUTH */}
            &lt;Route path="/login" element={&lt;Login /&gt;} /&gt;
            &lt;Route path="/auth" element={&lt;Login /&gt;} /&gt;
            &lt;Route path="/freelancer-login" element={&lt;FreelancerLogin /&gt;} /&gt;
            &lt;Route path="/freelancer" element={&lt;FreelancerLogin /&gt;} /&gt;

            {/* ADMIN LOGIN */}
            &lt;Route path="/adm/login" element={&lt;AdminLogin /&gt;} /&gt;

            {/* DASHBOARD (Client - Protected) */}
            &lt;Route path="/dashboard" element={&lt;DashboardLayout /&gt;}&gt;
              &lt;Route index element={&lt;DashboardHome /&gt;} /&gt;
              &lt;Route path="projects" element={&lt;Projects /&gt;} /&gt;
              &lt;Route path="projects/:id" element={&lt;ClientProjectDetail /&gt;} /&gt;
              &lt;Route path="invoices" element={&lt;Invoices /&gt;} /&gt;
              &lt;Route path="files" element={&lt;Files /&gt;} /&gt;
              &lt;Route path="messages" element={&lt;Messages /&gt;} /&gt;
              &lt;Route path="profile" element={&lt;Profile /&gt;} /&gt;
              &lt;Route path="company" element={&lt;Company /&gt;} /&gt;
              &lt;Route path="new-quote" element={&lt;NewQuote /&gt;} /&gt;
            &lt;/Route&gt;

            {/* FREELANCER WORKSPACE (Protected) */}
            &lt;Route path="/work" element={&lt;FreelancerLayout /&gt;}&gt;
              &lt;Route index element={&lt;FreelancerHome /&gt;} /&gt;
              &lt;Route path="projects" element={&lt;FreelancerProjects /&gt;} /&gt;
              &lt;Route path="payments" element={&lt;FreelancerPayments /&gt;} /&gt;
              &lt;Route path="contracts" element={&lt;FreelancerContracts /&gt;} /&gt;
              &lt;Route path="calendar" element={&lt;FreelancerCalendar /&gt;} /&gt;
              &lt;Route path="profile" element={&lt;FreelancerProfile /&gt;} /&gt;
            &lt;/Route&gt;

            {/* ADMIN PANEL (Protected) */}
            &lt;Route path="/adm" element={&lt;AdminLayout /&gt;}&gt;
              &lt;Route index element={&lt;AdminDashboard /&gt;} /&gt;
              &lt;Route path="companies" element={&lt;AdminCompanies /&gt;} /&gt;
              &lt;Route path="clients/:id" element={&lt;AdminClientProfile /&gt;} /&gt;
              &lt;Route path="projects" element={&lt;AdminProjects /&gt;} /&gt;
              &lt;Route path="projects/:id" element={&lt;AdminProjectDetail /&gt;} /&gt;
              &lt;Route path="gantt" element={&lt;AdminGantt /&gt;} /&gt;
              &lt;Route path="map" element={&lt;AdminMap /&gt;} /&gt;
              &lt;Route path="payments" element={&lt;AdminPayments /&gt;} /&gt;
              &lt;Route path="finance-overview" element={&lt;AdminFinanceOverview /&gt;} /&gt;
              &lt;Route path="milestones" element={&lt;AdminMilestones /&gt;} /&gt;
              &lt;Route path="activity" element={&lt;AdminActivity /&gt;} /&gt;
              &lt;Route path="freelancers" element={&lt;AdminFreelancers /&gt;} /&gt;
              &lt;Route path="tickets" element={&lt;AdminTickets /&gt;} /&gt;
              &lt;Route path="drawings" element={&lt;AdminDrawings /&gt;} /&gt;
              &lt;Route path="leads" element={&lt;AdminLeads /&gt;} /&gt;
              &lt;Route path="proposals" element={&lt;AdminProposals /&gt;} /&gt;
              &lt;Route path="contracts" element={&lt;AdminContracts /&gt;} /&gt;
              &lt;Route path="files" element={&lt;AdminFiles /&gt;} /&gt;
              &lt;Route path="permits" element={&lt;AdminPermits /&gt;} /&gt;
              &lt;Route path="site-updates" element={&lt;AdminSiteUpdates /&gt;} /&gt;
              &lt;Route path="ai-code-check" element={&lt;AdminAICodeCheck /&gt;} /&gt;
              &lt;Route path="client-portal" element={&lt;AdminClientPortal /&gt;} /&gt;
              &lt;Route path="analytics" element={&lt;AdminAnalytics /&gt;} /&gt;
              &lt;Route path="settings" element={&lt;AdminSettings /&gt;} /&gt;
            &lt;/Route&gt;

            {/* FALLBACK */}
            &lt;Route path="*" element={&lt;NotFound /&gt;} /&gt;
          &lt;/Routes&gt;
        &lt;/MainLayout&gt;
      &lt;/AuthProvider&gt;
    &lt;/BrowserRouter&gt;
  &lt;/TooltipProvider&gt;
&lt;/LanguageProvider&gt;
&lt;/QueryClientProvider&gt;
);

export default App;
```

src/components/LanguageSelector.tsx

```
import { useLanguage } from "@/hooks/useLanguage";
import { Button } from "@/components/ui/button";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { Globe } from "lucide-react";

const LanguageSelector = () =&gt; {
  const { language, setLanguage, t } = useLanguage();

  return (
    &lt;DropdownMenu&gt;
      &lt;DropdownMenuTrigger asChild&gt;
        &lt;Button variant="ghost" size="sm" className="gap-2"&gt;
          &lt;Globe className="h-4 w-4" /&gt;
          &lt;span className="hidden sm:inline"&gt;
            {language === "en" ? "■■" : "■■"}
          &lt;/span&gt;
        &lt;/Button&gt;
      &lt;/DropdownMenuTrigger&gt;
      &lt;DropdownMenuContent align="end"&gt;
        &lt;DropdownMenuItem
          onClick={() =&gt; setLanguage("en")}
          className={language === "en" ? "bg-accent" : ""}
        &gt;
          &lt;span className="mr-2"&gt;■■&lt;/span&gt;
          {t("language.english")}
        &lt;/DropdownMenuItem&gt;
        &lt;DropdownMenuItem
          onClick={() =&gt; setLanguage("pt")}
          className={language === "pt" ? "bg-accent" : ""}
        &gt;
          &lt;span className="mr-2"&gt;■■&lt;/span&gt;
          {t("language.portuguese")}
        &lt;/DropdownMenuItem&gt;
      &lt;/DropdownMenuContent&gt;
    &lt;/DropdownMenu&gt;
  );
};

export default LanguageSelector;
```

src/components/admin/AddressAutocomplete.tsx

```
import { useState, useRef, useEffect } from "react";
import { Input } from "@/components/ui/input";
import { Command, CommandEmpty, CommandGroup, CommandItem, CommandList } from "@/components/ui/command";
import { Popover, PopoverContent, PopoverTrigger } from "@/components/ui/popover";
import { MapPin } from "lucide-react";

interface AddressResult {
  display_name: string;
  address: {
    road?: string;
    house_number?: string;
    city?: string;
    town?: string;
    village?: string;
    state?: string;
    postcode?: string;
    country?: string;
    country_code?: string;
    suburb?: string;
    neighbourhood?: string;
  };
  lat: string;
  lon: string;
}

interface AddressData {
  street_name: string;
  street_number: string;
  neighborhood: string;
  city: string;
  state: string;
  zip_code: string;
  country: string;
  latitude: number;
  longitude: number;
}

interface Props {
  onSelect: (data: AddressData) =&gt; void;
  placeholder?: string;
}

const AddressAutocomplete = ({ onSelect, placeholder = "Search address..." }: Props) =&gt; {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState&lt;AddressResult[]&gt;([]);
  const [open, setOpen] = useState(false);
  const debounceRef = useRef&lt;NodeJS.Timeout&gt;();

  useEffect(() =&gt; {
    if (query.length &lt; 3) { setResults([]); return; }
    clearTimeout(debounceRef.current);
    debounceRef.current = setTimeout(async () =&gt; {
      try {
        const res = await fetch(
          `https://nominatim.openstreetmap.org/search?format=json&amp;addressdetails=1&amp;limit=5&amp;q=${encodeURIComponent(query)}`,
          { headers: { "Accept-Language": "en" } }
        );
        const data = await res.json();
        setResults(data);
        setOpen(data.length &gt; 0);
      } catch { setResults([]); }
    }, 400);
    return () =&gt; clearTimeout(debounceRef.current);
  }, [query]);

  const handleSelect = (r: AddressResult) =&gt; {
    const a = r.address;
    const cityName = a.city || a.town || a.village || "";
    let country = "Other";
    if (a.country_code === "us") country = "USA";
    else if (a.country_code === "br") country = "Brazil";

    onSelect({
      street_name: a.road || "",
      street_number: a.house_number || "",
      neighborhood: a.suburb || a.neighbourhood || "",
      city: cityName,
      state: a.state || "",
      zip_code: a.postcode || "",
      country,
      latitude: parseFloat(r.lat),
      longitude: parseFloat(r.lon),
    });
    setQuery(r.display_name);
```

```
    setOpen(false);
  };

  return (
    &lt;Popover open={open} onOpenChange={setOpen}&gt;
      &lt;PopoverTrigger asChild&gt;
        &lt;div className="relative"&gt;
          &lt;MapPin className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
          &lt;Input
            placeholder={placeholder}
            value={query}
            onChange={(e) =&gt; { setQuery(e.target.value); setOpen(true); }}
            className="pl-10"
          /&gt;
        &lt;/div&gt;
      &lt;/PopoverTrigger&gt;
      &lt;PopoverContent className="p-0 w-[var(--radix-popover-trigger-width)]" align="start"&gt;
        &lt;Command&gt;
          &lt;CommandList&gt;
            &lt;CommandEmpty&gt;No results found.&lt;/CommandEmpty&gt;
            &lt;CommandGroup&gt;
              {results.map((r, i) =&gt; (
                &lt;CommandItem key={i} onSelect={() =&gt; handleSelect(r)} className="cursor-pointer"&gt;
                  &lt;MapPin className="h-4 w-4 mr-2 shrink-0 text-muted-foreground" /&gt;
                  &lt;span className="truncate text-sm"&gt;{r.display_name}&lt;/span&gt;
                &lt;/CommandItem&gt;
              ))}
            &lt;/CommandGroup&gt;
          &lt;/CommandList&gt;
        &lt;/Command&gt;
      &lt;/PopoverContent&gt;
    &lt;/Popover&gt;
  );
};

export default AddressAutocomplete;
```

src/components/admin/AdminLayout.tsx

```
import { useEffect } from "react";
import { Outlet, useNavigate, Link } from "react-router-dom";
import daraLogo from "@/assets/dara-logo.jpg";
import { useAuth } from "@/hooks/useAuth";
import { useAdminRole } from "@/hooks/useAdminRole";
import { SidebarProvider, SidebarTrigger, SidebarInset } from "@/components/ui/sidebar";
import AdminSidebar from "./AdminSidebar";
import { Skeleton } from "@/components/ui/skeleton";
import LanguageSelector from "@/components/LanguageSelector";
import { useLanguage } from "@/hooks/useLanguage";

const AdminLayout = () =&gt; {
  const { user, loading: authLoading } = useAuth();
  const { isAdmin, loading: roleLoading } = useAdminRole();
  const navigate = useNavigate();
  const { t } = useLanguage();
  const loading = authLoading || roleLoading;

  useEffect(() =&gt; {
    if (!loading &amp;&amp; !user) {
      navigate("/adm/login");
    } else if (!loading &amp;&amp; user &amp;&amp; !roleLoading &amp;&amp; !isAdmin) {
      navigate("/");
    }
  }, [user, loading, isAdmin, roleLoading, navigate]);

  if (loading) {
    return (
      &lt;div className="min-h-screen flex items-center justify-center bg-background"&gt;
        &lt;div className="space-y-4 w-full max-w-md p-8"&gt;
          &lt;Skeleton className="h-8 w-3/4 mx-auto" /&gt;
          &lt;Skeleton className="h-4 w-1/2 mx-auto" /&gt;
          &lt;Skeleton className="h-32 w-full" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }

  if (!user) return null;

  if (!isAdmin) return null;

  return (
    &lt;SidebarProvider&gt;
      &lt;div className="min-h-screen flex w-full"&gt;
        &lt;AdminSidebar /&gt;
        &lt;SidebarInset className="flex flex-col flex-1"&gt;
          &lt;header className="h-14 border-b border-border flex items-center justify-between px-4 bg-background"&gt;
            &lt;div className="flex items-center gap-4"&gt;
              &lt;SidebarTrigger /&gt;
              &lt;Link to="/adm" className="flex items-center gap-2"&gt;
                &lt;img src={daraLogo} alt="DARA Studio" className="h-8 w-8 rounded" /&gt;
                &lt;span className="font-serif text-lg font-semibold text-primary"&gt;&lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt;
              &lt;/Link&gt;
              &lt;span className="text-xs font-medium text-destructive bg-destructive/10 px-2 py-0.5 rounded"&gt;Admin&lt;/span&gt;
            &lt;/div&gt;
            &lt;LanguageSelector /&gt;
          &lt;/header&gt;
          &lt;main className="flex-1 overflow-auto bg-muted/30"&gt;
            &lt;Outlet /&gt;
          &lt;/main&gt;
        &lt;/SidebarInset&gt;
      &lt;/div&gt;
    &lt;/SidebarProvider&gt;
  );
};

export default AdminLayout;
```

src/components/admin/AdminSidebar.tsx

```
import { useNavigate, useLocation } from "react-router-dom";
import { useAuth } from "@/hooks/useAuth";
import {
  Sidebar, SidebarContent, SidebarGroup, SidebarGroupContent, SidebarGroupLabel,
  SidebarMenu, SidebarMenuButton, SidebarMenuItem, SidebarFooter,
} from "@/components/ui/sidebar";
import {
  LayoutDashboard, FolderKanban, Building2, UserPlus, FileText, FileSignature,
  Receipt, PenTool, FolderOpen, BrainCircuit, Globe, BarChart3,
  Settings, LogOut, Ticket, PieChart, HardHat,
} from "lucide-react";

const sidebarGroups = [
  {
    label: "Dashboard",
    items: [
      { title: "Dashboard", url: "/adm", icon: LayoutDashboard },
    ],
  },
  {
    label: "CRM",
    items: [
      { title: "Leads", url: "/adm/leads", icon: UserPlus },
      { title: "Companies", url: "/adm/companies", icon: Building2 },
    ],
  },
  {
    label: "Projects",
    items: [
      { title: "All Projects", url: "/adm/projects", icon: FolderKanban },
      { title: "Drawings", url: "/adm/drawings", icon: PenTool },
      { title: "Files", url: "/adm/files", icon: FolderOpen },
      { title: "Site Updates", url: "/adm/site-updates", icon: HardHat },
    ],
  },
  {
    label: "Finance",
    items: [
      { title: "Finance Overview", url: "/adm/finance-overview", icon: PieChart },
      { title: "Proposals", url: "/adm/proposals", icon: FileText },
      { title: "Contracts", url: "/adm/contracts", icon: FileSignature },
      { title: "Invoices", url: "/adm/payments", icon: Receipt },
    ],
  },
  {
    label: "Clients",
    items: [
      { title: "Client Portal", url: "/adm/client-portal", icon: Globe },
      { title: "Client Tickets", url: "/adm/tickets", icon: Ticket },
    ],
  },
  {
    label: "Tools",
    items: [
      { title: "AI Code Check", url: "/adm/ai-code-check", icon: BrainCircuit },
    ],
  },
  {
    label: "Insights",
    items: [
      { title: "Analytics", url: "/adm/analytics", icon: BarChart3 },
    ],
  },
  {
    label: "Settings",
    items: [
      { title: "Settings", url: "/adm/settings", icon: Settings },
    ],
  },
];

const AdminSidebar = () =&gt; {
  const navigate = useNavigate();
  const location = useLocation();
  const { signOut } = useAuth();

  return (
    &lt;Sidebar&gt;
      &lt;SidebarContent&gt;
        {sidebarGroups.map((group) =&gt; (
          &lt;SidebarGroup key={group.label}&gt;
            &lt;SidebarGroupLabel className="text-xs uppercase tracking-wider text-muted-foreground/70"&gt;{group.label}&lt;/SidebarGroupLabel&gt
            &lt;SidebarGroupContent&gt;
              &lt;SidebarMenu&gt;
```

```
                {group.items.map((item) =&gt; (
                  &lt;SidebarMenuItem key={item.url}&gt;
                    &lt;SidebarMenuButton
                      onClick={() =&gt; navigate(item.url)}
                      isActive={location.pathname === item.url || (item.url !== "/adm" &amp;&amp; location.pathname.startsWith(item.url + "/"))}
                    &gt;
                      &lt;item.icon className="h-4 w-4" /&gt;
                      &lt;span&gt;{item.title}&lt;/span&gt;
                    &lt;/SidebarMenuButton&gt;
                  &lt;/SidebarMenuItem&gt;
                ))}
              &lt;/SidebarMenu&gt;
            &lt;/SidebarGroupContent&gt;
          &lt;/SidebarGroup&gt;
        ))}
      &lt;/SidebarContent&gt;
      &lt;SidebarFooter&gt;
        &lt;SidebarMenu&gt;
          &lt;SidebarMenuItem&gt;
            &lt;SidebarMenuButton onClick={async () =&gt; { await signOut(); navigate("/"); }}&gt;
              &lt;LogOut className="h-4 w-4" /&gt;
              &lt;span&gt;Sign Out&lt;/span&gt;
            &lt;/SidebarMenuButton&gt;
          &lt;/SidebarMenuItem&gt;
        &lt;/SidebarMenu&gt;
      &lt;/SidebarFooter&gt;
    &lt;/Sidebar&gt;
  );
};

export default AdminSidebar;
```

src/components/admin/CompanyEmployees.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Separator } from "@/components/ui/separator";
import { Plus, Trash2, Pencil, X, Check, UserPlus } from "lucide-react";
import { toast } from "sonner";

interface Employee {
  id: string;
  company_id: string;
  name: string;
  phone: string | null;
  email: string | null;
  instagram: string | null;
  linkedin: string | null;
}

const emptyEmployee = { name: "", phone: "", email: "", instagram: "", linkedin: "" };

const CompanyEmployees = ({ companyId }: { companyId: string }) =&gt; {
  const [employees, setEmployees] = useState&lt;Employee[]&gt;([]);
  const [adding, setAdding] = useState(false);
  const [editingId, setEditingId] = useState&lt;string | null&gt;(null);
  const [form, setForm] = useState(emptyEmployee);

  const fetch = async () =&gt; {
    const { data } = await supabase
      .from("company_employees")
      .select("*")
      .eq("company_id", companyId)
      .order("name");
    setEmployees((data as Employee[]) || []);
  };

  useEffect(() =&gt; { fetch(); }, [companyId]);

  const handleSave = async () =&gt; {
    if (!form.name.trim()) { toast.error("Nome é obrigatório"); return; }

    if (editingId) {
      const { error } = await supabase.from("company_employees").update({
        name: form.name, phone: form.phone || null, email: form.email || null,
        instagram: form.instagram || null, linkedin: form.linkedin || null,
      }).eq("id", editingId);
      if (error) { toast.error(error.message); return; }
      toast.success("Funcionário atualizado");
    } else {
      const { error } = await supabase.from("company_employees").insert({
        company_id: companyId, name: form.name,
        phone: form.phone || null, email: form.email || null,
        instagram: form.instagram || null, linkedin: form.linkedin || null,
      });
      if (error) { toast.error(error.message); return; }
      toast.success("Funcionário adicionado");
    }
    setForm(emptyEmployee); setAdding(false); setEditingId(null);
    fetch();
  };

  const handleDelete = async (id: string) =&gt; {
    if (!confirm("Excluir funcionário?")) return;
    const { error } = await supabase.from("company_employees").delete().eq("id", id);
    if (error) { toast.error(error.message); return; }
    toast.success("Funcionário excluído"); fetch();
  };

  const startEdit = (e: Employee) =&gt; {
    setEditingId(e.id);
    setAdding(true);
    setForm({ name: e.name, phone: e.phone || "", email: e.email || "", instagram: e.instagram || "", linkedin: e.linkedin || "" });
  };

  const cancel = () =&gt; { setAdding(false); setEditingId(null); setForm(emptyEmployee); };

  return (
    &lt;div className="space-y-4"&gt;
      {employees.length === 0 &amp;amp;&amp;amp; !adding &amp;amp;&amp;amp; (
        &lt;p className="text-sm text-muted-foreground text-center py-4"&gt;Nenhum funcionário cadastrado&lt;/p&gt;
      )}

      {employees.map(emp =&gt; (
        &lt;div key={emp.id} className="flex items-start justify-between p-3 rounded-md border border-border bg-muted/30"&gt;
```

```
          &lt;div className="space-y-0.5"&gt;
            &lt;p className="font-medium text-sm text-foreground"&gt;{emp.name}&lt;/p&gt;
            {emp.email &amp;&amp; &lt;p className="text-xs text-muted-foreground"&gt;{emp.email}&lt;/p&gt;}
            {emp.phone &amp;&amp; &lt;p className="text-xs text-muted-foreground"&gt;{emp.phone}&lt;/p&gt;}
            &lt;div className="flex gap-3 text-xs text-muted-foreground"&gt;
              {emp.instagram &amp;&amp; &lt;span&gt;@{emp.instagram.replace("@", "")}&lt;/span&gt;}
              {emp.linkedin &amp;&amp; &lt;span&gt;LinkedIn&lt;/span&gt;}
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div className="flex gap-1"&gt;
            &lt;Button variant="ghost" size="icon" className="h-7 w-7" onClick={() =&gt; startEdit(emp)}&gt;&lt;Pencil className="h-3.5 w-3.5" /&
            &lt;Button variant="ghost" size="icon" className="h-7 w-7" onClick={() =&gt; handleDelete(emp.id)}&gt;&lt;Trash2 className="h-3.5 w-3
          &lt;/div&gt;
        &lt;/div&gt;
      ))}

      {adding ? (
        &lt;div className="space-y-3 p-3 rounded-md border border-border"&gt;
          &lt;div&gt;&lt;Label&gt;Nome *&lt;/Label&gt;&lt;Input value={form.name} onChange={e =&gt; setForm({ ...form, name: e.target.value })} /
          &lt;div className="grid grid-cols-2 gap-3"&gt;
            &lt;div&gt;&lt;Label&gt;Telefone&lt;/Label&gt;&lt;Input value={form.phone} onChange={e =&gt; setForm({ ...form, phone: e.target.value
            &lt;div&gt;&lt;Label&gt;Email&lt;/Label&gt;&lt;Input type="email" value={form.email} onChange={e =&gt; setForm({ ...form, email: e.ta
          &lt;/div&gt;
          &lt;div className="grid grid-cols-2 gap-3"&gt;
            &lt;div&gt;&lt;Label&gt;Instagram&lt;/Label&gt;&lt;Input value={form.instagram} onChange={e =&gt; setForm({ ...form, instagram: e.tar
            &lt;div&gt;&lt;Label&gt;LinkedIn&lt;/Label&gt;&lt;Input value={form.linkedin} onChange={e =&gt; setForm({ ...form, linkedin: e.target
          &lt;/div&gt;
          &lt;div className="flex gap-2"&gt;
            &lt;Button size="sm" onClick={handleSave}&gt;&lt;Check className="h-3.5 w-3.5 mr-1" /&gt;{editingId ? "Salvar" : "Adicionar"}&lt;/But
            &lt;Button size="sm" variant="outline" onClick={cancel}&gt;&lt;X className="h-3.5 w-3.5 mr-1" /&gt;Cancelar&lt;/Button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      ) : (
        &lt;Button variant="outline" size="sm" className="w-full" onClick={() =&gt; setAdding(true)}&gt;
          &lt;UserPlus className="h-4 w-4 mr-2" /&gt;Adicionar Funcionário
        &lt;/Button&gt;
      )}
    &lt;/div&gt;
  );
};

export default CompanyEmployees;
```

src/components/admin/ProjectDocuments.tsx

```
import { useCallback, useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import {
  Download, Upload, FolderOpen, FileText, ChevronRight, Trash2, Plus,
} from "lucide-react";
import { format } from "date-fns";
import { toast } from "sonner";

const DOCUMENT_CATEGORIES = [
  { value: "pre_construction", label: "Pre-Construction", icon: "■" },
  { value: "rev_00", label: "REV 00", icon: "■" },
  { value: "rev_01", label: "REV 01", icon: "■" },
  { value: "rev_02", label: "REV 02", icon: "■" },
  { value: "rev_03", label: "REV 03 (Final)", icon: "■" },
  { value: "final_delivery", label: "Final Delivery", icon: "■" },
  { value: "general", label: "General", icon: "■" },
] as const;

type DocCategory = typeof DOCUMENT_CATEGORIES[number]["value"];

interface ProjectDocumentsProps {
  projectId: string;
}

const ProjectDocuments = ({ projectId }: ProjectDocumentsProps) =&gt; {
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [openFolders, setOpenFolders] = useState&lt;Set&lt;string&gt;&gt;(new Set(["pre_construction", "rev_00"]));
  const [uploading, setUploading] = useState(false);
  const [uploadCategory, setUploadCategory] = useState&lt;DocCategory&gt;("general");
  const [selectedFiles, setSelectedFiles] = useState&lt;FileList | null&gt;(null);

  const loadFiles = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_files")
      .select("*")
      .eq("project_id", projectId)
      .order("created_at", { ascending: false });
    setFiles(data || []);
  }, [projectId]);

  useEffect(() =&gt; { loadFiles(); }, [loadFiles]);

  const toggleFolder = (cat: string) =&gt; {
    setOpenFolders(prev =&gt; {
      const next = new Set(prev);
      next.has(cat) ? next.delete(cat) : next.add(cat);
      return next;
    });
  };

  const handleUpload = async () =&gt; {
    if (!selectedFiles || selectedFiles.length === 0) return;
    setUploading(true);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      for (const file of Array.from(selectedFiles)) {
        const ext = file.name.split(".").pop();
        const path = `${projectId}/${Date.now()}-${Math.random().toString(36).slice(2)}.${ext}`;
        const { error: uploadError } = await supabase.storage.from("project-files").upload(path, file);
        if (uploadError) { toast.error(`Upload failed: ${file.name}`); continue; }
        const { data: urlData } = supabase.storage.from("project-files").getPublicUrl(path);
        await supabase.from("project_files").insert({
          project_id: projectId,
          file_name: file.name,
          file_url: urlData.publicUrl,
          file_type: file.type || ext,
          file_size: file.size,
          uploaded_by: user?.id || null,
          document_category: uploadCategory,
        });
      }
      toast.success("Files uploaded");
      setSelectedFiles(null);
      loadFiles();
    } catch { toast.error("Upload error"); }
    setUploading(false);
  };
```

```
const handleDelete = async (fileId: string) =&gt; {
  const { error } = await supabase.from("project_files").delete().eq("id", fileId);
  if (error) { toast.error("Delete failed"); return; }
  toast.success("File deleted");
  loadFiles();
};

const handleCategoryChange = async (fileId: string, newCategory: string) =&gt; {
  const { error } = await supabase.from("project_files").update({ document_category: newCategory }).eq("id", fileId);
  if (error) { toast.error("Update failed"); return; }
  loadFiles();
};

const grouped = DOCUMENT_CATEGORIES.reduce((acc, cat) =&gt; {
  acc[cat.value] = files.filter(f =&gt; (f.document_category || "general") === cat.value);
  return acc;
}, {} as Record&lt;string, any[]&gt;);

const formatSize = (bytes: number | null) =&gt; {
  if (!bytes) return "—";
  if (bytes &lt; 1024) return `${bytes} B`;
  if (bytes &lt; 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
  return `${(bytes / 1048576).toFixed(1)} MB`;
};

return (
  &lt;div className="space-y-4"&gt;
    {/* Upload Bar */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-4"&gt;
        &lt;div className="flex items-center gap-3 flex-wrap"&gt;
          &lt;Select value={uploadCategory} onValueChange={(v) =&gt; setUploadCategory(v as DocCategory)}&gt;
            &lt;SelectTrigger className="w-48"&gt;
              &lt;SelectValue placeholder="Select folder" /&gt;
            &lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              {DOCUMENT_CATEGORIES.map(c =&gt; (
                &lt;SelectItem key={c.value} value={c.value}&gt;{c.icon} {c.label}&lt;/SelectItem&gt;
              ))}
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
          &lt;Input
            type="file"
            multiple
            className="max-w-xs"
            onChange={e =&gt; setSelectedFiles(e.target.files)}
          /&gt;
          &lt;Button onClick={handleUpload} disabled={uploading || !selectedFiles?.length} size="sm"&gt;
            &lt;Upload className="h-3.5 w-3.5 mr-1.5" /&gt;
            {uploading ? "Uploading..." : "Upload"}
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Folder Tree */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-2"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;CardTitle className="text-base"&gt;Project Documents&lt;/CardTitle&gt;
          &lt;Badge variant="secondary"&gt;{files.length} files&lt;/Badge&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;div className="divide-y divide-border"&gt;
          {DOCUMENT_CATEGORIES.map(cat =&gt; {
            const catFiles = grouped[cat.value];
            const isOpen = openFolders.has(cat.value);
            return (
              &lt;Collapsible key={cat.value} open={isOpen} onOpenChange={() =&gt; toggleFolder(cat.value)}&gt;
                &lt;CollapsibleTrigger className="flex items-center gap-3 w-full px-4 py-3 hover:bg-muted/50 transition-colors text-left"&gt;
                  &lt;ChevronRight className={`h-4 w-4 text-muted-foreground transition-transform ${isOpen ? "rotate-90" : ""}`} /&gt;
                  &lt;FolderOpen className="h-4 w-4 text-primary" /&gt;
                  &lt;span className="text-sm font-medium text-foreground flex-1"&gt;{cat.label}&lt;/span&gt;
                  &lt;Badge variant="outline" className="text-[10px]"&gt;{catFiles.length}&lt;/Badge&gt;
                &lt;/CollapsibleTrigger&gt;
                &lt;CollapsibleContent&gt;
                  {catFiles.length === 0 ? (
                    &lt;div className="px-12 py-4 text-xs text-muted-foreground italic"&gt;No files in this folder&lt;/div&gt;
                  ) : (
                    &lt;div className="pb-2"&gt;
                      {catFiles.map(f =&gt; (
                        &lt;div key={f.id} className="flex items-center gap-3 px-12 py-2 hover:bg-muted/30 transition-colors group"&gt;
                          &lt;FileText className="h-4 w-4 text-muted-foreground shrink-0" /&gt;
                          &lt;div className="flex-1 min-w-0"&gt;
                            &lt;p className="text-sm font-medium text-foreground truncate"&gt;{f.file_name}&lt;/p&gt;
                            &lt;p className="text-[11px] text-muted-foreground"&gt;
```

```
                                {formatSize(f.file_size)} · {format(new Date(f.created_at), "MMM d, yyyy")}
                              &lt;/p&gt;
                            &lt;/div&gt;
                            &lt;Select
                              value={f.document_category || "general"}
                              onValueChange={(v) =&gt; handleCategoryChange(f.id, v)}
                            &gt;
                              &lt;SelectTrigger className="w-32 h-7 text-[11px] opacity-0 group-hover:opacity-100 transition-opacity"&gt;
                                &lt;SelectValue /&gt;
                              &lt;/SelectTrigger&gt;
                              &lt;SelectContent&gt;
                                {DOCUMENT_CATEGORIES.map(c =&gt; (
                                  &lt;SelectItem key={c.value} value={c.value} className="text-xs"&gt;{c.label}&lt;/SelectItem&gt;
                                ))}
                              &lt;/SelectContent&gt;
                            &lt;/Select&gt;
                            &lt;Button variant="ghost" size="icon" className="h-7 w-7 shrink-0" asChild&gt;
                              &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-3.5 w-3.5" /&gt;&lt;
                            &lt;/Button&gt;
                            &lt;Button
                              variant="ghost"
                              size="icon"
                              className="h-7 w-7 shrink-0 opacity-0 group-hover:opacity-100 text-destructive hover:text-destructive"
                              onClick={() =&gt; handleDelete(f.id)}
                            &gt;
                              &lt;Trash2 className="h-3.5 w-3.5" /&gt;
                            &lt;/Button&gt;
                          &lt;/div&gt;
                        ))}
                      &lt;/div&gt;
                    )}
                  &lt;/CollapsibleContent&gt;
                &lt;/Collapsible&gt;
              );
            })}
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default ProjectDocuments;
```

src/components/admin/ProjectKanbanBoard.tsx

```
import { useEffect, useState, useRef, useCallback } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Popover, PopoverContent, PopoverTrigger } from "@/components/ui/popover";
import { Calendar } from "@/components/ui/calendar";
import { Checkbox } from "@/components/ui/checkbox";
import { ScrollArea } from "@/components/ui/scroll-area";
import {
  Plus, GripVertical, CalendarIcon, User, Flag, Pencil, Trash2,
  CheckCircle2, Clock, AlertTriangle, Eye,
} from "lucide-react";
import { format } from "date-fns";
import { cn } from "@/lib/utils";
import { toast } from "sonner";

import { KANBAN_STAGES, KANBAN_STAGE_COLORS as STAGE_COLORS, getStageLabel } from "@/constants/projectStages";

const PRIORITIES = [
  { value: "low", label: "Low", color: "text-muted-foreground bg-muted border-border" },
  { value: "medium", label: "Medium", color: "text-chart-4 bg-chart-4/15 border-chart-4/30" },
  { value: "high", label: "High", color: "text-chart-3 bg-chart-3/15 border-chart-3/30" },
  { value: "urgent", label: "Urgent", color: "text-destructive bg-destructive/15 border-destructive/30" },
];

interface ProjectTask {
  id: string;
  project_id: string;
  title: string;
  assigned_to: string | null;
  due_date: string | null;
  priority: string;
  stage: string;
  is_completed: boolean;
  completed_at: string | null;
  requires_client_review: boolean;
  sort_order: number;
  created_at: string;
  updated_at: string;
}

interface Props {
  projectId: string;
  onStageSync?: (stage: string) =&gt; void;
}

const emptyTask = {
  title: "",
  assigned_to: "",
  due_date: undefined as Date | undefined,
  priority: "medium",
  stage: "WAITING_CLIENT_INFO",
  requires_client_review: false,
};

export default function ProjectKanbanBoard({ projectId, onStageSync }: Props) {
  const [tasks, setTasks] = useState&lt;ProjectTask[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editingTask, setEditingTask] = useState&lt;ProjectTask | null&gt;(null);
  const [form, setForm] = useState(emptyTask);
  const [dragTaskId, setDragTaskId] = useState&lt;string | null&gt;(null);
  const [dragOverStage, setDragOverStage] = useState&lt;string | null&gt;(null);

  const fetchTasks = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_tasks")
      .select("*")
      .eq("project_id", projectId)
      .order("sort_order")
      .order("created_at");
    setTasks((data as ProjectTask[]) || []);
  }, [projectId]);

  useEffect(() =&gt; { fetchTasks(); }, [fetchTasks]);

  // Realtime subscription
  useEffect(() =&gt; {
    const channel = supabase
      .channel(`project_tasks_${projectId}`)
```

```
    .on("postgres_changes", { event: "*", schema: "public", table: "project_tasks", filter: `project_id=eq.${projectId}` }, () =&gt; {
      fetchTasks();
    })
    .subscribe();
  return () =&gt; { supabase.removeChannel(channel); };
}, [projectId, fetchTasks]);

const syncProjectStage = useCallback(async (updatedTasks: ProjectTask[]) =&gt; {
  // Count tasks per stage (excluding completed tasks)
  const activeTasks = updatedTasks.filter(t =&gt; !t.is_completed);
  if (activeTasks.length === 0) return;

  const stageCounts: Record&lt;string, number&gt; = {};
  activeTasks.forEach(t =&gt; {
    stageCounts[t.stage] = (stageCounts[t.stage] || 0) + 1;
  });

  // Find the stage with the most tasks
  let maxStage = "Lead";
  let maxCount = 0;
  for (const [stage, count] of Object.entries(stageCounts)) {
    if (count &gt; maxCount) {
      maxCount = count;
      maxStage = stage;
    }
  }

  onStageSync?.(maxStage);
}, [onStageSync]);

const openNew = (stage: string) =&gt; {
  setEditingTask(null);
  setForm({ ...emptyTask, stage });
  setDialogOpen(true);
};

const openEdit = (task: ProjectTask) =&gt; {
  setEditingTask(task);
  setForm({
    title: task.title,
    assigned_to: task.assigned_to || "",
    due_date: task.due_date ? new Date(task.due_date) : undefined,
    priority: task.priority,
    stage: task.stage,
    requires_client_review: task.requires_client_review,
  });
  setDialogOpen(true);
};

const handleSave = async () =&gt; {
  if (!form.title.trim()) { toast.error("Task title is required"); return; }

  const payload = {
    project_id: projectId,
    title: form.title.trim(),
    assigned_to: form.assigned_to?.trim() || null,
    due_date: form.due_date ? format(form.due_date, "yyyy-MM-dd") : null,
    priority: form.priority,
    stage: form.stage,
    requires_client_review: form.requires_client_review,
  };

  if (editingTask) {
    const { error } = await supabase.from("project_tasks").update(payload).eq("id", editingTask.id);
    if (error) { toast.error(error.message); return; }
    toast.success("Task updated");
  } else {
    const maxOrder = tasks.filter(t =&gt; t.stage === form.stage).reduce((max, t) =&gt; Math.max(max, t.sort_order), -1);
    const { error } = await supabase.from("project_tasks").insert({ ...payload, sort_order: maxOrder + 1 });
    if (error) { toast.error(error.message); return; }
    toast.success("Task created");
  }

  setDialogOpen(false);
  await fetchTasks();
};

const handleDelete = async (taskId: string) =&gt; {
  if (!confirm("Delete this task?")) return;
  await supabase.from("project_tasks").delete().eq("id", taskId);
  toast.success("Task deleted");
  fetchTasks();
};

const handleToggleComplete = async (task: ProjectTask) =&gt; {
  const nowCompleted = !task.is_completed;
  await supabase.from("project_tasks").update({
```

```
    is_completed: nowCompleted,
    completed_at: nowCompleted ? new Date().toISOString() : null,
  }).eq("id", task.id);

  // Log to activity_log when completed
  if (nowCompleted) {
    await supabase.from("activity_log").insert({
      project_id: projectId,
      description: `Task "${task.title}" completed`,
      log_type: "task_completed",
      log_date: new Date().toISOString(),
    });
  }

  await fetchTasks();
};

const handleDragStart = (e: React.DragEvent, taskId: string) =&gt; {
  setDragTaskId(taskId);
  e.dataTransfer.effectAllowed = "move";
  e.dataTransfer.setData("text/plain", taskId);
};

const handleDragOver = (e: React.DragEvent, stage: string) =&gt; {
  e.preventDefault();
  e.dataTransfer.dropEffect = "move";
  setDragOverStage(stage);
};

const handleDragLeave = () =&gt; {
  setDragOverStage(null);
};

const handleDrop = async (e: React.DragEvent, targetStage: string) =&gt; {
  e.preventDefault();
  setDragOverStage(null);
  const taskId = e.dataTransfer.getData("text/plain");
  if (!taskId) return;

  const task = tasks.find(t =&gt; t.id === taskId);
  if (!task || task.stage === targetStage) { setDragTaskId(null); return; }

  const maxOrder = tasks.filter(t =&gt; t.stage === targetStage).reduce((max, t) =&gt; Math.max(max, t.sort_order), -1);

  await supabase.from("project_tasks").update({
    stage: targetStage,
    sort_order: maxOrder + 1,
  }).eq("id", taskId);

  // Log stage change
  await supabase.from("activity_log").insert({
    project_id: projectId,
    description: `Task "${task.title}" moved to ${targetStage}`,
    log_type: "task_moved",
    log_date: new Date().toISOString(),
  });

  setDragTaskId(null);
  const updatedTasks = tasks.map(t =&gt; t.id === taskId ? { ...t, stage: targetStage } : t);
  setTasks(updatedTasks);
  syncProjectStage(updatedTasks);
  fetchTasks();
};

const getPriorityStyle = (priority: string) =&gt; {
  return PRIORITIES.find(p =&gt; p.value === priority)?.color || PRIORITIES[1].color;
};

const getTasksByStage = (stage: string) =&gt;
  tasks.filter(t =&gt; t.stage === stage).sort((a, b) =&gt; a.sort_order - b.sort_order);

const isOverdue = (dueDate: string | null) =&gt; {
  if (!dueDate) return false;
  return new Date(dueDate) &lt; new Date();
};

return (
  &lt;div className="space-y-4"&gt;
    &lt;ScrollArea className="w-full"&gt;
      &lt;div className="flex gap-3 pb-4 min-w-max"&gt;
        {KANBAN_STAGES.map(stage =&gt; {
          const stageTasks = getTasksByStage(stage);
          const colors = STAGE_COLORS[stage] || STAGE_COLORS["Lead"];
          const isDragTarget = dragOverStage === stage;

          return (
            &lt;div
```

```
    key={stage}
    className={cn(
      "w-[260px] flex-shrink-0 rounded-xl border transition-all",
      colors.bg,
      colors.border,
      isDragTarget &amp;&amp; "ring-2 ring-primary/40 scale-[1.01]"
    )}
    onDragOver={e =&gt; handleDragOver(e, stage)}
    onDragLeave={handleDragLeave}
    onDrop={e =&gt; handleDrop(e, stage)}
  &gt;
    {/* Column header */}
    &lt;div className={cn("px-3 py-2.5 rounded-t-xl flex items-center justify-between", colors.header)}&gt;
      &lt;div className="flex items-center gap-2"&gt;
        &lt;h3 className="text-xs font-semibold text-foreground uppercase tracking-wider"&gt;{getStageLabel(stage)}&lt;/h3&gt;
        &lt;span className="text-[10px] font-medium text-muted-foreground bg-background/60 rounded-full px-1.5 py-0.5"&gt;
          {stageTasks.length}
        &lt;/span&gt;
      &lt;/div&gt;
      &lt;Button variant="ghost" size="icon" className="h-6 w-6" onClick={() =&gt; openNew(stage)}&gt;
        &lt;Plus className="h-3.5 w-3.5" /&gt;
      &lt;/Button&gt;
    &lt;/div&gt;

    {/* Cards */}
    &lt;div className="p-2 space-y-2 min-h-[80px]"&gt;
      {stageTasks.map(task =&gt; (
        &lt;div
          key={task.id}
          draggable
          onDragStart={e =&gt; handleDragStart(e, task.id)}
          className={cn(
            "group rounded-lg border bg-card p-3 cursor-grab active:cursor-grabbing shadow-sm hover:shadow-md transition-all",
            task.is_completed &amp;&amp; "opacity-60",
            dragTaskId === task.id &amp;&amp; "opacity-40 scale-95"
          )}
        &gt;
          &lt;div className="flex items-start gap-2"&gt;
            &lt;button
              onClick={() =&gt; handleToggleComplete(task)}
              className="mt-0.5 shrink-0"
            &gt;
              &lt;CheckCircle2 className={cn(
                "h-4 w-4 transition-colors",
                task.is_completed ? "text-chart-2 fill-chart-2/20" : "text-muted-foreground/40 hover:text-chart-2"
              )} /&gt;
            &lt;/button&gt;
            &lt;div className="flex-1 min-w-0"&gt;
              &lt;p className={cn(
                "text-sm font-medium text-foreground leading-tight",
                task.is_completed &amp;&amp; "line-through text-muted-foreground"
              )}&gt;
                {task.title}
              &lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="flex gap-0.5 opacity-0 group-hover:opacity-100 transition-opacity"&gt;
              &lt;Button variant="ghost" size="icon" className="h-5 w-5" onClick={() =&gt; openEdit(task)}&gt;
                &lt;Pencil className="h-3 w-3" /&gt;
              &lt;/Button&gt;
              &lt;Button variant="ghost" size="icon" className="h-5 w-5" onClick={() =&gt; handleDelete(task.id)}&gt;
                &lt;Trash2 className="h-3 w-3 text-destructive" /&gt;
              &lt;/Button&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;div className="flex items-center gap-2 mt-2 flex-wrap"&gt;
            {task.assigned_to &amp;&amp; (
              &lt;span className="inline-flex items-center gap-1 text-[10px] text-muted-foreground bg-muted rounded-full px-1.5 py-0.
                &lt;User className="h-2.5 w-2.5" /&gt;
                {task.assigned_to}
              &lt;/span&gt;
            )}
            {task.due_date &amp;&amp; (
              &lt;span className={cn(
                "inline-flex items-center gap-1 text-[10px] rounded-full px-1.5 py-0.5",
                isOverdue(task.due_date) &amp;&amp; !task.is_completed
                  ? "text-destructive bg-destructive/10"
                  : "text-muted-foreground bg-muted"
              )}&gt;
                &lt;Clock className="h-2.5 w-2.5" /&gt;
                {format(new Date(task.due_date), "MMM d")}
              &lt;/span&gt;
            )}
            &lt;span className={cn(
              "inline-flex items-center gap-1 text-[10px] font-medium rounded-full px-1.5 py-0.5 border",
              getPriorityStyle(task.priority)
            )}&gt;
```

```
                &lt;Flag className="h-2.5 w-2.5" /&gt;
                {task.priority}
              &lt;/span&gt;
              {task.requires_client_review &amp;&amp; (
                &lt;span className="inline-flex items-center gap-1 text-[10px] font-medium text-chart-5 bg-chart-5/15 border-chart-5/30
                  &lt;Eye className="h-2.5 w-2.5" /&gt;
                  Client Review
                &lt;/span&gt;
              )}
            &lt;/div&gt;
          &lt;/div&gt;
        ))}

        {stageTasks.length === 0 &amp;&amp; (
          &lt;div className="text-center py-6 text-muted-foreground/40 text-xs"&gt;
            Drop tasks here
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
})}
&lt;/div&gt;
&lt;/ScrollArea&gt;

{/* Task Dialog */}
&lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
  &lt;DialogContent className="max-w-md"&gt;
    &lt;DialogHeader&gt;
      &lt;DialogTitle&gt;{editingTask ? "Edit Task" : "New Task"}&lt;/DialogTitle&gt;
    &lt;/DialogHeader&gt;
    &lt;div className="space-y-4"&gt;
      &lt;div&gt;
        &lt;Label&gt;Task Name&lt;/Label&gt;
        &lt;Input
          value={form.title}
          onChange={e =&gt; setForm(f =&gt; ({ ...f, title: e.target.value }))}
          placeholder="e.g. Kitchen Layout"
          className="mt-1"
        /&gt;
      &lt;/div&gt;
      &lt;div className="grid grid-cols-2 gap-4"&gt;
        &lt;div&gt;
          &lt;Label&gt;Assigned To&lt;/Label&gt;
          &lt;Input
            value={form.assigned_to}
            onChange={e =&gt; setForm(f =&gt; ({ ...f, assigned_to: e.target.value }))}
            placeholder="e.g. Emily"
            className="mt-1"
          /&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label&gt;Priority&lt;/Label&gt;
          &lt;Select value={form.priority} onValueChange={v =&gt; setForm(f =&gt; ({ ...f, priority: v }))}&gt;
            &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              {PRIORITIES.map(p =&gt; (
                &lt;SelectItem key={p.value} value={p.value}&gt;
                  &lt;span className="flex items-center gap-2"&gt;
                    &lt;Flag className={cn("h-3 w-3", p.color.split(" ")[0])} /&gt;
                    {p.label}
                  &lt;/span&gt;
                &lt;/SelectItem&gt;
              ))}
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;Label&gt;Due Date&lt;/Label&gt;
        &lt;Popover&gt;
          &lt;PopoverTrigger asChild&gt;
            &lt;Button
              variant="outline"
              className={cn(
                "w-full mt-1 justify-start text-left font-normal",
                !form.due_date &amp;&amp; "text-muted-foreground"
              )}
            &gt;
              &lt;CalendarIcon className="h-4 w-4 mr-2" /&gt;
              {form.due_date ? format(form.due_date, "PPP") : "Pick a date"}
            &lt;/Button&gt;
          &lt;/PopoverTrigger&gt;
          &lt;PopoverContent className="w-auto p-0" align="start"&gt;
            &lt;Calendar
              mode="single"
              selected={form.due_date}
```

```
                onSelect={d =&gt; setForm(f =&gt; ({ ...f, due_date: d }))}
                initialFocus
                className={cn("p-3 pointer-events-auto")}
              /&gt;
            &lt;/PopoverContent&gt;
          &lt;/Popover&gt;
        &lt;/div&gt;
        &lt;div&gt;
          &lt;Label&gt;Stage&lt;/Label&gt;
          &lt;Select value={form.stage} onValueChange={v =&gt; setForm(f =&gt; ({ ...f, stage: v }))}&gt;
            &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              {KANBAN_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/div&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Checkbox
            checked={form.requires_client_review}
            onCheckedChange={v =&gt; setForm(f =&gt; ({ ...f, requires_client_review: !!v }))}
          /&gt;
          &lt;Label className="text-sm font-normal"&gt;Requires Client Review&lt;/Label&gt;
        &lt;/div&gt;
        &lt;Button className="w-full" onClick={handleSave}&gt;
          {editingTask ? "Save Changes" : "Create Task"}
        &lt;/Button&gt;
      &lt;/div&gt;
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;
);
}
```

src/components/admin/ProjectTimeline.tsx

```
import { useEffect, useState, useCallback } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Badge } from "@/components/ui/badge";
import { Textarea } from "@/components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@/components/ui/dialog";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Progress } from "@/components/ui/progress";
import {
  Plus, CheckCircle2, Clock, DollarSign, FileText, Send,
  MessageSquare, Star, Package, Truck, ChevronDown, ChevronsUpDown,
  Trash2, RotateCcw, AlertTriangle,
} from "lucide-react";
import { format } from "date-fns";
import { cn } from "@/lib/utils";
import { toast } from "sonner";
import { useIsMobile } from "@/hooks/use-mobile";
import {
  REV_EVENTS, FINAL_EVENTS, TimelineEvent, STATUS_CONFIG, ALERT_CONFIG,
  getRevisionStatus, getRevisionAlert, getOverallAlert,
  getTimelineProgress, deriveStageFromTimeline, getRevisionDelay,
} from "./timeline/timelineUtils";

interface Props {
  projectId: string;
  deliveryDate?: string;
  dates?: {
    start_date?: string;
    delivery_date?: string;
    preview_delivery_date?: string;
    revision_return_date?: string;
    final_delivery_date?: string;
  };
  onDateChange?: (field: string, value: string) =&gt; void;
  onStageSync?: (stage: string) =&gt; void;
}

export default function ProjectTimeline({ projectId, deliveryDate, dates, onDateChange, onStageSync }: Props) {
  const [events, setEvents] = useState&lt;TimelineEvent[]&gt;([]);
  const [maxRevision, setMaxRevision] = useState(3);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [extraRevAmount, setExtraRevAmount] = useState("");
  const [addEventDialog, setAddEventDialog] = useState(false);
  const [newEvent, setNewEvent] = useState({ revision_number: 0, event_type: "", event_date: "", notes: "" });
  const [openRevisions, setOpenRevisions] = useState&lt;Record&lt;number, boolean&gt;&gt;({ 0: true });
  const [finalizationOpen, setFinalizationOpen] = useState(true);
  const isMobile = useIsMobile();

  const fetchEvents = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_timeline")
      .select("*")
      .eq("project_id", projectId)
      .order("revision_number")
      .order("created_at");
    const list = (data as TimelineEvent[]) || [];
    setEvents(list);
    const highestRev = list.reduce((m, e) =&gt; Math.max(m, e.revision_number), 3);
    setMaxRevision(highestRev &lt; 3 ? 3 : highestRev);
  }, [projectId]);

  useEffect(() =&gt; { fetchEvents(); }, [fetchEvents]);

  // Auto-sync stage from timeline events
  useEffect(() =&gt; {
    if (!onStageSync || events.length === 0) return;
    const derivedStage = deriveStageFromTimeline(events, maxRevision);
    if (derivedStage) onStageSync(derivedStage);
  }, [events, maxRevision, onStageSync]);

  // Auto-log activity on event changes
  const logActivity = async (description: string) =&gt; {
    await supabase.from("activity_log").insert({
      project_id: projectId,
      log_type: "timeline_event",
      description,
    });
  };

  const toggleAllRevisions = (expand: boolean) =&gt; {
```

```
  const newState: Record&lt;number, boolean&gt; = {};
  for (let i = 0; i &lt;= maxRevision; i++) newState[i] = expand;
  setOpenRevisions(newState);
  setFinalizationOpen(expand);
};

const markEvent = async (revNum: number, eventType: string) =&gt; {
  const existing = events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === eventType);
  if (existing) return; // Use delete button instead
  await supabase.from("project_timeline").insert({
    project_id: projectId,
    revision_number: revNum,
    event_type: eventType,
    event_date: format(new Date(), "yyyy-MM-dd"),
  });
  // Find label
  const allEvents = revNum === -1 ? FINAL_EVENTS : REV_EVENTS(revNum);
  const evDef = allEvents.find(e =&gt; e.type === eventType);
  await logActivity(`${evDef?.label || eventType} – ${format(new Date(), "MMM d")}`);
  toast.success("Event recorded");
  fetchEvents();
};

const deleteEvent = async (eventId: string, label: string) =&gt; {
  await supabase.from("project_timeline").delete().eq("id", eventId);
  await logActivity(`${label} removed – ${format(new Date(), "MMM d")}`);
  toast.success("Event deleted");
  fetchEvents();
};

const resetEvent = async (eventId: string, label: string) =&gt; {
  await supabase.from("project_timeline").update({ event_date: null }).eq("id", eventId);
  await logActivity(`${label} reset – ${format(new Date(), "MMM d")}`);
  toast.success("Event reset");
  fetchEvents();
};

const updateEventDate = async (eventId: string, date: string) =&gt; {
  await supabase.from("project_timeline").update({ event_date: date }).eq("id", eventId);
  fetchEvents();
};

const handleAddExtraRevision = async () =&gt; {
  const newRev = maxRevision + 1;
  const revEvents = REV_EVENTS(newRev);
  await supabase.from("project_timeline").insert(
    revEvents.map(e =&gt; ({
      project_id: projectId,
      revision_number: newRev,
      event_type: e.type,
      event_date: null,
      notes: null,
    }))
  );
  if (extraRevAmount &amp;&amp; Number(extraRevAmount) &gt; 0) {
    await supabase.from("payment_milestones").insert({
      project_id: projectId,
      stage: `Extra Revision REV ${String(newRev).padStart(2, "0")}`,
      percentage: 0,
      amount: Number(extraRevAmount),
      status: "pending",
    });
  }
  await logActivity(`Extra Revision REV ${String(newRev).padStart(2, "0")} added – ${format(new Date(), "MMM d")}`);
  toast.success(`Extra Revision REV ${String(newRev).padStart(2, "0")} added`);
  setDialogOpen(false);
  setExtraRevAmount("");
  setOpenRevisions(prev =&gt; ({ ...prev, [newRev]: true }));
  fetchEvents();
};

const handleAddCustomEvent = async () =&gt; {
  if (!newEvent.event_type.trim()) { toast.error("Event type required"); return; }
  await supabase.from("project_timeline").insert({
    project_id: projectId,
    revision_number: newEvent.revision_number,
    event_type: newEvent.event_type,
    event_date: newEvent.event_date || null,
    notes: newEvent.notes || null,
  });
  await logActivity(`Custom event: ${newEvent.event_type} – ${format(new Date(), "MMM d")}`);
  toast.success("Event added");
  setAddEventDialog(false);
  setNewEvent({ revision_number: 0, event_type: "", event_date: "", notes: "" });
  fetchEvents();
};
```

```
const getEventRecord = (revNum: number, eventType: string) =&gt;
  events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === eventType);

// Overall stats
const overallAlert = getOverallAlert(events, deliveryDate || dates?.delivery_date);
const alertCfg = ALERT_CONFIG[overallAlert];
const progressPct = getTimelineProgress(events, maxRevision);

const renderEventRow = (ev: { type: string; label: string; icon: React.ElementType }, revNum: number) =&gt; {
  const record = getEventRecord(revNum, ev.type);
  const Icon = ev.icon;
  return (
    &lt;div
      key={ev.type}
      className={cn(
        "flex items-center gap-2 px-3 py-2 rounded-md transition-colors group",
        record?.event_date ? "bg-chart-2/5" : "bg-transparent hover:bg-muted/50"
      )}
    &gt;
      {/* Click to mark */}
      &lt;button
        className={cn(
          "h-6 w-6 rounded-full flex items-center justify-center shrink-0 transition-colors",
          record?.event_date
            ? "bg-chart-2/20 text-chart-2"
            : "bg-muted text-muted-foreground hover:bg-primary/20 hover:text-primary cursor-pointer"
        )}
        onClick={() =&gt; !record?.event_date &amp;&amp; markEvent(revNum, ev.type)}
        disabled={!!record?.event_date}
      &gt;
        {record?.event_date ? &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt; : &lt;Icon className="h-3.5 w-3.5" /&gt;}
      &lt;/button&gt;

      &lt;div className="flex-1 min-w-0"&gt;
        &lt;p className={cn("text-sm", record?.event_date ? "text-foreground font-medium" : "text-muted-foreground")}&gt;
          {ev.label}
        &lt;/p&gt;
      &lt;/div&gt;

      {record?.event_date &amp;&amp; (
        &lt;span className="text-xs text-muted-foreground shrink-0 hidden sm:block"&gt;
          {format(new Date(record.event_date), "MMM d, yyyy")}
        &lt;/span&gt;
      )}

      {record &amp;&amp; (
        &lt;&gt;
          &lt;Input
            type="date"
            className="w-[120px] h-7 text-xs hidden sm:block"
            value={record.event_date || ""}
            onClick={e =&gt; e.stopPropagation()}
            onChange={e =&gt; updateEventDate(record.id, e.target.value)}
          /&gt;
          &lt;div className="flex items-center gap-0.5 opacity-0 group-hover:opacity-100 transition-opacity"&gt;
            &lt;Button
              variant="ghost"
              size="icon"
              className="h-6 w-6"
              title="Reset event"
              onClick={() =&gt; resetEvent(record.id, ev.label)}
            &gt;
              &lt;RotateCcw className="h-3 w-3 text-muted-foreground" /&gt;
            &lt;/Button&gt;
            &lt;Button
              variant="ghost"
              size="icon"
              className="h-6 w-6"
              title="Delete event"
              onClick={() =&gt; deleteEvent(record.id, ev.label)}
            &gt;
              &lt;Trash2 className="h-3 w-3 text-destructive" /&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/&gt;
      )}
    &lt;/div&gt;
  );
};

const renderRevisionBlock = (revNum: number) =&gt; {
  const revEvents = REV_EVENTS(revNum);
  const isExtra = revNum &gt; 3;
  const revLabel = `REV ${String(revNum).padStart(2, "0")}`;
  const status = getRevisionStatus(revNum, events);
  const statusCfg = STATUS_CONFIG[status];
  const alert = getRevisionAlert(revNum, events);
```

```
  const delay = getRevisionDelay(revNum, events);
  const completedCount = revEvents.filter(ev =&gt; getEventRecord(revNum, ev.type)?.event_date).length;
  const isOpen = openRevisions[revNum] ?? false;

  return (
    &lt;Collapsible
      key={`rev-${revNum}`}
      open={isOpen}
      onOpenChange={(open) =&gt; setOpenRevisions(prev =&gt; ({ ...prev, [revNum]: open }))}
    &gt;
      &lt;Card className={cn(
        "border shadow-sm",
        isExtra &amp;&amp; "border-l-4 border-l-chart-4",
        alert === "delayed" &amp;&amp; "border-destructive/50",
        alert === "waiting_client" &amp;&amp; "border-yellow-400/50 dark:border-yellow-600/50"
      )}&gt;
        &lt;CollapsibleTrigger asChild&gt;
          &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
            &lt;CardTitle className="text-sm flex items-center gap-2 flex-wrap"&gt;
              &lt;Package className="h-4 w-4 text-muted-foreground" /&gt;
              {revLabel}
              {revNum === 0 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Início&lt;/Badge&gt;}
              {revNum === 3 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Final&lt;/Badge&gt;}
              {isExtra &amp;&amp; &lt;Badge className="text-[10px] bg-chart-4/15 text-chart-4 border-chart-4/30"&gt;Extra&lt;/Badge&gt;}
              &lt;Badge className={cn("text-[10px] border-0", statusCfg.classes)}&gt;
                {statusCfg.icon} {statusCfg.label}
              &lt;/Badge&gt;
              {alert &amp;&amp; (
                &lt;Badge className={cn("text-[10px] border-0", ALERT_CONFIG[alert].classes)}&gt;
                  {ALERT_CONFIG[alert].icon} {ALERT_CONFIG[alert].label}
                &lt;/Badge&gt;
              )}
              {delay?.delayed &amp;&amp; (
                &lt;span className="text-[10px] text-destructive font-medium"&gt;
                  +{delay.days}d late
                &lt;/span&gt;
              )}
              &lt;span className="ml-auto text-xs text-muted-foreground font-normal"&gt;{completedCount}/{revEvents.length}&lt;/span&gt;
              &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground transition-transform", isOpen &amp;&amp; "rotate-180")} /&gt;
            &lt;/CardTitle&gt;
          &lt;/CardHeader&gt;
        &lt;/CollapsibleTrigger&gt;
        &lt;CollapsibleContent&gt;
          &lt;CardContent className="space-y-0.5 pt-0"&gt;
            {revEvents.map(ev =&gt; renderEventRow(ev, revNum))}
          &lt;/CardContent&gt;
        &lt;/CollapsibleContent&gt;
      &lt;/Card&gt;
    &lt;/Collapsible&gt;
  );
};

const renderFinalBlock = () =&gt; (
  &lt;Collapsible open={finalizationOpen} onOpenChange={setFinalizationOpen}&gt;
    &lt;Card className="border shadow-sm"&gt;
      &lt;CollapsibleTrigger asChild&gt;
        &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
          &lt;CardTitle className="text-sm flex items-center gap-2"&gt;
            &lt;Star className="h-4 w-4 text-chart-2" /&gt; Finalização
            &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground ml-auto transition-transform", finalizationOpen &amp;&amp; "rotate-180
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
      &lt;/CollapsibleTrigger&gt;
      &lt;CollapsibleContent&gt;
        &lt;CardContent className="space-y-0.5 pt-0"&gt;
          {FINAL_EVENTS.map(ev =&gt; {
            const isAutoType = ev.type === "invoice_sent_auto" || ev.type === "payment_received_auto";
            const autoRecords = isAutoType ? events.filter(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type) : [];
            const Icon = ev.icon;

            if (isAutoType &amp;&amp; autoRecords.length &gt; 0) {
              return autoRecords.map(ar =&gt; (
                &lt;div key={ar.id} className="flex items-center gap-2 px-3 py-2 rounded-md bg-chart-2/5"&gt;
                  &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-chart-2/20 text-chart-2"&gt;
                    &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;div className="flex-1 min-w-0"&gt;
                    &lt;p className="text-sm text-foreground font-medium"&gt;{ev.label}&lt;/p&gt;
                    {ar.notes &amp;&amp; &lt;p className="text-xs text-muted-foreground truncate"&gt;{ar.notes}&lt;/p&gt;}
                  &lt;/div&gt;
                  {ar.event_date &amp;&amp; (
                    &lt;span className="text-xs text-muted-foreground shrink-0"&gt;
                      {format(new Date(ar.event_date), "MMM d, yyyy")}
                    &lt;/span&gt;
                  )}
                &lt;/div&gt;
              ));
```

```
            }

            if (isAutoType) {
              return (
                &lt;div key={ev.type} className="flex items-center gap-2 px-3 py-2 rounded-md bg-transparent opacity-50"&gt;
                  &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-muted text-muted-foreground"&gt;
                    &lt;Icon className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;p className="text-sm text-muted-foreground"&gt;{ev.label}&lt;/p&gt;
                  &lt;span className="text-[10px] text-muted-foreground ml-auto"&gt;Auto-tracked&lt;/span&gt;
                &lt;/div&gt;
              );
            }

            return renderEventRow(ev, -1);
          })}
        &lt;/CardContent&gt;
      &lt;/CollapsibleContent&gt;
    &lt;/Card&gt;
  &lt;/Collapsible&gt;
);

const revisionNumbers = Array.from({ length: maxRevision + 1 }, (_, i) =&gt; i);

return (
  &lt;div className="space-y-4"&gt;
    {/* Overall Status Bar */}
    &lt;Card className="border shadow-sm"&gt;
      &lt;CardContent className="py-4 space-y-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;div className="flex items-center gap-3"&gt;
            &lt;Badge className={cn("text-xs border-0", alertCfg.classes)}&gt;
              {alertCfg.icon} {alertCfg.label}
            &lt;/Badge&gt;
            &lt;span className="text-sm text-muted-foreground"&gt;Timeline Progress&lt;/span&gt;
          &lt;/div&gt;
          &lt;span className="text-sm font-semibold"&gt;{progressPct}%&lt;/span&gt;
        &lt;/div&gt;
        &lt;Progress value={progressPct} className="h-2" /&gt;
        {overallAlert === "overdue" &amp;&amp; (
          &lt;div className="flex items-center gap-2 text-xs text-destructive"&gt;
            &lt;AlertTriangle className="h-3.5 w-3.5" /&gt;
            Project is past deadline
          &lt;/div&gt;
        )}
        {overallAlert === "waiting_client" &amp;&amp; (
          &lt;div className="flex items-center gap-2 text-xs text-yellow-700 dark:text-yellow-400"&gt;
            &lt;Clock className="h-3.5 w-3.5" /&gt;
            Waiting for client feedback
          &lt;/div&gt;
        )}
        {overallAlert === "delayed" &amp;&amp; (
          &lt;div className="flex items-center gap-2 text-xs text-destructive"&gt;
            &lt;AlertTriangle className="h-3.5 w-3.5" /&gt;
            One or more revisions are delayed
          &lt;/div&gt;
        )}
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Date fields */}
    {dates &amp;&amp; onDateChange &amp;&amp; (
      &lt;Card className="border shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;CardTitle className="text-base"&gt;Key Dates&lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          &lt;div className="grid grid-cols-2 lg:grid-cols-3 gap-4"&gt;
            &lt;div&gt;
              &lt;Label className="text-xs text-muted-foreground"&gt;Start Date&lt;/Label&gt;
              &lt;Input type="date" className="mt-1" value={dates.start_date || ""} onChange={e =&gt; onDateChange("start_date", e.target.value
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label className="text-xs text-muted-foreground"&gt;Preview Delivery&lt;/Label&gt;
              &lt;Input type="date" className="mt-1" value={dates.preview_delivery_date || ""} onChange={e =&gt; onDateChange("preview_delivery
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label className="text-xs text-muted-foreground"&gt;Client Revision Return&lt;/Label&gt;
              &lt;Input type="date" className="mt-1" value={dates.revision_return_date || ""} onChange={e =&gt; onDateChange("revision_return_d
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label className="text-xs text-muted-foreground"&gt;Final Delivery&lt;/Label&gt;
              &lt;Input type="date" className="mt-1" value={dates.final_delivery_date || ""} onChange={e =&gt; onDateChange("final_delivery_dat
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label className="text-xs text-muted-foreground"&gt;Project Deadline&lt;/Label&gt;
              &lt;Input type="date" className="mt-1" value={dates.delivery_date || ""} onChange={e =&gt; onDateChange("delivery_date", e.target
```

```
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    )}

    {/* Expand/Collapse All */}
    &lt;div className="flex items-center justify-between"&gt;
      &lt;Button variant="ghost" size="sm" onClick={() =&gt; toggleAllRevisions(true)} className="text-xs gap-1"&gt;
        &lt;ChevronsUpDown className="h-3.5 w-3.5" /&gt; Expand All
      &lt;/Button&gt;
      &lt;Button variant="ghost" size="sm" onClick={() =&gt; toggleAllRevisions(false)} className="text-xs gap-1"&gt;
        Collapse All
      &lt;/Button&gt;
    &lt;/div&gt;

    {/* Revision blocks in responsive grid */}
    &lt;div className={cn("grid gap-4", isMobile ? "grid-cols-1" : "grid-cols-1 lg:grid-cols-2")}&gt;
      {revisionNumbers.map(revNum =&gt; renderRevisionBlock(revNum))}
      {renderFinalBlock()}
    &lt;/div&gt;

    {/* Actions */}
    &lt;div className="flex gap-2"&gt;
      &lt;Button variant="outline" onClick={() =&gt; setDialogOpen(true)}&gt;
        &lt;Plus className="h-3.5 w-3.5 mr-1" /&gt; Add Extra Revision
      &lt;/Button&gt;
      &lt;Button variant="ghost" onClick={() =&gt; setAddEventDialog(true)}&gt;
        &lt;Plus className="h-3.5 w-3.5 mr-1" /&gt; Add Custom Event
      &lt;/Button&gt;
    &lt;/div&gt;

    {/* Extra Revision Dialog */}
    &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
      &lt;DialogContent className="max-w-sm"&gt;
        &lt;DialogHeader&gt;
          &lt;DialogTitle&gt;Add Extra Revision (REV {String(maxRevision + 1).padStart(2, "0")})&lt;/DialogTitle&gt;
        &lt;/DialogHeader&gt;
        &lt;div className="space-y-4"&gt;
          &lt;div&gt;
            &lt;Label&gt;Invoice Amount (optional)&lt;/Label&gt;
            &lt;Input type="number" placeholder="e.g. 500" value={extraRevAmount} onChange={e =&gt; setExtraRevAmount(e.target.value)} classNam
            &lt;p className="text-xs text-muted-foreground mt-1"&gt;A payment milestone will be created for this extra revision.&lt;/p&gt;
          &lt;/div&gt;
          &lt;Button className="w-full" onClick={handleAddExtraRevision}&gt;Create Extra Revision&lt;/Button&gt;
        &lt;/div&gt;
      &lt;/DialogContent&gt;
    &lt;/Dialog&gt;

    {/* Custom Event Dialog */}
    &lt;Dialog open={addEventDialog} onOpenChange={setAddEventDialog}&gt;
      &lt;DialogContent className="max-w-sm"&gt;
        &lt;DialogHeader&gt;
          &lt;DialogTitle&gt;Add Custom Event&lt;/DialogTitle&gt;
        &lt;/DialogHeader&gt;
        &lt;div className="space-y-4"&gt;
          &lt;div&gt;
            &lt;Label&gt;Event Type&lt;/Label&gt;
            &lt;Input value={newEvent.event_type} onChange={e =&gt; setNewEvent(n =&gt; ({ ...n, event_type: e.target.value }))} placeholder="e
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label&gt;Revision #&lt;/Label&gt;
            &lt;Select value={String(newEvent.revision_number)} onValueChange={v =&gt; setNewEvent(n =&gt; ({ ...n, revision_number: Number(v)
              &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                &lt;SelectItem value="-1"&gt;Finalização&lt;/SelectItem&gt;
                {Array.from({ length: maxRevision + 1 }, (_, i) =&gt; (
                  &lt;SelectItem key={i} value={String(i)}&gt;REV {String(i).padStart(2, "0")}&lt;/SelectItem&gt;
                ))}
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label&gt;Date&lt;/Label&gt;
            &lt;Input type="date" value={newEvent.event_date} onChange={e =&gt; setNewEvent(n =&gt; ({ ...n, event_date: e.target.value }))} cl
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label&gt;Notes&lt;/Label&gt;
            &lt;Textarea value={newEvent.notes} onChange={e =&gt; setNewEvent(n =&gt; ({ ...n, notes: e.target.value }))} className="mt-1" /&gt
          &lt;/div&gt;
          &lt;Button className="w-full" onClick={handleAddCustomEvent}&gt;Add Event&lt;/Button&gt;
        &lt;/div&gt;
      &lt;/DialogContent&gt;
    &lt;/Dialog&gt;
  &lt;/div&gt;
  );
}
```

src/components/admin/dashboard/ActivityFeed.tsx

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { MessageSquare, FileUp, CreditCard, CheckCircle } from "lucide-react";
import { formatDistanceToNow } from "date-fns";

interface ActivityFeedProps {
  activities: any[];
}

const iconMap: Record&lt;string, any&gt; = {
  message: MessageSquare,
  file_upload: FileUp,
  payment: CreditCard,
  default: CheckCircle,
};

const ActivityFeed = ({ activities }: ActivityFeedProps) =&gt; {
  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;CardTitle className="text-base font-semibold"&gt;Recent Activity&lt;/CardTitle&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;div className="space-y-3 max-h-[300px] overflow-y-auto"&gt;
          {activities.length === 0 &amp;&amp; (
            &lt;p className="text-sm text-muted-foreground text-center py-6"&gt;No recent activity&lt;/p&gt;
          )}
          {activities.slice(0, 8).map((a) =&gt; {
            const Icon = iconMap[a.log_type] || iconMap.default;
            return (
              &lt;div key={a.id} className="flex items-start gap-3 p-2 rounded-md hover:bg-muted/50 transition-colors"&gt;
                &lt;div className="p-1.5 rounded-md bg-primary/10 text-primary mt-0.5"&gt;
                  &lt;Icon className="h-3.5 w-3.5" /&gt;
                &lt;/div&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className="text-sm text-foreground line-clamp-1"&gt;{a.description}&lt;/p&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;
                    {formatDistanceToNow(new Date(a.created_at), { addSuffix: true })}
                  &lt;/p&gt;
                &lt;/div&gt;
                &lt;Badge variant="outline" className="text-[10px] shrink-0"&gt;{a.log_type}&lt;/Badge&gt;
              &lt;/div&gt;
            );
          })}
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default ActivityFeed;
```

src/components/admin/dashboard/ClientActivityFeed.tsx

```
import { useNavigate } from "react-router-dom";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Avatar, AvatarFallback } from "@/components/ui/avatar";
import { ScrollArea } from "@/components/ui/scroll-area";
import {
  CheckCircle,
  Upload,
  RotateCcw,
  MessageSquare,
  FileCheck,
  Download,
  Eye,
  Bell,
} from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import { getCountryFlag } from "@/utils/countryFlags";

interface ClientActivity {
  id: string;
  client_name: string;
  client_country?: string | null;
  company_id?: string | null;
  project_name: string;
  action: string;
  action_type: string;
  created_at: string;
}

interface ClientActivityFeedProps {
  activities: ClientActivity[];
}

const ACTION_CONFIG: Record&lt;string, { icon: any; color: string; label: string }&gt; = {
  approval: { icon: CheckCircle, color: "text-emerald-500 bg-emerald-500/10", label: "Approved" },
  upload: { icon: Upload, color: "text-blue-500 bg-blue-500/10", label: "Uploaded" },
  revision: { icon: RotateCcw, color: "text-amber-500 bg-amber-500/10", label: "Revision" },
  message: { icon: MessageSquare, color: "text-violet-500 bg-violet-500/10", label: "Message" },
  invoice: { icon: FileCheck, color: "text-emerald-600 bg-emerald-600/10", label: "Invoice" },
  download: { icon: Download, color: "text-cyan-500 bg-cyan-500/10", label: "Download" },
  view: { icon: Eye, color: "text-muted-foreground bg-muted", label: "Viewed" },
  default: { icon: Bell, color: "text-muted-foreground bg-muted", label: "Activity" },
};

const getInitials = (name: string) =&gt;
  name.split(" ").map((w) =&gt; w[0]).join("").toUpperCase().slice(0, 2);

const ClientActivityFeed = ({ activities }: ClientActivityFeedProps) =&gt; {
  const navigate = useNavigate();
  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;Client Activity&lt;/CardTitle&gt;
          {activities.length &gt; 0 &amp;&amp; (
            &lt;Badge variant="secondary" className="text-[10px]"&gt;
              {activities.length} recent
            &lt;/Badge&gt;
          )}
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;ScrollArea className="h-[340px] pr-3"&gt;
          {activities.length === 0 &amp;&amp; (
            &lt;div className="flex flex-col items-center justify-center py-12 text-center"&gt;
              &lt;Bell className="h-8 w-8 text-muted-foreground/40 mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;No client activity yet&lt;/p&gt;
            &lt;/div&gt;
          )}
          &lt;div className="space-y-1"&gt;
            {activities.map((a) =&gt; {
              const config = ACTION_CONFIG[a.action_type] || ACTION_CONFIG.default;
              const Icon = config.icon;
              return (
                &lt;div
                  key={a.id}
                  className="flex items-start gap-3 p-2.5 rounded-lg hover:bg-muted/50 transition-colors group"
                &gt;
                  &lt;Avatar className="h-8 w-8 shrink-0 mt-0.5"&gt;
                    &lt;AvatarFallback className="text-[10px] font-medium bg-primary/10 text-primary"&gt;
                      {getInitials(a.client_name)}
                    &lt;/AvatarFallback&gt;
                  &lt;/Avatar&gt;
                  &lt;div className="flex-1 min-w-0"&gt;
```

```
                &lt;p className="text-sm leading-snug"&gt;
                  &lt;span
                    className="font-medium text-foreground hover:text-primary cursor-pointer hover:underline"
                    onClick={() =&gt; a.company_id &amp;&amp; navigate(`/adm/clients/${a.company_id}`)}
                  &gt;{a.client_name} {getCountryFlag(a.client_country)}&lt;/span&gt;{" "}
                  &lt;span className="text-muted-foreground"&gt;{a.action}&lt;/span&gt;
                &lt;/p&gt;
                &lt;p className="text-xs text-muted-foreground mt-0.5 truncate"&gt;{a.project_name}&lt;/p&gt;
              &lt;/div&gt;
              &lt;div className="flex flex-col items-end gap-1 shrink-0"&gt;
                &lt;div className={`p-1 rounded-md ${config.color}`}&gt;
                  &lt;Icon className="h-3.5 w-3.5" /&gt;
                &lt;/div&gt;
                &lt;span className="text-[10px] text-muted-foreground whitespace-nowrap"&gt;
                  {formatDistanceToNow(new Date(a.created_at), { addSuffix: true })}
                &lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/ScrollArea&gt;
  &lt;/CardContent&gt;
&lt;/Card&gt;
);
};

export default ClientActivityFeed;
```

src/components/admin/dashboard/ClientTicketsWidget.tsx

```
import { useEffect, useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Button } from "@/components/ui/button";
import { supabase } from "@/integrations/supabase/client";
import { Ticket, MessageSquare, ArrowRight } from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import { useNavigate } from "react-router-dom";

interface TicketRow {
  id: string;
  subject: string;
  status: string;
  updated_at: string;
  projects: { name: string } | null;
  companies: { name: string } | null;
}

const STATUS_STYLES: Record&lt;string, { label: string; variant: "default" | "secondary" | "outline" | "destructive" }&gt; = {
  open: { label: "Open", variant: "destructive" },
  waiting: { label: "Waiting", variant: "secondary" },
  resolved: { label: "Resolved", variant: "outline" },
};

const ClientTicketsWidget = () =&gt; {
  const [tickets, setTickets] = useState&lt;TicketRow[]&gt;([]);
  const navigate = useNavigate();

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      const { data } = await supabase
        .from("client_tickets")
        .select("id, subject, status, updated_at, projects(name), companies(name)")
        .order("updated_at", { ascending: false })
        .limit(8);
      setTickets((data as any[]) || []);
    };
    fetch();
  }, []);

  const openCount = tickets.filter((t) =&gt; t.status === "open").length;

  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;Client Tickets&lt;/CardTitle&gt;
          &lt;div className="flex items-center gap-2"&gt;
            {openCount &gt; 0 &amp;&amp; (
              &lt;Badge variant="destructive" className="text-[10px]"&gt;
                {openCount} open
              &lt;/Badge&gt;
            )}
            &lt;Button variant="ghost" size="sm" className="h-7 text-xs" onClick={() =&gt; navigate("/adm/tickets")}&gt;
              View All &lt;ArrowRight className="h-3 w-3 ml-1" /&gt;
            &lt;/Button&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;ScrollArea className="h-[300px] pr-3"&gt;
          {tickets.length === 0 &amp;&amp; (
            &lt;div className="flex flex-col items-center justify-center py-12 text-center"&gt;
              &lt;Ticket className="h-8 w-8 text-muted-foreground/40 mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;No tickets yet&lt;/p&gt;
            &lt;/div&gt;
          )}
          &lt;div className="space-y-1"&gt;
            {tickets.map((t) =&gt; {
              const style = STATUS_STYLES[t.status] || STATUS_STYLES.open;
              return (
                &lt;div
                  key={t.id}
                  className="flex items-start gap-3 p-2.5 rounded-lg hover:bg-muted/50 transition-colors cursor-pointer group"
                  onClick={() =&gt; navigate(`/adm/tickets?id=${t.id}`)}
                &gt;
                  &lt;div className="p-1.5 rounded-md bg-accent text-accent-foreground mt-0.5"&gt;
                    &lt;MessageSquare className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;div className="flex-1 min-w-0"&gt;
                    &lt;p className="text-sm font-medium text-foreground line-clamp-1"&gt;{t.subject}&lt;/p&gt;
                    &lt;p className="text-xs text-muted-foreground truncate"&gt;
                      {t.companies?.name || "Client"} · {t.projects?.name || "Project"}
```

```
                &lt;/p&gt;
              &lt;/div&gt;
              &lt;div className="flex flex-col items-end gap-1 shrink-0"&gt;
                &lt;Badge variant={style.variant} className="text-[10px]"&gt;{style.label}&lt;/Badge&gt;
                &lt;span className="text-[10px] text-muted-foreground"&gt;
                  {formatDistanceToNow(new Date(t.updated_at), { addSuffix: true })}
                &lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/ScrollArea&gt;
  &lt;/CardContent&gt;
&lt;/Card&gt;
);
};

export default ClientTicketsWidget;
```

src/components/admin/dashboard/FinancialChart.tsx

```
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import {
  BarChart, Bar, LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, Legend,
} from "recharts";
import { DollarSign, TrendingUp, BarChart3 } from "lucide-react";

interface FinancialChartProps {
  payments: any[];
  projects: any[];
}

const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];

const tooltipStyle = {
  backgroundColor: "hsl(var(--card))",
  border: "1px solid hsl(var(--border))",
  borderRadius: "8px",
  fontSize: 12,
};

const FinancialChart = ({ payments, projects }: FinancialChartProps) =&gt; {
  // 1. Revenue per Month
  const monthlyRevenue: Record&lt;string, number&gt; = {};
  payments.forEach((p) =&gt; {
    if (p.status === "paid") {
      const date = new Date(p.payment_date || p.created_at);
      const key = months[date.getMonth()];
      monthlyRevenue[key] = (monthlyRevenue[key] || 0) + Number(p.amount_paid || 0);
    }
  });
  const revenueData = months
    .filter((m) =&gt; monthlyRevenue[m])
    .map((m) =&gt; ({ month: m, revenue: monthlyRevenue[m] }));

  // 2. Projects Started vs Completed per Month
  const projectMonthly: Record&lt;string, { started: number; completed: number }&gt; = {};
  projects.forEach((p) =&gt; {
    if (p.start_date) {
      const m = months[new Date(p.start_date).getMonth()];
      if (!projectMonthly[m]) projectMonthly[m] = { started: 0, completed: 0 };
      projectMonthly[m].started++;
    }
    if (p.stage === "Completed" &amp;&amp; p.updated_at) {
      const m = months[new Date(p.updated_at).getMonth()];
      if (!projectMonthly[m]) projectMonthly[m] = { started: 0, completed: 0 };
      projectMonthly[m].completed++;
    }
  });
  const projectData = months
    .filter((m) =&gt; projectMonthly[m])
    .map((m) =&gt; ({ month: m, ...projectMonthly[m] }));

  // 3. Pipeline Value
  const pipelineTotal = projects
    .filter((p) =&gt; p.stage !== "Completed" &amp;&amp; p.stage !== "Lead" &amp;&amp; p.stage !== "briefing")
    .reduce((s, p) =&gt; s + Number(p.total_value || 0), 0);
  const totalRevenue = payments
    .filter((p) =&gt; p.status === "paid")
    .reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const totalPending = payments
    .filter((p) =&gt; p.status !== "paid")
    .reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);

  const fmt = (v: number) =&gt;
    v &gt;= 1000 ? `$${(v / 1000).toFixed(1)}k` : `$${v.toFixed(0)}`;

  return (
    &lt;div className="space-y-6"&gt;
      {/* Revenue per Month */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;DollarSign className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base font-semibold"&gt;Revenue per Month&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          {revenueData.length === 0 ? (
            &lt;p className="text-sm text-muted-foreground text-center py-10"&gt;No payment data yet&lt;/p&gt;
          ) : (
            &lt;ResponsiveContainer width="100%" height={220}&gt;
              &lt;BarChart data={revenueData}&gt;
                &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
```

```
                &lt;XAxis dataKey="month" tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                &lt;YAxis tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" tickFormatter={fmt} /&gt;
                &lt;Tooltip contentStyle={tooltipStyle} formatter={(v: number) =&gt; [`$${v.toLocaleString()}`, "Revenue"]} /&gt;
                &lt;Bar dataKey="revenue" fill="hsl(var(--chart-2))" radius={[4, 4, 0, 0]} /&gt;
              &lt;/BarChart&gt;
            &lt;/ResponsiveContainer&gt;
          )}
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Projects Started vs Completed */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;TrendingUp className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base font-semibold"&gt;Projects Started vs Completed&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          {projectData.length === 0 ? (
            &lt;p className="text-sm text-muted-foreground text-center py-10"&gt;No project timeline data yet&lt;/p&gt;
          ) : (
            &lt;ResponsiveContainer width="100%" height={220}&gt;
              &lt;LineChart data={projectData}&gt;
                &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
                &lt;XAxis dataKey="month" tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                &lt;YAxis tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" allowDecimals={false} /&gt;
                &lt;Tooltip contentStyle={tooltipStyle} /&gt;
                &lt;Legend wrapperStyle={{ fontSize: 12 }} /&gt;
                &lt;Line type="monotone" dataKey="started" stroke="hsl(var(--chart-1))" strokeWidth={2} dot={{ r: 4 }} name="Started" /&gt;
                &lt;Line type="monotone" dataKey="completed" stroke="hsl(var(--chart-2))" strokeWidth={2} dot={{ r: 4 }} name="Completed" /&gt;
              &lt;/LineChart&gt;
            &lt;/ResponsiveContainer&gt;
          )}
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Pipeline Value Summary */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;BarChart3 className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base font-semibold"&gt;Revenue Pipeline&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent&gt;
          &lt;div className="grid grid-cols-3 gap-4"&gt;
            &lt;div className="text-center p-4 rounded-lg bg-muted/50"&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{fmt(totalRevenue)}&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground mt-1"&gt;Total Received&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="text-center p-4 rounded-lg bg-muted/50"&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{fmt(totalPending)}&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground mt-1"&gt;Pending Payments&lt;/p&gt;
            &lt;/div&gt;
            &lt;div className="text-center p-4 rounded-lg bg-muted/50"&gt;
              &lt;p className="text-2xl font-bold text-foreground"&gt;{fmt(pipelineTotal)}&lt;/p&gt;
              &lt;p className="text-xs text-muted-foreground mt-1"&gt;Pipeline Value&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default FinancialChart;
```

src/components/admin/dashboard/KPICards.tsx

```
import { useNavigate } from "react-router-dom";
import { Card, CardContent } from "@/components/ui/card";
import { TrendingUp, TrendingDown, Minus, UserPlus, FolderKanban, FileText, DollarSign, AlertTriangle, MessageSquareWarning } from "lucide-react"

interface KPICardsProps {
  totalProjects: number;
  activeProjects: number;
  totalClients: number;
  newLeads: number;
  revenue: number;
  pendingPayments: number;
  delayedProjects?: number;
  proposalsSent?: number;
  clientTickets?: number;
}

const KPICards = ({
  activeProjects,
  newLeads,
  revenue,
  delayedProjects = 0,
  proposalsSent = 0,
  clientTickets = 0,
}: KPICardsProps) =&gt; {
  const navigate = useNavigate();

  const cards = [
    {
      label: "Leads Received", sublabel: "Last 30 days", value: newLeads,
      icon: UserPlus, color: "text-blue-600", bg: "bg-blue-50 dark:bg-blue-950/40",
      iconBg: "bg-blue-100 dark:bg-blue-900/50", borderAccent: "border-l-blue-500",
      change: "+4", changeType: "up" as const, url: "/adm/leads",
    },
    {
      label: "Active Projects", sublabel: "In progress", value: activeProjects,
      icon: FolderKanban, color: "text-emerald-600", bg: "bg-emerald-50 dark:bg-emerald-950/40",
      iconBg: "bg-emerald-100 dark:bg-emerald-900/50", borderAccent: "border-l-emerald-500",
      change: "+2", changeType: "up" as const, url: "/adm/projects",
    },
    {
      label: "Proposals Sent", sublabel: "This month", value: proposalsSent,
      icon: FileText, color: "text-amber-600", bg: "bg-amber-50 dark:bg-amber-950/40",
      iconBg: "bg-amber-100 dark:bg-amber-900/50", borderAccent: "border-l-amber-500",
      change: "0", changeType: "neutral" as const, url: "/adm/proposals",
    },
    {
      label: "Revenue Pipeline", sublabel: "Total value", value: `$${revenue.toLocaleString()}`,
      icon: DollarSign, color: "text-purple-600", bg: "bg-purple-50 dark:bg-purple-950/40",
      iconBg: "bg-purple-100 dark:bg-purple-900/50", borderAccent: "border-l-purple-500",
      change: "+12%", changeType: "up" as const, url: "/adm/analytics",
    },
    {
      label: "Delayed Projects", sublabel: "Needs attention", value: delayedProjects,
      icon: AlertTriangle, color: "text-red-600", bg: "bg-red-50 dark:bg-red-950/40",
      iconBg: "bg-red-100 dark:bg-red-900/50", borderAccent: "border-l-red-500",
      change: delayedProjects &gt; 0 ? `${delayedProjects}` : "0",
      changeType: delayedProjects &gt; 0 ? "down" as const : "neutral" as const,
      url: "/adm/projects",
    },
    {
      label: "Client Tickets", sublabel: "Open requests", value: clientTickets,
      icon: MessageSquareWarning, color: "text-orange-600", bg: "bg-orange-50 dark:bg-orange-950/40",
      iconBg: "bg-orange-100 dark:bg-orange-900/50", borderAccent: "border-l-orange-500",
      change: clientTickets &gt; 0 ? `${clientTickets} open` : "0",
      changeType: clientTickets &gt; 0 ? "down" as const : "neutral" as const,
      url: "/adm/tickets",
    },
  ];

  return (
    &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-6 gap-4"&gt;
      {cards.map((card) =&gt; (
        &lt;Card
          key={card.label}
          onClick={() =&gt; navigate(card.url)}
          className={`border-none shadow-sm hover:shadow-md transition-all border-l-4 ${card.borderAccent} ${card.bg} cursor-pointer`}
        &gt;
          &lt;CardContent className="p-4"&gt;
            &lt;div className="flex items-start justify-between mb-3"&gt;
              &lt;div className={`p-2 rounded-lg ${card.iconBg}`}&gt;
                &lt;card.icon className={`h-4 w-4 ${card.color}`} /&gt;
              &lt;/div&gt;
              &lt;div className="flex items-center gap-1 text-[11px] font-medium"&gt;
                {card.changeType === "up" &amp;&amp; &lt;TrendingUp className="h-3 w-3 text-emerald-500" /&gt;}
```

```
                {card.changeType === "down" &amp;&amp; &lt;TrendingDown className="h-3 w-3 text-red-500" /&gt;}
                {card.changeType === "neutral" &amp;&amp; &lt;Minus className="h-3 w-3 text-muted-foreground" /&gt;}
                &lt;span className={
                  card.changeType === "up" ? "text-emerald-600" :
                  card.changeType === "down" ? "text-red-600" :
                  "text-muted-foreground"
                }&gt;
                  {card.change}
                &lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;p className="text-2xl font-bold text-foreground leading-none mb-1"&gt;{card.value}&lt;/p&gt;
            &lt;p className="text-xs font-medium text-muted-foreground"&gt;{card.label}&lt;/p&gt;
            &lt;p className="text-[10px] text-muted-foreground/70"&gt;{card.sublabel}&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ))}
    &lt;/div&gt;
  );
};

export default KPICards;
```

src/components/admin/dashboard/NewLeadsTable.tsx

```
import { useEffect, useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger } from "@/components/ui/dropdown-menu";
import { formatDistanceToNow } from "date-fns";
import { supabase } from "@/integrations/supabase/client";
import { MoreHorizontal, UserPlus, Send, Phone, Inbox } from "lucide-react";
import { toast } from "sonner";

const NewLeadsTable = () =&gt; {
  const [leads, setLeads] = useState&lt;any[]&gt;([]);

  const fetchLeads = async () =&gt; {
    const { data } = await supabase
      .from("quote_requests")
      .select("*")
      .order("created_at", { ascending: false })
      .limit(10);
    setLeads(data || []);
  };

  useEffect(() =&gt; {
    fetchLeads();
  }, []);

  const handleConvert = async (lead: any) =&gt; {
    // Create company + project from lead
    const { data: company, error: compErr } = await supabase
      .from("companies")
      .insert({ name: lead.company_name || lead.full_name, email: lead.email, phone: lead.phone })
      .select()
      .single();

    if (compErr || !company) {
      toast.error("Failed to create client");
      return;
    }

    const { error: projErr } = await supabase.from("projects").insert({
      company_id: company.id,
      name: `${lead.full_name} - ${lead.service_type}`,
      service_type: lead.service_type,
      stage: "Lead",
      description: lead.description,
      address: lead.project_address,
    });

    if (!projErr) {
      await supabase.from("quote_requests").update({ processed: true }).eq("id", lead.id);
      toast.success("Lead converted to project");
      fetchLeads();
    } else {
      toast.error("Failed to create project");
    }
  };

  const handleMarkProcessed = async (id: string) =&gt; {
    await supabase.from("quote_requests").update({ processed: true }).eq("id", id);
    toast.success("Marked as processed");
    fetchLeads();
  };

  const serviceLabel = (s: string) =&gt;
    s?.replace(/_/g, " ").replace(/\b\w/g, (c) =&gt; c.toUpperCase()) || "—";

  return (
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Inbox className="h-4 w-4 text-muted-foreground" /&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;New Leads&lt;/CardTitle&gt;
          &lt;Badge variant="secondary" className="ml-auto"&gt;
            {leads.filter((l) =&gt; !l.processed).length} new
          &lt;/Badge&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead className="text-xs"&gt;Lead Name&lt;/TableHead&gt;
              &lt;TableHead className="text-xs"&gt;Project Type&lt;/TableHead&gt;
```

```
            &lt;TableHead className="text-xs"&gt;Status&lt;/TableHead&gt;
            &lt;TableHead className="text-xs"&gt;Received&lt;/TableHead&gt;
            &lt;TableHead className="text-xs w-[50px]"&gt;&lt;/TableHead&gt;
          &lt;/TableRow&gt;
        &lt;/TableHeader&gt;
        &lt;TableBody&gt;
          {leads.length === 0 &amp;&amp; (
            &lt;TableRow&gt;
              &lt;TableCell colSpan={5} className="text-center text-sm text-muted-foreground py-8"&gt;
                No leads from intake form yet
              &lt;/TableCell&gt;
            &lt;/TableRow&gt;
          )}
          {leads.map((lead) =&gt; (
            &lt;TableRow key={lead.id}&gt;
              &lt;TableCell&gt;
                &lt;div&gt;
                  &lt;p className="text-sm font-medium text-foreground"&gt;{lead.full_name}&lt;/p&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;{lead.email}&lt;/p&gt;
                &lt;/div&gt;
              &lt;/TableCell&gt;
              &lt;TableCell className="text-sm text-muted-foreground"&gt;
                {serviceLabel(lead.service_type)}
              &lt;/TableCell&gt;
              &lt;TableCell&gt;
                {lead.processed ? (
                  &lt;Badge variant="secondary" className="text-[10px]"&gt;Processed&lt;/Badge&gt;
                ) : (
                  &lt;Badge className="text-[10px] bg-chart-1/10 text-chart-1 border-chart-1/20" variant="outline"&gt;
                    New
                  &lt;/Badge&gt;
                )}
              &lt;/TableCell&gt;
              &lt;TableCell className="text-xs text-muted-foreground"&gt;
                {formatDistanceToNow(new Date(lead.created_at), { addSuffix: true })}
              &lt;/TableCell&gt;
              &lt;TableCell&gt;
                &lt;DropdownMenu&gt;
                  &lt;DropdownMenuTrigger asChild&gt;
                    &lt;Button variant="ghost" size="icon" className="h-7 w-7"&gt;
                      &lt;MoreHorizontal className="h-4 w-4" /&gt;
                    &lt;/Button&gt;
                  &lt;/DropdownMenuTrigger&gt;
                  &lt;DropdownMenuContent align="end"&gt;
                    &lt;DropdownMenuItem onClick={() =&gt; handleConvert(lead)} className="gap-2"&gt;
                      &lt;UserPlus className="h-3.5 w-3.5" /&gt; Convert to Project
                    &lt;/DropdownMenuItem&gt;
                    &lt;DropdownMenuItem onClick={() =&gt; window.open(`mailto:${lead.email}?subject=Proposal`, "_blank")} className="gap-2"&gt
                      &lt;Send className="h-3.5 w-3.5" /&gt; Send Proposal
                    &lt;/DropdownMenuItem&gt;
                    &lt;DropdownMenuItem onClick={() =&gt; lead.phone &amp;&amp; window.open(`tel:${lead.phone}`, "_blank")} className="gap-2"&
                      &lt;Phone className="h-3.5 w-3.5" /&gt; Schedule Call
                    &lt;/DropdownMenuItem&gt;
                    {!lead.processed &amp;&amp; (
                      &lt;DropdownMenuItem onClick={() =&gt; handleMarkProcessed(lead.id)} className="gap-2 text-muted-foreground"&gt;
                        Mark as Processed
                      &lt;/DropdownMenuItem&gt;
                    )}
                  &lt;/DropdownMenuContent&gt;
                &lt;/DropdownMenu&gt;
              &lt;/TableCell&gt;
            &lt;/TableRow&gt;
          ))}
        &lt;/TableBody&gt;
      &lt;/Table&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  );
};

export default NewLeadsTable;
```

src/components/admin/dashboard/ProjectPipeline.tsx

```
import { useState, useRef, DragEvent } from "react";
import { useNavigate } from "react-router-dom";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { ScrollArea, ScrollBar } from "@/components/ui/scroll-area";
import { Calendar, DollarSign, GripVertical, ImageIcon } from "lucide-react";
import { getCountryFlag } from "@/utils/countryFlags";
import { supabase } from "@/integrations/supabase/client";
import { toast } from "sonner";

const PIPELINE_STAGES = [
  { key: "Lead", color: "bg-slate-400", dot: "bg-slate-500" },
  { key: "Discovery Call", color: "bg-sky-400", dot: "bg-sky-500" },
  { key: "Proposal Sent", color: "bg-amber-400", dot: "bg-amber-500" },
  { key: "Contract Signed", color: "bg-violet-400", dot: "bg-violet-500" },
  { key: "Concept Design", color: "bg-indigo-400", dot: "bg-indigo-500" },
  { key: "Design Development", color: "bg-blue-400", dot: "bg-blue-500" },
  { key: "Construction Documents", color: "bg-cyan-400", dot: "bg-cyan-500" },
  { key: "Permit Submission", color: "bg-teal-400", dot: "bg-teal-500" },
  { key: "Construction", color: "bg-emerald-400", dot: "bg-emerald-500" },
  { key: "Completed", color: "bg-green-500", dot: "bg-green-600" },
];

interface ProjectPipelineProps {
  projects: any[];
  onProjectMoved?: () =&gt; void;
}

const ProjectPipeline = ({ projects, onProjectMoved }: ProjectPipelineProps) =&gt; {
  const navigate = useNavigate();
  const [draggedId, setDraggedId] = useState&lt;string | null&gt;(null);
  const [dragOverStage, setDragOverStage] = useState&lt;string | null&gt;(null);
  const scrollRef = useRef&lt;HTMLDivElement&gt;(null);

  const grouped: Record&lt;string, any[]&gt; = {};
  PIPELINE_STAGES.forEach((s) =&gt; (grouped[s.key] = []));
  projects.forEach((p) =&gt; {
    const stage = p.stage || "Lead";
    if (grouped[stage]) {
      grouped[stage].push(p);
    } else {
      // Map old stages to new pipeline
      if (stage === "briefing" || stage === "Budget Sent") grouped["Lead"].push(p);
      else if (stage === "Budget Approved") grouped["Proposal Sent"].push(p);
      else if (stage === "In Progress") grouped["Design Development"].push(p);
      else if (stage === "Preview Sent" || stage === "Revision" || stage === "Final Review") grouped["Construction Documents"].push(p);
      else grouped["Lead"].push(p);
    }
  });

  const handleDragStart = (e: DragEvent, projectId: string) =&gt; {
    setDraggedId(projectId);
    e.dataTransfer.effectAllowed = "move";
    e.dataTransfer.setData("text/plain", projectId);
  };

  const handleDragOver = (e: DragEvent, stageKey: string) =&gt; {
    e.preventDefault();
    e.dataTransfer.dropEffect = "move";
    setDragOverStage(stageKey);
  };

  const handleDragLeave = () =&gt; {
    setDragOverStage(null);
  };

  const handleDrop = async (e: DragEvent, stageKey: string) =&gt; {
    e.preventDefault();
    setDragOverStage(null);
    const projectId = e.dataTransfer.getData("text/plain");
    if (!projectId) return;

    const { error } = await supabase
      .from("projects")
      .update({ stage: stageKey })
      .eq("id", projectId);

    if (error) {
      toast.error("Failed to move project");
    } else {
      toast.success(`Moved to ${stageKey}`);
      onProjectMoved?.();
    }
    setDraggedId(null);
```

```
};

const handleDragEnd = () =&gt; {
  setDraggedId(null);
  setDragOverStage(null);
};

const getStatusTag = (p: any) =&gt; {
  if (p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date() &amp;&amp; p.stage !== "Completed") {
    return &lt;Badge variant="destructive" className="text-[9px] px-1.5 py-0"&gt;Overdue&lt;/Badge&gt;;
  }
  if (p.stage === "Completed") {
    return &lt;Badge className="text-[9px] px-1.5 py-0 bg-emerald-100 text-emerald-700 dark:bg-emerald-900/40 dark:text-emerald-300 border-0"&g
  }
  return &lt;Badge variant="secondary" className="text-[9px] px-1.5 py-0"&gt;Active&lt;/Badge&gt;;
};

return (
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardHeader className="pb-2"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;CardTitle className="text-base font-semibold"&gt;Project Pipeline&lt;/CardTitle&gt;
        &lt;span className="text-xs text-muted-foreground"&gt;{projects.length} projects&lt;/span&gt;
      &lt;/div&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent className="p-0"&gt;
      &lt;ScrollArea className="w-full" ref={scrollRef}&gt;
        &lt;div className="flex gap-3 p-4 min-w-max"&gt;
          {PIPELINE_STAGES.map((stage) =&gt; {
            const items = grouped[stage.key] || [];
            const isOver = dragOverStage === stage.key;
            return (
              &lt;div
                key={stage.key}
                className={`flex-shrink-0 w-[240px] rounded-lg transition-colors ${
                  isOver ? "bg-primary/5 ring-2 ring-primary/20" : "bg-muted/30"
                }`}
                onDragOver={(e) =&gt; handleDragOver(e, stage.key)}
                onDragLeave={handleDragLeave}
                onDrop={(e) =&gt; handleDrop(e, stage.key)}
              &gt;
                {/* Column header */}
                &lt;div className="flex items-center gap-2 p-3 pb-2"&gt;
                  &lt;span className={`w-2.5 h-2.5 rounded-full ${stage.dot}`} /&gt;
                  &lt;span className="text-xs font-semibold text-foreground uppercase tracking-wide truncate"&gt;
                    {stage.key}
                  &lt;/span&gt;
                  &lt;Badge variant="outline" className="text-[10px] h-4 px-1.5 ml-auto shrink-0"&gt;
                    {items.length}
                  &lt;/Badge&gt;
                &lt;/div&gt;

                {/* Cards */}
                &lt;div className="space-y-2 p-2 pt-0 min-h-[100px]"&gt;
                  {items.map((p) =&gt; (
                    &lt;div
                      key={p.id}
                      draggable
                      onDragStart={(e) =&gt; handleDragStart(e, p.id)}
                      onDragEnd={handleDragEnd}
                      className={`group rounded-lg border bg-card p-3 cursor-grab active:cursor-grabbing shadow-sm hover:shadow-md transition-a
                        draggedId === p.id ? "opacity-40 scale-95" : "opacity-100"
                      }`}
                    &gt;
                      {/* Top row: thumbnail + grip */}
                      &lt;div className="flex items-start gap-2 mb-2"&gt;
                        &lt;div className="w-8 h-8 rounded bg-muted flex items-center justify-center flex-shrink-0 overflow-hidden"&gt;
                          {p.cover_image ? (
                            &lt;img src={p.cover_image} alt="" className="w-full h-full object-cover" /&gt;
                          ) : (
                            &lt;ImageIcon className="h-3.5 w-3.5 text-muted-foreground/50" /&gt;
                          )}
                        &lt;/div&gt;
                        &lt;div className="flex-1 min-w-0"&gt;
                          &lt;p
                            className="text-xs font-semibold text-foreground truncate leading-tight hover:text-primary cursor-pointer"
                            onClick={(e) =&gt; { e.stopPropagation(); navigate(`/adm/projects/${p.id}`); }}
                          &gt;
                            {p.name || "Untitled Project"}
                          &lt;/p&gt;
                          &lt;p className="text-[11px] text-muted-foreground truncate"&gt;
                            &lt;span
                              className="hover:text-primary cursor-pointer hover:underline"
                              onClick={(e) =&gt; { e.stopPropagation(); if (p.company_id) navigate(`/adm/clients/${p.company_id}`); }}
                            &gt;
                              {p.companies?.name || "No client"}
                            &lt;/span&gt; {getCountryFlag(p.companies?.country)}
```

```
                    &lt;/p&gt;
                  &lt;/div&gt;
                  &lt;GripVertical className="h-3.5 w-3.5 text-muted-foreground/30 group-hover:text-muted-foreground/60 shrink-0 mt-0.5"
                &lt;/div&gt;

                {/* Meta row */}
                &lt;div className="flex items-center gap-2 flex-wrap"&gt;
                  {p.total_value &gt; 0 &amp;&amp; (
                    &lt;span className="flex items-center gap-0.5 text-[10px] text-muted-foreground"&gt;
                      &lt;DollarSign className="h-2.5 w-2.5" /&gt;
                      {Number(p.total_value).toLocaleString()}
                    &lt;/span&gt;
                  )}
                  {p.delivery_date &amp;&amp; (
                    &lt;span className="flex items-center gap-0.5 text-[10px] text-muted-foreground"&gt;
                      &lt;Calendar className="h-2.5 w-2.5" /&gt;
                      {new Date(p.delivery_date).toLocaleDateString("en-US", { month: "short", day: "numeric" })}
                    &lt;/span&gt;
                  )}
                  &lt;span className="ml-auto"&gt;{getStatusTag(p)}&lt;/span&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            ))}

            {items.length === 0 &amp;&amp; (
              &lt;div className="flex items-center justify-center h-[80px] border border-dashed border-border/50 rounded-lg"&gt;
                &lt;p className="text-[11px] text-muted-foreground/50"&gt;Drop here&lt;/p&gt;
              &lt;/div&gt;
            )}
          &lt;/div&gt;
        &lt;/div&gt;
      );
    })}
  &lt;/div&gt;
  &lt;ScrollBar orientation="horizontal" /&gt;
&lt;/ScrollArea&gt;
&lt;/CardContent&gt;
&lt;/Card&gt;
);
};

export default ProjectPipeline;
```

src/components/admin/dashboard/QuickActions.tsx

```
import { useNavigate } from "react-router-dom";
import { Button } from "@/components/ui/button";
import { FolderKanban, Send, Upload, UserPlus, Receipt } from "lucide-react";

const QuickActions = () =&gt; {
  const navigate = useNavigate();

  const actions = [
    { label: "New Project", icon: FolderKanban, url: "/adm/projects" },
    { label: "Send Proposal", icon: Send, url: "/adm/projects" },
    { label: "Upload Drawings", icon: Upload, url: "/adm/drawings" },
    { label: "Invite Client", icon: UserPlus, url: "/adm/companies" },
    { label: "Create Invoice", icon: Receipt, url: "/adm/payments" },
  ];

  return (
    &lt;div className="flex flex-wrap gap-2"&gt;
      {actions.map((action) =&gt; (
        &lt;Button
          key={action.label}
          variant="outline"
          onClick={() =&gt; navigate(action.url)}
          className="gap-2 h-9 px-4 text-sm font-medium"
        &gt;
          &lt;action.icon className="h-4 w-4" /&gt;
          {action.label}
        &lt;/Button&gt;
      ))}
    &lt;/div&gt;
  );
};

export default QuickActions;
```

src/components/admin/dashboard/TodaysTasks.tsx

```
import { useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Checkbox } from "@/components/ui/checkbox";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { CalendarDays, Plus, Flag, Filter } from "lucide-react";
import { supabase } from "@/integrations/supabase/client";
import { toast } from "sonner";

interface Task {
  id: string;
  title: string;
  project_id: string | null;
  priority: string;
  due_date: string;
  is_completed: boolean;
  project_name?: string;
}

interface TodaysTasksProps {
  tasks: Task[];
  projects: { id: string; name: string | null; companies?: { name: string } | null }[];
  onRefresh: () =&gt; void;
}

const priorityConfig: Record&lt;string, { label: string; className: string }&gt; = {
  high: { label: "High", className: "bg-destructive/10 text-destructive border-destructive/20" },
  medium: { label: "Medium", className: "bg-yellow-500/10 text-yellow-700 border-yellow-500/20" },
  low: { label: "Low", className: "bg-muted text-muted-foreground border-border" },
};

const TodaysTasks = ({ tasks, projects, onRefresh }: TodaysTasksProps) =&gt; {
  const [filter, setFilter] = useState&lt;string&gt;("all");
  const [dialogOpen, setDialogOpen] = useState(false);
  const [newTitle, setNewTitle] = useState("");
  const [newProjectId, setNewProjectId] = useState("");
  const [newPriority, setNewPriority] = useState("medium");
  const [newDueDate, setNewDueDate] = useState(new Date().toISOString().split("T")[0]);

  const filtered = tasks.filter((t) =&gt; {
    if (filter === "all") return true;
    if (filter === "pending") return !t.is_completed;
    if (filter === "completed") return t.is_completed;
    return t.priority === filter;
  });

  const handleToggle = async (task: Task) =&gt; {
    const nowCompleted = !task.is_completed;
    const { error } = await supabase
      .from("admin_tasks")
      .update({
        is_completed: nowCompleted,
        completed_at: nowCompleted ? new Date().toISOString() : null,
      })
      .eq("id", task.id);
    if (!error) onRefresh();
    else toast.error("Failed to update task");
  };

  const handleAdd = async () =&gt; {
    if (!newTitle.trim()) return;
    const { error } = await supabase.from("admin_tasks").insert({
      title: newTitle.trim(),
      project_id: newProjectId || null,
      priority: newPriority,
      due_date: newDueDate,
    });
    if (!error) {
      toast.success("Task added");
      setNewTitle("");
      setNewProjectId("");
      setNewPriority("medium");
      setNewDueDate(new Date().toISOString().split("T")[0]);
      setDialogOpen(false);
      onRefresh();
    } else {
      toast.error("Failed to add task");
    }
  };

  const pendingCount = tasks.filter((t) =&gt; !t.is_completed).length;
```

```
return (
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;div className="flex items-center gap-2"&gt;
        &lt;CalendarDays className="h-4 w-4 text-muted-foreground" /&gt;
        &lt;CardTitle className="text-base font-semibold"&gt;Today's Tasks&lt;/CardTitle&gt;
        &lt;Badge variant="secondary" className="ml-auto"&gt;{pendingCount} pending&lt;/Badge&gt;
      &lt;/div&gt;
      &lt;div className="flex items-center gap-2 pt-2"&gt;
        &lt;Filter className="h-3.5 w-3.5 text-muted-foreground" /&gt;
        &lt;Select value={filter} onValueChange={setFilter}&gt;
          &lt;SelectTrigger className="h-7 text-xs w-[120px]"&gt;
            &lt;SelectValue /&gt;
          &lt;/SelectTrigger&gt;
          &lt;SelectContent&gt;
            &lt;SelectItem value="all"&gt;All&lt;/SelectItem&gt;
            &lt;SelectItem value="pending"&gt;Pending&lt;/SelectItem&gt;
            &lt;SelectItem value="completed"&gt;Completed&lt;/SelectItem&gt;
            &lt;SelectItem value="high"&gt;High Priority&lt;/SelectItem&gt;
            &lt;SelectItem value="medium"&gt;Medium Priority&lt;/SelectItem&gt;
            &lt;SelectItem value="low"&gt;Low Priority&lt;/SelectItem&gt;
          &lt;/SelectContent&gt;
        &lt;/Select&gt;
        &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
          &lt;DialogTrigger asChild&gt;
            &lt;Button variant="ghost" size="sm" className="h-7 ml-auto gap-1 text-xs"&gt;
              &lt;Plus className="h-3.5 w-3.5" /&gt; Add
            &lt;/Button&gt;
          &lt;/DialogTrigger&gt;
          &lt;DialogContent className="max-w-md"&gt;
            &lt;DialogHeader&gt;
              &lt;DialogTitle&gt;New Task&lt;/DialogTitle&gt;
            &lt;/DialogHeader&gt;
            &lt;div className="space-y-3"&gt;
              &lt;Input placeholder="Task title" value={newTitle} onChange={(e) =&gt; setNewTitle(e.target.value)} /&gt;
              &lt;Select value={newProjectId} onValueChange={setNewProjectId}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Related project (optional)" /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
                  {projects.map((p) =&gt; (
                    &lt;SelectItem key={p.id} value={p.id}&gt;{p.name || p.companies?.name || "Untitled"}&lt;/SelectItem&gt;
                  ))}
                &lt;/SelectContent&gt;
              &lt;/Select&gt;
              &lt;div className="flex gap-3"&gt;
                &lt;Select value={newPriority} onValueChange={setNewPriority}&gt;
                  &lt;SelectTrigger className="flex-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;
                    &lt;SelectItem value="high"&gt;High&lt;/SelectItem&gt;
                    &lt;SelectItem value="medium"&gt;Medium&lt;/SelectItem&gt;
                    &lt;SelectItem value="low"&gt;Low&lt;/SelectItem&gt;
                  &lt;/SelectContent&gt;
                &lt;/Select&gt;
                &lt;Input type="date" className="flex-1" value={newDueDate} onChange={(e) =&gt; setNewDueDate(e.target.value)} /&gt;
              &lt;/div&gt;
              &lt;Button className="w-full" onClick={handleAdd} disabled={!newTitle.trim()}&gt;Add Task&lt;/Button&gt;
            &lt;/div&gt;
          &lt;/DialogContent&gt;
        &lt;/Dialog&gt;
      &lt;/div&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent&gt;
      &lt;div className="space-y-1.5 max-h-[280px] overflow-y-auto"&gt;
        {filtered.length === 0 &amp;amp;&amp;amp; (
          &lt;p className="text-sm text-muted-foreground text-center py-6"&gt;
            {filter === "all" ? "No tasks yet — add one!" : "No tasks match this filter"}
          &lt;/p&gt;
        )}
        {filtered.map((task) =&gt; {
          const prio = priorityConfig[task.priority] || priorityConfig.medium;
          return (
            &lt;div
              key={task.id}
              className={`flex items-center gap-3 p-2 rounded-md hover:bg-muted/50 transition-colors ${task.is_completed ? "opacity-50" : ""}`}
            &gt;
              &lt;Checkbox checked={task.is_completed} onCheckedChange={() =&gt; handleToggle(task)} /&gt;
              &lt;div className="flex-1 min-w-0"&gt;
                &lt;p className={`text-sm font-medium truncate ${task.is_completed ? "line-through text-muted-foreground" : "text-foreground"}`
                  {task.title}
                &lt;/p&gt;
                {task.project_name &amp;amp;&amp;amp; (
                  &lt;p className="text-xs text-muted-foreground truncate"&gt;{task.project_name}&lt;/p&gt;
                )}
              &lt;/div&gt;
              &lt;div className="flex items-center gap-1.5 shrink-0"&gt;
                &lt;Badge variant="outline" className={`text-[10px] gap-1 ${prio.className}`}&gt;
                  &lt;Flag className="h-2.5 w-2.5" /&gt;
                  {prio.label}
```

```
                &lt;/Badge&gt;
                &lt;span className="text-[10px] text-muted-foreground whitespace-nowrap"&gt;{task.due_date}&lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
);
};

export default TodaysTasks;
```

src/components/admin/timeline/timelineUtils.ts

```
import { LucideIcon } from "lucide-react";
import {
  CheckCircle2, Clock, DollarSign, FileText, Send,
  MessageSquare, Star, Package, Truck,
} from "lucide-react";

// ■■ Event definitions ■■

export interface EventDef {
  type: string;
  label: string;
  icon: LucideIcon;
}

export const REV_EVENTS = (rev: number): EventDef[] =&gt; {
  if (rev === 0) return [
    { type: "estimate_created", label: "Estimate Created", icon: FileText },
    { type: "initial_invoice_sent", label: "Initial Invoice Sent", icon: Send },
    { type: "initial_payment_received", label: "Initial Payment Received", icon: DollarSign },
    { type: "rev_started", label: `REV 0${rev} Started`, icon: Clock },
    { type: "rev_preview_sent", label: `REV 0${rev} Preview Sent`, icon: Send },
    { type: "client_feedback", label: `Client Feedback REV 0${rev}`, icon: MessageSquare },
  ];
  return [
    { type: "rev_started", label: `REV ${String(rev).padStart(2, "0")} Started`, icon: Clock },
    { type: "rev_preview_sent", label: `REV ${String(rev).padStart(2, "0")} Preview Sent`, icon: Send },
    { type: "client_feedback", label: `Client Feedback REV ${String(rev).padStart(2, "0")}`, icon: MessageSquare },
  ];
};

export const FINAL_EVENTS: EventDef[] = [
  { type: "invoice_sent_auto", label: "Invoice Sent", icon: Send },
  { type: "payment_received_auto", label: "Payment Received", icon: DollarSign },
  { type: "project_approved", label: "Project Approved", icon: Star },
  { type: "final_documents", label: "Final Documents", icon: FileText },
  { type: "final_payment", label: "Final Payment", icon: DollarSign },
  { type: "project_completed", label: "Project Completed", icon: CheckCircle2 },
  { type: "project_delivered", label: "Project Delivered", icon: Truck },
];

// ■■ Types ■■

export interface TimelineEvent {
  id: string;
  project_id: string;
  revision_number: number;
  event_type: string;
  event_date: string | null;
  notes: string | null;
  created_at: string;
}

export type RevStatus = "not_started" | "in_progress" | "completed";
export type AlertLevel = "on_track" | "waiting_client" | "delayed" | "overdue";

export const STATUS_CONFIG: Record&lt;RevStatus, { label: string; icon: string; classes: string }&gt; = {
  not_started: { label: "Not Started", icon: "■", classes: "bg-muted text-muted-foreground" },
  in_progress: { label: "In Progress", icon: "■", classes: "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-400" },
  completed: { label: "Completed", icon: "■", classes: "bg-chart-2/15 text-chart-2" },
};

export const ALERT_CONFIG: Record&lt;AlertLevel, { label: string; icon: string; classes: string }&gt; = {
  on_track: { label: "On Track", icon: "■", classes: "bg-chart-2/15 text-chart-2" },
  waiting_client: { label: "Waiting Client", icon: "■", classes: "bg-yellow-100 text-yellow-800 dark:bg-yellow-900/30 dark:text-yellow-400" },
  delayed: { label: "Delayed", icon: "■", classes: "bg-destructive/15 text-destructive" },
  overdue: { label: "Overdue", icon: "■", classes: "bg-destructive/15 text-destructive" },
};

// ■■ Status logic ■■

export function getRevisionStatus(revNum: number, events: TimelineEvent[]): RevStatus {
  const revEvents = REV_EVENTS(revNum);
  const completed = revEvents.filter(ev =&gt;
    events.some(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === ev.type &amp;&amp; e.event_date)
  ).length;
  if (completed === 0) return "not_started";
  if (completed === revEvents.length) return "completed";
  return "in_progress";
}

// ■■ Alert logic ■■

const WAITING_CLIENT_DAYS = 3;
const DELAYED_DAYS = 5;
```

```
export function getRevisionAlert(revNum: number, events: TimelineEvent[]): AlertLevel | null {
  const revEvents = REV_EVENTS(revNum);
  const status = getRevisionStatus(revNum, events);
  if (status === "completed" || status === "not_started") return null;

  const getRecord = (type: string) =&gt;
    events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === type);

  // Check: Preview sent but no feedback
  const previewSent = getRecord("rev_preview_sent");
  const feedback = getRecord("client_feedback");
  if (previewSent?.event_date &amp;&amp; !feedback?.event_date) {
    const daysSince = Math.floor((Date.now() - new Date(previewSent.event_date).getTime()) / (1000 * 60 * 60 * 24));
    if (daysSince &gt;= WAITING_CLIENT_DAYS) return "waiting_client";
  }

  // Check: Rev started but no preview sent
  const revStarted = getRecord("rev_started");
  const previewSentCheck = getRecord("rev_preview_sent");
  if (revStarted?.event_date &amp;&amp; !previewSentCheck?.event_date) {
    const daysSince = Math.floor((Date.now() - new Date(revStarted.event_date).getTime()) / (1000 * 60 * 60 * 24));
    if (daysSince &gt;= DELAYED_DAYS) return "delayed";
  }

  return null;
}

export function getOverallAlert(events: TimelineEvent[], deliveryDate?: string): AlertLevel {
  // Check overdue
  if (deliveryDate) {
    const deadline = new Date(deliveryDate);
    const isCompleted = events.some(e =&gt; e.event_type === "project_completed" &amp;&amp; e.event_date);
    const isDelivered = events.some(e =&gt; e.event_type === "project_delivered" &amp;&amp; e.event_date);
    if (deadline &lt; new Date() &amp;&amp; !isCompleted &amp;&amp; !isDelivered) return "overdue";
  }

  // Check all revisions for worst alert
  const maxRev = events.reduce((m, e) =&gt; Math.max(m, e.revision_number), 3);
  let worstAlert: AlertLevel = "on_track";
  for (let i = 0; i &lt;= maxRev; i++) {
    const alert = getRevisionAlert(i, events);
    if (alert === "delayed") return "delayed";
    if (alert === "waiting_client") worstAlert = "waiting_client";
  }
  return worstAlert;
}

// ■■ Timeline progress ■■

export function getTimelineProgress(events: TimelineEvent[], maxRevision: number): number {
  let totalEvents = 0;
  let completedEvents = 0;

  for (let i = 0; i &lt;= maxRevision; i++) {
    const revEvents = REV_EVENTS(i);
    totalEvents += revEvents.length;
    completedEvents += revEvents.filter(ev =&gt;
      events.some(e =&gt; e.revision_number === i &amp;&amp; e.event_type === ev.type &amp;&amp; e.event_date)
    ).length;
  }

  // Final events (non-auto only)
  const manualFinals = FINAL_EVENTS.filter(e =&gt; e.type !== "invoice_sent_auto" &amp;&amp; e.type !== "payment_received_auto");
  totalEvents += manualFinals.length;
  completedEvents += manualFinals.filter(ev =&gt;
    events.some(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type &amp;&amp; e.event_date)
  ).length;

  return totalEvents &gt; 0 ? Math.round((completedEvents / totalEvents) * 100) : 0;
}

// ■■ Auto stage derivation ■■

export function deriveStageFromTimeline(events: TimelineEvent[], maxRevision: number): string | null {
  const has = (rev: number, type: string) =&gt;
    events.some(e =&gt; e.revision_number === rev &amp;&amp; e.event_type === type &amp;&amp; e.event_date);
  const hasFinal = (type: string) =&gt;
    events.some(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === type &amp;&amp; e.event_date);

  // Check from end to start
  if (hasFinal("project_delivered")) return "DELIVERED";
  if (hasFinal("project_completed")) return "COMPLETED";
  if (hasFinal("project_approved")) return "APPROVED";

  // Check revisions from highest to lowest
  for (let rev = maxRevision; rev &gt;= 0; rev--) {
    if (has(rev, "rev_preview_sent")) return `PREVIEW_SENT_REV_${String(rev).padStart(2, "0")}`;
```

```
    if (has(rev, "rev_started")) return `IN_PROGRESS_REV_${String(rev).padStart(2, "0")}`;
  }

  // Check initial events
  if (has(0, "initial_payment_received")) return "READY_TO_START";
  if (has(0, "initial_invoice_sent")) return "WAITING_INITIAL_PAYMENT";

  return null;
}

// ■■ Deadline helpers ■■

export function getRevisionDelay(revNum: number, events: TimelineEvent[], expectedDaysPerRev: number = 3): { delayed: boolean; days: number } | n
  const revStarted = events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === "rev_started" &amp;&amp; e.event_date);
  const previewSent = events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === "rev_preview_sent" &amp;&amp; e.event_date);

  if (!revStarted?.event_date) return null;
  if (previewSent?.event_date) return { delayed: false, days: 0 };

  const startDate = new Date(revStarted.event_date);
  const expected = new Date(startDate);
  expected.setDate(expected.getDate() + expectedDaysPerRev);

  const today = new Date();
  if (today &gt; expected) {
    const delayDays = Math.floor((today.getTime() - expected.getTime()) / (1000 * 60 * 60 * 24));
    return { delayed: true, days: delayDays };
  }
  return { delayed: false, days: 0 };
}
```

src/components/dashboard/ClientProjectDocuments.tsx

```
import { useCallback, useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Download, FolderOpen, FileText, ChevronRight, PenTool } from "lucide-react";
import { format } from "date-fns";

const DOCUMENT_CATEGORIES = [
  { value: "pre_construction", label: "Pre-Construction" },
  { value: "rev_00", label: "REV 00" },
  { value: "rev_01", label: "REV 01" },
  { value: "rev_02", label: "REV 02" },
  { value: "rev_03", label: "REV 03 (Final)" },
  { value: "final_delivery", label: "Final Delivery" },
  { value: "general", label: "General" },
] as const;

interface ClientProjectDocumentsProps {
  projectId: string;
}

const ClientProjectDocuments = ({ projectId }: ClientProjectDocumentsProps) =&gt; {
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [drawings, setDrawings] = useState&lt;any[]&gt;([]);
  const [openFolders, setOpenFolders] = useState&lt;Set&lt;string&gt;&gt;(new Set(DOCUMENT_CATEGORIES.map(c =&gt; c.value)));

  const loadData = useCallback(async () =&gt; {
    const [fRes, dRes] = await Promise.all([
      supabase.from("project_files").select("*").eq("project_id", projectId).order("created_at", { ascending: false }),
      supabase.from("drawing_versions").select("*").eq("project_id", projectId).eq("is_current", true).order("created_at", { ascending: false }),
    ]);
    setFiles(fRes.data || []);
    setDrawings(dRes.data || []);
  }, [projectId]);

  useEffect(() =&gt; { loadData(); }, [loadData]);

  const toggleFolder = (cat: string) =&gt; {
    setOpenFolders(prev =&gt; {
      const next = new Set(prev);
      next.has(cat) ? next.delete(cat) : next.add(cat);
      return next;
    });
  };

  const grouped = DOCUMENT_CATEGORIES.reduce((acc, cat) =&gt; {
    acc[cat.value] = files.filter(f =&gt; (f.document_category || "general") === cat.value);
    return acc;
  }, {} as Record&lt;string, any[]&gt;);

  const totalCount = files.length + drawings.length;

  const formatSize = (bytes: number | null) =&gt; {
    if (!bytes) return "";
    if (bytes &lt; 1024) return `${bytes} B`;
    if (bytes &lt; 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
    return `${(bytes / 1048576).toFixed(1)} MB`;
  };

  if (totalCount === 0) {
    return (
      &lt;Card className="border-dashed"&gt;
        &lt;CardContent className="py-10 text-center text-muted-foreground"&gt;
          &lt;FolderOpen className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;
          No files available yet
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    );
  }

  const hasDrawings = drawings.length &gt; 0;
  const nonEmptyFolders = DOCUMENT_CATEGORIES.filter(cat =&gt; grouped[cat.value].length &gt; 0);

  return (
    &lt;div className="space-y-4"&gt;
      {/* Drawings Section */}
      {hasDrawings &amp;&amp; (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-2"&gt;
            &lt;CardTitle className="text-sm"&gt;Drawings&lt;/CardTitle&gt;
          &lt;/CardHeader&gt;
          &lt;CardContent className="p-0 pb-2"&gt;
```

```
            {drawings.map(d =&gt; (
              &lt;div key={d.id} className="flex items-center gap-3 px-4 py-2 hover:bg-muted/30 transition-colors"&gt;
                &lt;PenTool className="h-4 w-4 text-primary shrink-0" /&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;p className="text-sm font-medium text-foreground truncate"&gt;{d.drawing_name}&lt;/p&gt;
                  &lt;p className="text-[11px] text-muted-foreground"&gt;
                    v{d.version_number} · {format(new Date(d.created_at), "MMM d, yyyy")}
                  &lt;/p&gt;
                &lt;/div&gt;
                &lt;Button size="sm" variant="ghost" asChild&gt;
                  &lt;a href={d.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-4 w-4" /&gt;&lt;/a&gt;
                &lt;/Button&gt;
              &lt;/div&gt;
            ))}
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      )}

      {/* Document Folders */}
      {nonEmptyFolders.length &gt; 0 &amp;&amp; (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-2"&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;CardTitle className="text-sm"&gt;Documents&lt;/CardTitle&gt;
              &lt;Badge variant="secondary" className="text-[10px]"&gt;{files.length} files&lt;/Badge&gt;
            &lt;/div&gt;
          &lt;/CardHeader&gt;
          &lt;CardContent className="p-0"&gt;
            &lt;div className="divide-y divide-border"&gt;
              {nonEmptyFolders.map(cat =&gt; {
                const catFiles = grouped[cat.value];
                const isOpen = openFolders.has(cat.value);
                return (
                  &lt;Collapsible key={cat.value} open={isOpen} onOpenChange={() =&gt; toggleFolder(cat.value)}&gt;
                    &lt;CollapsibleTrigger className="flex items-center gap-3 w-full px-4 py-3 hover:bg-muted/50 transition-colors text-left"&gt;
                      &lt;ChevronRight className={`h-4 w-4 text-muted-foreground transition-transform ${isOpen ? "rotate-90" : ""}`} /&gt;
                      &lt;FolderOpen className="h-4 w-4 text-primary" /&gt;
                      &lt;span className="text-sm font-medium text-foreground flex-1"&gt;{cat.label}&lt;/span&gt;
                      &lt;Badge variant="outline" className="text-[10px]"&gt;{catFiles.length}&lt;/Badge&gt;
                    &lt;/CollapsibleTrigger&gt;
                    &lt;CollapsibleContent&gt;
                      &lt;div className="pb-2"&gt;
                        {catFiles.map(f =&gt; (
                          &lt;div key={f.id} className="flex items-center gap-3 px-12 py-2 hover:bg-muted/30 transition-colors"&gt;
                            &lt;FileText className="h-4 w-4 text-muted-foreground shrink-0" /&gt;
                            &lt;div className="flex-1 min-w-0"&gt;
                              &lt;p className="text-sm font-medium text-foreground truncate"&gt;{f.file_name}&lt;/p&gt;
                              &lt;p className="text-[11px] text-muted-foreground"&gt;
                                {formatSize(f.file_size)} · {format(new Date(f.created_at), "MMM d, yyyy")}
                              &lt;/p&gt;
                            &lt;/div&gt;
                            &lt;Button variant="ghost" size="icon" className="h-7 w-7 shrink-0" asChild&gt;
                              &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download className="h-3.5 w-3.5" /&gt;&lt;
                            &lt;/Button&gt;
                          &lt;/div&gt;
                        ))}
                      &lt;/div&gt;
                    &lt;/CollapsibleContent&gt;
                  &lt;/Collapsible&gt;
                );
              })}
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      )}
    &lt;/div&gt;
  );
};

export default ClientProjectDocuments;
```

src/components/dashboard/ClientTimeline.tsx

```
import { useEffect, useState, useCallback } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Progress } from "@/components/ui/progress";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import {
  CheckCircle2, Clock, DollarSign, FileText, Send,
  MessageSquare, Star, Package, Truck, ChevronDown, AlertTriangle,
} from "lucide-react";
import { format } from "date-fns";
import { cn } from "@/lib/utils";
import { useIsMobile } from "@/hooks/use-mobile";
import {
  REV_EVENTS, FINAL_EVENTS, TimelineEvent, STATUS_CONFIG, ALERT_CONFIG,
  getRevisionStatus, getRevisionAlert, getOverallAlert,
  getTimelineProgress, getRevisionDelay,
} from "@/components/admin/timeline/timelineUtils";

interface Props {
  projectId: string;
  deliveryDate?: string;
}

export default function ClientTimeline({ projectId, deliveryDate }: Props) {
  const [events, setEvents] = useState&lt;TimelineEvent[]&gt;([]);
  const [maxRevision, setMaxRevision] = useState(3);
  const [openRevisions, setOpenRevisions] = useState&lt;Record&lt;number, boolean&gt;&gt;({ 0: true });
  const [finalizationOpen, setFinalizationOpen] = useState(true);
  const isMobile = useIsMobile();

  const fetchEvents = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("project_timeline")
      .select("*")
      .eq("project_id", projectId)
      .order("revision_number")
      .order("created_at");
    const list = (data as TimelineEvent[]) || [];
    setEvents(list);
    const highestRev = list.reduce((m, e) =&gt; Math.max(m, e.revision_number), 3);
    setMaxRevision(highestRev &lt; 3 ? 3 : highestRev);
  }, [projectId]);

  useEffect(() =&gt; { fetchEvents(); }, [fetchEvents]);

  const getEventRecord = (revNum: number, eventType: string) =&gt;
    events.find(e =&gt; e.revision_number === revNum &amp;&amp; e.event_type === eventType);

  // Find the "next step" for the client
  const getNextStep = (): string | null =&gt; {
    for (let rev = 0; rev &lt;= maxRevision; rev++) {
      const alert = getRevisionAlert(rev, events);
      if (alert === "waiting_client") {
        return `Waiting for your feedback on REV ${String(rev).padStart(2, "0")}`;
      }
    }
    // Check if there's an in-progress revision
    for (let rev = 0; rev &lt;= maxRevision; rev++) {
      const status = getRevisionStatus(rev, events);
      if (status === "in_progress") {
        const previewSent = events.find(e =&gt; e.revision_number === rev &amp;&amp; e.event_type === "rev_preview_sent" &amp;&amp; e.event_date)
        if (previewSent) return `Please review REV ${String(rev).padStart(2, "0")} preview`;
        return `REV ${String(rev).padStart(2, "0")} is being worked on`;
      }
    }
    return null;
  };

  const renderEvent = (ev: { type: string; label: string; icon: React.ElementType }, record: TimelineEvent | undefined) =&gt; {
    const Icon = ev.icon;
    const done = !!record?.event_date;
    return (
      &lt;div key={ev.type} className={cn("flex items-center gap-3 px-3 py-2 rounded-md", done ? "bg-chart-2/5" : "bg-transparent")}&gt;
        &lt;div className={cn(
          "h-6 w-6 rounded-full flex items-center justify-center shrink-0",
          done ? "bg-chart-2/20 text-chart-2" : "bg-muted text-muted-foreground"
        )}&gt;
          {done ? &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt; : &lt;Icon className="h-3.5 w-3.5" /&gt;}
        &lt;/div&gt;
        &lt;p className={cn("text-sm flex-1", done ? "text-foreground font-medium" : "text-muted-foreground")}&gt;{ev.label}&lt;/p&gt;
        {record?.event_date &amp;&amp; (
          &lt;span className="text-xs text-muted-foreground shrink-0"&gt;
            {format(new Date(record.event_date), "MMM d, yyyy")}
```

```
        &lt;/span&gt;
      )}
    &lt;/div&gt;
  );
};

const renderRevisionBlock = (revNum: number) =&gt; {
  const revEvents = REV_EVENTS(revNum);
  const isExtra = revNum &gt; 3;
  const revLabel = `REV ${String(revNum).padStart(2, "0")}`;
  const status = getRevisionStatus(revNum, events);
  const statusCfg = STATUS_CONFIG[status];
  const alert = getRevisionAlert(revNum, events);
  const completedCount = revEvents.filter(ev =&gt; getEventRecord(revNum, ev.type)?.event_date).length;
  const isOpen = openRevisions[revNum] ?? false;

  return (
    &lt;Collapsible
      key={`rev-${revNum}`}
      open={isOpen}
      onOpenChange={(open) =&gt; setOpenRevisions(prev =&gt; ({ ...prev, [revNum]: open }))}
    &gt;
      &lt;Card className={cn(
        "border shadow-sm",
        isExtra &amp;&amp; "border-l-4 border-l-chart-4",
        alert === "waiting_client" &amp;&amp; "border-yellow-400/50 dark:border-yellow-600/50"
      )}&gt;
        &lt;CollapsibleTrigger asChild&gt;
          &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
            &lt;CardTitle className="text-sm flex items-center gap-2"&gt;
              &lt;Package className="h-4 w-4 text-muted-foreground" /&gt;
              {revLabel}
              {revNum === 0 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Initial&lt;/Badge&gt;}
              {revNum === 3 &amp;&amp; &lt;Badge variant="secondary" className="text-[10px]"&gt;Final&lt;/Badge&gt;}
              {isExtra &amp;&amp; &lt;Badge className="text-[10px] bg-chart-4/15 text-chart-4 border-chart-4/30"&gt;Extra&lt;/Badge&gt;}
              &lt;Badge className={cn("text-[10px] ml-1 border-0", statusCfg.classes)}&gt;
                {statusCfg.icon} {statusCfg.label}
              &lt;/Badge&gt;
              {alert === "waiting_client" &amp;&amp; (
                &lt;Badge className={cn("text-[10px] border-0", ALERT_CONFIG.waiting_client.classes)}&gt;
                  Action Required
                &lt;/Badge&gt;
              )}
              &lt;span className="ml-auto text-xs text-muted-foreground font-normal"&gt;{completedCount}/{revEvents.length}&lt;/span&gt;
              &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground transition-transform", isOpen &amp;&amp; "rotate-180")} /&gt;
            &lt;/CardTitle&gt;
          &lt;/CardHeader&gt;
        &lt;/CollapsibleTrigger&gt;
        &lt;CollapsibleContent&gt;
          &lt;CardContent className="space-y-1 pt-0"&gt;
            {revEvents.map(ev =&gt; renderEvent(ev, getEventRecord(revNum, ev.type)))}
          &lt;/CardContent&gt;
        &lt;/CollapsibleContent&gt;
      &lt;/Card&gt;
    &lt;/Collapsible&gt;
  );
};

const renderFinalBlock = () =&gt; (
  &lt;Collapsible open={finalizationOpen} onOpenChange={setFinalizationOpen}&gt;
    &lt;Card className="border shadow-sm"&gt;
      &lt;CollapsibleTrigger asChild&gt;
        &lt;CardHeader className="pb-2 cursor-pointer hover:bg-muted/30 transition-colors"&gt;
          &lt;CardTitle className="text-sm flex items-center gap-2"&gt;
            &lt;Star className="h-4 w-4 text-chart-2" /&gt; Completion
            &lt;ChevronDown className={cn("h-4 w-4 text-muted-foreground ml-auto transition-transform", finalizationOpen &amp;&amp; "rotate-180
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
      &lt;/CollapsibleTrigger&gt;
      &lt;CollapsibleContent&gt;
        &lt;CardContent className="space-y-1 pt-0"&gt;
          {FINAL_EVENTS.map(ev =&gt; {
            const isAutoType = ev.type === "invoice_sent_auto" || ev.type === "payment_received_auto";
            if (isAutoType) {
              const autoRecords = events.filter(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type);
              if (autoRecords.length &gt; 0) {
                return autoRecords.map(ar =&gt; (
                  &lt;div key={ar.id} className="flex items-center gap-3 px-3 py-2 rounded-md bg-chart-2/5"&gt;
                    &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-chart-2/20 text-chart-2"&gt;
                      &lt;CheckCircle2 className="h-3.5 w-3.5" /&gt;
                    &lt;/div&gt;
                    &lt;div className="flex-1 min-w-0"&gt;
                      &lt;p className="text-sm text-foreground font-medium"&gt;{ev.label}&lt;/p&gt;
                      {ar.notes &amp;&amp; &lt;p className="text-xs text-muted-foreground truncate"&gt;{ar.notes}&lt;/p&gt;}
                    &lt;/div&gt;
                    {ar.event_date &amp;&amp; (
                      &lt;span className="text-xs text-muted-foreground shrink-0"&gt;
```

```
                        {format(new Date(ar.event_date), "MMM d, yyyy")}
                      &lt;/span&gt;
                    )}
                  &lt;/div&gt;
                ));
              }
              return (
                &lt;div key={ev.type} className="flex items-center gap-3 px-3 py-2 rounded-md bg-transparent opacity-50"&gt;
                  &lt;div className="h-6 w-6 rounded-full flex items-center justify-center shrink-0 bg-muted text-muted-foreground"&gt;
                    &lt;ev.icon className="h-3.5 w-3.5" /&gt;
                  &lt;/div&gt;
                  &lt;p className="text-sm text-muted-foreground"&gt;{ev.label}&lt;/p&gt;
                &lt;/div&gt;
              );
            }
            return renderEvent(ev, events.find(e =&gt; e.revision_number === -1 &amp;&amp; e.event_type === ev.type));
          })}
        &lt;/CardContent&gt;
      &lt;/CollapsibleContent&gt;
    &lt;/Card&gt;
  &lt;/Collapsible&gt;
);

const overallAlert = getOverallAlert(events, deliveryDate);
const alertCfg = ALERT_CONFIG[overallAlert];
const progressPct = getTimelineProgress(events, maxRevision);
const nextStep = getNextStep();
const revisionNumbers = Array.from({ length: maxRevision + 1 }, (_, i) =&gt; i);

return (
  &lt;div className="space-y-4"&gt;
    {/* Overall status */}
    &lt;Card className="border shadow-sm"&gt;
      &lt;CardContent className="py-4 space-y-3"&gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;div className="flex items-center gap-3"&gt;
            &lt;Badge className={cn("text-xs border-0", alertCfg.classes)}&gt;
              {alertCfg.icon} {alertCfg.label}
            &lt;/Badge&gt;
            &lt;span className="text-sm text-muted-foreground"&gt;Project Progress&lt;/span&gt;
          &lt;/div&gt;
          &lt;span className="text-sm font-semibold"&gt;{progressPct}%&lt;/span&gt;
        &lt;/div&gt;
        &lt;Progress value={progressPct} className="h-2" /&gt;
        {nextStep &amp;&amp; (
          &lt;div className="flex items-center gap-2 text-xs text-yellow-700 dark:text-yellow-400 bg-yellow-50 dark:bg-yellow-900/20 px-3 py-2
            &lt;AlertTriangle className="h-3.5 w-3.5 shrink-0" /&gt;
            {nextStep}
          &lt;/div&gt;
        )}
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Revision grid */}
    &lt;div className={cn("grid gap-4", isMobile ? "grid-cols-1" : "grid-cols-1 lg:grid-cols-2")}&gt;
      {revisionNumbers.map(revNum =&gt; renderRevisionBlock(revNum))}
      {renderFinalBlock()}
    &lt;/div&gt;
  &lt;/div&gt;
);
}
```

src/components/dashboard/DashboardLayout.tsx

```
import { useEffect } from "react";
import { Outlet, useNavigate, Link } from "react-router-dom";
import daraLogo from "@/assets/dara-logo.jpg";
import { useAuth } from "@/hooks/useAuth";
import { useLanguage } from "@/hooks/useLanguage";
import { SidebarProvider, SidebarTrigger, SidebarInset } from "@/components/ui/sidebar";
import DashboardSidebar from "./DashboardSidebar";
import { Skeleton } from "@/components/ui/skeleton";

const DashboardLayout = () =&gt; {
  const { user, loading } = useAuth();
  const { t } = useLanguage();
  const navigate = useNavigate();

  useEffect(() =&gt; {
    if (!loading &amp;&amp; !user) {
      navigate("/auth");
    }
  }, [user, loading, navigate]);

  if (loading) {
    return (
      &lt;div className="min-h-screen flex items-center justify-center bg-background"&gt;
        &lt;div className="space-y-4 w-full max-w-md p-8"&gt;
          &lt;Skeleton className="h-8 w-3/4 mx-auto" /&gt;
          &lt;Skeleton className="h-4 w-1/2 mx-auto" /&gt;
          &lt;Skeleton className="h-32 w-full" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }

  if (!user) {
    return null;
  }

  return (
    &lt;SidebarProvider&gt;
      &lt;div className="min-h-screen flex w-full"&gt;
        &lt;DashboardSidebar /&gt;
        &lt;SidebarInset className="flex flex-col flex-1"&gt;
          {/* Top Header */}
          &lt;header className="h-14 border-b border-border flex items-center justify-between px-4 bg-background"&gt;
            &lt;div className="flex items-center gap-4"&gt;
              &lt;SidebarTrigger /&gt;
              &lt;Link to="/" className="flex items-center gap-2"&gt;
                &lt;img src={daraLogo} alt="DARA Studio" className="h-8 w-8 rounded" /&gt;
                &lt;span className="font-serif text-lg font-semibold text-primary"&gt;&lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt;
              &lt;/Link&gt;
            &lt;/div&gt;
          &lt;/header&gt;

          {/* Main Content */}
          &lt;main className="flex-1 overflow-auto bg-muted/30"&gt;
            &lt;Outlet /&gt;
          &lt;/main&gt;
        &lt;/SidebarInset&gt;
      &lt;/div&gt;
    &lt;/SidebarProvider&gt;
  );
};

export default DashboardLayout;
```

src/components/dashboard/DashboardSidebar.tsx

```
import { NavLink, useLocation } from "react-router-dom";
import { LayoutDashboard, FolderOpen, FileText, MessageSquare, User, Building2, LogOut, Plus, Receipt, Upload } from "lucide-react";
import { useAuth } from "@/hooks/useAuth";
import { Button } from "@/components/ui/button";
import { Sidebar, SidebarContent, SidebarGroup, SidebarGroupContent, SidebarGroupLabel, SidebarMenu, SidebarMenuButton, SidebarMenuItem, SidebarH
import { cn } from "@/lib/utils";

const DashboardSidebar = () =&gt; {
  const { state } = useSidebar();
  const collapsed = state === "collapsed";
  const location = useLocation();
  const { signOut, user } = useAuth();

  const mainNavItems = [
    { title: "Dashboard", url: "/dashboard", icon: LayoutDashboard },
    { title: "My Projects", url: "/dashboard/projects", icon: FolderOpen },
    { title: "Invoices", url: "/dashboard/invoices", icon: Receipt },
    { title: "Files", url: "/dashboard/files", icon: Upload },
    { title: "Messages", url: "/dashboard/messages", icon: MessageSquare },
  ];

  const settingsNavItems = [
    { title: "Profile", url: "/dashboard/profile", icon: User },
    { title: "Company", url: "/dashboard/company", icon: Building2 },
  ];

  const isActive = (path: string) =&gt; {
    if (path === "/dashboard") return location.pathname === "/dashboard";
    return location.pathname.startsWith(path);
  };

  const initials = user?.user_metadata?.full_name
    ? user.user_metadata.full_name.split(" ").map((n: string) =&gt; n[0]).join("").toUpperCase().slice(0, 2)
    : user?.email?.charAt(0).toUpperCase() || "C";

  return (
    &lt;Sidebar collapsible="icon"&gt;
      &lt;SidebarHeader className="border-b border-sidebar-border p-4"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;div className="w-9 h-9 rounded-full bg-primary flex items-center justify-center flex-shrink-0"&gt;
            &lt;span className="text-primary-foreground font-semibold text-sm"&gt;{initials}&lt;/span&gt;
          &lt;/div&gt;
          {!collapsed &amp;&amp; (
            &lt;div className="flex flex-col min-w-0"&gt;
              &lt;span className="font-semibold text-sm text-sidebar-foreground truncate"&gt;{user?.user_metadata?.full_name || "Client"}&lt;/spa
              &lt;span className="text-xs text-sidebar-foreground/60 truncate"&gt;{user?.email}&lt;/span&gt;
            &lt;/div&gt;
          )}
        &lt;/div&gt;
      &lt;/SidebarHeader&gt;
      &lt;SidebarContent&gt;
        &lt;div className="p-3"&gt;
          &lt;Button asChild className="w-full" size={collapsed ? "icon" : "default"}&gt;
            &lt;NavLink to="/dashboard/new-quote"&gt;
              &lt;Plus className={cn("h-4 w-4", !collapsed &amp;&amp; "mr-2")} /&gt;
              {!collapsed &amp;&amp; "New Project"}
            &lt;/NavLink&gt;
          &lt;/Button&gt;
        &lt;/div&gt;
        &lt;SidebarGroup&gt;
          &lt;SidebarGroupLabel&gt;Menu&lt;/SidebarGroupLabel&gt;
          &lt;SidebarGroupContent&gt;
            &lt;SidebarMenu&gt;
              {mainNavItems.map((item) =&gt; (
                &lt;SidebarMenuItem key={item.url}&gt;
                  &lt;SidebarMenuButton asChild isActive={isActive(item.url)} tooltip={item.title}&gt;
                    &lt;NavLink to={item.url}&gt;&lt;item.icon className="h-4 w-4" /&gt;&lt;span&gt;{item.title}&lt;/span&gt;&lt;/NavLink&gt;
                  &lt;/SidebarMenuButton&gt;
                &lt;/SidebarMenuItem&gt;
              ))}
            &lt;/SidebarMenu&gt;
          &lt;/SidebarGroupContent&gt;
        &lt;/SidebarGroup&gt;
        &lt;SidebarGroup&gt;
          &lt;SidebarGroupLabel&gt;Settings&lt;/SidebarGroupLabel&gt;
          &lt;SidebarGroupContent&gt;
            &lt;SidebarMenu&gt;
              {settingsNavItems.map((item) =&gt; (
                &lt;SidebarMenuItem key={item.url}&gt;
                  &lt;SidebarMenuButton asChild isActive={isActive(item.url)} tooltip={item.title}&gt;
                    &lt;NavLink to={item.url}&gt;&lt;item.icon className="h-4 w-4" /&gt;&lt;span&gt;{item.title}&lt;/span&gt;&lt;/NavLink&gt;
                  &lt;/SidebarMenuButton&gt;
                &lt;/SidebarMenuItem&gt;
              ))}
```

```
            &lt;/SidebarMenu&gt;
          &lt;/SidebarGroupContent&gt;
        &lt;/SidebarGroup&gt;
      &lt;/SidebarContent&gt;
      &lt;SidebarFooter className="border-t border-sidebar-border p-3"&gt;
        &lt;Button variant="ghost" className="w-full justify-start text-sidebar-foreground/70 hover:text-sidebar-foreground" onClick={signOut}&gt
          &lt;LogOut className={cn("h-4 w-4", !collapsed &amp;&amp; "mr-2")} /&gt;
          {!collapsed &amp;&amp; "Sign Out"}
        &lt;/Button&gt;
      &lt;/SidebarFooter&gt;
    &lt;/Sidebar&gt;
  );
};

export default DashboardSidebar;
```

src/components/dashboard/ProjectCard.tsx

```
import { Link } from "react-router-dom";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { MapPin, ArrowRight, FolderOpen, MessageSquare, DollarSign } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

interface ProjectCardProps {
  id: string;
  name: string;
  address: string;
  status: "pending" | "in_progress" | "under_review" | "completed" | "cancelled";
  serviceType: string;
  filesCount?: number;
  messagesCount?: number;
  quotesCount?: number;
}

const ProjectCard = ({ id, name, address, status, filesCount = 0, messagesCount = 0, quotesCount = 0 }: ProjectCardProps) =&gt; {
  const { t } = useLanguage();

  const statusConfig = {
    pending: { label: t("projectCard.briefing"), variant: "secondary" as const },
    in_progress: { label: t("projectCard.inDevelopment"), variant: "default" as const },
    under_review: { label: t("projectCard.underReview"), variant: "outline" as const },
    completed: { label: t("projectCard.completed"), variant: "default" as const },
    cancelled: { label: t("projectCard.cancelled"), variant: "destructive" as const },
  };

  const statusInfo = statusConfig[status] || statusConfig.pending;

  return (
    &lt;Card className="hover:border-primary/50 transition-all duration-300 overflow-hidden"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;div className="p-4 border-b border-border bg-muted/30"&gt;
          &lt;div className="flex items-start justify-between gap-2"&gt;
            &lt;div className="flex-1 min-w-0"&gt;
              &lt;h3 className="font-semibold text-foreground truncate"&gt;{name}&lt;/h3&gt;
              &lt;div className="flex items-center gap-1 mt-1 text-muted-foreground"&gt;
                &lt;MapPin className="h-3 w-3 flex-shrink-0" /&gt;
                &lt;span className="text-sm truncate"&gt;{address}&lt;/span&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;Badge variant={statusInfo.variant} className="flex-shrink-0"&gt;{statusInfo.label}&lt;/Badge&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div className="p-4 grid grid-cols-3 gap-4 border-b border-border"&gt;
          &lt;div className="flex items-center gap-2 text-sm text-muted-foreground"&gt;&lt;FolderOpen className="h-4 w-4" /&gt;&lt;span&gt;{files
          &lt;div className="flex items-center gap-2 text-sm text-muted-foreground"&gt;&lt;MessageSquare className="h-4 w-4" /&gt;&lt;span&gt;{me
          &lt;div className="flex items-center gap-2 text-sm text-muted-foreground"&gt;&lt;DollarSign className="h-4 w-4" /&gt;&lt;span&gt;{quote
        &lt;/div&gt;
        &lt;div className="p-4"&gt;
          &lt;Button asChild className="w-full"&gt;
            &lt;Link to={`/dashboard/projects/${id}`}&gt;{t("dashboard.openProject")}&lt;ArrowRight className="ml-2 h-4 w-4" /&gt;&lt;/Link&gt;
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  );
};

export default ProjectCard;
```

src/components/layout/Footer.tsx

```
import { Link } from "react-router-dom";
import { Mail, Phone, MapPin } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const Footer = () =&gt; {
  const currentYear = new Date().getFullYear();
  const { t } = useLanguage();

  const services = [t("footer.service1"), t("footer.service2"), t("footer.service3"), t("footer.service4")];

  return (
    &lt;footer className="bg-secondary text-secondary-foreground"&gt;
      &lt;div className="container mx-auto px-4 py-12"&gt;
        &lt;div className="grid grid-cols-1 md:grid-cols-4 gap-8"&gt;
          &lt;div className="space-y-4"&gt;
            &lt;span className="font-serif text-2xl font-bold text-primary-foreground"&gt;DA•RA Studio&lt;/span&gt;
            &lt;p className="text-sm text-secondary-foreground/80"&gt;{t("footer.tagline")}&lt;/p&gt;
          &lt;/div&gt;
          &lt;div className="space-y-4"&gt;
            &lt;h3 className="font-semibold text-lg"&gt;{t("footer.quickLinks")}&lt;/h3&gt;
            &lt;ul className="space-y-2"&gt;
              &lt;li&gt;&lt;Link to="/servicos" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"&
              &lt;li&gt;&lt;Link to="/portfolio" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"
              &lt;li&gt;&lt;Link to="/orcamento" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"
              &lt;li&gt;&lt;Link to="/auth" className="text-sm text-secondary-foreground/80 hover:text-primary-foreground transition-colors"&gt;{
            &lt;/ul&gt;
          &lt;/div&gt;
          &lt;div className="space-y-4"&gt;
            &lt;h3 className="font-semibold text-lg"&gt;{t("footer.services")}&lt;/h3&gt;
            &lt;ul className="space-y-2"&gt;
              {services.map((service) =&gt; (&lt;li key={service} className="text-sm text-secondary-foreground/80"&gt;{service}&lt;/li&gt;))}
            &lt;/ul&gt;
          &lt;/div&gt;
          &lt;div className="space-y-4"&gt;
            &lt;h3 className="font-semibold text-lg"&gt;{t("footer.contact")}&lt;/h3&gt;
            &lt;ul className="space-y-3"&gt;
              &lt;li className="flex items-center gap-2 text-sm text-secondary-foreground/80"&gt;&lt;Mail className="h-4 w-4" /&gt;darastudioofic
              &lt;li className="flex items-center gap-2 text-sm text-secondary-foreground/80"&gt;&lt;Phone className="h-4 w-4" /&gt;+1 (508) 523-
              &lt;li className="flex items-center gap-2 text-sm text-secondary-foreground/80"&gt;&lt;MapPin className="h-4 w-4" /&gt;Worcester, M
            &lt;/ul&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div className="mt-12 pt-8 border-t border-secondary-foreground/20"&gt;
          &lt;p class="text-center text-sm text-secondary-foreground/60"&gt;© {currentYear} DA•RA Studio. {t("footer.rights")}&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/footer&gt;
  );
};

export default Footer;
```

src/components/layout/Header.tsx

```
import { useState } from "react";
import { Link } from "react-router-dom";
import { Menu, X } from "lucide-react";
import daraLogo from "@/assets/dara-logo.jpg";
import { Button } from "@/components/ui/button";
import { useLanguage } from "@/hooks/useLanguage";
import LanguageSelector from "@/components/LanguageSelector";

const Header = () =&gt; {
  const [isMenuOpen, setIsMenuOpen] = useState(false);
  const { t } = useLanguage();

  const navLinks = [
    { href: "/how-we-work", label: t("nav.howWeWork") },
    { href: "/portfolio", label: t("nav.portfolio") },
    { href: "/orcamento", label: t("nav.requestQuote") },
    { href: "/contato", label: t("nav.contact") },
  ];

  return (
    &lt;header className="sticky top-0 z-50 w-full border-b border-border bg-background/95 backdrop-blur supports-[backdrop-filter]:bg-backgroun
      &lt;div className="container mx-auto flex h-16 items-center justify-between px-4"&gt;
        {/* Logo */}
        &lt;Link to="/" className="flex items-center gap-2"&gt;
          &lt;img src={daraLogo} alt="DARA Studio" className="h-9 w-9 rounded" /&gt;
          &lt;span className="font-serif text-lg font-semibold text-primary"&gt;
            &lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt; | Drafting &amp; 3D Support
          &lt;/span&gt;
        &lt;/Link&gt;

        {/* Desktop Navigation */}
        &lt;nav className="hidden md:flex items-center gap-6"&gt;
          {navLinks.map((link) =&gt; (
            &lt;Link
              key={link.href}
              to={link.href}
              className="text-sm font-medium text-muted-foreground transition-colors hover:text-primary"
            &gt;
              {link.label}
            &lt;/Link&gt;
          ))}
          &lt;LanguageSelector /&gt;
          &lt;Button asChild&gt;
            &lt;Link to="/auth"&gt;{t("nav.clientArea")}&lt;/Link&gt;
          &lt;/Button&gt;
        &lt;/nav&gt;

        {/* Mobile Menu Button */}
        &lt;div className="flex items-center gap-2 md:hidden"&gt;
          &lt;LanguageSelector /&gt;
          &lt;button
            className="p-2"
            onClick={() =&gt; setIsMenuOpen(!isMenuOpen)}
            aria-label="Toggle menu"
          &gt;
            {isMenuOpen ? (
              &lt;X className="h-6 w-6 text-foreground" /&gt;
            ) : (
              &lt;Menu className="h-6 w-6 text-foreground" /&gt;
            )}
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Mobile Navigation */}
      {isMenuOpen &amp;&amp; (
        &lt;div className="md:hidden border-t border-border bg-background"&gt;
          &lt;nav className="container mx-auto flex flex-col gap-4 px-4 py-4"&gt;
            {navLinks.map((link) =&gt; (
              &lt;Link
                key={link.href}
                to={link.href}
                className="text-sm font-medium text-muted-foreground transition-colors hover:text-primary"
                onClick={() =&gt; setIsMenuOpen(false)}
              &gt;
                {link.label}
              &lt;/Link&gt;
            ))}
            &lt;Button asChild className="w-full"&gt;
              &lt;Link to="/auth" onClick={() =&gt; setIsMenuOpen(false)}&gt;
                {t("nav.clientArea")}
              &lt;/Link&gt;
            &lt;/Button&gt;
          &lt;/nav&gt;
```

```
        &lt;/div&gt;
      )}
    &lt;/header&gt;
  );
};

export default Header;
```

src/components/layout/MainLayout.tsx

```
import { ReactNode } from "react";

interface MainLayoutProps {
  children: ReactNode;
}

const MainLayout = ({ children }: MainLayoutProps) =&gt; {
  return &lt;&gt;{children}&lt;/&gt;;
};

export default MainLayout;
```

src/components/layout/PublicLayout.tsx

```
import { ReactNode, forwardRef } from "react";
import Header from "./Header";
import Footer from "./Footer";

interface PublicLayoutProps {
  children: ReactNode;
}

const PublicLayout = forwardRef&lt;HTMLDivElement, PublicLayoutProps&gt;(({ children }, ref) =&gt; {
  return (
    &lt;div ref={ref} className="flex min-h-screen flex-col"&gt;
      &lt;Header /&gt;
      &lt;main className="flex-1"&gt;{children}&lt;/main&gt;
      &lt;Footer /&gt;
    &lt;/div&gt;
  );
});

PublicLayout.displayName = "PublicLayout";

export default PublicLayout;
```

src/components/ui/sidebar.tsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot";
import { VariantProps, cva } from "class-variance-authority";
import { PanelLeft } from "lucide-react";

import { useIsMobile } from "@/hooks/use-mobile";
import { cn } from "@/lib/utils";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Separator } from "@/components/ui/separator";
import { Sheet, SheetContent } from "@/components/ui/sheet";
import { Skeleton } from "@/components/ui/skeleton";
import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger } from "@/components/ui/tooltip";

const SIDEBAR_COOKIE_NAME = "sidebar:state";
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7;
const SIDEBAR_WIDTH = "16rem";
const SIDEBAR_WIDTH_MOBILE = "18rem";
const SIDEBAR_WIDTH_ICON = "3rem";
const SIDEBAR_KEYBOARD_SHORTCUT = "b";

type SidebarContext = {
  state: "expanded" | "collapsed";
  open: boolean;
  setOpen: (open: boolean) =&gt; void;
  openMobile: boolean;
  setOpenMobile: (open: boolean) =&gt; void;
  isMobile: boolean;
  toggleSidebar: () =&gt; void;
};

const SidebarContext = React.createContext&lt;SidebarContext | null&gt;(null);

function useSidebar() {
  const context = React.useContext(SidebarContext);
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.");
  }

  return context;
}

const SidebarProvider = React.forwardRef&lt;
  HTMLDivElement,
  React.ComponentProps&lt;"div"&gt; &amp; {
    defaultOpen?: boolean;
    open?: boolean;
    onOpenChange?: (open: boolean) =&gt; void;
  }
&gt;(({ defaultOpen = true, open: openProp, onOpenChange: setOpenProp, className, style, children, ...props }, ref) =&gt; {
  const isMobile = useIsMobile();
  const [openMobile, setOpenMobile] = React.useState(false);

  // This is the internal state of the sidebar.
  // We use openProp and setOpenProp for control from outside the component.
  const [_open, _setOpen] = React.useState(defaultOpen);
  const open = openProp ?? _open;
  const setOpen = React.useCallback(
    (value: boolean | ((value: boolean) =&gt; boolean)) =&gt; {
      const openState = typeof value === "function" ? value(open) : value;
      if (setOpenProp) {
        setOpenProp(openState);
      } else {
        _setOpen(openState);
      }

      // This sets the cookie to keep the sidebar state.
      document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`;
    },
    [setOpenProp, open],
  );

  // Helper to toggle the sidebar.
  const toggleSidebar = React.useCallback(() =&gt; {
    return isMobile ? setOpenMobile((open) =&gt; !open) : setOpen((open) =&gt; !open);
  }, [isMobile, setOpen, setOpenMobile]);

  // Adds a keyboard shortcut to toggle the sidebar.
  React.useEffect(() =&gt; {
    const handleKeyDown = (event: KeyboardEvent) =&gt; {
      if (event.key === SIDEBAR_KEYBOARD_SHORTCUT &amp;&amp; (event.metaKey || event.ctrlKey)) {
        event.preventDefault();
        toggleSidebar();
      }
```

```
    };

    window.addEventListener("keydown", handleKeyDown);
    return () =&gt; window.removeEventListener("keydown", handleKeyDown);
  }, [toggleSidebar]);

  // We add a state so that we can do data-state="expanded" or "collapsed".
  // This makes it easier to style the sidebar with Tailwind classes.
  const state = open ? "expanded" : "collapsed";

  const contextValue = React.useMemo&lt;SidebarContext&gt;(
    () =&gt; ({
      state,
      open,
      setOpen,
      isMobile,
      openMobile,
      setOpenMobile,
      toggleSidebar,
    }),
    [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar],
  );

  return (
    &lt;SidebarContext.Provider value={contextValue}&gt;
      &lt;TooltipProvider delayDuration={0}&gt;
        &lt;div
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH,
              "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
              ...style,
            } as React.CSSProperties
          }
          className={cn("group/sidebar-wrapper flex min-h-svh w-full has-[[data-variant=inset]]:bg-sidebar", className)}
          ref={ref}
          {...props}
        &gt;
          {children}
        &lt;/div&gt;
      &lt;/TooltipProvider&gt;
    &lt;/SidebarContext.Provider&gt;
  );
});
SidebarProvider.displayName = "SidebarProvider";

const Sidebar = React.forwardRef&lt;
  HTMLDivElement,
  React.ComponentProps&lt;"div"&gt; &amp; {
    side?: "left" | "right";
    variant?: "sidebar" | "floating" | "inset";
    collapsible?: "offcanvas" | "icon" | "none";
  }
&gt;(({ side = "left", variant = "sidebar", collapsible = "offcanvas", className, children, ...props }, ref) =&gt; {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar();

  if (collapsible === "none") {
    return (
      &lt;div
        className={cn("flex h-full w-[--sidebar-width] flex-col bg-sidebar text-sidebar-foreground", className)}
        ref={ref}
        {...props}
      &gt;
        {children}
      &lt;/div&gt;
    );
  }

  if (isMobile) {
    return (
      &lt;Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}&gt;
        &lt;SheetContent
          data-sidebar="sidebar"
          data-mobile="true"
          className="w-[--sidebar-width] bg-sidebar p-0 text-sidebar-foreground [&amp;&gt;button]:hidden"
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
            } as React.CSSProperties
          }
          side={side}
        &gt;
          &lt;div className="flex h-full w-full flex-col"&gt;{children}&lt;/div&gt;
        &lt;/SheetContent&gt;
      &lt;/Sheet&gt;
    );
  }
```

```
  return (
    &lt;div
      ref={ref}
      className="group peer hidden text-sidebar-foreground md:block"
      data-state={state}
      data-collapsible={state === "collapsed" ? collapsible : ""}
      data-variant={variant}
      data-side={side}
    &gt;
      {/* This is what handles the sidebar gap on desktop */}
      &lt;div
        className={cn(
          "relative h-svh w-[--sidebar-width] bg-transparent transition-[width] duration-200 ease-linear",
          "group-data-[collapsible=offcanvas]:w-0",
          "group-data-[side=right]:rotate-180",
          variant === "floating" || variant === "inset"
            ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spacing.4))]"
            : "group-data-[collapsible=icon]:w-[--sidebar-width-icon]",
        )}
      /&gt;
      &lt;div
        className={cn(
          "fixed inset-y-0 z-10 hidden h-svh w-[--sidebar-width] transition-[left,right,width] duration-200 ease-linear md:flex",
          side === "left"
            ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"
            : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",
          // Adjust the padding for floating and inset variants.
          variant === "floating" || variant === "inset"
            ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spacing.4)_+2px)]"
            : "group-data-[collapsible=icon]:w-[--sidebar-width-icon] group-data-[side=left]:border-r group-data-[side=right]:border-l",
          className,
        )}
        {...props}
      &gt;
        &lt;div
          data-sidebar="sidebar"
          className="flex h-full w-full flex-col bg-sidebar group-data-[variant=floating]:rounded-lg group-data-[variant=floating]:border group-d
        &gt;
          {children}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
});
Sidebar.displayName = "Sidebar";

const SidebarTrigger = React.forwardRef&lt;React.ElementRef&lt;typeof Button&gt;, React.ComponentProps&lt;typeof Button&gt;&gt;(
  ({ className, onClick, ...props }, ref) =&gt; {
    const { toggleSidebar } = useSidebar();

    return (
      &lt;Button
        ref={ref}
        data-sidebar="trigger"
        variant="ghost"
        size="icon"
        className={cn("h-7 w-7", className)}
        onClick={(event) =&gt; {
          onClick?.(event);
          toggleSidebar();
        }}
        {...props}
      &gt;
        &lt;PanelLeft /&gt;
        &lt;span className="sr-only"&gt;Toggle Sidebar&lt;/span&gt;
      &lt;/Button&gt;
    );
  },
);
SidebarTrigger.displayName = "SidebarTrigger";

const SidebarRail = React.forwardRef&lt;HTMLButtonElement, React.ComponentProps&lt;"button"&gt;&gt;(
  ({ className, ...props }, ref) =&gt; {
    const { toggleSidebar } = useSidebar();

    return (
      &lt;button
        ref={ref}
        data-sidebar="rail"
        aria-label="Toggle Sidebar"
        tabIndex={-1}
        onClick={toggleSidebar}
        title="Toggle Sidebar"
        className={cn(
          "absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all ease-linear after:absolute after:inset-y-0 after:left-1/2 after:w-[
          "[[data-side=left]_&amp;]:cursor-w-resize [[data-side=right]_&amp;]:cursor-e-resize",
          "[[data-side=left][data-state=collapsed]_&amp;]:cursor-e-resize [[data-side=right][data-state=collapsed]_&amp;]:cursor-w-resize",
```

```
          "group-data-[collapsible=offcanvas]:translate-x-0 group-data-[collapsible=offcanvas]:after:left-full group-data-[collapsible=offcanvas]
          "[[data-side=left][data-collapsible=offcanvas]_&amp;]:-right-2",
          "[[data-side=right][data-collapsible=offcanvas]_&amp;]:-left-2",
          className,
        )}
        {...props}
      /&gt;
    );
  },
);
SidebarRail.displayName = "SidebarRail";

const SidebarInset = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"main"&gt;&gt;(({ className, ...props }, ref) =&gt; {
  return (
    &lt;main
      ref={ref}
      className={cn(
        "relative flex min-h-svh flex-1 flex-col bg-background",
        "peer-data-[variant=inset]:min-h-[calc(100svh-theme(spacing.4))] md:peer-data-[variant=inset]:m-2 md:peer-data-[state=collapsed]:peer-dat
        className,
      )}
      {...props}
    /&gt;
  );
});
SidebarInset.displayName = "SidebarInset";

const SidebarInput = React.forwardRef&lt;React.ElementRef&lt;typeof Input&gt;, React.ComponentProps&lt;typeof Input&gt;&gt;(
  ({ className, ...props }, ref) =&gt; {
    return (
      &lt;Input
        ref={ref}
        data-sidebar="input"
        className={cn(
          "h-8 w-full bg-background shadow-none focus-visible:ring-2 focus-visible:ring-sidebar-ring",
          className,
        )}
        {...props}
      /&gt;
    );
  },
);
SidebarInput.displayName = "SidebarInput";

const SidebarHeader = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"div"&gt;&gt;(({ className, ...props }, ref) =&gt; {
  return &lt;div ref={ref} data-sidebar="header" className={cn("flex flex-col gap-2 p-2", className)} {...props} /&gt;;
});
SidebarHeader.displayName = "SidebarHeader";

const SidebarFooter = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"div"&gt;&gt;(({ className, ...props }, ref) =&gt; {
  return &lt;div ref={ref} data-sidebar="footer" className={cn("flex flex-col gap-2 p-2", className)} {...props} /&gt;;
});
SidebarFooter.displayName = "SidebarFooter";

const SidebarSeparator = React.forwardRef&lt;React.ElementRef&lt;typeof Separator&gt;, React.ComponentProps&lt;typeof Separator&gt;&gt;(
  ({ className, ...props }, ref) =&gt; {
    return (
      &lt;Separator
        ref={ref}
        data-sidebar="separator"
        className={cn("mx-2 w-auto bg-sidebar-border", className)}
        {...props}
      /&gt;
    );
  },
);
SidebarSeparator.displayName = "SidebarSeparator";

const SidebarContent = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"div"&gt;&gt;(({ className, ...props }, ref) =&gt; {
  return (
    &lt;div
      ref={ref}
      data-sidebar="content"
      className={cn(
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto group-data-[collapsible=icon]:overflow-hidden",
        className,
      )}
      {...props}
    /&gt;
  );
});
SidebarContent.displayName = "SidebarContent";

const SidebarGroup = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"div"&gt;&gt;(({ className, ...props }, ref) =&gt; {
  return (
    &lt;div
      ref={ref}
```

```
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props}
    /&gt;
  );
});
SidebarGroup.displayName = "SidebarGroup";

const SidebarGroupLabel = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"div"&gt; &amp; { asChild?: boolean }&gt;(
  ({ className, asChild = false, ...props }, ref) =&gt; {
    const Comp = asChild ? Slot : "div";

    return (
      &lt;Comp
        ref={ref}
        data-sidebar="group-label"
        className={cn(
          "flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium text-sidebar-foreground/70 outline-none ring-sidebar-ring transitio
          "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",
          className,
        )}
        {...props}
      /&gt;
    );
  },
);
SidebarGroupLabel.displayName = "SidebarGroupLabel";

const SidebarGroupAction = React.forwardRef&lt;HTMLButtonElement, React.ComponentProps&lt;"button"&gt; &amp; { asChild?: boolean }&gt;(
  ({ className, asChild = false, ...props }, ref) =&gt; {
    const Comp = asChild ? Slot : "button";

    return (
      &lt;Comp
        ref={ref}
        data-sidebar="group-action"
        className={cn(
          "absolute right-3 top-3.5 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreground outline-none ring-s
          // Increases the hit area of the button on mobile.
          "after:absolute after:-inset-2 after:md:hidden",
          "group-data-[collapsible=icon]:hidden",
          className,
        )}
        {...props}
      /&gt;
    );
  },
);
SidebarGroupAction.displayName = "SidebarGroupAction";

const SidebarGroupContent = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"div"&gt;&gt;(
  ({ className, ...props }, ref) =&gt; (
    &lt;div ref={ref} data-sidebar="group-content" className={cn("w-full text-sm", className)} {...props} /&gt;
  ),
);
SidebarGroupContent.displayName = "SidebarGroupContent";

const SidebarMenu = React.forwardRef&lt;HTMLUListElement, React.ComponentProps&lt;"ul"&gt;&gt;(({ className, ...props }, ref) =&gt; (
  &lt;ul ref={ref} data-sidebar="menu" className={cn("flex w-full min-w-0 flex-col gap-1", className)} {...props} /&gt;
));
SidebarMenu.displayName = "SidebarMenu";

const SidebarMenuItem = React.forwardRef&lt;HTMLLIElement, React.ComponentProps&lt;"li"&gt;&gt;(({ className, ...props }, ref) =&gt; (
  &lt;li ref={ref} data-sidebar="menu-item" className={cn("group/menu-item relative", className)} {...props} /&gt;
));
SidebarMenuItem.displayName = "SidebarMenuItem";

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm outline-none ring-sidebar-ring transition-[wi
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_0_1px_hsl(var(--sidebar-border))] hover:bg-sidebar-accent hover:text-sidebar-accent-foreground hover:shadow-
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:!p-0",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  },
```

```
);

const SidebarMenuButton = React.forwardRef&lt;
  HTMLButtonElement,
  React.ComponentProps&lt;"button"&gt; &amp; {
    asChild?: boolean;
    isActive?: boolean;
    tooltip?: string | React.ComponentProps&lt;typeof TooltipContent&gt;;
  } &amp; VariantProps&lt;typeof sidebarMenuButtonVariants&gt;
&gt;(({ asChild = false, isActive = false, variant = "default", size = "default", tooltip, className, ...props }, ref) =&gt; {
  const Comp = asChild ? Slot : "button";
  const { isMobile, state } = useSidebar();

  const button = (
    &lt;Comp
      ref={ref}
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props}
    /&gt;
  );

  if (!tooltip) {
    return button;
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    };
  }

  return (
    &lt;Tooltip&gt;
      &lt;TooltipTrigger asChild&gt;{button}&lt;/TooltipTrigger&gt;
      &lt;TooltipContent side="right" align="center" hidden={state !== "collapsed" || isMobile} {...tooltip} /&gt;
    &lt;/Tooltip&gt;
  );
});
SidebarMenuButton.displayName = "SidebarMenuButton";

const SidebarMenuAction = React.forwardRef&lt;
  HTMLButtonElement,
  React.ComponentProps&lt;"button"&gt; &amp; {
    asChild?: boolean;
    showOnHover?: boolean;
  }
&gt;(({ className, asChild = false, showOnHover = false, ...props }, ref) =&gt; {
  const Comp = asChild ? Slot : "button";

  return (
    &lt;Comp
      ref={ref}
      data-sidebar="menu-action"
      className={cn(
        "absolute right-1 top-1.5 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreground outline-none ring-sid
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 after:md:hidden",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        showOnHover &amp;&amp;
          "group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100 data-[state=open]:opacity-100 peer-data-[active=true]/menu-
        className,
      )}
      {...props}
    /&gt;
  );
});
SidebarMenuAction.displayName = "SidebarMenuAction";

const SidebarMenuBadge = React.forwardRef&lt;HTMLDivElement, React.ComponentProps&lt;"div"&gt;&gt;(
  ({ className, ...props }, ref) =&gt; (
    &lt;div
      ref={ref}
      data-sidebar="menu-badge"
      className={cn(
        "pointer-events-none absolute right-1 flex h-5 min-w-5 select-none items-center justify-center rounded-md px-1 text-xs font-medium tabula
        "peer-hover/menu-button:text-sidebar-accent-foreground peer-data-[active=true]/menu-button:text-sidebar-accent-foreground",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        className,
```

```
      )}
      {...props}
    /&gt;
  ),
);
SidebarMenuBadge.displayName = "SidebarMenuBadge";

const SidebarMenuSkeleton = React.forwardRef&lt;
  HTMLDivElement,
  React.ComponentProps&lt;"div"&gt; &amp; {
    showIcon?: boolean;
  }
&gt;(({ className, showIcon = false, ...props }, ref) =&gt; {
  // Random width between 50 to 90%.
  const width = React.useMemo(() =&gt; {
    return `${Math.floor(Math.random() * 40) + 50}%`;
  }, []);

  return (
    &lt;div
      ref={ref}
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
      {...props}
    &gt;
      {showIcon &amp;&amp; &lt;Skeleton className="size-4 rounded-md" data-sidebar="menu-skeleton-icon" /&gt;}
      &lt;Skeleton
        className="h-4 max-w-[--skeleton-width] flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            "--skeleton-width": width,
          } as React.CSSProperties
        }
      /&gt;
    &lt;/div&gt;
  );
});
SidebarMenuSkeleton.displayName = "SidebarMenuSkeleton";

const SidebarMenuSub = React.forwardRef&lt;HTMLUListElement, React.ComponentProps&lt;"ul"&gt;&gt;(
  ({ className, ...props }, ref) =&gt; (
    &lt;ul
      ref={ref}
      data-sidebar="menu-sub"
      className={cn(
        "mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l border-sidebar-border px-2.5 py-0.5",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    /&gt;
  ),
);
SidebarMenuSub.displayName = "SidebarMenuSub";

const SidebarMenuSubItem = React.forwardRef&lt;HTMLLIElement, React.ComponentProps&lt;"li"&gt;&gt;(({ ...props }, ref) =&gt; (
  &lt;li ref={ref} {...props} /&gt;
));
SidebarMenuSubItem.displayName = "SidebarMenuSubItem";

const SidebarMenuSubButton = React.forwardRef&lt;
  HTMLAnchorElement,
  React.ComponentProps&lt;"a"&gt; &amp; {
    asChild?: boolean;
    size?: "sm" | "md";
    isActive?: boolean;
  }
&gt;(({ asChild = false, size = "md", isActive, className, ...props }, ref) =&gt; {
  const Comp = asChild ? Slot : "a";

  return (
    &lt;Comp
      ref={ref}
      data-sidebar="menu-sub-button"
      data-size={size}
      data-active={isActive}
      className={cn(
        "flex h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden rounded-md px-2 text-sidebar-foreground outline-none ring-sidebar-ri
        "data-[active=true]:bg-sidebar-accent data-[active=true]:text-sidebar-accent-foreground",
        size === "sm" &amp;&amp; "text-xs",
        size === "md" &amp;&amp; "text-sm",
        "group-data-[collapsible=icon]:hidden",
        className,
      )}
      {...props}
    /&gt;
```

```
  );
});
SidebarMenuSubButton.displayName = "SidebarMenuSubButton";

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
  SidebarTrigger,
  useSidebar,
};
```

src/constants/projectStages.ts

```
// Unified project stages used across the entire system
// SYSTEM VALUES — stored in DB
export const PROJECT_STAGES = [
  "STOPPED",
  "CANCELED",
  "WAITING_INITIAL_PAYMENT",
  "WAITING_CLIENT_INFO",
  "READY_TO_START",
  "IN_PROGRESS_REV_00",
  "PREVIEW_SENT_REV_00",
  "IN_PROGRESS_REV_01",
  "PREVIEW_SENT_REV_01",
  "IN_PROGRESS_REV_02",
  "PREVIEW_SENT_REV_02",
  "IN_PROGRESS_REV_03",
  "PREVIEW_SENT_REV_03",
  "APPROVED",
  "FINALIZING",
  "COMPLETED",
  "DELIVERED",
] as const;

export type ProjectStage = typeof PROJECT_STAGES[number];

// UI LABELS — display only (PT-BR)
export const PROJECT_STAGE_LABELS: Record&lt;ProjectStage, string&gt; = {
  STOPPED: "Parado",
  CANCELED: "Cancelado",
  WAITING_INITIAL_PAYMENT: "Esperando pagamento inicial",
  WAITING_CLIENT_INFO: "Esperando informações do cliente",
  READY_TO_START: "Pronto para iniciar",
  IN_PROGRESS_REV_00: "Em andamento – REV 00",
  PREVIEW_SENT_REV_00: "Preview REV 00 enviado",
  IN_PROGRESS_REV_01: "Em andamento – REV 01",
  PREVIEW_SENT_REV_01: "Preview REV 01 enviado",
  IN_PROGRESS_REV_02: "Em andamento – REV 02",
  PREVIEW_SENT_REV_02: "Preview REV 02 enviado",
  IN_PROGRESS_REV_03: "Em andamento – REV 03 (final)",
  PREVIEW_SENT_REV_03: "Preview REV 03 enviado",
  APPROVED: "Aprovado",
  FINALIZING: "Em finalização",
  COMPLETED: "Concluído",
  DELIVERED: "Entregue",
};

// Helper to get display label for any stage value (graceful fallback)
export const getStageLabel = (stage: string): string =&gt; {
  return PROJECT_STAGE_LABELS[stage as ProjectStage] || stage;
};

// Color tokens for stage badges / indicators
export const PROJECT_STAGE_COLORS: Record&lt;string, { dot: string; bg: string; text: string; border: string }&gt; = {
  STOPPED: { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" },
  CANCELED: { dot: "bg-destructive", bg: "bg-destructive/10", text: "text-destructive", border: "border-destructive/20" },
  WAITING_INITIAL_PAYMENT: { dot: "bg-chart-4", bg: "bg-chart-4/15", text: "text-chart-4", border: "border-chart-4/30" },
  WAITING_CLIENT_INFO: { dot: "bg-chart-4", bg: "bg-chart-4/15", text: "text-chart-4", border: "border-chart-4/30" },
  READY_TO_START: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  IN_PROGRESS_REV_00: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_00: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  IN_PROGRESS_REV_01: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_01: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  IN_PROGRESS_REV_02: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_02: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  IN_PROGRESS_REV_03: { dot: "bg-primary", bg: "bg-primary/10", text: "text-primary", border: "border-primary/20" },
  PREVIEW_SENT_REV_03: { dot: "bg-chart-5", bg: "bg-chart-5/15", text: "text-chart-5", border: "border-chart-5/30" },
  APPROVED: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  FINALIZING: { dot: "bg-chart-3", bg: "bg-chart-3/15", text: "text-chart-3", border: "border-chart-3/30" },
  COMPLETED: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  DELIVERED: { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
};

// Kanban board stages — the workflow stages shown as columns
export const KANBAN_STAGES = [
  "WAITING_INITIAL_PAYMENT",
  "WAITING_CLIENT_INFO",
  "READY_TO_START",
  "IN_PROGRESS_REV_00",
  "PREVIEW_SENT_REV_00",
  "IN_PROGRESS_REV_01",
  "PREVIEW_SENT_REV_01",
  "IN_PROGRESS_REV_02",
  "PREVIEW_SENT_REV_02",
  "IN_PROGRESS_REV_03",
  "PREVIEW_SENT_REV_03",
```

```
  "APPROVED",
  "FINALIZING",
  "COMPLETED",
] as const;

export const KANBAN_STAGE_COLORS: Record&lt;string, { bg: string; border: string; header: string }&gt; = {
  WAITING_INITIAL_PAYMENT: { bg: "bg-chart-4/5", border: "border-chart-4/20", header: "bg-chart-4/15" },
  WAITING_CLIENT_INFO: { bg: "bg-chart-4/5", border: "border-chart-4/20", header: "bg-chart-4/15" },
  READY_TO_START: { bg: "bg-chart-2/5", border: "border-chart-2/20", header: "bg-chart-2/15" },
  IN_PROGRESS_REV_00: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_00: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  IN_PROGRESS_REV_01: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_01: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  IN_PROGRESS_REV_02: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_02: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  IN_PROGRESS_REV_03: { bg: "bg-primary/5", border: "border-primary/20", header: "bg-primary/15" },
  PREVIEW_SENT_REV_03: { bg: "bg-chart-5/5", border: "border-chart-5/20", header: "bg-chart-5/15" },
  APPROVED: { bg: "bg-chart-2/5", border: "border-chart-2/20", header: "bg-chart-2/15" },
  FINALIZING: { bg: "bg-chart-3/5", border: "border-chart-3/20", header: "bg-chart-3/15" },
  COMPLETED: { bg: "bg-chart-2/5", border: "border-chart-2/20", header: "bg-chart-2/15" },
};

// Stages considered "completed" for financial grouping
export const COMPLETED_STAGES: ProjectStage[] = ["COMPLETED", "DELIVERED"];

// Stages considered "active" (not stopped/canceled/completed)
export const isActiveStage = (stage: string): boolean =&gt; {
  return !["STOPPED", "CANCELED", "COMPLETED", "DELIVERED"].includes(stage);
};
```

src/hooks/useAdminRole.tsx

```
import { useState, useEffect } from "react";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";

export const useAdminRole = () =&gt; {
  const { user } = useAuth();
  const [isAdmin, setIsAdmin] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const checkRole = async () =&gt; {
      if (!user) {
        setIsAdmin(false);
        setLoading(false);
        return;
      }
      const { data, error } = await supabase
        .from("user_roles")
        .select("role")
        .eq("user_id", user.id)
        .eq("role", "admin")
        .maybeSingle();
      setIsAdmin(!!data &amp;&amp; !error);
      setLoading(false);
    };
    checkRole();
  }, [user]);

  return { isAdmin, loading };
};
```

src/hooks/useAuth.tsx

```
import { useState, useEffect, createContext, useContext, ReactNode } from "react";
import { User, Session } from "@supabase/supabase-js";
import { supabase } from "@/integrations/supabase/client";

interface AuthContextType {
  user: User | null;
  session: Session | null;
  loading: boolean;
  signIn: (email: string, password: string) =&gt; Promise&lt;{ error: Error | null }&gt;;
  signUp: (email: string, password: string, fullName: string) =&gt; Promise&lt;{ error: Error | null }&gt;;
  signOut: () =&gt; Promise&lt;void&gt;;
}

const AuthContext = createContext&lt;AuthContextType | undefined&gt;(undefined);

export const AuthProvider = ({ children }: { children: ReactNode }) =&gt; {
  const [user, setUser] = useState&lt;User | null&gt;(null);
  const [session, setSession] = useState&lt;Session | null&gt;(null);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    // Set up auth state listener FIRST
    const { data: { subscription } } = supabase.auth.onAuthStateChange(
      (event, session) =&gt; {
        setSession(session);
        setUser(session?.user ?? null);
        setLoading(false);
      }
    );

    // THEN check for existing session
    supabase.auth.getSession().then(({ data: { session } }) =&gt; {
      setSession(session);
      setUser(session?.user ?? null);
      setLoading(false);
    });

    return () =&gt; subscription.unsubscribe();
  }, []);

  const signIn = async (email: string, password: string) =&gt; {
    const { error } = await supabase.auth.signInWithPassword({
      email,
      password,
    });
    return { error: error as Error | null };
  };

  const signUp = async (email: string, password: string, fullName: string) =&gt; {
    const redirectUrl = `${window.location.origin}/`;

    const { error } = await supabase.auth.signUp({
      email,
      password,
      options: {
        emailRedirectTo: redirectUrl,
        data: {
          full_name: fullName,
        },
      },
    });
    return { error: error as Error | null };
  };

  const signOut = async () =&gt; {
    await supabase.auth.signOut();
  };

  return (
    &lt;AuthContext.Provider value={{ user, session, loading, signIn, signUp, signOut }}&gt;
      {children}
    &lt;/AuthContext.Provider&gt;
  );
};

export const useAuth = () =&gt; {
  const context = useContext(AuthContext);
  if (context === undefined) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context;
};
```

src/hooks/useFreelancerRole.tsx

```
import { useState, useEffect } from "react";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";

export const useFreelancerRole = () =&gt; {
  const { user } = useAuth();
  const [isFreelancer, setIsFreelancer] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const checkRole = async () =&gt; {
      if (!user) {
        setIsFreelancer(false);
        setLoading(false);
        return;
      }

      const { data, error } = await supabase
        .from("user_roles")
        .select("role")
        .eq("user_id", user.id)
        .eq("role", "freelancer")
        .maybeSingle();

      setIsFreelancer(!!data &amp;&amp; !error);
      setLoading(false);
    };

    checkRole();
  }, [user]);

  return { isFreelancer, loading };
};
```

src/hooks/useLanguage.tsx

```
import { createContext, useContext, useState, useEffect, ReactNode } from "react";
import { translations, Language } from "@/i18n/translations";

interface LanguageContextType {
  language: Language;
  setLanguage: (lang: Language) =&gt; void;
  t: (key: string) =&gt; string;
}

const LanguageContext = createContext&lt;LanguageContextType | undefined&gt;(undefined);

export const LanguageProvider = ({ children }: { children: ReactNode }) =&gt; {
  const [language, setLanguageState] = useState&lt;Language&gt;(() =&gt; {
    const saved = localStorage.getItem("language");
    return (saved as Language) || "en";
  });

  useEffect(() =&gt; {
    localStorage.setItem("language", language);
  }, [language]);

  const setLanguage = (lang: Language) =&gt; {
    setLanguageState(lang);
  };

  const t = (key: string): string =&gt; {
    return translations[language][key] || key;
  };

  return (
    &lt;LanguageContext.Provider value={{ language, setLanguage, t }}&gt;
      {children}
    &lt;/LanguageContext.Provider&gt;
  );
};

export const useLanguage = () =&gt; {
  const context = useContext(LanguageContext);
  if (context === undefined) {
    throw new Error("useLanguage must be used within a LanguageProvider");
  }
  return context;
};
```

src/i18n/translations.ts

```
export type Language = "en" | "pt";

export const translations: Record&lt;Language, Record&lt;string, string&gt;&gt; = {
  en: {
    // Header
    "nav.howWeWork": "How We Work",
    "nav.portfolio": "Portfolio",
    "nav.requestQuote": "Request a Quote",
    "nav.contact": "Contact",
    "nav.clientArea": "Client Area",

    // Hero Section
    "hero.badge": "Excellence in Architectural Projects",
    "hero.title": "Transforming your ideas into",
    "hero.titleHighlight": "exceptional projects",
    "hero.subtitle": "Specialists in technical drafting, executive projects, and wood frame structures. From conception to final detailing.",
    "hero.cta.quote": "Request a Quote",
    "hero.cta.portfolio": "View Portfolio",
    "hero.stats.projects": "Projects Delivered",
    "hero.stats.experience": "Years of Experience",
    "hero.stats.clients": "Satisfied Clients",

    // Services Section
    "services.title": "Design-Led Architectural Solutions",
    "services.subtitle": "We deliver design-driven architectural solutions from concept development to construction-ready documentation, aligned
    "services.learnMore": "Learn more",
    "services.cta": "Request Free Quote",

    // Portfolio Section
    "portfolio.title": "Selected Projects",
    "portfolio.subtitle": "A curated selection of residential and commercial projects developed for builders, investors, and private clients acro
    "portfolio.badge": "Designed to meet U.S. building codes and construction standards.",
    "portfolio.viewProject": "View Project",
    "portfolio.viewAll": "View all projects",
    "portfolio.item1.title": "Modern Single-Family Residence",
    "portfolio.item1.category": "Residential Architecture",
    "portfolio.item2.title": "Wood Frame Residential Build",
    "portfolio.item2.category": "Wood Framing Layout",
    "portfolio.item3.title": "Commercial Retail Building",
    "portfolio.item3.category": "Commercial Design",

    // CTA Section
    "cta.title": "Let's Build Something Exceptional",
    "cta.subtitle": "Tell us about your project. We'll guide you through the next steps with clarity, strategy, and precision.",
    "cta.proposal": "Request a Proposal",
    "cta.call": "Schedule a Call",

    // Footer
    "footer.tagline": "Transforming ideas into exceptional projects with precision and excellence.",
    "footer.quickLinks": "Quick Links",
    "footer.services": "Services",
    "footer.contact": "Contact",
    "footer.rights": "All rights reserved.",
    "footer.service1": "Technical Drafting",
    "footer.service2": "Executive Project",
    "footer.service3": "Wood Frame Structures",
    "footer.service4": "Project Budgeting",

    // Auth
    "auth.title": "Client Portal",
    "auth.subtitle": "Access your projects, files, and communications",
    "auth.signIn": "Sign In",
    "auth.createAccount": "Create Account",
    "auth.welcomeBack": "Welcome Back",
    "auth.enterCredentials": "Enter your credentials to access your account",
    "auth.register": "Register to access your project portal",
    "auth.email": "Email",
    "auth.password": "Password",
    "auth.fullName": "Full Name",
    "auth.confirmPassword": "Confirm Password",
    "auth.signingIn": "Signing in...",
    "auth.creatingAccount": "Creating account...",
    "auth.forgotPassword": "Forgot your password?",
    "auth.resetPassword": "Reset Password",
    "auth.resetInstructions": "Enter your email and we'll send you a reset link",
    "auth.sendResetLink": "Send Reset Link",
    "auth.sending": "Sending...",
    "auth.backToLogin": "Back to Login",
    "auth.checkEmail": "Check your email for the reset link",

    // Quote Form
    "quote.title": "Request a Quote",
    "quote.subtitle": "Complete the form below to receive a personalized proposal",
    "quote.step.location": "Location",
```

```
"quote.step.project": "Project",
"quote.step.client": "Your Info",
"quote.step.details": "Details",
"quote.locationDesc": "Where will the project be located?",
"quote.projectDesc": "Tell us about your project",
"quote.clientDesc": "Your contact information",
"quote.detailsDesc": "Additional information (optional)",
"quote.back": "Back",
"quote.next": "Next",
"quote.submit": "Submit Request",
"quote.submitting": "Submitting...",

// Thank You Page
"thankyou.title": "Project Received",
"thankyou.message": "Thank you for your submission.",
"thankyou.projectCreated": "Your project has been created under:",
"thankyou.proposalTime": "You will receive a detailed proposal within 2-3 business days.",
"thankyou.portalAccess": "We've sent you an email with instructions to access your Client Portal.",
"thankyou.goHome": "Return Home",
"thankyou.goPortal": "Go to Client Portal",

// Dashboard
"dashboard.welcome": "Welcome",
"dashboard.welcomeBack": "Welcome back",
"dashboard.overview": "Here's an overview of your projects and activity",
"dashboard.home": "Home",
"dashboard.projects": "Projects",
"dashboard.quotes": "Quotes",
"dashboard.messages": "Messages",
"dashboard.profile": "Profile",
"dashboard.company": "Company",
"dashboard.newQuote": "New Quote",
"dashboard.signOut": "Sign Out",
"dashboard.noProjects": "No projects yet",
"dashboard.noProjectsDesc": "Start by requesting a quote for your first project",
"dashboard.openProject": "Open Project",
"dashboard.activeProjects": "Active Projects",
"dashboard.quotesInvoices": "Quotes &amp;amp; Invoices",
"dashboard.newQuoteRequest": "New Quote Request",
"dashboard.yourProjects": "Your Projects",
"dashboard.requestQuote": "Request a Quote",
"dashboard.myProjects": "My Projects",
"dashboard.viewManageProjects": "View and manage all your projects",
"dashboard.comingSoon": "Coming Soon",
"dashboard.comingSoonDesc": "This section is under development. Check back soon!",
"dashboard.myProfile": "My Profile",
"dashboard.manageProfile": "Manage your personal information",
"dashboard.companyInfo": "Company Information",
"dashboard.manageCompany": "Manage your company details",
"dashboard.communicateTeam": "Communicate with our team about your projects",
"dashboard.viewQuotes": "View your quotes, invoices, and payment history",
"dashboard.menu": "Menu",
"dashboard.settings": "Settings",
"dashboard.backToDashboard": "Back to Dashboard",

// Dashboard New Quote
"newQuote.title": "New Quote Request",
"newQuote.subtitle": "Submit a new project for a quote estimate",
"newQuote.projectDetails": "Project Details",
"newQuote.projectDetailsDesc": "Provide the basic information about your project",
"newQuote.projectAddress": "Project Address",
"newQuote.city": "City",
"newQuote.state": "State",
"newQuote.projectType": "Project Type",
"newQuote.selectProjectType": "Select project type",
"newQuote.estimatedSize": "Estimated Project Size (sq ft)",
"newQuote.additionalNotes": "Additional Notes (optional)",
"newQuote.additionalNotesPlaceholder": "Describe your project requirements, special requests, or any other relevant information...",
"newQuote.cancel": "Cancel",
"newQuote.submitting": "Submitting...",
"newQuote.submitRequest": "Submit Request",

// Project Card
"projectCard.briefing": "Briefing",
"projectCard.inDevelopment": "In Development",
"projectCard.underReview": "Under Review",
"projectCard.completed": "Completed",
"projectCard.cancelled": "Cancelled",
"projectCard.files": "files",
"projectCard.msgs": "msgs",
"projectCard.quotes": "quotes",

// How We Work
"howWeWork.title": "HOW WE WORK",
"howWeWork.subtitle": "Project Process &amp;amp; Professional Scope",
"howWeWork.ourProcess": "Our Process",
"howWeWork.ourProcessDesc": "We follow a structured and efficient workflow designed for builders, developers, and serious homeowners.",
```

"howWeWork.whatsIncluded": "What's Included",
"howWeWork.whatsIncludedDesc": "Our services focus on architectural design development and drawing documentation. Depending on the selected p
"howWeWork.whatsNotIncluded": "What's Not Included",
"howWeWork.whatsNotIncludedDesc": "To maintain clarity and compliance, the following services are not part of our scope unless explicitly con
"howWeWork.compliance": "Compliance & Responsibility",
"howWeWork.complianceDesc": "Our drawings are intended for design development and permit documentation support. Final approval is subject to
"howWeWork.clientResponsible": "The client is responsible for:",
"howWeWork.complianceNote": "We design in alignment with U.S. residential construction standards; however, final compliance depends on local
"howWeWork.meetings": "Meetings & Communication",
"howWeWork.meetingsDesc": "DA•RA Studio operates remotely with a structured digital workflow.",
"howWeWork.meetingsNote": "We have proudly supported builders and investors across multiple U.S. states.",
"howWeWork.whyDara": "Why DA•RA Studio",
"howWeWork.drawingsNote": "All drawings are prepared in accordance with standard U.S. residential drafting practices.",
"howWeWork.licensedNote": "When required, licensed professionals must be hired locally by the client.",

// Process Steps
"process.step1.title": "Proposal Approval",
"process.step1.desc": "Project scope and pricing are confirmed.",
"process.step1.detail": "During this phase, we finalize all project parameters including scope of work, deliverables, timeline estimates, and
"process.step2.title": "Design Preview",
"process.step2.desc": "Initial layout and visual direction are delivered for review.",
"process.step2.timeline": "8–16 Business Days",
"process.step2.note": "Timeline starts from receipt of required project information, including proposal approval, initial payment, and site c
"process.step2.detail": "Our design team develops the initial conceptual layouts based on your project requirements, site conditions, and des
"process.step3.title": "Revisions Phase",
"process.step3.desc": "Two (2) revision rounds are included to refine layout and design intent.",
"process.step3.note": "Additional revisions are available upon request.",
"process.step3.detail": "Collaboration is key to a successful project. During this phase, we work closely with you to refine the design based
"process.step4.title": "Final Drawing Set",
"process.step4.desc": "Complete architectural drawing package is delivered in digital format.",
"process.step4.timeline": "25–30 Business Days After Approval",
"process.step4.note": "Timeline may vary depending on project complexity, requested revisions, and technical coordination during project deve
"process.step4.detail": "Once the design preview is approved, our team develops the complete construction documentation package. This include
"process.step5.title": "Ongoing Coordination",
"process.step5.desc": "We provide remote support and coordination with your contractor or engineer as needed.",
"process.step5.detail": "Our commitment doesn't end with drawing delivery. We remain available to answer questions from your construction tea

// Included Items
"included.1": "Architectural floor plans",
"included.2": "Exterior elevations",
"included.3": "Basic building sections",
"included.4": "Interior layout planning",
"included.5": "Kitchen and bathroom layouts",
"included.6": "Fixture and equipment placement",
"included.7": "Door and window schedules",
"included.8": "Dimensioned construction drawings (for permit submission when applicable)",
"included.9": "3D visualization (if included in selected package)",

// Not Included Items
"notIncluded.1": "Structural engineering calculations",
"notIncluded.2": "Engineer stamp or sealed drawings",
"notIncluded.3": "Load diagrams or structural reinforcement detailing",
"notIncluded.4": "Full electrical system design",
"notIncluded.5": "Full plumbing system design (we indicate fixture locations only)",
"notIncluded.6": "Mechanical/HVAC design",
"notIncluded.7": "Site surveys, plot plans, or topographic mapping",
"notIncluded.8": "Soil testing or geotechnical reports",
"notIncluded.9": "Energy compliance reports (HERS, blower door, etc.)",
"notIncluded.10": "In-person site visits or inspections",
"notIncluded.11": "Material procurement or interior décor shopping lists",

// Client Responsibilities
"responsibility.1": "Hiring licensed structural engineers (if required)",
"responsibility.2": "Coordinating required technical reports",
"responsibility.3": "Verifying local zoning and code restrictions",

// Communication Features
"communication.1": "Full online project management",
"communication.2": "Scheduled video meetings",
"communication.3": "Direct communication during business hours",
"communication.4": "Continuous updates throughout each phase",

// Why DARA
"whyDara.1": "Code-conscious documentation",
"whyDara.2": "Builder-friendly drawing sets",
"whyDara.3": "Efficient turnaround",
"whyDara.4": "Clear scope and professional boundaries",
"whyDara.5": "Trusted by developers and construction teams",

// 404
"notFound.title": "404",
"notFound.message": "Oops! Page not found",
"notFound.goHome": "Return to Home",

// Language Selector
"language.english": "English",
"language.portuguese": "Português",

```
// Freelancer
"freelancer.workspace": "Freelancer Workspace",
"freelancer.dashboard": "Dashboard",
"freelancer.projects": "Projects",
"freelancer.payments": "Payments",
"freelancer.contracts": "Contracts",
"freelancer.calendar": "Calendar",
"freelancer.workspaceLabel": "Workspace",
"freelancer.welcome": "Welcome",
"freelancer.overviewDesc": "Here's your workspace overview",
"freelancer.activeProjects": "Active Projects",
"freelancer.pendingPayments": "Pending Payments",
"freelancer.totalEarnings": "Total Earnings",
"freelancer.upcomingDeadlines": "Upcoming Deadlines",
"freelancer.noUpcomingDeadlines": "No upcoming deadlines",
"freelancer.myProjects": "My Projects",
"freelancer.loading": "Loading...",
"freelancer.noProjectsAssigned": "No projects assigned yet",
"freelancer.backToProjects": "← Back to Projects",
"freelancer.address": "Address",
"freelancer.status": "Status",
"freelancer.service": "Service",
"freelancer.deadline": "Deadline",
"freelancer.notes": "Notes",
"freelancer.referenceFiles": "Reference Files",
"freelancer.noFiles": "No files available",
"freelancer.submitDelivery": "Submit Delivery",
"freelancer.fileName": "File Name",
"freelancer.fileUrl": "File URL",
"freelancer.notesOptional": "Notes (optional)",
"freelancer.submitting": "Submitting...",
"freelancer.yourDeliveries": "Your Deliveries",
"freelancer.view": "View",
"freelancer.pending": "Pending",
"freelancer.totalPaid": "Total Paid",
"freelancer.paymentHistory": "Payment History",
"freelancer.noPayments": "No payments yet",
"freelancer.due": "Due",
"freelancer.noDueDate": "No due date",
"freelancer.agreedAmount": "Agreed Amount",
"freelancer.paidAmount": "Paid",
"freelancer.remainingAmount": "Balance",
"freelancer.partial": "Partial",
"freelancer.paid": "Paid",
"freelancer.totalReceivable": "Total Receivable",
"freelancer.totalAlreadyPaid": "Total Already Paid",
"freelancer.nextPayment": "Next Payment",
"freelancer.project": "Project",
"freelancer.paymentMethod": "Method",
"freelancer.noContracts": "No contracts available",
"freelancer.signed": "Signed",
"freelancer.unsigned": "Unsigned",
"freelancer.download": "Download",
"freelancer.unknownProject": "Unknown project",
"freelancer.selectDate": "Select a date",
"freelancer.noDeadlinesOnDate": "No deadlines on this date",
"freelancer.allUpcomingDeadlines": "All Upcoming Deadlines",
"freelancer.noUpcoming": "No upcoming deadlines",
"freelancer.signOut": "Sign Out",

// Freelancer Auth
"freelancerAuth.title": "Freelancer Portal",
"freelancerAuth.subtitle": "Access your workspace, projects, and payments",
"freelancerAuth.welcomeBack": "Welcome, Collaborator",
"freelancerAuth.enterCredentials": "Sign in with the credentials provided by the admin",
"freelancerAuth.signIn": "Access Workspace",
"freelancerAuth.noAccess": "You don't have freelancer access. Contact the administrator.",
"freelancerAuth.invalidCredentials": "Invalid email or password. Please try again.",
"freelancerAuth.accessNote": "Access is provided by the DA•RA Studio team. If you don't have credentials, contact your project manager.",

// Quote Form - Location Step
"form.projectAddress": "Project Address",
"form.streetAddress": "Street address",
"form.city": "City",
"form.state": "State",
"form.statePlaceholder": "e.g., CA, TX, FL",
"form.zipCode": "ZIP Code",
"form.zipPlaceholder": "e.g., 90210",

// Quote Form - Project Step
"form.projectInfo": "Project Information",
"form.projectType": "Project Type",
"form.selectOne": "Select one option",
"form.hasExistingPlans": "Do you have existing architectural plans?",
"form.yes": "Yes",
"form.no": "No",
"form.hasEngineer": "Do you already have an engineer or need a referral?",
```

"form.haveOne": "I already have one",
"form.needReferral": "I need a referral",
"form.buildingStatus": "Is the existing building occupied or under construction?",
"form.projectSizeLevels": "Project Size & Levels",
"form.existingArea": "Existing Area (if applicable)",
"form.sqftOptional": "sq ft (optional)",
"form.proposedArea": "Proposed / Estimated Project Area",
"form.proposedAreaDesc": "Approximate size of the new or modified area",
"form.projectLevels": "Project Levels (check all that apply):",
"form.fileUploads": "File Uploads & References",
"form.uploadDesc": "Upload photos, sketches, or reference files",
"form.acceptsFormats": "Accepts PDF, JPG, PNG, DWG",
"form.referenceNotesPlaceholder": "Describe your reference files or include notes about the documents you'll be sharing",
"form.referenceLinks": "Reference links (optional)",
"form.referenceLinksDesc": "Google Drive, Dropbox, etc.",
"form.referenceLinksPlaceholder": "Paste links to your files here...",
"form.programRequirements": "Program of Requirements",
"form.programRequirementsDesc": "For residential projects, including ADU when applicable",
"form.other": "Other:",
"form.otherPlaceholder": "Specify other spaces needed",
"form.servicesOffered": "Services Offered by DARA",
"form.designDrafting": "Design & Drafting Services",
"form.selectServices": "Select the services you're interested in",
"form.disclaimer": "Professional Disclaimer",
"form.disclaimerText": "DARA provides design and drafting services only. All drawings are conceptual and illustrative in nature and are not i
"form.ifOtherDescribe": 'If "Other", please describe:',
"form.describeProjectType": "Describe your project type",

// Quote Form - Client Step
"form.iAmA": "I am a...",
"form.contactInfo": "Contact Information",
"form.fullName": "Full Name",
"form.fullNamePlaceholder": "Your full name",
"form.email": "Email",
"form.phone": "Phone Number",
"form.companyInfo": "Company Information",
"form.companyName": "Company Name",
"form.companyNamePlaceholder": "Your company name",
"form.companyAddress": "Company Address",
"form.companyAddressPlaceholder": "Full business address",
"form.instagram": "Instagram",
"form.companyPhone": "Company Phone",
"form.companyEmail": "Company Email",
"form.optional": "(optional)",

// Quote Form - Details Step
"form.desiredTimeline": "Desired Timeline",
"form.whenStart": "When would you like to start?",
"form.estimatedInvestment": "Estimated Investment for Design Services",
"form.investmentDesc": "Optional — helps us prepare the right proposal",
"form.investmentNote": "This refers to design and drafting services only (not construction costs).",
"form.howHear": "How did you hear about us?",
"form.additionalNotes": "Additional Notes",
"form.additionalNotesDesc": "Tell us any important details about your project",
"form.additionalNotesPlaceholder": "Share any special requirements, preferences, or questions...",
"form.agreement": "I agree to the processing of my information for quote purposes and accept the",
"form.termsOfUse": "Terms of Use",
"form.and": "and",
"form.privacyPolicy": "Privacy Policy",

// Client Types
"clientType.homeowner": "Homeowner",
"clientType.contractor": "General Contractor",
"clientType.architect": "Architect",
"clientType.engineer": "Engineer",
"clientType.realEstate": "Real Estate Developer",

// Project Types
"projectType.singleFamily": "Single-Family Residential",
"projectType.multiFamily": "Multi-Family Residential",
"projectType.commercial": "Commercial",
"projectType.renovation": "Renovation / Remodel",
"projectType.addition": "Addition / Extension",
"projectType.adu": "ADU (Accessory Dwelling Unit)",
"projectType.deck": "Deck",
"projectType.garage": "Garage (Attached / Detached)",
"projectType.other": "Other",

// Building Status
"buildingStatus.occupied": "Occupied",
"buildingStatus.underConstruction": "Under Construction",
"buildingStatus.vacant": "Vacant",
"buildingStatus.newConstruction": "N/A (New Construction)",

// Project Levels
"level.basement": "Basement",
"level.firstFloor": "First Floor",

"level.secondFloor": "Second Floor",
"level.atticLoft": "Attic / Loft",
"level.garageLevel": "Garage Level",

// Timeline Options
"timeline.asap": "ASAP",
"timeline.within30": "Within 30 days",
"timeline.1to3months": "1–3 months",
"timeline.3plusMonths": "3+ months",
"timeline.flexible": "Flexible",

// Budget Options
"budget.under2500": "Under $2,500",
"budget.2500to5000": "$2,500 – $5,000",
"budget.5000to10000": "$5,000 – $10,000",
"budget.10000plus": "$10,000+",
"budget.notSure": "Not sure yet",

// Referral Options
"referral.google": "Google",
"referral.referral": "Referral",
"referral.socialMedia": "Social Media",
"referral.contractor": "Contractor",
"referral.returningClient": "Returning Client",
"referral.other": "Other",

// Required Spaces
"spaces.private": "Private Spaces",
"spaces.bathrooms": "Bathrooms",
"spaces.social": "Social Areas",
"spaces.kitchen": "Kitchen & Support Spaces",
"spaces.work": "Work & Wellness",
"spaces.circulation": "Circulation & Storage",
"spaces.technical": "Technical Areas",
"spaces.outdoor": "Outdoor Areas",

// DARA Services Categories
"daraService.architecture": "ARCHITECTURAL DESIGN & DRAFTING",
"daraService.interiors": "INTERIOR DESIGN & SPACE PLANNING",
"daraService.visualization": "3D VISUALIZATION",
"daraService.documentation": "TECHNICAL DRAFTING & DOCUMENTATION",
"daraService.construction": "CONSTRUCTION COORDINATION SUPPORT",
"daraService.additional": "ADDITIONAL DESIGN SERVICES",

// Admin Panel
"admin.dashboard": "Dashboard",
"admin.studioOverview": "Studio overview",
"admin.projects": "Projects",
"admin.manageProjects": "Manage all studio projects",
"admin.companies": "Companies",
"admin.payments": "Payments",
"admin.milestones": "Milestones",
"admin.activity": "Activity",
"admin.map": "Map",
"admin.signOut": "Sign Out",
"admin.newProject": "New Project",
"admin.editProject": "Edit Project",
"admin.company": "Company",
"admin.select": "Select",
"admin.projectStage": "Project Stage",
"admin.paymentStage": "Payment Stage",
"admin.package": "Package",
"admin.none": "None",
"admin.services": "Services",
"admin.projectAddress": "Project Address",
"admin.copyFromCompany": "Copy from Company",
"admin.street": "Street",
"admin.number": "Number",
"admin.unit": "Unit",
"admin.neighborhood": "Neighborhood",
"admin.city": "City",
"admin.state": "State",
"admin.zip": "Zip",
"admin.countryLabel": "Country",
"admin.dates": "Dates",
"admin.startDate": "Start Date",
"admin.deliveryDate": "Delivery Date",
"admin.paymentEffective": "Payment Effective",
"admin.previewDelivery": "Preview Delivery",
"admin.previewSent": "Preview Sent",
"admin.revisionReturn": "Revision Return",
"admin.finalDelivery": "Final Delivery",
"admin.revisionCount": "Revision Count",
"admin.financials": "Financials",
"admin.currency": "Currency",
"admin.totalValue": "Total Value",
"admin.entryAmount": "Entry Amount",

```
    "admin.finalAmount": "Final Amount",
    "admin.save": "Save",
    "admin.create": "Create",
    "admin.search": "Search...",
    "admin.stage": "Stage",
    "admin.allStages": "All Stages",
    "admin.allCompanies": "All Companies",
    "admin.address": "Address",
    "admin.delivery": "Delivery",
    "admin.value": "Value",
    "admin.paid": "Paid",
    "admin.balance": "Balance",
    "admin.noProjects": "No projects found",
    "admin.noCompany": "No company",
    "admin.selectCompanyFirst": "Select a company first",
    "admin.addressCopied": "Address copied from company",
    "admin.projectUpdated": "Project updated",
    "admin.projectCreated": "Project created",
    "admin.delayedProjects": "Delayed Projects",
    "admin.waitingPayment": "Waiting Payment",
    "admin.overduePayment": "Overdue Payment",
    "admin.inProgress": "In Progress",
    "admin.cancelled": "Cancelled",
    "admin.pendingRevenue": "Pending Revenue",
    "admin.receivedThisMonth": "Received This Month",
    "admin.mapView": "Map View",
    "admin.loadingMap": "Loading map...",
    "admin.noProjectsWithLocation": "No projects with location data. Add latitude and longitude to your projects to see them on the map.",
    "admin.projectsWithLocation": "projects with location data",
    "admin.noAddress": "No address",
    "admin.financialControl": "Financial control of projects",
    "admin.newPayment": "New Payment",
    "admin.registerPayment": "Register Payment",
    "admin.project": "Project",
    "admin.selectProject": "Select a project",
    "admin.type": "Type",
    "admin.method": "Method",
    "admin.amountTotal": "Total Amount",
    "admin.amountPaid": "Amount Paid",
    "admin.stripeFee": "Stripe Fee",
    "admin.status": "Status",
    "admin.paymentDate": "Payment Date",
    "admin.register": "Register",
    "admin.paymentRegistered": "Payment registered",
    "admin.totalReceived": "Total Received",
    "admin.pendingLabel": "Pending",
    "admin.records": "Records",
    "admin.total": "Total",
    "admin.net": "Net",
    "admin.date": "Date",
    "admin.noPayments": "No payments registered",
    "admin.paidStatus": "Paid",
    "admin.partialStatus": "Partial",
    "admin.pendingStatus": "Pending",
    "admin.restrictedAccess": "Restricted Access",
    "admin.noPermission": "You do not have permission to access this area.",
    "admin.backToHome": "Back to home",
    "admin.allCities": "All Cities",
    "admin.previewSentDate": "Preview Sent",
    "admin.projectFinancials": "Project Financial Summary",
    "admin.paymentRecords": "Payment Records",

    //

... [TRUNCATED - file too large] ...
```

src/index.css

```
@import url("https://fonts.googleapis.com/css2?family=Roboto&amp;display=swap");
@import url("https://fonts.googleapis.com/css2?family=Libre+Caslon+Text&amp;display=swap");
@import url("https://fonts.googleapis.com/css2?family=Roboto+Mono&amp;display=swap");
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Definition of the design system. All colors, gradients, fonts, etc should be defined here.
All colors MUST be HSL.
*/

@layer base {
  :root {
    --background: 0 0% 96%;
    --foreground: 0 0% 9%;

    --card: 0 0% 98%;
    --card-foreground: 0 0% 9%;

    --popover: 0 0% 89%;
    --popover-foreground: 0 0% 9%;

    --primary: 42 55% 44%;
    --primary-foreground: 33 100% 96%;

    --secondary: 0 0% 32%;
    --secondary-foreground: 0 0% 98%;

    --muted: 0 0% 63%;
    --muted-foreground: 0 0% 9%;

    --accent: 47 100% 96%;
    --accent-foreground: 42 55% 44%;

    --destructive: 0 72% 50%;
    --destructive-foreground: 0 85% 97%;

    --border: 0 0% 83%;
    --input: 0 0% 83%;
    --ring: 42 55% 44%;

    --radius: 0.75rem;

    --sidebar-background: 0 0% 98%;

    --sidebar-foreground: 0 0% 9%;

    --sidebar-primary: 20 90% 48%;

    --sidebar-primary-foreground: 33 100% 96%;

    --sidebar-accent: 0 0% 32%;

    --sidebar-accent-foreground: 0 0% 98%;

    --sidebar-border: 0 0% 83%;

    --sidebar-ring: 20 90% 48%;

    --chart-1: 42 55% 44%;

    --chart-2: 43 96% 56%;

    --chart-3: 47 95% 53%;

    --chart-4: 42 55% 44%;

    --chart-5: 0 0% 45%;

    --sidebar: 0 0% 98%;

    --font-sans: 'Roboto', ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Helvetica Neue', Arial, 'Noto Sans', sans-se

    --font-serif: 'Libre Caslon Text', ui-serif, Georgia, Cambria, 'Times New Roman', Times, serif;

    --font-mono: 'Roboto Mono', ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, 'Liberation Mono', 'Courier New', monospace;

    --shadow-2xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);

    --shadow-xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);

    --shadow-sm: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);

    --shadow: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);
```

```
    --shadow-md: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 2px 4px -1px hsl(0 0% 0% / 0.1);

    --shadow-lg: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 4px 6px -1px hsl(0 0% 0% / 0.1);

    --shadow-xl: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 8px 10px -1px hsl(0 0% 0% / 0.1);

    --shadow-2xl: 0 1px 3px 0px hsl(0 0% 0% / 0.25);

    --tracking-normal: 0em;

    --spacing: 0.25rem;
  }

  .dark {
    --background: 0 0% 9%;
    --foreground: 0 0% 98%;

    --card: 0 0% 14%;
    --card-foreground: 0 0% 98%;

    --popover: 0 0% 25%;
    --popover-foreground: 0 0% 98%;

    --primary: 27 95% 60%;
    --primary-foreground: 12 81% 14%;

    --secondary: 0 0% 45%;
    --secondary-foreground: 0 0% 98%;

    --muted: 0 0% 45%;
    --muted-foreground: 0 0% 98%;

    --accent: 20 91% 14%;
    --accent-foreground: 43 96% 56%;

    --destructive: 0 84% 60%;
    --destructive-foreground: 0 85% 97%;

    --border: 0 0% 32%;
    --input: 0 0% 32%;
    --ring: 27 95% 60%;
    --sidebar-background: 240 5.9% 10%;
    --sidebar-foreground: 0 0% 98%;
    --sidebar-primary: 27 95% 60%;
    --sidebar-primary-foreground: 12 81% 14%;
    --sidebar-accent: 43 96% 56%;
    --sidebar-accent-foreground: 20 91% 14%;
    --sidebar-border: 0 0% 32%;
    --sidebar-ring: 27 95% 60%;
    --chart-1: 30 97% 72%;
    --chart-2: 45 96% 64%;
    --chart-3: 48 96% 76%;
    --chart-4: 27 95% 60%;
    --chart-5: 0 0% 45%;
    --sidebar: 0 0% 14%;
    --shadow-2xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);
    --shadow-xs: 0 1px 3px 0px hsl(0 0% 0% / 0.05);
    --shadow-sm: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);
    --shadow: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 1px 2px -1px hsl(0 0% 0% / 0.1);
    --shadow-md: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 2px 4px -1px hsl(0 0% 0% / 0.1);
    --shadow-lg: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 4px 6px -1px hsl(0 0% 0% / 0.1);
    --shadow-xl: 0 1px 3px 0px hsl(0 0% 0% / 0.1), 0 8px 10px -1px hsl(0 0% 0% / 0.1);
    --shadow-2xl: 0 1px 3px 0px hsl(0 0% 0% / 0.25);
  }
}

@layer base {
  * {
    @apply border-border;
  }

  body {
    @apply bg-background text-foreground;
  }
}
```

src/integrations/supabase/client.ts

```
// This file is automatically generated. Do not edit it directly.
import { createClient } from '@supabase/supabase-js';
import type { Database } from './types';

const SUPABASE_URL = import.meta.env.VITE_SUPABASE_URL;
const SUPABASE_PUBLISHABLE_KEY = import.meta.env.VITE_SUPABASE_PUBLISHABLE_KEY;

// Import the supabase client like this:
// import { supabase } from "@/integrations/supabase/client";

export const supabase = createClient&lt;Database&gt;(SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, {
  auth: {
    storage: localStorage,
    persistSession: true,
    autoRefreshToken: true,
  }
});
```

src/integrations/supabase/types.ts

```
export type Json =
  | string
  | number
  | boolean
  | null
  | { [key: string]: Json | undefined }
  | Json[]

export type Database = {
  // Allows to automatically instantiate createClient with right options
  // instead of createClient&lt;Database, { PostgrestVersion: 'XX' }&gt;(URL, KEY)
  __InternalSupabase: {
    PostgrestVersion: "14.1"
  }
  public: {
    Tables: {
      activity_log: {
        Row: {
          created_at: string
          description: string
          id: string
          log_date: string
          log_type: string
          project_id: string
        }
        Insert: {
          created_at?: string
          description: string
          id?: string
          log_date?: string
          log_type?: string
          project_id: string
        }
        Update: {
          created_at?: string
          description?: string
          id?: string
          log_date?: string
          log_type?: string
          project_id?: string
        }
        Relationships: [
          {
            foreignKeyName: "activity_log_project_id_fkey"
            columns: ["project_id"]
            isOneToOne: false
            referencedRelation: "projects"
            referencedColumns: ["id"]
          },
        ]
      }
      admin_payments: {
        Row: {
          amount_paid: number
          amount_total: number
          created_at: string
          id: string
          installment_type: string
          net_received: number | null
          payment_date: string | null
          payment_method: string | null
          payment_notes: string | null
          price_table: string | null
          project_id: string
          status: string
          stripe_fee_percent: number | null
          third_party_costs: number | null
          updated_at: string
          value_brl: number | null
        }
        Insert: {
          amount_paid?: number
          amount_total?: number
          created_at?: string
          id?: string
          installment_type?: string
          net_received?: number | null
          payment_date?: string | null
          payment_method?: string | null
          payment_notes?: string | null
          price_table?: string | null
          project_id: string
          status?: string
          stripe_fee_percent?: number | null
```

```
      third_party_costs?: number | null
      updated_at?: string
      value_brl?: number | null
    }
    Update: {
      amount_paid?: number
      amount_total?: number
      created_at?: string
      id?: string
      installment_type?: string
      net_received?: number | null
      payment_date?: string | null
      payment_method?: string | null
      payment_notes?: string | null
      price_table?: string | null
      project_id?: string
      status?: string
      stripe_fee_percent?: number | null
      third_party_costs?: number | null
      updated_at?: string
      value_brl?: number | null
    }
    Relationships: [
      {
        foreignKeyName: "admin_payments_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  admin_tasks: {
    Row: {
      completed_at: string | null
      created_at: string
      created_by: string | null
      due_date: string
      id: string
      is_completed: boolean
      priority: string
      project_id: string | null
      title: string
      updated_at: string
    }
    Insert: {
      completed_at?: string | null
      created_at?: string
      created_by?: string | null
      due_date?: string
      id?: string
      is_completed?: boolean
      priority?: string
      project_id?: string | null
      title: string
      updated_at?: string
    }
    Update: {
      completed_at?: string | null
      created_at?: string
      created_by?: string | null
      due_date?: string
      id?: string
      is_completed?: boolean
      priority?: string
      project_id?: string | null
      title?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "admin_tasks_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  client_ticket_messages: {
    Row: {
      content: string
      created_at: string
      id: string
      is_from_admin: boolean
      sender_id: string | null
      ticket_id: string
```

```
    }
    Insert: {
      content: string
      created_at?: string
      id?: string
      is_from_admin?: boolean
      sender_id?: string | null
      ticket_id: string
    }
    Update: {
      content?: string
      created_at?: string
      id?: string
      is_from_admin?: boolean
      sender_id?: string | null
      ticket_id?: string
    }
    Relationships: [
      {
        foreignKeyName: "client_ticket_messages_ticket_id_fkey"
        columns: ["ticket_id"]
        isOneToOne: false
        referencedRelation: "client_tickets"
        referencedColumns: ["id"]
      },
    ]
  }
  client_tickets: {
    Row: {
      company_id: string
      created_at: string
      created_by: string | null
      id: string
      project_id: string
      status: string
      subject: string
      updated_at: string
    }
    Insert: {
      company_id: string
      created_at?: string
      created_by?: string | null
      id?: string
      project_id: string
      status?: string
      subject: string
      updated_at?: string
    }
    Update: {
      company_id?: string
      created_at?: string
      created_by?: string | null
      id?: string
      project_id?: string
      status?: string
      subject?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "client_tickets_company_id_fkey"
        columns: ["company_id"]
        isOneToOne: false
        referencedRelation: "companies"
        referencedColumns: ["id"]
      },
      {
        foreignKeyName: "client_tickets_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  companies: {
    Row: {
      address: string | null
      avatar_url: string | null
      city: string | null
      company_type: string | null
      country: string | null
      created_at: string
      email: string | null
      id: string
      instagram: string | null
      language: string | null
```

```
      linkedin: string | null
      name: string
      neighborhood: string | null
      owner_name: string | null
      personal_email: string | null
      personal_instagram: string | null
      phone: string | null
      state: string | null
      status: string | null
      street_name: string | null
      street_number: string | null
      unit: string | null
      updated_at: string
      website: string | null
      zip_code: string | null
    }
    Insert: {
      address?: string | null
      avatar_url?: string | null
      city?: string | null
      company_type?: string | null
      country?: string | null
      created_at?: string
      email?: string | null
      id?: string
      instagram?: string | null
      language?: string | null
      linkedin?: string | null
      name: string
      neighborhood?: string | null
      owner_name?: string | null
      personal_email?: string | null
      personal_instagram?: string | null
      phone?: string | null
      state?: string | null
      status?: string | null
      street_name?: string | null
      street_number?: string | null
      unit?: string | null
      updated_at?: string
      website?: string | null
      zip_code?: string | null
    }
    Update: {
      address?: string | null
      avatar_url?: string | null
      city?: string | null
      company_type?: string | null
      country?: string | null
      created_at?: string
      email?: string | null
      id?: string
      instagram?: string | null
      language?: string | null
      linkedin?: string | null
      name?: string
      neighborhood?: string | null
      owner_name?: string | null
      personal_email?: string | null
      personal_instagram?: string | null
      phone?: string | null
      state?: string | null
      status?: string | null
      street_name?: string | null
      street_number?: string | null
      unit?: string | null
      updated_at?: string
      website?: string | null
      zip_code?: string | null
    }
    Relationships: []
  }
  company_employees: {
    Row: {
      company_id: string
      created_at: string
      email: string | null
      id: string
      instagram: string | null
      linkedin: string | null
      name: string
      phone: string | null
    }
    Insert: {
      company_id: string
      created_at?: string
      email?: string | null
```

```
      id?: string
      instagram?: string | null
      linkedin?: string | null
      name: string
      phone?: string | null
    }
    Update: {
      company_id?: string
      created_at?: string
      email?: string | null
      id?: string
      instagram?: string | null
      linkedin?: string | null
      name?: string
      phone?: string | null
    }
    Relationships: [
      {
        foreignKeyName: "company_employees_company_id_fkey"
        columns: ["company_id"]
        isOneToOne: false
        referencedRelation: "companies"
        referencedColumns: ["id"]
      },
    ]
  }
  drawing_versions: {
    Row: {
      created_at: string
      drawing_name: string
      file_name: string
      file_url: string
      id: string
      is_current: boolean
      project_id: string
      revision_notes: string | null
      uploaded_by: string | null
      version_number: number
    }
    Insert: {
      created_at?: string
      drawing_name: string
      file_name: string
      file_url: string
      id?: string
      is_current?: boolean
      project_id: string
      revision_notes?: string | null
      uploaded_by?: string | null
      version_number?: number
    }
    Update: {
      created_at?: string
      drawing_name?: string
      file_name?: string
      file_url?: string
      id?: string
      is_current?: boolean
      project_id?: string
      revision_notes?: string | null
      uploaded_by?: string | null
      version_number?: number
    }
    Relationships: [
      {
        foreignKeyName: "drawing_versions_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_assignments: {
    Row: {
      assigned_at: string
      deadline: string | null
      freelancer_id: string
      id: string
      notes: string | null
      project_id: string
    }
    Insert: {
      assigned_at?: string
      deadline?: string | null
      freelancer_id: string
      id?: string
```

```
      notes?: string | null
      project_id: string
    }
    Update: {
      assigned_at?: string
      deadline?: string | null
      freelancer_id?: string
      id?: string
      notes?: string | null
      project_id?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_assignments_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_contracts: {
    Row: {
      created_at: string
      file_url: string | null
      freelancer_id: string
      id: string
      is_signed: boolean
      project_id: string
      signed_at: string | null
      title: string
    }
    Insert: {
      created_at?: string
      file_url?: string | null
      freelancer_id: string
      id?: string
      is_signed?: boolean
      project_id: string
      signed_at?: string | null
      title: string
    }
    Update: {
      created_at?: string
      file_url?: string | null
      freelancer_id?: string
      id?: string
      is_signed?: boolean
      project_id?: string
      signed_at?: string | null
      title?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_contracts_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_deliveries: {
    Row: {
      created_at: string
      file_name: string
      file_url: string
      freelancer_id: string
      id: string
      notes: string | null
      project_id: string
    }
    Insert: {
      created_at?: string
      file_name: string
      file_url: string
      freelancer_id: string
      id?: string
      notes?: string | null
      project_id: string
    }
    Update: {
      created_at?: string
      file_name?: string
      file_url?: string
      freelancer_id?: string
      id?: string
```

```
      notes?: string | null
      project_id?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_deliveries_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  freelancer_payments: {
    Row: {
      agreed_amount: number
      amount: number
      created_at: string
      description: string | null
      due_date: string | null
      freelancer_id: string
      id: string
      paid_amount: number
      payment_method: string | null
      project_id: string
      status: string
      updated_at: string
    }
    Insert: {
      agreed_amount?: number
      amount: number
      created_at?: string
      description?: string | null
      due_date?: string | null
      freelancer_id: string
      id?: string
      paid_amount?: number
      payment_method?: string | null
      project_id: string
      status?: string
      updated_at?: string
    }
    Update: {
      agreed_amount?: number
      amount?: number
      created_at?: string
      description?: string | null
      due_date?: string | null
      freelancer_id?: string
      id?: string
      paid_amount?: number
      payment_method?: string | null
      project_id?: string
      status?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "freelancer_payments_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  messages: {
    Row: {
      content: string
      created_at: string
      id: string
      is_from_admin: boolean
      project_id: string
      sender_id: string | null
    }
    Insert: {
      content: string
      created_at?: string
      id?: string
      is_from_admin?: boolean
      project_id: string
      sender_id?: string | null
    }
    Update: {
      content?: string
      created_at?: string
      id?: string
```

```
    is_from_admin?: boolean
    project_id?: string
    sender_id?: string | null
  }
  Relationships: [
    {
      foreignKeyName: "messages_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
  ]
}
milestones: {
  Row: {
    approved_date: string | null
    created_at: string
    id: string
    name: string
    notes: string | null
    project_id: string
    sent_date: string | null
    status: string
  }
  Insert: {
    approved_date?: string | null
    created_at?: string
    id?: string
    name: string
    notes?: string | null
    project_id: string
    sent_date?: string | null
    status?: string
  }
  Update: {
    approved_date?: string | null
    created_at?: string
    id?: string
    name?: string
    notes?: string | null
    project_id?: string
    sent_date?: string | null
    status?: string
  }
  Relationships: [
    {
      foreignKeyName: "milestones_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
  ]
}
packages: {
  Row: {
    created_at: string
    description: string | null
    id: string
    name: string
    slug: string
  }
  Insert: {
    created_at?: string
    description?: string | null
    id?: string
    name: string
    slug: string
  }
  Update: {
    created_at?: string
    description?: string | null
    id?: string
    name?: string
    slug?: string
  }
  Relationships: []
}
payment_milestones: {
  Row: {
    amount: number
    created_at: string
    id: string
    payment_date: string | null
    percentage: number
    project_id: string
```

```
      stage: string
      status: string
      updated_at: string
    }
    Insert: {
      amount?: number
      created_at?: string
      id?: string
      payment_date?: string | null
      percentage?: number
      project_id: string
      stage: string
      status?: string
      updated_at?: string
    }
    Update: {
      amount?: number
      created_at?: string
      id?: string
      payment_date?: string | null
      percentage?: number
      project_id?: string
      stage?: string
      status?: string
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "payment_milestones_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  profiles: {
    Row: {
      avatar_url: string | null
      company_id: string | null
      created_at: string
      email: string | null
      full_name: string | null
      id: string
      instagram: string | null
      phone: string | null
      updated_at: string
    }
    Insert: {
      avatar_url?: string | null
      company_id?: string | null
      created_at?: string
      email?: string | null
      full_name?: string | null
      id: string
      instagram?: string | null
      phone?: string | null
      updated_at?: string
    }
    Update: {
      avatar_url?: string | null
      company_id?: string | null
      created_at?: string
      email?: string | null
      full_name?: string | null
      id?: string
      instagram?: string | null
      phone?: string | null
      updated_at?: string
    }
    Relationships: [
      {
        foreignKeyName: "profiles_company_id_fkey"
        columns: ["company_id"]
        isOneToOne: false
        referencedRelation: "companies"
        referencedColumns: ["id"]
      },
    ]
  }
  project_files: {
    Row: {
      created_at: string
      document_category: string
      file_name: string
      file_size: number | null
      file_type: string | null
```

```
      file_url: string
      id: string
      project_id: string
      uploaded_by: string | null
    }
    Insert: {
      created_at?: string
      document_category?: string
      file_name: string
      file_size?: number | null
      file_type?: string | null
      file_url: string
      id?: string
      project_id: string
      uploaded_by?: string | null
    }
    Update: {
      created_at?: string
      document_category?: string
      file_name?: string
      file_size?: number | null
      file_type?: string | null
      file_url?: string
      id?: string
      project_id?: string
      uploaded_by?: string | null
    }
    Relationships: [
      {
        foreignKeyName: "project_files_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  project_homeowners: {
    Row: {
      address: string | null
      created_at: string
      email: string | null
      id: string
      name: string
      phone: string | null
      project_id: string
    }
    Insert: {
      address?: string | null
      created_at?: string
      email?: string | null
      id?: string
      name: string
      phone?: string | null
      project_id: string
    }
    Update: {
      address?: string | null
      created_at?: string
      email?: string | null
      id?: string
      name?: string
      phone?: string | null
      project_id?: string
    }
    Relationships: [
      {
        foreignKeyName: "project_homeowners_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  project_services: {
    Row: {
      id: string
      project_id: string
      service_id: string
    }
    Insert: {
      id?: string
      project_id: string
      service_id: string
    }
    Update: {
```

```
    id?: string
    project_id?: string
    service_id?: string
  }
  Relationships: [
    {
      foreignKeyName: "project_services_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
    {
      foreignKeyName: "project_services_service_id_fkey"
      columns: ["service_id"]
      isOneToOne: false
      referencedRelation: "services"
      referencedColumns: ["id"]
    },
  ]
}
project_tasks: {
  Row: {
    assigned_to: string | null
    completed_at: string | null
    created_at: string
    due_date: string | null
    id: string
    is_completed: boolean
    priority: string
    project_id: string
    requires_client_review: boolean
    sort_order: number
    stage: string
    title: string
    updated_at: string
  }
  Insert: {
    assigned_to?: string | null
    completed_at?: string | null
    created_at?: string
    due_date?: string | null
    id?: string
    is_completed?: boolean
    priority?: string
    project_id: string
    requires_client_review?: boolean
    sort_order?: number
    stage?: string
    title: string
    updated_at?: string
  }
  Update: {
    assigned_to?: string | null
    completed_at?: string | null
    created_at?: string
    due_date?: string | null
    id?: string
    is_completed?: boolean
    priority?: string
    project_id?: string
    requires_client_review?: boolean
    sort_order?: number
    stage?: string
    title?: string
    updated_at?: string
  }
  Relationships: [
    {
      foreignKeyName: "project_tasks_project_id_fkey"
      columns: ["project_id"]
      isOneToOne: false
      referencedRelation: "projects"
      referencedColumns: ["id"]
    },
  ]
}
project_timeline: {
  Row: {
    created_at: string
    event_date: string | null
    event_type: string
    id: string
    notes: string | null
    project_id: string
    revision_number: number
  }
```

```
    Insert: {
      created_at?: string
      event_date?: string | null
      event_type: string
      id?: string
      notes?: string | null
      project_id: string
      revision_number?: number
    }
    Update: {
      created_at?: string
      event_date?: string | null
      event_type?: string
      id?: string
      notes?: string | null
      project_id?: string
      revision_number?: number
    }
    Relationships: [
      {
        foreignKeyName: "project_timeline_project_id_fkey"
        columns: ["project_id"]
        isOneToOne: false
        referencedRelation: "projects"
        referencedColumns: ["id"]
      },
    ]
  }
  projects: {
    Row: {
      address: string | null
      city: string | null
      company_id: string
      country: string | null
      cover_image: string | null
      created_at: string
      currency: string
      date_notes: Json | null
      delivery_date: string | null
      description: string | null
      display_address: string | null
      entry_payment_percent: number
      final_delivery_date: string | null
      homeowner_address: string | null
      homeowner_email: string | null
      homeowner_name: string | null
      homeowner_phone: string | null
      id: string
      internal_notes: string | null
      is_portfolio: boolean
      latitude: number | null
      longitude: number | null
      name: string | null
      neighborhood: string | null
      package_id: string | null
      payment_effective_date: string | null
      payment_stage: string
      preview_delivery_date: string | null
      preview_sent_date: string | null
      project_number: string | null
      revision_count: number
      revision_return_date: string | null
      service_type: string
      square_feet: number | null
      stage: string | null
      start_date: string | null
      state: string | null
      status: Database["public"]["Enums"]["project_status"]
      street_name: string | null
      street_number: string | null
      total_value: number | null
      unit: string | null
      updated_at: string
      zip_code: string | null
    }
    Insert: {
      address?: string | null
      city?: string | null
      company_id: string
      country?: string | null
      cover_image?: string | null
      created_at?: string
      currency?: string
      date_notes?: Json | null
      delivery_date?: string | null
      description?: string | null
      display_address?: string | null
```

```
          entry_payment_percent?: number
          final_delivery_date?: string | null
          homeowner_address?: string | null
          homeowner_email?: string | null
          homeowner_name?: string | null
          homeowner_phone?: string | null
          id?: string
          internal_notes?: string | null
          is_portfolio?: boolean
          latitude?: number | null
          longitude?: number | null
          name?: string | null
          neighborhood?: string | null
          package_id?: string | null
          payment_effective_date?: string | null
          payment_stage?: string
          preview_delivery_date?: string | null
          preview_sent_date?: string | null
          project_number?: string | null
          revision_count?: number
          revision_return_date?: string | null
          service_type?: string
          square_feet?: number | null
          stage

... [TRUNCATED - file too large] ...
```

src/main.tsx

```
import { createRoot } from "react-dom/client";
import App from "./App.tsx";
import "./index.css";

createRoot(document.getElementById("root")!).render(&lt;App /&gt;);
```

src/pages/admin/AdminAICodeCheck.tsx

```
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { BrainCircuit } from "lucide-react";

const AdminAICodeCheck = () =&gt; {
  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;AI Code Check&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Upload drawings for automated compliance analysis&lt;/p&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="flex flex-col items-center justify-center py-16"&gt;
          &lt;BrainCircuit className="h-12 w-12 text-muted-foreground/30 mb-4" /&gt;
          &lt;h3 className="text-lg font-medium text-foreground mb-2"&gt;AI Drawing Analysis&lt;/h3&gt;
          &lt;p className="text-sm text-muted-foreground text-center max-w-md"&gt;
            Upload architectural drawings for automated building code compliance checks. AI will analyze setbacks, egress, accessibility, and str
          &lt;/p&gt;
          &lt;Badge variant="secondary" className="mt-4"&gt;Coming Soon&lt;/Badge&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminAICodeCheck;
```

src/pages/admin/AdminActivity.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Badge } from "@/components/ui/badge";
import { Plus, CreditCard, Package, RotateCcw, StickyNote } from "lucide-react";
import { toast } from "sonner";
import { format } from "date-fns";

const LOG_TYPES = [
  { value: "payment", label: "Pagamento", icon: CreditCard },
  { value: "delivery", label: "Entrega", icon: Package },
  { value: "revision", label: "Revisão", icon: RotateCcw },
  { value: "internal_note", label: "Nota Interna", icon: StickyNote },
];

const AdminActivity = () =&gt; {
  const [logs, setLogs] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [form, setForm] = useState({ project_id: "", log_type: "internal_note", description: "" });

  const fetchData = async () =&gt; {
    const [logRes, projRes] = await Promise.all([
      supabase.from("activity_log").select("*, projects(name)").order("log_date", { ascending: false }).limit(100),
      supabase.from("projects").select("id, name").order("name"),
    ]);
    setLogs(logRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  const handleSave = async () =&gt; {
    if (!form.project_id || !form.description.trim()) { toast.error("Projeto e descrição são obrigatórios"); return; }
    const { error } = await supabase.from("activity_log").insert({
      project_id: form.project_id, log_type: form.log_type, description: form.description,
    });
    if (error) { toast.error(error.message); return; }
    toast.success("Registro adicionado");
    setDialogOpen(false); setForm({ project_id: "", log_type: "internal_note", description: "" }); fetchData();
  };

  const getTypeInfo = (type: string) =&gt; LOG_TYPES.find(t =&gt; t.value === type) || LOG_TYPES[3];

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Log de Atividades&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Histórico de ações dos projetos&lt;/p&gt;
        &lt;/div&gt;
        &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
          &lt;DialogTrigger asChild&gt;&lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;Novo Registro&lt;/Button&gt;&lt;/DialogTrigger&gt;
          &lt;DialogContent&gt;
            &lt;DialogHeader&gt;&lt;DialogTitle&gt;Novo Registro&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
            &lt;div className="space-y-4"&gt;
              &lt;div&gt;&lt;Label&gt;Projeto *&lt;/Label&gt;
                &lt;Select value={form.project_id} onValueChange={v =&gt; setForm({ ...form, project_id: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Selecione" /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{projects.map(p =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{p.name}&lt;/SelectItem&gt;)}&lt;/SelectCo
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Tipo&lt;/Label&gt;
                &lt;Select value={form.log_type} onValueChange={v =&gt; setForm({ ...form, log_type: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{LOG_TYPES.map(t =&gt; &lt;SelectItem key={t.value} value={t.value}&gt;{t.label}&lt;/SelectItem&gt;)}&lt;/
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Descrição *&lt;/Label&gt;&lt;Textarea value={form.description} onChange={e =&gt; setForm({ ...form, descrip
              &lt;Button className="w-full" onClick={handleSave}&gt;Registrar&lt;/Button&gt;
            &lt;/div&gt;
          &lt;/DialogContent&gt;
        &lt;/Dialog&gt;
      &lt;/div&gt;

      &lt;div className="space-y-3"&gt;
        {logs.map(log =&gt; {
          const typeInfo = getTypeInfo(log.log_type);
```

```
          const Icon = typeInfo.icon;
          return (
            &lt;Card key={log.id}&gt;
              &lt;CardContent className="flex items-start gap-4 py-4"&gt;
                &lt;div className="p-2 rounded-lg bg-muted"&gt;
                  &lt;Icon className="h-4 w-4 text-muted-foreground" /&gt;
                &lt;/div&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;div className="flex items-center gap-2 mb-1"&gt;
                    &lt;span className="font-medium text-foreground text-sm"&gt;{log.projects?.name}&lt;/span&gt;
                    &lt;Badge variant="outline" className="text-xs"&gt;{typeInfo.label}&lt;/Badge&gt;
                  &lt;/div&gt;
                  &lt;p className="text-sm text-muted-foreground"&gt;{log.description}&lt;/p&gt;
                &lt;/div&gt;
                &lt;span className="text-xs text-muted-foreground whitespace-nowrap"&gt;
                  {format(new Date(log.log_date), "dd/MM/yyyy")}
                &lt;/span&gt;
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          );
        })}
        {logs.length === 0 &amp;amp;&amp;amp; (
          &lt;div className="text-center text-muted-foreground py-12"&gt;Nenhum registro de atividade&lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminActivity;
```

src/pages/admin/AdminAnalytics.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { BarChart, Bar, LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer, PieChart, Pie, Cell, Legend } from "recharts"
import { BarChart3 } from "lucide-react";

const months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"];
const COLORS = ["hsl(var(--chart-1))", "hsl(var(--chart-2))", "hsl(var(--chart-3))", "hsl(var(--chart-4))", "hsl(var(--chart-5))"];
const tooltipStyle = { backgroundColor: "hsl(var(--card))", border: "1px solid hsl(var(--border))", borderRadius: "8px", fontSize: 12 };

const AdminAnalytics = () =&gt; {
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [payments, setPayments] = useState&lt;any[]&gt;([]);

  useEffect(() =&gt; {
    Promise.all([
      supabase.from("projects").select("*"),
      supabase.from("admin_payments").select("*"),
    ]).then(([p, pay]) =&gt; {
      setProjects(p.data || []);
      setPayments(pay.data || []);
    });
  }, []);

  // Revenue by month
  const revenueByMonth: Record&lt;string, number&gt; = {};
  payments.filter(p =&gt; p.status === "paid").forEach(p =&gt; {
    const m = months[new Date(p.payment_date || p.created_at).getMonth()];
    revenueByMonth[m] = (revenueByMonth[m] || 0) + Number(p.amount_paid || 0);
  });
  const revenueData = months.filter(m =&gt; revenueByMonth[m]).map(m =&gt; ({ month: m, revenue: revenueByMonth[m] }));

  // Projects by stage
  const stageCount: Record&lt;string, number&gt; = {};
  projects.forEach(p =&gt; { stageCount[p.stage || "Unknown"] = (stageCount[p.stage || "Unknown"] || 0) + 1; });
  const stageData = Object.entries(stageCount).map(([name, value]) =&gt; ({ name, value }));

  // Summary
  const totalRevenue = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const totalPipeline = projects.filter(p =&gt; p.stage !== "Completed").reduce((s, p) =&gt; s + Number(p.total_value || 0), 0);

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Analytics&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Financial and project performance analytics&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;${totalRevenue.toLocaleString()}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Total Revenue&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;${totalPipeline.toLocaleString()}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Pipeline Value&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 text-center"&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{projects.length}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Total Projects&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;

      &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-3"&gt;&lt;CardTitle className="text-base"&gt;Revenue by Month&lt;/CardTitle&gt;&lt;/CardHeader&gt;
          &lt;CardContent&gt;
            {revenueData.length === 0 ? (
              &lt;div className="flex flex-col items-center py-12"&gt;&lt;BarChart3 className="h-8 w-8 text-muted-foreground/30 mb-2" /&gt;&lt;p
            ) : (
              &lt;ResponsiveContainer width="100%" height={250}&gt;
                &lt;BarChart data={revenueData}&gt;
                  &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
                  &lt;XAxis dataKey="month" tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                  &lt;YAxis tick={{ fontSize: 11 }} stroke="hsl(var(--muted-foreground))" /&gt;
                  &lt;Tooltip contentStyle={tooltipStyle} /&gt;
                  &lt;Bar dataKey="revenue" fill="hsl(var(--chart-2))" radius={[4, 4, 0, 0]} /&gt;
                &lt;/BarChart&gt;
```

```
              &lt;/ResponsiveContainer&gt;
            )}
          &lt;/CardContent&gt;
        &lt;/Card&gt;

        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardHeader className="pb-3"&gt;&lt;CardTitle className="text-base"&gt;Projects by Stage&lt;/CardTitle&gt;&lt;/CardHeader&gt;
          &lt;CardContent&gt;
            {stageData.length === 0 ? (
              &lt;div className="flex flex-col items-center py-12"&gt;&lt;BarChart3 className="h-8 w-8 text-muted-foreground/30 mb-2" /&gt;&lt;p
            ) : (
              &lt;ResponsiveContainer width="100%" height={250}&gt;
                &lt;PieChart&gt;
                  &lt;Pie data={stageData} dataKey="value" nameKey="name" cx="50%" cy="50%" outerRadius={90} label={(e) =&gt; e.name}&gt;
                    {stageData.map((_, i) =&gt; &lt;Cell key={i} fill={COLORS[i % COLORS.length]} /&gt;)}
                  &lt;/Pie&gt;
                  &lt;Tooltip contentStyle={tooltipStyle} /&gt;
                  &lt;Legend wrapperStyle={{ fontSize: 11 }} /&gt;
                &lt;/PieChart&gt;
              &lt;/ResponsiveContainer&gt;
            )}
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminAnalytics;
```

src/pages/admin/AdminClientPortal.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Globe, Users, Ticket, Activity } from "lucide-react";
import { format } from "date-fns";

const AdminClientPortal = () =&gt; {
  const [tickets, setTickets] = useState&lt;any[]&gt;([]);
  const [profiles, setProfiles] = useState&lt;any[]&gt;([]);

  useEffect(() =&gt; {
    supabase.from("client_tickets").select("*, companies(name), projects(name)").order("updated_at", { ascending: false }).limit(20)
      .then(({ data }) =&gt; setTickets(data || []));
    supabase.from("profiles").select("*, companies:company_id(name)").order("created_at", { ascending: false }).limit(20)
      .then(({ data }) =&gt; setProfiles(data || []));
  }, []);

  const statusColor: Record&lt;string, string&gt; = {
    open: "bg-blue-500/10 text-blue-600", waiting: "bg-amber-500/10 text-amber-600", resolved: "bg-emerald-500/10 text-emerald-600",
  };

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Client Portal&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Monitor client activity and portal access&lt;/p&gt;
      &lt;/div&gt;

      &lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="p-2 rounded-lg bg-blue-50 dark:bg-blue-950/40"&gt;&lt;Users className="h-5 w-5 text-blue-600" /&gt;&lt;/div&gt;
            &lt;div&gt;&lt;p className="text-2xl font-bold"&gt;{profiles.length}&lt;/p&gt;&lt;p className="text-xs text-muted-foreground"&gt;Port
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="p-2 rounded-lg bg-amber-50 dark:bg-amber-950/40"&gt;&lt;Ticket className="h-5 w-5 text-amber-600" /&gt;&lt;/div&gt
            &lt;div&gt;&lt;p className="text-2xl font-bold"&gt;{tickets.filter(t =&gt; t.status === "open").length}&lt;/p&gt;&lt;p className="tex
          &lt;/CardContent&gt;
        &lt;/Card&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4 flex items-center gap-3"&gt;
            &lt;div className="p-2 rounded-lg bg-emerald-50 dark:bg-emerald-950/40"&gt;&lt;Activity className="h-5 w-5 text-emerald-600" /&gt;&lt
            &lt;div&gt;&lt;p className="text-2xl font-bold"&gt;{tickets.filter(t =&gt; t.status === "resolved").length}&lt;/p&gt;&lt;p className=
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;

      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;&lt;CardTitle className="text-base"&gt;Recent Tickets&lt;/CardTitle&gt;&lt;/CardHeader&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Subject&lt;/TableHead&gt;
                &lt;TableHead&gt;Client&lt;/TableHead&gt;
                &lt;TableHead&gt;Project&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Updated&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {tickets.length === 0 &amp;amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={5} className="text-center text-muted-foreground py-12"&gt;&lt;Globe className="h-8 w-8 mx-
              )}
              {tickets.map(t =&gt; (
                &lt;TableRow key={t.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{t.subject}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{t.companies?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{t.projects?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;&lt;Badge variant="outline" className={statusColor[t.status] || ""}&gt;{t.status}&lt;/Badge&gt;&lt;/TableCell&
                  &lt;TableCell className="text-xs text-muted-foreground"&gt;{format(new Date(t.updated_at), "MMM d, yyyy")}&lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};
```

```
export default AdminClientPortal;
```

src/pages/admin/AdminClientProfile.tsx

```
import { useEffect, useState } from "react";
import { useParams, useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { ArrowLeft, Building2, Globe, Languages, Mail, Phone, FolderKanban, FileText, MessageSquare, FolderOpen } from "lucide-react";
import { getCountryFlag, getLanguageLabel } from "@/utils/countryFlags";
import { format } from "date-fns";

const AdminClientProfile = () =&gt; {
  const { id } = useParams&lt;{ id: string }&gt;();
  const navigate = useNavigate();
  const [company, setCompany] = useState&lt;any&gt;(null);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [messages, setMessages] = useState&lt;any[]&gt;([]);
  const [employees, setEmployees] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    if (!id) return;
    const fetch = async () =&gt; {
      const [compRes, projRes, empRes] = await Promise.all([
        supabase.from("companies").select("*").eq("id", id).single(),
        supabase.from("projects").select("*").eq("company_id", id).order("created_at", { ascending: false }),
        supabase.from("company_employees").select("*").eq("company_id", id),
      ]);
      setCompany(compRes.data);
      setProjects(projRes.data || []);
      setEmployees(empRes.data || []);

      // Fetch files &amp; messages for all projects
      const projectIds = (projRes.data || []).map((p: any) =&gt; p.id);
      if (projectIds.length &gt; 0) {
        const [filesRes, msgsRes] = await Promise.all([
          supabase.from("project_files").select("*, projects(name)").in("project_id", projectIds).order("created_at", { ascending: false }),
          supabase.from("messages").select("*, projects(name)").in("project_id", projectIds).order("created_at", { ascending: false }).limit(50),
        ]);
        setFiles(filesRes.data || []);
        setMessages(msgsRes.data || []);
      }
      setLoading(false);
    };
    fetch();
  }, [id]);

  if (loading) return &lt;div className="p-6 text-muted-foreground"&gt;Loading...&lt;/div&gt;;
  if (!company) return &lt;div className="p-6 text-muted-foreground"&gt;Client not found&lt;/div&gt;;

  const initials = (company.owner_name || company.name || "C")
    .split(" ").map((w: string) =&gt; w[0]).join("").toUpperCase().slice(0, 2);

  return (
    &lt;div className="p-6 space-y-6 max-w-[1200px] mx-auto"&gt;
      {/* Back */}
      &lt;Button variant="ghost" size="sm" onClick={() =&gt; navigate("/adm/companies")}&gt;
        &lt;ArrowLeft className="h-4 w-4 mr-1" /&gt; Back to Clients
      &lt;/Button&gt;

      {/* Header */}
      &lt;div className="flex items-start gap-4"&gt;
        &lt;Avatar className="h-16 w-16"&gt;
          {company.avatar_url &amp;&amp; &lt;AvatarImage src={company.avatar_url} /&gt;}
          &lt;AvatarFallback className="text-lg bg-primary/10 text-primary"&gt;{initials}&lt;/AvatarFallback&gt;
        &lt;/Avatar&gt;
        &lt;div className="flex-1"&gt;
          &lt;h1 className="text-xl font-semibold text-foreground flex items-center gap-2"&gt;
            {company.owner_name || company.name} {getCountryFlag(company.country)}
          &lt;/h1&gt;
          {company.owner_name &amp;&amp; company.name &amp;&amp; (
            &lt;p className="text-sm text-muted-foreground flex items-center gap-1"&gt;
              &lt;Building2 className="h-3.5 w-3.5" /&gt; {company.name}
            &lt;/p&gt;
          )}
          &lt;div className="flex flex-wrap gap-3 mt-2 text-xs text-muted-foreground"&gt;
            {company.language &amp;&amp; (
              &lt;span className="flex items-center gap-1"&gt;
                &lt;Languages className="h-3.5 w-3.5" /&gt; {getLanguageLabel(company.language)}
              &lt;/span&gt;
            )}
```

```
      {company.country &amp;&amp; (
        &lt;span className="flex items-center gap-1"&gt;
          &lt;Globe className="h-3.5 w-3.5" /&gt; {company.country}
        &lt;/span&gt;
      )}
      {company.email &amp;&amp; (
        &lt;span className="flex items-center gap-1"&gt;
          &lt;Mail className="h-3.5 w-3.5" /&gt; {company.email}
        &lt;/span&gt;
      )}
      {company.phone &amp;&amp; (
        &lt;span className="flex items-center gap-1"&gt;
          &lt;Phone className="h-3.5 w-3.5" /&gt; {company.phone}
        &lt;/span&gt;
      )}
    &lt;/div&gt;
    &lt;div className="flex gap-2 mt-2"&gt;
      &lt;Badge variant={company.status === "active" ? "default" : "secondary"}&gt;
        {company.status || "active"}
      &lt;/Badge&gt;
      {company.company_type &amp;&amp; &lt;Badge variant="outline"&gt;{company.company_type}&lt;/Badge&gt;}
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/div&gt;

{/* Tabs */}
&lt;Tabs defaultValue="projects"&gt;
  &lt;TabsList&gt;
    &lt;TabsTrigger value="projects" className="gap-1"&gt;&lt;FolderKanban className="h-3.5 w-3.5" /&gt; Projects ({projects.length})&lt;/T
    &lt;TabsTrigger value="contacts" className="gap-1"&gt;&lt;Building2 className="h-3.5 w-3.5" /&gt; Contacts ({employees.length})&lt;/Tab
    &lt;TabsTrigger value="files" className="gap-1"&gt;&lt;FolderOpen className="h-3.5 w-3.5" /&gt; Files ({files.length})&lt;/TabsTrigger&
    &lt;TabsTrigger value="messages" className="gap-1"&gt;&lt;MessageSquare className="h-3.5 w-3.5" /&gt; Messages ({messages.length})&lt;/
  &lt;/TabsList&gt;

  &lt;TabsContent value="projects"&gt;
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead&gt;Project&lt;/TableHead&gt;
              &lt;TableHead&gt;Stage&lt;/TableHead&gt;
              &lt;TableHead&gt;Service&lt;/TableHead&gt;
              &lt;TableHead&gt;Value&lt;/TableHead&gt;
              &lt;TableHead&gt;Created&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {projects.length === 0 &amp;&amp; (
              &lt;TableRow&gt;&lt;TableCell colSpan={5} className="text-center text-muted-foreground py-8"&gt;No projects&lt;/TableCell&gt;
            )}
            {projects.map((p) =&gt; (
              &lt;TableRow key={p.id} className="cursor-pointer hover:bg-muted/50" onClick={() =&gt; navigate(`/adm/projects/${p.id}`)}&gt;
                &lt;TableCell className="font-medium"&gt;{p.name || p.display_address || "Untitled"}&lt;/TableCell&gt;
                &lt;TableCell&gt;&lt;Badge variant="outline"&gt;{p.stage}&lt;/Badge&gt;&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{p.service_type}&lt;/TableCell&gt;
                &lt;TableCell&gt;{p.currency} {Number(p.total_value || 0).toLocaleString()}&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{format(new Date(p.created_at), "MMM d, yyyy")}&lt;/TableCell&gt
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/TabsContent&gt;

  &lt;TabsContent value="contacts"&gt;
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead&gt;Name&lt;/TableHead&gt;
              &lt;TableHead&gt;Email&lt;/TableHead&gt;
              &lt;TableHead&gt;Phone&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {employees.length === 0 &amp;&amp; (
              &lt;TableRow&gt;&lt;TableCell colSpan={3} className="text-center text-muted-foreground py-8"&gt;No contacts&lt;/TableCell&gt;
            )}
            {employees.map((e) =&gt; (
              &lt;TableRow key={e.id}&gt;
                &lt;TableCell className="font-medium"&gt;{e.name}&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{e.email || "—"}&lt;/TableCell&gt;
                &lt;TableCell className="text-sm text-muted-foreground"&gt;{e.phone || "—"}&lt;/TableCell&gt;
              &lt;/TableRow&gt;
```

```
                ))}
              &lt;/TableBody&gt;
            &lt;/Table&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/TabsContent&gt;

      &lt;TabsContent value="files"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-0"&gt;
            &lt;Table&gt;
              &lt;TableHeader&gt;
                &lt;TableRow&gt;
                  &lt;TableHead&gt;File&lt;/TableHead&gt;
                  &lt;TableHead&gt;Project&lt;/TableHead&gt;
                  &lt;TableHead&gt;Date&lt;/TableHead&gt;
                &lt;/TableRow&gt;
              &lt;/TableHeader&gt;
              &lt;TableBody&gt;
                {files.length === 0 &amp;&amp; (
                  &lt;TableRow&gt;&lt;TableCell colSpan={3} className="text-center text-muted-foreground py-8"&gt;No files&lt;/TableCell&gt;&lt
                )}
                {files.map((f) =&gt; (
                  &lt;TableRow key={f.id}&gt;
                    &lt;TableCell&gt;
                      &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer" className="text-primary hover:underline text-sm font-me
                        {f.file_name}
                      &lt;/a&gt;
                    &lt;/TableCell&gt;
                    &lt;TableCell className="text-sm text-muted-foreground"&gt;{f.projects?.name || "—"}&lt;/TableCell&gt;
                    &lt;TableCell className="text-sm text-muted-foreground"&gt;{format(new Date(f.created_at), "MMM d, yyyy")}&lt;/TableCell&gt
                  &lt;/TableRow&gt;
                ))}
              &lt;/TableBody&gt;
            &lt;/Table&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/TabsContent&gt;

      &lt;TabsContent value="messages"&gt;
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="p-4"&gt;
            {messages.length === 0 &amp;&amp; (
              &lt;p className="text-center text-muted-foreground py-8"&gt;No messages&lt;/p&gt;
            )}
            &lt;div className="space-y-3 max-h-[500px] overflow-y-auto"&gt;
              {messages.map((m) =&gt; (
                &lt;div key={m.id} className="flex gap-3 p-2 rounded-md hover:bg-muted/50"&gt;
                  &lt;div className="flex-1"&gt;
                    &lt;p className="text-sm"&gt;{m.content}&lt;/p&gt;
                    &lt;p className="text-xs text-muted-foreground mt-1"&gt;
                      {m.projects?.name} · {format(new Date(m.created_at), "MMM d, h:mm a")}
                      {m.is_from_admin &amp;&amp; &lt;Badge variant="outline" className="ml-2 text-[9px]"&gt;Admin&lt;/Badge&gt;}
                    &lt;/p&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              ))}
            &lt;/div&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/TabsContent&gt;
    &lt;/Tabs&gt;
  &lt;/div&gt;
  );
};

export default AdminClientProfile;
```

src/pages/admin/AdminCompanies.tsx

```
import { useEffect, useState, useRef } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Plus, Pencil, Trash2, Search, Camera, Building2, Users, UserPlus } from "lucide-react";
import { toast } from "sonner";
import { Separator } from "@/components/ui/separator";
import CompanyEmployees from "@/components/admin/CompanyEmployees";
import { getCountryFlag, getLanguageLabel } from "@/utils/countryFlags";

interface Company {
  id: string;
  name: string;
  owner_name: string | null;
  email: string | null;
  personal_email: string | null;
  phone: string | null;
  company_type: string | null;
  country: string | null;
  status: string | null;
  website: string | null;
  instagram: string | null;
  personal_instagram: string | null;
  linkedin: string | null;
  street_name: string | null;
  street_number: string | null;
  unit: string | null;
  neighborhood: string | null;
  city: string | null;
  state: string | null;
  zip_code: string | null;
  avatar_url: string | null;
  language: string | null;
}

const TYPES = ["Construtora", "Cliente Final", "Real Estate", "Builder", "Contractor", "Client", "Architect", "Engineering"];
const COUNTRIES = ["USA", "Brazil", "Other"];
const STATUSES = ["active", "inactive"];

const LANGUAGES = [
  { value: "en", label: "English ■■" },
  { value: "pt", label: "Portuguese ■■" },
];

const emptyForm = {
  name: "", owner_name: "", email: "", personal_email: "", phone: "",
  company_type: "Client", country: "USA", status: "active", language: "en",
  website: "", instagram: "", personal_instagram: "", linkedin: "",
  street_name: "", street_number: "", unit: "", neighborhood: "",
  city: "", state: "", zip_code: "", suggested_password: "",
};

const AdminCompanies = () =&gt; {
  const [companies, setCompanies] = useState&lt;Company[]&gt;([]);
  const [search, setSearch] = useState("");
  const [filterStatus, setFilterStatus] = useState&lt;string&gt;("all");
  const [filterCountry, setFilterCountry] = useState&lt;string&gt;("all");
  const [groupBy, setGroupBy] = useState&lt;"none" | "country" | "company_type" | "city"&gt;("none");
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editing, setEditing] = useState&lt;Company | null&gt;(null);
  const [form, setForm] = useState(emptyForm);
  const [avatarPreview, setAvatarPreview] = useState&lt;string | null&gt;(null);
  const [avatarFile, setAvatarFile] = useState&lt;File | null&gt;(null);
  const fileInputRef = useRef&lt;HTMLInputElement&gt;(null);
  const [employeesDialogOpen, setEmployeesDialogOpen] = useState(false);
  const [selectedCompany, setSelectedCompany] = useState&lt;Company | null&gt;(null);

  const fetchCompanies = async () =&gt; {
    const { data } = await supabase.from("companies").select("*").order("name");
    setCompanies((data as Company[]) || []);
  };

  useEffect(() =&gt; { fetchCompanies(); }, []);

  const uploadAvatar = async (companyId: string): Promise&lt;string | null&gt; =&gt; {
    if (!avatarFile) return editing?.avatar_url || null;
    const ext = avatarFile.name.split(".").pop();
```

```
  const path = `${companyId}.${ext}`;
  const { error } = await supabase.storage.from("company-avatars").upload(path, avatarFile, { upsert: true });
  if (error) { toast.error("Erro ao enviar imagem"); return null; }
  const { data } = supabase.storage.from("company-avatars").getPublicUrl(path);
  return `${data.publicUrl}?t=${Date.now()}`;
};

const handleAvatarChange = (e: React.ChangeEvent&lt;HTMLInputElement&gt;) =&gt; {
  const file = e.target.files?.[0];
  if (!file) return;
  setAvatarFile(file);
  setAvatarPreview(URL.createObjectURL(file));
};

const handleSave = async () =&gt; {
  const payload: any = {
    name: form.name || null, owner_name: form.owner_name || null,
    email: form.email || null, personal_email: form.personal_email || null,
    phone: form.phone || null,
    company_type: form.company_type, country: form.country, status: form.status, language: form.language,
    website: form.website || null,
    instagram: form.instagram || null, personal_instagram: form.personal_instagram || null,
    linkedin: form.linkedin || null,
    street_name: form.street_name || null, street_number: form.street_number || null,
    unit: form.unit || null, neighborhood: form.neighborhood || null,
    city: form.city || null, state: form.state || null, zip_code: form.zip_code || null,
  };

  let companyId = editing?.id;

  if (editing) {
    const avatarUrl = await uploadAvatar(editing.id);
    if (avatarUrl) payload.avatar_url = avatarUrl;
    const { error } = await supabase.from("companies").update(payload).eq("id", editing.id);
    if (error) { toast.error(error.message); return; }
    companyId = editing.id;
    toast.success("Company updated");
  } else {
    const { data, error } = await supabase.from("companies").insert({ ...payload, name: payload.name || "No name" }).select("id").single();
    if (error) { toast.error(error.message); return; }
    companyId = data.id;
    if (avatarFile) {
      const avatarUrl = await uploadAvatar(companyId);
      if (avatarUrl) {
        await supabase.from("companies").update({ avatar_url: avatarUrl }).eq("id", companyId);
      }
    }
    toast.success("Company created");
  }

  // Create client login account if password is provided
  if (form.suggested_password &amp;amp;&amp;amp; form.email &amp;amp;&amp;amp; companyId) {
    try {
      const { data: fnData, error: fnError } = await supabase.functions.invoke("create-client-account", {
        body: {
          email: form.email,
          password: form.suggested_password,
          full_name: form.owner_name || form.name,
          company_id: companyId,
        },
      });
      if (fnError) {
        toast.error(`Account error: ${fnError.message}`);
      } else if (fnData?.error) {
        toast.error(`Account error: ${fnData.error}`);
      } else {
        toast.success(`Client login created: ${form.email}`);
      }
    } catch (err: any) {
      toast.error(`Failed to create client login: ${err.message}`);
    }
  }

  setDialogOpen(false); setEditing(null); setForm(emptyForm);
  setAvatarFile(null); setAvatarPreview(null);
  fetchCompanies();
};

const handleDelete = async (id: string) =&gt; {
  if (!confirm("Tem certeza que deseja excluir?")) return;
  const { error } = await supabase.from("companies").delete().eq("id", id);
  if (error) { toast.error(error.message); return; }
  toast.success("Empresa excluída"); fetchCompanies();
};

const openEdit = (c: Company) =&gt; {
  setEditing(c);
```

```
  setAvatarPreview(c.avatar_url || null);
  setAvatarFile(null);
  setForm({
    name: c.name || "", owner_name: c.owner_name || "", email: c.email || "",
    personal_email: c.personal_email || "",
    phone: c.phone || "",
    company_type: c.company_type || "Client", country: c.country || "USA", status: c.status || "active", language: (c as any).language || "en",
    website: c.website || "",
    instagram: c.instagram || "", personal_instagram: c.personal_instagram || "",
    linkedin: c.linkedin || "",
    street_name: c.street_name || "", street_number: c.street_number || "", unit: c.unit || "",
    neighborhood: c.neighborhood || "", city: c.city || "", state: c.state || "", zip_code: c.zip_code || "",
    suggested_password: "",
  });
  setDialogOpen(true);
};

const openNew = () =&gt; {
  setEditing(null); setForm(emptyForm);
  setAvatarFile(null); setAvatarPreview(null);
  setDialogOpen(true);
};

const openEmployees = (c: Company) =&gt; {
  setSelectedCompany(c);
  setEmployeesDialogOpen(true);
};




const filtered = companies.filter(c =&gt; {
  const matchSearch = (c.name || "").toLowerCase().includes(search.toLowerCase()) ||
    (c.owner_name || "").toLowerCase().includes(search.toLowerCase()) ||
    (c.city || "").toLowerCase().includes(search.toLowerCase());
  const matchStatus = filterStatus === "all" || c.status === filterStatus;
  const matchCountry = filterCountry === "all" || c.country === filterCountry;
  return matchSearch &amp;&amp; matchStatus &amp;&amp; matchCountry;
});

const grouped = () =&gt; {
  if (groupBy === "none") return { "": filtered };
  const map: Record&lt;string, Company[]&gt; = {};
  filtered.forEach(c =&gt; {
    const key = (c as any)[groupBy] || "Sem definição";
    if (!map[key]) map[key] = [];
    map[key].push(c);
  });
  return Object.fromEntries(Object.entries(map).sort(([a], [b]) =&gt; a.localeCompare(b)));
};

const getInitials = (name: string) =&gt; {
  return name.split(" ").map(w =&gt; w[0]).join("").toUpperCase().slice(0, 2);
};

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Empresas&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Gerencie seus clientes e parceiros&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={(o) =&gt; { setDialogOpen(o); if (!o) { setEditing(null); setForm(emptyForm); setAvatarFile(nu
        &lt;DialogTrigger asChild&gt;
          &lt;Button onClick={openNew}&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;Nova Empresa&lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent className="max-w-lg max-h-[85vh] overflow-y-auto"&gt;
          &lt;DialogHeader&gt;
            &lt;DialogTitle&gt;{editing ? "Editar Empresa" : "Nova Empresa"}&lt;/DialogTitle&gt;
          &lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            {/* Avatar Upload */}
            &lt;div className="flex flex-col items-center gap-2"&gt;
              &lt;div className="relative cursor-pointer" onClick={() =&gt; fileInputRef.current?.click()}&gt;
                &lt;Avatar className="h-20 w-20 border-2 border-border"&gt;
                  &lt;AvatarImage src={avatarPreview || undefined} /&gt;
                  &lt;AvatarFallback className="bg-muted text-muted-foreground text-lg"&gt;
                    {form.name ? getInitials(form.name) : &lt;Building2 className="h-8 w-8" /&gt;}
                  &lt;/AvatarFallback&gt;
                &lt;/Avatar&gt;
                &lt;div className="absolute bottom-0 right-0 bg-primary text-primary-foreground rounded-full p-1"&gt;
                  &lt;Camera className="h-3 w-3" /&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;p className="text-xs text-muted-foreground"&gt;Clique para adicionar foto&lt;/p&gt;
              &lt;input ref={fileInputRef} type="file" accept="image/*" className="hidden" onChange={handleAvatarChange} /&gt;
            &lt;/div&gt;
```

```
        {/* Section 1 — Responsável (FIRST) */}
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Responsável&lt;/p&gt;
        &lt;div&gt;&lt;Label&gt;Nome do Responsável&lt;/Label&gt;&lt;Input value={form.owner_name} onChange={e =&gt; setForm({ ...form, own
        &lt;div className="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Telefone&lt;/Label&gt;&lt;Input value={form.phone} onChange={e =&gt; setForm({ ...form, phone: e.target.v
          &lt;div&gt;&lt;Label&gt;Email Pessoal&lt;/Label&gt;&lt;Input type="email" value={form.personal_email} onChange={e =&gt; setForm({
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;Instagram Pessoal&lt;/Label&gt;&lt;Input value={form.personal_instagram} onChange={e =&gt; setForm({ ...for

        &lt;Separator /&gt;
        {/* Section 2 — Empresa (AFTER) */}
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Empresa&lt;/p&gt;
        &lt;div&gt;&lt;Label&gt;Nome da Empresa&lt;/Label&gt;&lt;Input value={form.name} onChange={e =&gt; setForm({ ...form, name: e.targe
        &lt;div className="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Tipo&lt;/Label&gt;
            &lt;Select value={form.company_type} onValueChange={v =&gt; setForm({ ...form, company_type: v })}&gt;
              &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{TYPES.map(t =&gt; &lt;SelectItem key={t} value={t}&gt;{t}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
          &lt;div&gt;&lt;Label&gt;Status&lt;/Label&gt;
            &lt;Select value={form.status} onValueChange={v =&gt; setForm({ ...form, status: v })}&gt;
              &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{STATUSES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{s === "active" ? "Ativo" : "Inativo"}&lt;/Sel
            &lt;/Select&gt;
          &lt;/div&gt;
          &lt;div&gt;&lt;Label&gt;Language&lt;/Label&gt;
            &lt;Select value={form.language} onValueChange={v =&gt; setForm({ ...form, language: v })}&gt;
              &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{LANGUAGES.map(l =&gt; &lt;SelectItem key={l.value} value={l.value}&gt;{l.label}&lt;/SelectItem&gt;)}&lt
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;Email Profissional&lt;/Label&gt;&lt;Input type="email" value={form.email} onChange={e =&gt; setForm({ ...fo
        &lt;div&gt;&lt;Label&gt;Instagram Profissional&lt;/Label&gt;&lt;Input value={form.instagram} onChange={e =&gt; setForm({ ...form, i
        &lt;div&gt;&lt;Label&gt;Website&lt;/Label&gt;&lt;Input value={form.website} onChange={e =&gt; setForm({ ...form, website: e.target.
        &lt;div&gt;&lt;Label&gt;LinkedIn&lt;/Label&gt;&lt;Input value={form.linkedin} onChange={e =&gt; setForm({ ...form, linkedin: e.targ

        &lt;Separator /&gt;
        {/* Section 3 — Client Area */}
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Área do Cliente&lt;/p&gt;
        &lt;div className="p-3 rounded-md bg-muted border border-border space-y-2"&gt;
          &lt;p className="text-xs text-muted-foreground"&gt;Login e senha para acesso à área do cliente&lt;/p&gt;
          &lt;div&gt;&lt;Label&gt;Login (Email)&lt;/Label&gt;&lt;Input value={form.email} disabled className="bg-background" /&gt;&lt;/div&
          &lt;div&gt;&lt;Label&gt;Senha Sugerida&lt;/Label&gt;
            &lt;div className="flex gap-2"&gt;
              &lt;Input value={form.suggested_password || ""} onChange={e =&gt; setForm({ ...form, suggested_password: e.target.value })} p
              &lt;Button type="button" variant="outline" size="sm" onClick={() =&gt; {
                const pwd = `Dara${Math.random().toString(36).slice(2, 8)}!`;
                setForm({ ...form, suggested_password: pwd });
              }}&gt;Gerar&lt;/Button&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        &lt;Separator /&gt;
        {/* Section 4 — Address */}
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Endereço&lt;/p&gt;
        &lt;div class="grid grid-cols-3 gap-4"&gt;
          &lt;div class="col-span-2"&gt;&lt;Label&gt;Rua&lt;/Label&gt;&lt;Input value={form.street_name} onChange={e =&gt; setForm({ ..
          &lt;div&gt;&lt;Label&gt;Número&lt;/Label&gt;&lt;Input value={form.street_number} onChange={e =&gt; setForm({ ...form, street_numb
        &lt;/div&gt;
        &lt;div class="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Complemento&lt;/Label&gt;&lt;Input value={form.unit} onChange={e =&gt; setForm({ ...form, unit: e.target.
          &lt;div&gt;&lt;Label&gt;Bairro&lt;/Label&gt;&lt;Input value={form.neighborhood} onChange={e =&gt; setForm({ ...form, neighborhood
        &lt;/div&gt;
        &lt;div class="grid grid-cols-3 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Cidade&lt;/Label&gt;&lt;Input value={form.city} onChange={e =&gt; setForm({ ...form, city: e.target.value
          &lt;div&gt;&lt;Label&gt;Estado&lt;/Label&gt;&lt;Input value={form.state} onChange={e =&gt; setForm({ ...form, state: e.target.val
          &lt;div&gt;&lt;Label&gt;CEP&lt;/Label&gt;&lt;Input value={form.zip_code} onChange={e =&gt; setForm({ ...form, zip_code: e.target.
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;País&lt;/Label&gt;
          &lt;Select value={form.country} onValueChange={v =&gt; setForm({ ...form, country: v })}&gt;
            &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;{COUNTRIES.map(c =&gt; &lt;SelectItem key={c} value={c}&gt;{c}&lt;/SelectItem&gt;)}&lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/div&gt;

        &lt;Button className="w-full" onClick={handleSave}&gt;{editing ? "Salvar" : "Criar"}&lt;/Button&gt;
      &lt;/div&gt;
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;

&lt;div className="flex flex-wrap gap-3"&gt;
  &lt;div className="relative max-w-xs"&gt;
    &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
```

```
    &lt;Input placeholder="Buscar empresa..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
  &lt;/div&gt;
  &lt;Select value={filterStatus} onValueChange={setFilterStatus}&gt;
    &lt;SelectTrigger className="w-[140px]"&gt;&lt;SelectValue placeholder="Status" /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="all"&gt;All Status&lt;/SelectItem&gt;
      &lt;SelectItem value="active"&gt;■ Active&lt;/SelectItem&gt;
      &lt;SelectItem value="inactive"&gt;■ Inactive&lt;/SelectItem&gt;
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
  &lt;Select value={filterCountry} onValueChange={setFilterCountry}&gt;
    &lt;SelectTrigger className="w-[140px]"&gt;&lt;SelectValue placeholder="Country" /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="all"&gt;All Countries&lt;/SelectItem&gt;
      &lt;SelectItem value="USA"&gt;■■ USA&lt;/SelectItem&gt;
      &lt;SelectItem value="Brazil"&gt;■■ Brazil&lt;/SelectItem&gt;
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
  &lt;Select value={groupBy} onValueChange={v =&gt; setGroupBy(v as any)}&gt;
    &lt;SelectTrigger className="w-[180px]"&gt;&lt;SelectValue placeholder="Agrupar por" /&gt;&lt;/SelectTrigger&gt;
    &lt;SelectContent&gt;
      &lt;SelectItem value="none"&gt;Sem agrupamento&lt;/SelectItem&gt;
      &lt;SelectItem value="country"&gt;País&lt;/SelectItem&gt;
      &lt;SelectItem value="company_type"&gt;Tipo&lt;/SelectItem&gt;
      &lt;SelectItem value="city"&gt;Cidade&lt;/SelectItem&gt;
    &lt;/SelectContent&gt;
  &lt;/Select&gt;
&lt;/div&gt;

{Object.entries(grouped()).map(([group, items]) =&gt; (
  &lt;Card key={group}&gt;
    {group &amp;&amp; (
      &lt;div className="px-6 pt-4 pb-2"&gt;
        &lt;h3 className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;{group}&lt;/h3&gt;
      &lt;/div&gt;
    )}
    &lt;CardContent className="p-0"&gt;
      &lt;Table&gt;
        &lt;TableHeader&gt;
          &lt;TableRow&gt;
            &lt;TableHead&gt;Responsável / Empresa&lt;/TableHead&gt;
            &lt;TableHead&gt;Tipo&lt;/TableHead&gt;
            &lt;TableHead&gt;Cidade&lt;/TableHead&gt;
            &lt;TableHead&gt;Status&lt;/TableHead&gt;
            &lt;TableHead className="w-[140px]"&gt;Ações&lt;/TableHead&gt;
          &lt;/TableRow&gt;
        &lt;/TableHeader&gt;
        &lt;TableBody&gt;
          {items.map(c =&gt; (
            &lt;TableRow key={c.id}&gt;
              &lt;TableCell className="font-medium"&gt;
                &lt;div className="flex items-center gap-3"&gt;
                  &lt;Avatar className="h-8 w-8"&gt;
                    &lt;AvatarImage src={c.avatar_url || undefined} /&gt;
                    &lt;AvatarFallback className="bg-muted text-muted-foreground text-xs"&gt;
                      {getInitials(c.name)}
                    &lt;/AvatarFallback&gt;
                  &lt;/Avatar&gt;
                  &lt;div&gt;
                    &lt;p className="font-medium text-foreground"&gt;
                      {c.owner_name || "—"} {getCountryFlag(c.country)}
                    &lt;/p&gt;
                    &lt;p className="text-xs text-muted-foreground"&gt;{c.name}&lt;/p&gt;
                  &lt;/div&gt;
                &lt;/div&gt;
              &lt;/TableCell&gt;
              &lt;TableCell&gt;{c.company_type}&lt;/TableCell&gt;
              &lt;TableCell className="text-muted-foreground"&gt;{c.city ? `${c.city}, ${c.state || ""}` : "—"}&lt;/TableCell&gt;
              &lt;TableCell&gt;
                &lt;Badge variant={c.status === "active" ? "default" : "secondary"}&gt;
                  {c.status === "active" ? "Ativo" : "Inativo"}
                &lt;/Badge&gt;
              &lt;/TableCell&gt;
              &lt;TableCell&gt;
                &lt;div className="flex gap-1"&gt;
                  &lt;Button variant="ghost" size="sm" onClick={() =&gt; openEmployees(c)} title="Funcionários"&gt;&lt;Users className="h-4
                  &lt;Button variant="ghost" size="icon" onClick={() =&gt; openEdit(c)}&gt;&lt;Pencil className="h-4 w-4" /&gt;&lt;/Button&
                  &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleDelete(c.id)}&gt;&lt;Trash2 className="h-4 w-4 text-destru
                &lt;/div&gt;
              &lt;/TableCell&gt;
            &lt;/TableRow&gt;
          ))}
          {items.length === 0 &amp;&amp; (
            &lt;TableRow&gt;&lt;TableCell colSpan={5} className="text-center text-muted-foreground py-8"&gt;Nenhuma empresa encontrada&lt;/
          )}
        &lt;/TableBody&gt;
      &lt;/Table&gt;
```

```
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ))}

      {/* Employees Dialog */}
      &lt;Dialog open={employeesDialogOpen} onOpenChange={setEmployeesDialogOpen}&gt;
        &lt;DialogContent className="max-w-lg max-h-[85vh] overflow-y-auto"&gt;
          &lt;DialogHeader&gt;
            &lt;DialogTitle&gt;Funcionários — {selectedCompany?.name}&lt;/DialogTitle&gt;
          &lt;/DialogHeader&gt;
          {selectedCompany &amp;&amp; &lt;CompanyEmployees companyId={selectedCompany.id} /&gt;}
        &lt;/DialogContent&gt;
      &lt;/Dialog&gt;
    &lt;/div&gt;
  );
};

export default AdminCompanies;
```

src/pages/admin/AdminContracts.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Search, FileSignature, ExternalLink } from "lucide-react";
import { Button } from "@/components/ui/button";
import { format } from "date-fns";

const AdminContracts = () =&gt; {
  const [contracts, setContracts] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");

  useEffect(() =&gt; {
    supabase.from("freelancer_contracts").select("*, projects(name, companies(name))").order("created_at", { ascending: false })
      .then(({ data }) =&gt; setContracts(data || []));
  }, []);

  const filtered = contracts.filter(c =&gt;
    (c.title || "").toLowerCase().includes(search.toLowerCase()) ||
    (c.projects?.name || "").toLowerCase().includes(search.toLowerCase())
  );

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Contracts&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Track contracts and agreements&lt;/p&gt;
        &lt;/div&gt;
        &lt;Badge variant="secondary"&gt;{contracts.length} total&lt;/Badge&gt;
      &lt;/div&gt;
      &lt;div className="relative max-w-xs"&gt;
        &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
        &lt;Input placeholder="Search contracts..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Title&lt;/TableHead&gt;
                &lt;TableHead&gt;Project&lt;/TableHead&gt;
                &lt;TableHead&gt;Client&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Created&lt;/TableHead&gt;
                &lt;TableHead className="w-[50px]"&gt;&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {filtered.length === 0 &amp;amp;&amp;amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={6} className="text-center text-muted-foreground py-12"&gt;&lt;FileSignature className="h-8
              )}
              {filtered.map(c =&gt; (
                &lt;TableRow key={c.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{c.title}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{c.projects?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{c.projects?.companies?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    {c.is_signed
                      ? &lt;Badge className="bg-emerald-500/10 text-emerald-600" variant="outline"&gt;Signed&lt;/Badge&gt;
                      : &lt;Badge variant="outline"&gt;Pending&lt;/Badge&gt;}
                  &lt;/TableCell&gt;
                  &lt;TableCell className="text-xs text-muted-foreground"&gt;{format(new Date(c.created_at), "MMM d, yyyy")}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    {c.file_url &amp;amp;&amp;amp; (
                      &lt;Button variant="ghost" size="icon" className="h-7 w-7" asChild&gt;
                        &lt;a href={c.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;ExternalLink className="h-3.5 w-3.5" /&gt;&lt;/a
                      &lt;/Button&gt;
                    )}
                  &lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminContracts;
```

src/pages/admin/AdminDashboard.tsx

```
import { useEffect, useState } from "react";
import { useAuth } from "@/hooks/useAuth";
import { supabase } from "@/integrations/supabase/client";
import KPICards from "@/components/admin/dashboard/KPICards";
import ProjectPipeline from "@/components/admin/dashboard/ProjectPipeline";
import ActivityFeed from "@/components/admin/dashboard/ActivityFeed";
import ClientActivityFeed from "@/components/admin/dashboard/ClientActivityFeed";
import TodaysTasks from "@/components/admin/dashboard/TodaysTasks";
import FinancialChart from "@/components/admin/dashboard/FinancialChart";
import NewLeadsTable from "@/components/admin/dashboard/NewLeadsTable";
import QuickActions from "@/components/admin/dashboard/QuickActions";
import ClientTicketsWidget from "@/components/admin/dashboard/ClientTicketsWidget";

const AdminDashboard = () =&gt; {
  const { user } = useAuth();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [companies, setCompanies] = useState&lt;any[]&gt;([]);
  const [activities, setActivities] = useState&lt;any[]&gt;([]);
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [clientActivities, setClientActivities] = useState&lt;any[]&gt;([]);
  const [tasks, setTasks] = useState&lt;any[]&gt;([]);

  const fetchData = async () =&gt; {
    const [projRes, compRes, actRes, payRes, taskRes] = await Promise.all([
      supabase.from("projects").select("*, companies(name, country, language)"),
      supabase.from("companies").select("id, name"),
      supabase.from("activity_log").select("*, projects(name, companies(name, country))").order("created_at", { ascending: false }).limit(20),
      supabase.from("admin_payments").select("*"),
      supabase.from("admin_tasks").select("*, projects(name, companies(name))").order("due_date", { ascending: true }),
    ]);
    setProjects(projRes.data || []);
    setCompanies(compRes.data || []);
    setActivities(actRes.data || []);
    setPayments(payRes.data || []);

    // Build tasks with project name
    const taskList = (taskRes.data || []).map((t: any) =&gt; ({
      ...t,
      project_name: t.projects?.name || t.projects?.companies?.name || null,
    }));
    setTasks(taskList);

    // Build client activity feed from activity_log entries
    const clientTypes = ["approval", "upload", "revision", "message", "invoice", "download", "view"];
    const clientActs = (actRes.data || [])
      .filter((a: any) =&gt; clientTypes.includes(a.log_type) || a.log_type === "file_upload")
      .map((a: any) =&gt; ({
        id: a.id,
        client_name: a.projects?.companies?.name || "Client",
        client_country: a.projects?.companies?.country || null,
        company_id: a.project_id ? (projRes.data || []).find((p: any) =&gt; p.id === a.project_id)?.company_id : null,
        project_name: a.projects?.name || "Project",
        action: a.description,
        action_type: a.log_type === "file_upload" ? "upload" : a.log_type,
        created_at: a.created_at,
      }));
    setClientActivities(clientActs);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  const activeStages = ["In Progress", "Preview Sent", "Revision", "Final Review"];
  const activeCount = projects.filter((p) =&gt; activeStages.includes(p.stage)).length;
  const revenue = payments.filter((p) =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const pendingPayments = payments.filter((p) =&gt; p.status !== "paid").reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);
  const thirtyDaysAgo = new Date(); thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);
  const newLeads = projects.filter((p) =&gt; (p.stage === "Lead" || p.stage === "briefing") &amp;&amp; new Date(p.created_at) &gt;= thirtyDaysAgo
  const delayedProjects = projects.filter((p) =&gt; p.stage === "Delayed" || (p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date(
  const proposalsSent = projects.filter((p) =&gt; p.stage === "Budget Sent" || p.stage === "Proposal Sent").length;

  return (
    &lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
      {/* Header */}
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;
            Welcome back, {user?.user_metadata?.full_name?.split(" ")[0] || "Admin"}
          &lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Here's what's happening with your studio today.&lt;/p&gt;
        &lt;/div&gt;
        &lt;QuickActions /&gt;
      &lt;/div&gt;

      {/* KPI Cards */}
```

```
      &lt;KPICards
        totalProjects={projects.length}
        activeProjects={activeCount}
        totalClients={companies.length}
        newLeads={newLeads}
        revenue={revenue}
        pendingPayments={pendingPayments}
        delayedProjects={delayedProjects}
        proposalsSent={proposalsSent}
      /&gt;

      {/* Project Pipeline */}
      &lt;ProjectPipeline projects={projects} onProjectMoved={fetchData} /&gt;

      {/* Activity + Client Activity + Today's Tasks */}
      &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
        &lt;ActivityFeed activities={activities} /&gt;
        &lt;ClientActivityFeed activities={clientActivities} /&gt;
      &lt;/div&gt;

      {/* Tickets + Today's Tasks */}
      &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
        &lt;ClientTicketsWidget /&gt;
        &lt;TodaysTasks tasks={tasks} projects={projects} onRefresh={fetchData} /&gt;
      &lt;/div&gt;

      {/* Financial Performance */}
      &lt;FinancialChart payments={payments} projects={projects} /&gt;

      {/* New Leads */}
      &lt;NewLeadsTable /&gt;
    &lt;/div&gt;
  );
};

export default AdminDashboard;
```

src/pages/admin/AdminDrawings.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Upload, FileImage, Download, Star, Clock, Plus, ChevronRight } from "lucide-react";
import { toast } from "sonner";
import { format } from "date-fns";

const AdminDrawings = () =&gt; {
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [selectedProject, setSelectedProject] = useState&lt;string&gt;("");
  const [drawings, setDrawings] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [newDrawingName, setNewDrawingName] = useState("");
  const [newNotes, setNewNotes] = useState("");
  const [file, setFile] = useState&lt;File | null&gt;(null);
  const [uploading, setUploading] = useState(false);

  useEffect(() =&gt; {
    supabase.from("projects").select("id, name, companies(name)").then(({ data }) =&gt; {
      setProjects(data || []);
    });
  }, []);

  useEffect(() =&gt; {
    if (selectedProject) fetchDrawings();
  }, [selectedProject]);

  const fetchDrawings = async () =&gt; {
    const { data } = await supabase
      .from("drawing_versions")
      .select("*")
      .eq("project_id", selectedProject)
      .order("drawing_name", { ascending: true })
      .order("version_number", { ascending: false });
    setDrawings(data || []);
  };

  const handleUpload = async () =&gt; {
    if (!file || !selectedProject || !newDrawingName.trim()) return;
    setUploading(true);

    const ext = file.name.split(".").pop();
    const path = `${selectedProject}/${Date.now()}.${ext}`;
    const { error: uploadErr } = await supabase.storage
      .from("project-files")
      .upload(path, file);

    if (uploadErr) {
      toast.error("Upload failed");
      setUploading(false);
      return;
    }

    const { data: urlData } = supabase.storage.from("project-files").getPublicUrl(path);

    // Get next version number for this drawing name
    const existing = drawings.filter(
      (d) =&gt; d.drawing_name.toLowerCase() === newDrawingName.trim().toLowerCase()
    );
    const nextVersion = existing.length &gt; 0
      ? Math.max(...existing.map((d) =&gt; d.version_number)) + 1
      : 1;

    // Unmark previous current
    if (existing.length &gt; 0) {
      await supabase
        .from("drawing_versions")
        .update({ is_current: false })
        .eq("project_id", selectedProject)
        .eq("drawing_name", newDrawingName.trim());
    }

    const { error } = await supabase.from("drawing_versions").insert({
      project_id: selectedProject,
      drawing_name: newDrawingName.trim(),
      version_number: nextVersion,
      file_url: urlData.publicUrl,
```

```
    file_name: file.name,
    revision_notes: newNotes || null,
    is_current: true,
  });

  if (!error) {
    toast.success(`Uploaded v${nextVersion}`);
    setDialogOpen(false);
    setNewDrawingName("");
    setNewNotes("");
    setFile(null);
    fetchDrawings();
  } else {
    toast.error("Failed to save version");
  }
  setUploading(false);
};

const handleSetCurrent = async (drawing: any) =&gt; {
  await supabase
    .from("drawing_versions")
    .update({ is_current: false })
    .eq("project_id", drawing.project_id)
    .eq("drawing_name", drawing.drawing_name);

  await supabase
    .from("drawing_versions")
    .update({ is_current: true })
    .eq("id", drawing.id);

  toast.success(`v${drawing.version_number} set as current`);
  fetchDrawings();
};

// Group drawings by name
const grouped: Record&lt;string, any[]&gt; = {};
drawings.forEach((d) =&gt; {
  if (!grouped[d.drawing_name]) grouped[d.drawing_name] = [];
  grouped[d.drawing_name].push(d);
});

return (
  &lt;div className="p-6 space-y-6 max-w-[1200px] mx-auto"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Drawing Version Control&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Manage drawing versions for each project&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
        &lt;DialogTrigger asChild&gt;
          &lt;Button className="gap-2" disabled={!selectedProject}&gt;
            &lt;Plus className="h-4 w-4" /&gt; Upload Drawing
          &lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent&gt;
          &lt;DialogHeader&gt;
            &lt;DialogTitle&gt;Upload New Drawing Version&lt;/DialogTitle&gt;
          &lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            &lt;Input
              placeholder="Drawing name (e.g. Floor Plan, Elevations)"
              value={newDrawingName}
              onChange={(e) =&gt; setNewDrawingName(e.target.value)}
            /&gt;
            &lt;div className="border-2 border-dashed rounded-lg p-6 text-center"&gt;
              &lt;input
                type="file"
                accept=".pdf,.dwg,.dxf,.png,.jpg,.jpeg"
                onChange={(e) =&gt; setFile(e.target.files?.[0] || null)}
                className="hidden"
                id="drawing-upload"
              /&gt;
              &lt;label htmlFor="drawing-upload" className="cursor-pointer space-y-2"&gt;
                &lt;Upload className="h-8 w-8 mx-auto text-muted-foreground" /&gt;
                &lt;p className="text-sm text-muted-foreground"&gt;
                  {file ? file.name : "Click to select file (PDF, DWG, DXF, PNG, JPG)"}
                &lt;/p&gt;
              &lt;/label&gt;
            &lt;/div&gt;
            &lt;Textarea
              placeholder="Revision notes — what changed?"
              value={newNotes}
              onChange={(e) =&gt; setNewNotes(e.target.value)}
              rows={3}
            /&gt;
            &lt;Button
              className="w-full"
```

```
          onClick={handleUpload}
          disabled={!file || !newDrawingName.trim() || uploading}
        &gt;
          {uploading ? "Uploading..." : "Upload Version"}
        &lt;/Button&gt;
      &lt;/div&gt;
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;

{/* Project Selector */}
&lt;Select value={selectedProject} onValueChange={setSelectedProject}&gt;
  &lt;SelectTrigger className="max-w-sm"&gt;
    &lt;SelectValue placeholder="Select a project" /&gt;
  &lt;/SelectTrigger&gt;
  &lt;SelectContent&gt;
    {projects.map((p) =&gt; (
      &lt;SelectItem key={p.id} value={p.id}&gt;
        {p.name || p.companies?.name || "Untitled"}
      &lt;/SelectItem&gt;
    ))}
  &lt;/SelectContent&gt;
&lt;/Select&gt;

{!selectedProject &amp;&amp; (
  &lt;p className="text-sm text-muted-foreground text-center py-16"&gt;Select a project to view drawings&lt;/p&gt;
)}

{selectedProject &amp;&amp; Object.keys(grouped).length === 0 &amp;&amp; (
  &lt;p className="text-sm text-muted-foreground text-center py-16"&gt;No drawings uploaded yet&lt;/p&gt;
)}

{/* Drawing Groups */}
&lt;div className="space-y-6"&gt;
  {Object.entries(grouped).map(([name, versions]) =&gt; (
    &lt;Card key={name} className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;FileImage className="h-4 w-4 text-muted-foreground" /&gt;
          &lt;CardTitle className="text-base font-semibold"&gt;{name}&lt;/CardTitle&gt;
          &lt;Badge variant="secondary" className="ml-auto"&gt;{versions.length} versions&lt;/Badge&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;ScrollArea className="max-h-[300px]"&gt;
          &lt;div className="space-y-2"&gt;
            {versions.map((v) =&gt; (
              &lt;div
                key={v.id}
                className={`flex items-center gap-3 p-3 rounded-lg border transition-colors ${
                  v.is_current ? "border-primary/30 bg-primary/5" : "border-border hover:bg-muted/50"
                }`}
              &gt;
                &lt;div className="flex items-center justify-center h-9 w-9 rounded-md bg-muted text-xs font-bold text-muted-foreground shr
                  v{v.version_number}
                &lt;/div&gt;
                &lt;div className="flex-1 min-w-0"&gt;
                  &lt;div className="flex items-center gap-2"&gt;
                    &lt;p className="text-sm font-medium text-foreground truncate"&gt;{v.file_name}&lt;/p&gt;
                    {v.is_current &amp;&amp; (
                      &lt;Badge className="text-[10px] bg-primary/10 text-primary border-primary/20" variant="outline"&gt;
                        &lt;Star className="h-2.5 w-2.5 mr-0.5" /&gt; Current
                      &lt;/Badge&gt;
                    )}
                  &lt;/div&gt;
                  {v.revision_notes &amp;&amp; (
                    &lt;p className="text-xs text-muted-foreground mt-0.5 truncate"&gt;{v.revision_notes}&lt;/p&gt;
                  )}
                  &lt;div className="flex items-center gap-2 mt-1 text-xs text-muted-foreground"&gt;
                    &lt;Clock className="h-3 w-3" /&gt;
                    {format(new Date(v.created_at), "MMM d, yyyy 'at' h:mm a")}
                  &lt;/div&gt;
                &lt;/div&gt;
                &lt;div className="flex items-center gap-1 shrink-0"&gt;
                  {!v.is_current &amp;&amp; (
                    &lt;Button
                      variant="ghost"
                      size="sm"
                      className="h-7 text-xs gap-1"
                      onClick={() =&gt; handleSetCurrent(v)}
                    &gt;
                      &lt;Star className="h-3 w-3" /&gt; Set Current
                    &lt;/Button&gt;
                  )}
                  &lt;Button
                    variant="ghost"
                    size="icon"
```

```
                          className="h-7 w-7"
                          asChild
                        &gt;
                          &lt;a href={v.file_url} target="_blank" rel="noopener noreferrer" download&gt;
                            &lt;Download className="h-3.5 w-3.5" /&gt;
                          &lt;/a&gt;
                        &lt;/Button&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                  ))}
                &lt;/div&gt;
              &lt;/ScrollArea&gt;
            &lt;/CardContent&gt;
          &lt;/Card&gt;
        ))}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminDrawings;
```

src/pages/admin/AdminFiles.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Button } from "@/components/ui/button";
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Search, FolderOpen, Download, FileIcon, ChevronRight, Building2 } from "lucide-react";
import { format } from "date-fns";

const CATEGORY_LABELS: Record&lt;string, string&gt; = {
  pre_construction: "■ Pre-Construction",
  rev_00: "■ REV 00",
  rev_01: "■ REV 01",
  rev_02: "■ REV 02",
  rev_03: "■ REV 03 (Final)",
  final_delivery: "■ Final Delivery",
  general: "■ General",
};

const CATEGORY_ORDER = ["pre_construction", "rev_00", "rev_01", "rev_02", "rev_03", "final_delivery", "general"];

const AdminFiles = () =&gt; {
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");
  const [openProjects, setOpenProjects] = useState&lt;Set&lt;string&gt;&gt;(new Set());

  useEffect(() =&gt; {
    supabase.from("project_files").select("*, projects(name, companies(name))").order("created_at", { ascending: false })
      .then(({ data }) =&gt; setFiles(data || []));
  }, []);

  const filtered = files.filter(f =&gt;
    (f.file_name || "").toLowerCase().includes(search.toLowerCase()) ||
    (f.projects?.name || "").toLowerCase().includes(search.toLowerCase()) ||
    (f.projects?.companies?.name || "").toLowerCase().includes(search.toLowerCase())
  );

  // Group by project
  const byProject = filtered.reduce((acc: Record&lt;string, { project: any; files: any[] }&gt;, f) =&gt; {
    const pid = f.project_id;
    if (!acc[pid]) {
      acc[pid] = { project: { id: pid, name: f.projects?.name || "Unnamed Project", company: f.projects?.companies?.name || "" }, files: [] };
    }
    acc[pid].files.push(f);
    return acc;
  }, {});

  const formatSize = (bytes: number | null) =&gt; {
    if (!bytes) return "—";
    if (bytes &lt; 1024) return `${bytes} B`;
    if (bytes &lt; 1048576) return `${(bytes / 1024).toFixed(1)} KB`;
    return `${(bytes / 1048576).toFixed(1)} MB`;
  };

  const toggleProject = (pid: string) =&gt; {
    setOpenProjects(prev =&gt; {
      const next = new Set(prev);
      next.has(pid) ? next.delete(pid) : next.add(pid);
      return next;
    });
  };

  const downloadAll = (projectFiles: any[]) =&gt; {
    projectFiles.forEach((f, i) =&gt; {
      setTimeout(() =&gt; {
        const a = document.createElement("a");
        a.href = f.file_url;
        a.target = "_blank";
        a.rel = "noopener noreferrer";
        a.click();
      }, i * 300);
    });
  };

  const projectEntries = Object.entries(byProject).sort((a, b) =&gt; (a[1] as any).project.name.localeCompare((b[1] as any).project.name));

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Project Files&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;All files organized by project and revision&lt;/p&gt;
        &lt;/div&gt;
```

```
  &lt;Badge variant="secondary"&gt;{files.length} files • {projectEntries.length} projects&lt;/Badge&gt;
&lt;/div&gt;

&lt;div className="relative max-w-xs"&gt;
  &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
  &lt;Input placeholder="Search files, projects, companies..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)
&lt;/div&gt;

{projectEntries.length === 0 &amp;amp;&amp;amp; (
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="py-12 text-center text-muted-foreground"&gt;
      &lt;FolderOpen className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;
      No files yet
    &lt;/CardContent&gt;
  &lt;/Card&gt;
)}

&lt;div className="space-y-3"&gt;
  {projectEntries.map(([pid, entry]) =&gt; {
    const { project, files: pFiles } = entry as { project: any; files: any[] };
    const isOpen = openProjects.has(pid);
    const categorized = CATEGORY_ORDER.reduce((acc: Record&lt;string, any[]&gt;, cat) =&gt; {
      const catFiles = pFiles.filter(f =&gt; (f.document_category || "general") === cat);
      if (catFiles.length &gt; 0) acc[cat] = catFiles;
      return acc;
    }, {});

    return (
      &lt;Collapsible key={pid} open={isOpen} onOpenChange={() =&gt; toggleProject(pid)}&gt;
        &lt;Card className="border shadow-sm"&gt;
          &lt;CollapsibleTrigger asChild&gt;
            &lt;CardHeader className="cursor-pointer hover:bg-muted/50 transition-colors py-3 px-4"&gt;
              &lt;div className="flex items-center justify-between"&gt;
                &lt;div className="flex items-center gap-3"&gt;
                  &lt;ChevronRight className={`h-4 w-4 text-muted-foreground transition-transform ${isOpen ? "rotate-90" : ""}`} /&gt;
                  &lt;div&gt;
                    &lt;CardTitle className="text-sm font-semibold"&gt;{project.name}&lt;/CardTitle&gt;
                    {project.company &amp;amp;&amp;amp; (
                      &lt;p className="text-xs text-muted-foreground flex items-center gap-1 mt-0.5"&gt;
                        &lt;Building2 className="h-3 w-3" /&gt; {project.company}
                      &lt;/p&gt;
                    )}
                  &lt;/div&gt;
                &lt;/div&gt;
                &lt;div className="flex items-center gap-2"&gt;
                  &lt;Badge variant="outline" className="text-xs"&gt;{pFiles.length} files&lt;/Badge&gt;
                  &lt;Button
                    variant="outline"
                    size="sm"
                    className="h-7 gap-1.5 text-xs"
                    onClick={(e) =&gt; { e.stopPropagation(); downloadAll(pFiles); }}
                  &gt;
                    &lt;Download className="h-3 w-3" /&gt; Download All
                  &lt;/Button&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            &lt;/CardHeader&gt;
          &lt;/CollapsibleTrigger&gt;
          &lt;CollapsibleContent&gt;
            &lt;CardContent className="pt-0 pb-3 px-4 space-y-3"&gt;
              {Object.entries(categorized).map(([cat, catFiles]) =&gt; (
                &lt;div key={cat} className="space-y-1"&gt;
                  &lt;p className="text-xs font-medium text-muted-foreground pl-1"&gt;{CATEGORY_LABELS[cat] || cat}&lt;/p&gt;
                  &lt;div class="space-y-0.5"&gt;
                    {catFiles.map((f: any) =&gt; (
                      &lt;div key={f.id} class="flex items-center justify-between py-1.5 px-2 rounded hover:bg-muted/50 group"&gt;
                        &lt;div class="flex items-center gap-2 min-w-0 flex-1"&gt;
                          &lt;FileIcon class="h-3.5 w-3.5 text-muted-foreground flex-shrink-0" /&gt;
                          &lt;span class="text-sm truncate"&gt;{f.file_name}&lt;/span&gt;
                          &lt;Badge variant="outline" class="text-[9px] flex-shrink-0"&gt;{f.file_type || "—"}&lt;/Badge&gt;
                        &lt;/div&gt;
                        &lt;div class="flex items-center gap-3 text-xs text-muted-foreground flex-shrink-0"&gt;
                          &lt;span&gt;{formatSize(f.file_size)}&lt;/span&gt;
                          &lt;span&gt;{format(new Date(f.created_at), "MMM d, yyyy")}&lt;/span&gt;
                          &lt;Button variant="ghost" size="icon" class="h-6 w-6 opacity-0 group-hover:opacity-100 transition-opacity" a
                            &lt;a href={f.file_url} target="_blank" rel="noopener noreferrer"&gt;&lt;Download class="h-3 w-3" /&gt;&lt;
                          &lt;/Button&gt;
                        &lt;/div&gt;
                      &lt;/div&gt;
                    ))}
                  &lt;/div&gt;
                &lt;/div&gt;
              ))}
            &lt;/CardContent&gt;
          &lt;/CollapsibleContent&gt;
        &lt;/Card&gt;
      &lt;/Collapsible&gt;
```

```
          );
        })}
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminFiles;
```

src/pages/admin/AdminFinanceOverview.tsx

```
import { useEffect, useState, useMemo } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Progress } from "@/components/ui/progress";
import {
  DollarSign, TrendingUp, Clock, CheckCircle2, Receipt,
  ExternalLink, FileText, Download, Bell,
} from "lucide-react";
import { format, startOfMonth, endOfMonth, addDays, subMonths } from "date-fns";
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer,
} from "recharts";
import { toast } from "sonner";

const AdminFinanceOverview = () =&gt; {
  const navigate = useNavigate();
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [milestones, setMilestones] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);

  const fetchData = async () =&gt; {
    const [payRes, msRes, projRes] = await Promise.all([
      supabase.from("admin_payments").select("*, projects(name, street_name, street_number, companies(name))").order("created_at", { ascending: f
      supabase.from("payment_milestones").select("*, projects(name, street_name, street_number, companies(name))").order("created_at", { ascendir
      supabase.from("projects").select("id, name, street_name, street_number, total_value, currency, companies(name)"),
    ]);
    setPayments(payRes.data || []);
    setMilestones(msRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  // Realtime
  useEffect(() =&gt; {
    const channel = supabase
      .channel("finance-overview")
      .on("postgres_changes", { event: "*", schema: "public", table: "admin_payments" }, () =&gt; fetchData())
      .on("postgres_changes", { event: "*", schema: "public", table: "payment_milestones" }, () =&gt; fetchData())
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, []);

  const now = new Date();
  const monthStart = startOfMonth(now);
  const monthEnd = endOfMonth(now);
  const next30 = addDays(now, 30);

  // KPI calculations
  const totalRevenue = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const revenueThisMonth = payments
    .filter(p =&gt; p.status === "paid" &amp;&amp; p.payment_date &amp;&amp; new Date(p.payment_date) &gt;= monthStart &amp;&amp; new Date(p.paym
    .reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const outstandingFromPayments = payments.filter(p =&gt; p.status === "pending").reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);
  const outstandingFromMilestones = milestones.filter(m =&gt; m.status === "pending" || m.status === "overdue").reduce((s, m) =&gt; s + Number(m.
  const outstandingRevenue = outstandingFromPayments + outstandingFromMilestones;
  const paidInvoicesCount = payments.filter(p =&gt; p.status === "paid").length;

  // Outstanding invoices (pending milestones + pending payments)
  const outstandingInvoices = useMemo(() =&gt; {
    const fromMilestones = milestones
      .filter(m =&gt; m.status === "pending" || m.status === "overdue")
      .map(m =&gt; ({
        id: m.id,
        type: "milestone" as const,
        label: m.stage,
        project: [m.projects?.street_number, m.projects?.street_name].filter(Boolean).join(" ") || m.projects?.name || "—",
        client: m.projects?.companies?.name || "—",
        amount: Number(m.amount || 0),
        status: m.status,
        date: m.created_at,
      }));
    const fromPayments = payments
      .filter(p =&gt; p.status === "pending")
      .map(p =&gt; ({
        id: p.id,
        type: "payment" as const,
        label: p.installment_type || "Invoice",
        project: [p.projects?.street_number, p.projects?.street_name].filter(Boolean).join(" ") || p.projects?.name || "—",
        client: p.projects?.companies?.name || "—",
```

```
      amount: Number(p.amount_total || 0),
      status: p.status,
      date: p.created_at,
    }));
  return [...fromMilestones, ...fromPayments];
}, [milestones, payments]);

// Recent payments (last 10 paid)
const recentPayments = useMemo(() =&gt;
  payments.filter(p =&gt; p.status === "paid").slice(0, 10), [payments]);

// Revenue forecast (pending with dates in next 30 days)
const revenueForecast = useMemo(() =&gt; {
  const pendingMs = milestones
    .filter(m =&gt; (m.status === "pending" || m.status === "overdue"))
    .reduce((s, m) =&gt; s + Number(m.amount || 0), 0);
  const pendingPay = payments
    .filter(p =&gt; p.status === "pending")
    .reduce((s, p) =&gt; s + Number(p.amount_total || 0), 0);
  return pendingMs + pendingPay;
}, [milestones, payments]);

// Monthly revenue chart (last 6 months)
const monthlyData = useMemo(() =&gt; {
  const months: { label: string; revenue: number }[] = [];
  for (let i = 5; i &gt;= 0; i--) {
    const d = subMonths(now, i);
    const ms = startOfMonth(d);
    const me = endOfMonth(d);
    const rev = payments
      .filter(p =&gt; p.status === "paid" &amp;&amp; p.payment_date &amp;&amp; new Date(p.payment_date) &gt;= ms &amp;&amp; new Date(p.payment_
      .reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
    months.push({ label: format(ms, "MMM yyyy"), revenue: rev });
  }
  return months;
}, [payments]);

const fmtCurrency = (v: number) =&gt; `$${v.toLocaleString("en-US", { minimumFractionDigits: 2, maximumFractionDigits: 2 })}`;

return (
  &lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
    {/* Header */}
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Finance Overview&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Complete financial status of your business&lt;/p&gt;
      &lt;/div&gt;
      &lt;div className="flex items-center gap-2"&gt;
        &lt;Button variant="outline" size="sm" onClick={() =&gt; navigate("/adm/payments")}&gt;
          &lt;Receipt className="h-3.5 w-3.5 mr-1" /&gt; Payments Page
        &lt;/Button&gt;
        &lt;Button variant="outline" size="sm" onClick={() =&gt; {
          toast.info("Export coming soon");
        }}&gt;
          &lt;Download className="h-3.5 w-3.5 mr-1" /&gt; Export Report
        &lt;/Button&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* KPI Cards */}
    &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4"&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4 flex items-center gap-3"&gt;
          &lt;div className="p-2.5 rounded-lg bg-chart-2/15"&gt;
            &lt;DollarSign className="h-5 w-5 text-chart-2" /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{fmtCurrency(totalRevenue)}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Total Revenue&lt;/p&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4 flex items-center gap-3"&gt;
          &lt;div className="p-2.5 rounded-lg bg-chart-4/15"&gt;
            &lt;TrendingUp className="h-5 w-5 text-chart-4" /&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;p className="text-2xl font-bold text-foreground"&gt;{fmtCurrency(revenueThisMonth)}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground"&gt;Revenue This Month&lt;/p&gt;
          &lt;/div&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-4 flex items-center gap-3"&gt;
          &lt;div className="p-2.5 rounded-lg bg-destructive/15"&gt;
            &lt;Clock className="h-5 w-5 text-destructive" /&gt;
```

```
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;{fmtCurrency(outstandingRevenue)}&lt;/p&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;Outstanding Revenue&lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-4 flex items-center gap-3"&gt;
      &lt;div className="p-2.5 rounded-lg bg-primary/15"&gt;
        &lt;CheckCircle2 className="h-5 w-5 text-primary" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;{paidInvoicesCount}&lt;/p&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;Paid Invoices&lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Revenue Forecast + Monthly Chart */}
&lt;div className="grid grid-cols-1 lg:grid-cols-3 gap-6"&gt;
  {/* Forecast card */}
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;CardTitle className="text-base"&gt;Revenue Forecast&lt;/CardTitle&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent className="space-y-4"&gt;
      &lt;div className="text-center py-4"&gt;
        &lt;p className="text-3xl font-bold text-foreground"&gt;{fmtCurrency(revenueForecast)}&lt;/p&gt;
        &lt;p className="text-sm text-muted-foreground mt-1"&gt;Projected (pending invoices)&lt;/p&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;div className="space-y-2"&gt;
          &lt;div className="flex justify-between text-sm"&gt;
            &lt;span className="text-muted-foreground"&gt;Pending Milestones&lt;/span&gt;
            &lt;span className="font-medium"&gt;{fmtCurrency(outstandingFromMilestones)}&lt;/span&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;span className="font-medium"&gt;{fmtCurrency(outstandingFromPayments)}&lt;/span&gt;
          &lt;/div&gt;
      &lt;/div&gt;
      {outstandingRevenue &gt; 0 &amp;&amp; totalRevenue &gt; 0 &amp;&amp; (
        &lt;div&gt;
          &lt;div className="flex justify-between text-xs text-muted-foreground mb-1"&gt;
            &lt;span&gt;Collection Progress&lt;/span&gt;
            &lt;span&gt;{Math.round((totalRevenue / (totalRevenue + outstandingRevenue)) * 100)}%&lt;/span&gt;
          &lt;/div&gt;
          &lt;Progress value={(totalRevenue / (totalRevenue + outstandingRevenue)) * 100} className="h-2" /&gt;
        &lt;/div&gt;
      )}
    &lt;/CardContent&gt;
  &lt;/Card&gt;

  {/* Monthly Revenue Chart */}
  &lt;Card className="border-none shadow-sm lg:col-span-2"&gt;
    &lt;CardHeader className="pb-3"&gt;
      &lt;CardTitle className="text-base"&gt;Monthly Revenue&lt;/CardTitle&gt;
    &lt;/CardHeader&gt;
    &lt;CardContent&gt;
      &lt;div className="h-[250px]"&gt;
        &lt;ResponsiveContainer width="100%" height="100%"&gt;
          &lt;BarChart data={monthlyData}&gt;
            &lt;CartesianGrid strokeDasharray="3 3" stroke="hsl(var(--border))" /&gt;
            &lt;XAxis dataKey="label" tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" /&gt;
            &lt;YAxis tick={{ fontSize: 12 }} stroke="hsl(var(--muted-foreground))" tickFormatter={(v) =&gt; `$${(v / 1000).toFixed(0)}k`}
            &lt;Tooltip
              formatter={(value: number) =&gt; [fmtCurrency(value), "Revenue"]}
              contentStyle={{
                background: "hsl(var(--card))",
                border: "1px solid hsl(var(--border))",
                borderRadius: "8px",
                fontSize: "12px",
              }}
            /&gt;
            &lt;Bar dataKey="revenue" fill="hsl(var(--chart-2))" radius={[4, 4, 0, 0]} /&gt;
          &lt;/BarChart&gt;
        &lt;/ResponsiveContainer&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Outstanding Invoices */}
&lt;Card className="border-none shadow-sm"&gt;
  &lt;CardHeader className="pb-3"&gt;
    &lt;CardTitle className="text-base"&gt;Outstanding Invoices ({outstandingInvoices.length})&lt;/CardTitle&gt;
```

```
  &lt;/CardHeader&gt;
  &lt;CardContent className="p-0"&gt;
    &lt;Table&gt;
      &lt;TableHeader&gt;
        &lt;TableRow&gt;
          &lt;TableHead&gt;Invoice&lt;/TableHead&gt;
          &lt;TableHead&gt;Project&lt;/TableHead&gt;
          &lt;TableHead&gt;Client&lt;/TableHead&gt;
          &lt;TableHead&gt;Amount&lt;/TableHead&gt;
          &lt;TableHead&gt;Status&lt;/TableHead&gt;
          &lt;TableHead&gt;Action&lt;/TableHead&gt;
        &lt;/TableRow&gt;
      &lt;/TableHeader&gt;
      &lt;TableBody&gt;
        {outstandingInvoices.length === 0 &amp;&amp; (
          &lt;TableRow&gt;
            &lt;TableCell colSpan={6} className="text-center text-muted-foreground py-12"&gt;
              &lt;CheckCircle2 className="h-8 w-8 mx-auto mb-2 opacity-40" /&gt;
              No outstanding invoices
            &lt;/TableCell&gt;
          &lt;/TableRow&gt;
        )}
        {outstandingInvoices.map((inv) =&gt; (
          &lt;TableRow key={inv.id}&gt;
            &lt;TableCell className="font-medium"&gt;{inv.label}&lt;/TableCell&gt;
            &lt;TableCell className="text-muted-foreground"&gt;{inv.project}&lt;/TableCell&gt;
            &lt;TableCell className="text-muted-foreground"&gt;{inv.client}&lt;/TableCell&gt;
            &lt;TableCell className="font-medium"&gt;{fmtCurrency(inv.amount)}&lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;Badge variant="outline" className={
                inv.status === "overdue"
                  ? "bg-destructive/15 text-destructive border-destructive/30"
                  : "bg-chart-3/15 text-chart-3 border-chart-3/30"
              }&gt;
                {inv.status}
              &lt;/Badge&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;Button
                size="sm"
                variant="ghost"
                className="text-xs"
                onClick={() =&gt; toast.info(`Reminder for "${inv.label}" would be sent`)}
              &gt;
                &lt;Bell className="h-3 w-3 mr-1" /&gt; Remind
              &lt;/Button&gt;
            &lt;/TableCell&gt;
          &lt;/TableRow&gt;
        ))}
      &lt;/TableBody&gt;
    &lt;/Table&gt;
  &lt;/CardContent&gt;
&lt;/Card&gt;

{/* Recent Payments */}
&lt;Card className="border-none shadow-sm"&gt;
  &lt;CardHeader className="pb-3"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;CardTitle className="text-base"&gt;Recent Payments&lt;/CardTitle&gt;
      &lt;Button variant="ghost" size="sm" className="text-xs" onClick={() =&gt; navigate("/adm/payments")}&gt;
        View All &lt;ExternalLink className="h-3 w-3 ml-1" /&gt;
      &lt;/Button&gt;
    &lt;/div&gt;
  &lt;/CardHeader&gt;
  &lt;CardContent className="p-0"&gt;
    &lt;Table&gt;
      &lt;TableHeader&gt;
        &lt;TableRow&gt;
          &lt;TableHead&gt;Project&lt;/TableHead&gt;
          &lt;TableHead&gt;Client&lt;/TableHead&gt;
          &lt;TableHead&gt;Amount&lt;/TableHead&gt;
          &lt;TableHead&gt;Method&lt;/TableHead&gt;
          &lt;TableHead&gt;Date&lt;/TableHead&gt;
        &lt;/TableRow&gt;
      &lt;/TableHeader&gt;
      &lt;TableBody&gt;
        {recentPayments.length === 0 &amp;&amp; (
          &lt;TableRow&gt;
            &lt;TableCell colSpan={5} className="text-center text-muted-foreground py-12"&gt;
              No payments yet
            &lt;/TableCell&gt;
          &lt;/TableRow&gt;
        )}
        {recentPayments.map((p) =&gt; (
          &lt;TableRow key={p.id}&gt;
            &lt;TableCell className="font-medium"&gt;
              {[p.projects?.street_number, p.projects?.street_name].filter(Boolean).join(" ") || p.projects?.name || "—"}
```

```
                &lt;/TableCell&gt;
                &lt;TableCell className="text-muted-foreground"&gt;{p.projects?.companies?.name || "—"}&lt;/TableCell&gt;
                &lt;TableCell className="font-medium text-chart-2"&gt;{fmtCurrency(Number(p.amount_paid || 0))}&lt;/TableCell&gt;
                &lt;TableCell&gt;
                  &lt;Badge variant="secondary" className="text-xs"&gt;{p.payment_method || "—"}&lt;/Badge&gt;
                &lt;/TableCell&gt;
                &lt;TableCell className="text-muted-foreground text-sm"&gt;
                  {p.payment_date ? format(new Date(p.payment_date), "MMM d, yyyy") : "—"}
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
  );
};

export default AdminFinanceOverview;
```

src/pages/admin/AdminFreelancers.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Users, FolderOpen, DollarSign } from "lucide-react";
import { format } from "date-fns";

const AdminFreelancers = () =&gt; {
  const [freelancers, setFreelancers] = useState&lt;any[]&gt;([]);
  const [loading, setLoading] = useState(true);

  useEffect(() =&gt; {
    const fetchData = async () =&gt; {
      // Get all freelancer roles
      const { data: roles } = await supabase
        .from("user_roles")
        .select("user_id")
        .eq("role", "freelancer");

      if (!roles || roles.length === 0) {
        setLoading(false);
        return;
      }

      const freelancerIds = roles.map(r =&gt; r.user_id);

      // Get profiles, assignments, and payments in parallel
      const [profilesRes, assignmentsRes, paymentsRes] = await Promise.all([
        supabase
          .from("profiles")
          .select("id, full_name, email, avatar_url")
          .in("id", freelancerIds),
        supabase
          .from("freelancer_assignments")
          .select("freelancer_id, project_id, deadline, projects(display_address, name, stage, status, companies(name))")
          .in("freelancer_id", freelancerIds),
        supabase
          .from("freelancer_payments")
          .select("freelancer_id, agreed_amount, paid_amount, status, due_date, project_id, projects(display_address, name)")
          .in("freelancer_id", freelancerIds),
      ]);

      const profiles = profilesRes.data || [];
      const assignments = assignmentsRes.data || [];
      const payments = paymentsRes.data || [];

      const freelancerData = profiles.map(profile =&gt; {
        const myAssignments = assignments.filter((a: any) =&gt; a.freelancer_id === profile.id);
        const myPayments = payments.filter((p: any) =&gt; p.freelancer_id === profile.id);

        const activeProjects = myAssignments.filter(
          (a: any) =&gt; a.projects?.status !== "completed" &amp;&amp; a.projects?.status !== "cancelled"
        ).length;

        const totalAgreed = myPayments.reduce((s: number, p: any) =&gt; s + Number(p.agreed_amount || 0), 0);
        const totalPaid = myPayments.reduce((s: number, p: any) =&gt; s + Number(p.paid_amount || 0), 0);

        return {
          ...profile,
          assignments: myAssignments,
          payments: myPayments,
          activeProjects,
          totalAgreed,
          totalPaid,
          totalReceivable: Math.max(totalAgreed - totalPaid, 0),
        };
      });

      setFreelancers(freelancerData);
      setLoading(false);
    };

    fetchData();
  }, []);

  if (loading) {
    return &lt;div className="p-6 text-muted-foreground"&gt;Loading...&lt;/div&gt;;
  }

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Freelancers&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;View all freelancers, their assigned projects and payment status&lt;/p&gt;
```

```
&lt;/div&gt;

{/* Summary */}
&lt;div className="grid grid-cols-1 md:grid-cols-3 gap-4"&gt;
  &lt;Card&gt;
    &lt;CardContent className="flex items-center gap-4 p-6"&gt;
      &lt;div className="w-12 h-12 rounded-lg bg-primary/10 flex items-center justify-center"&gt;
        &lt;Users className="h-6 w-6 text-primary" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Total Freelancers&lt;/p&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;{freelancers.length}&lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card&gt;
    &lt;CardContent className="flex items-center gap-4 p-6"&gt;
      &lt;div className="w-12 h-12 rounded-lg bg-chart-2/10 flex items-center justify-center"&gt;
        &lt;DollarSign className="h-6 w-6 text-chart-2" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Total Paid&lt;/p&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;
          ${freelancers.reduce((s, f) =&gt; s + f.totalPaid, 0).toLocaleString(undefined, { minimumFractionDigits: 2 })}
        &lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card&gt;
    &lt;CardContent className="flex items-center gap-4 p-6"&gt;
      &lt;div className="w-12 h-12 rounded-lg bg-destructive/10 flex items-center justify-center"&gt;
        &lt;DollarSign className="h-6 w-6 text-destructive" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Total Receivable&lt;/p&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;
          ${freelancers.reduce((s, f) =&gt; s + f.totalReceivable, 0).toLocaleString(undefined, { minimumFractionDigits: 2 })}
        &lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Freelancer Cards */}
{freelancers.length === 0 ? (
  &lt;Card&gt;
    &lt;CardContent className="p-8 text-center text-muted-foreground"&gt;
      No freelancers registered yet
    &lt;/CardContent&gt;
  &lt;/Card&gt;
) : (
  &lt;div className="space-y-6"&gt;
    {freelancers.map(f =&gt; (
      &lt;Card key={f.id}&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center justify-between"&gt;
            &lt;div className="flex items-center gap-3"&gt;
              &lt;div className="w-10 h-10 rounded-full bg-primary flex items-center justify-center"&gt;
                {f.avatar_url ? (
                  &lt;img src={f.avatar_url} alt="" className="w-10 h-10 rounded-full object-cover" /&gt;
                ) : (
                  &lt;span className="text-primary-foreground font-semibold text-sm"&gt;
                    {f.full_name?.charAt(0)?.toUpperCase() || "F"}
                  &lt;/span&gt;
                )}
              &lt;/div&gt;
              &lt;div&gt;
                &lt;CardTitle className="text-lg"&gt;{f.full_name || "Unnamed"}&lt;/CardTitle&gt;
                &lt;p className="text-xs text-muted-foreground"&gt;{f.email}&lt;/p&gt;
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div className="flex items-center gap-4 text-sm"&gt;
              &lt;div className="text-center"&gt;
                &lt;p className="text-muted-foreground text-xs"&gt;Active&lt;/p&gt;
                &lt;p className="font-bold text-foreground"&gt;{f.activeProjects}&lt;/p&gt;
              &lt;/div&gt;
              &lt;div className="text-center"&gt;
                &lt;p className="text-muted-foreground text-xs"&gt;Paid&lt;/p&gt;
                &lt;p className="font-bold text-chart-2"&gt;${f.totalPaid.toLocaleString(undefined, { minimumFractionDigits: 2 })}&lt;/p&gt
              &lt;/div&gt;
              &lt;div&gt;
                &lt;p className="text-muted-foreground text-xs"&gt;Receivable&lt;/p&gt;
                &lt;p className="font-bold text-destructive"&gt;${f.totalReceivable.toLocaleString(undefined, { minimumFractionDigits: 2 }
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
```

```
&lt;CardContent&gt;
  {/* Assigned Projects */}
  {f.assignments.length &gt; 0 &amp;&amp; (
    &lt;div className="space-y-2"&gt;
      &lt;p className="text-xs font-medium text-muted-foreground uppercase tracking-wider"&gt;Assigned Projects&lt;/p&gt;
      &lt;div className="space-y-2"&gt;
        {f.assignments.map((a: any, i: number) =&gt; {
          const projectLabel = a.projects?.display_address || a.projects?.name || "Unknown";
          const stage = a.projects?.stage || "—";
          const company = a.projects?.companies?.name || "";
          const stageColors: Record&lt;string, string&gt; = {
            "In Progress": "bg-chart-2/15 text-chart-2 border-chart-2/30",
            "Completed": "bg-chart-2/15 text-chart-2 border-chart-2/30",
            "Revision": "bg-chart-5/15 text-chart-5 border-chart-5/30",
            "Delayed": "bg-destructive/15 text-destructive border-destructive/30",
          };
          const badgeClass = stageColors[stage] || "bg-muted text-muted-foreground border-border";

          return (
            &lt;div key={i} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
              &lt;div className="flex items-center gap-2"&gt;
                &lt;FolderOpen className="h-4 w-4 text-muted-foreground" /&gt;
                &lt;div&gt;
                  &lt;span className="text-sm font-medium text-foreground"&gt;{projectLabel}&lt;/span&gt;
                  {company &amp;&amp; &lt;span className="text-xs text-muted-foreground ml-2"&gt;({company})&lt;/span&gt;}
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;div className="flex items-center gap-3"&gt;
                {a.deadline &amp;&amp; (
                  &lt;span className="text-xs text-muted-foreground"&gt;
                    Due: {format(new Date(a.deadline), "MMM dd")}
                  &lt;/span&gt;
                )}
                &lt;Badge className={`${badgeClass} border rounded-full px-3 py-1 text-xs font-semibold`}&gt;
                  {stage}
                &lt;/Badge&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/div&gt;
  )}

  {/* Payments */}
  {f.payments.length &gt; 0 &amp;&amp; (
    &lt;div className="space-y-2 mt-4"&gt;
      &lt;p className="text-xs font-medium text-muted-foreground uppercase tracking-wider"&gt;Payments&lt;/p&gt;
      &lt;div className="space-y-2"&gt;
        {f.payments.map((p: any, i: number) =&gt; {
          const agreed = Number(p.agreed_amount || 0);
          const paid = Number(p.paid_amount || 0);
          const status = paid &lt;= 0 ? "pending" : paid &lt; agreed ? "partial" : "paid";
          const statusConfig: Record&lt;string, { label: string; className: string }&gt; = {
            pending: { label: "Pending", className: "bg-destructive/15 text-destructive border-destructive/30" },
            partial: { label: "Partial", className: "bg-chart-4/15 text-chart-4 border-chart-4/30" },
            paid: { label: "Paid", className: "bg-chart-2/15 text-chart-2 border-chart-2/30" },
          };
          const config = statusConfig[status];
          const projectLabel = p.projects?.display_address || p.projects?.name || "—";

          return (
            &lt;div key={i} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
              &lt;div class="text-sm"&gt;
                &lt;span class="font-medium text-foreground"&gt;{projectLabel}&lt;/span&gt;
                &lt;span class="text-muted-foreground ml-2"&gt;
                  ${agreed.toLocaleString(undefined, { minimumFractionDigits: 2 })}
                &lt;/span&gt;
              &lt;/div&gt;
              &lt;div class="flex items-center gap-3"&gt;
                {p.due_date &amp;&amp; (
                  &lt;span class="text-xs text-muted-foreground"&gt;
                    {format(new Date(p.due_date), "MMM dd")}
                  &lt;/span&gt;
                )}
                &lt;Badge class={`${config.class} border rounded-full px-3 py-1 text-xs font-semibold`}&gt;
                  {config.label}
                &lt;/Badge&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/div&gt;
  )}

  {f.assignments.length === 0 &amp;&amp; f.payments.length === 0 &amp;&amp; (
```

```
                &lt;p className="text-sm text-muted-foreground"&gt;No assignments or payments yet&lt;/p&gt;
              )}
            &lt;/CardContent&gt;
          &lt;/Card&gt;
        ))}
      &lt;/div&gt;
    )}
  &lt;/div&gt;
);
};

export default AdminFreelancers;
```

src/pages/admin/AdminGantt.tsx

```
import { useEffect, useState, useMemo } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { useLanguage } from "@/hooks/useLanguage";
import { ChevronLeft, ChevronRight, GanttChart, CalendarDays } from "lucide-react";
import { format, addDays, startOfWeek, startOfMonth, endOfMonth, differenceInDays, isSameDay, isSameMonth, getDay } from "date-fns";
import { PROJECT_STAGE_COLORS as STAGE_COLOR_MAP, getStageLabel } from "@/constants/projectStages";

const PROJECT_STAGE_COLORS: Record&lt;string, string&gt; = Object.fromEntries(
  Object.entries(STAGE_COLOR_MAP).map(([k, v]) =&gt; [k, v.dot])
);

const AdminGantt = () =&gt; {
  const { t } = useLanguage();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [currentWeekStart, setCurrentWeekStart] = useState(() =&gt; startOfWeek(new Date(), { weekStartsOn: 1 }));
  const [viewMode, setViewMode] = useState&lt;"week" | "month"&gt;("week");
  const [calendarMonth, setCalendarMonth] = useState(new Date());
  const [selectedDate, setSelectedDate] = useState&lt;Date | null&gt;(null);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      const { data } = await supabase
        .from("projects")
        .select("id, stage, start_date, delivery_date, companies(name), street_name, street_number, city, display_address")
        .order("start_date", { ascending: true });
      setProjects(data || []);
    };
    fetch();
  }, []);

  // Gantt logic
  const daysCount = viewMode === "week" ? 7 : 28;
  const days = useMemo(() =&gt; Array.from({ length: daysCount }, (_, i) =&gt; addDays(currentWeekStart, i)), [currentWeekStart, daysCount]);

  const navigate = (dir: number) =&gt; {
    setCurrentWeekStart(prev =&gt; addDays(prev, dir &gt; 0 ? daysCount : -daysCount));
  };

  const rangeEnd = addDays(currentWeekStart, daysCount - 1);

  const visibleProjects = projects.filter(p =&gt; {
    if (!p.start_date &amp;&amp; !p.delivery_date) return false;
    const start = p.start_date ? new Date(p.start_date) : new Date(p.delivery_date);
    const end = p.delivery_date ? new Date(p.delivery_date) : new Date(p.start_date);
    return start &lt;= rangeEnd &amp;&amp; end &gt;= currentWeekStart;
  });

  const getBarStyle = (project: any) =&gt; {
    const start = new Date(project.start_date || project.delivery_date);
    const end = new Date(project.delivery_date || project.start_date);
    const startOffset = Math.max(0, differenceInDays(start, currentWeekStart));
    const endOffset = Math.min(daysCount - 1, differenceInDays(end, currentWeekStart));
    const width = endOffset - startOffset + 1;
    return { left: `${(startOffset / daysCount) * 100}%`, width: `${(width / daysCount) * 100}%` };
  };

  const projectName = (p: any) =&gt; {
    const addr = p.display_address || [p.street_number, p.street_name].filter(Boolean).join(" ");
    return addr || p.companies?.name || "—";
  };

  const todayOffset = differenceInDays(new Date(), currentWeekStart);
  const todayInRange = todayOffset &gt;= 0 &amp;&amp; todayOffset &lt; daysCount;

  // Calendar logic
  const monthStart = startOfMonth(calendarMonth);
  const monthEnd = endOfMonth(calendarMonth);
  const calendarStartDay = getDay(monthStart); // 0=Sun
  const totalDaysInMonth = differenceInDays(monthEnd, monthStart) + 1;
  const calendarDays = Array.from({ length: 42 }, (_, i) =&gt; {
    const day = addDays(monthStart, i - calendarStartDay);
    return day;
  });

  const getProjectsOnDate = (date: Date) =&gt; {
    return projects.filter(p =&gt; {
      if (!p.start_date &amp;&amp; !p.delivery_date) return false;
      const start = p.start_date ? new Date(p.start_date) : new Date(p.delivery_date);
      const end = p.delivery_date ? new Date(p.delivery_date) : new Date(p.start_date);
      return date &gt;= start &amp;&amp; date &lt;= end;
```

```
  });
};

const navigateCalendar = (dir: number) =&gt; {
  setCalendarMonth(prev =&gt; {
    const d = new Date(prev);
    d.setMonth(d.getMonth() + dir);
    return d;
  });
};

const selectedDateProjects = selectedDate ? getProjectsOnDate(selectedDate) : [];

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div&gt;
      &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.gantt")} &amp; Calendar&lt;/h1&gt;
      &lt;p className="text-sm text-muted-foreground"&gt;Visualize project timelines and overlapping work&lt;/p&gt;
    &lt;/div&gt;

    &lt;Tabs defaultValue="gantt"&gt;
      &lt;TabsList&gt;
        &lt;TabsTrigger value="gantt" className="gap-2"&gt;&lt;GanttChart className="h-4 w-4" /&gt;Gantt&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="calendar" className="gap-2"&gt;&lt;CalendarDays className="h-4 w-4" /&gt;Calendar&lt;/TabsTrigger&gt;
      &lt;/TabsList&gt;

      {/* GANTT TAB */}
      &lt;TabsContent value="gantt" className="space-y-4"&gt;
        &lt;div className="flex items-center justify-end gap-2"&gt;
          &lt;Select value={viewMode} onValueChange={v =&gt; setViewMode(v as any)}&gt;
            &lt;SelectTrigger className="w-[140px]"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              &lt;SelectItem value="week"&gt;{t("admin.weekView")}&lt;/SelectItem&gt;
              &lt;SelectItem value="month"&gt;{t("admin.monthView")}&lt;/SelectItem&gt;
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
          &lt;Button variant="outline" size="icon" onClick={() =&gt; navigate(-1)}&gt;&lt;ChevronLeft className="h-4 w-4" /&gt;&lt;/Button&gt;
          &lt;Button variant="outline" size="sm" onClick={() =&gt; setCurrentWeekStart(startOfWeek(new Date(), { weekStartsOn: 1 }))}&gt;{t("ad
          &lt;Button variant="outline" size="icon" onClick={() =&gt; navigate(1)}&gt;&lt;ChevronRight className="h-4 w-4" /&gt;&lt;/Button&gt;
        &lt;/div&gt;

        &lt;Card&gt;
          &lt;CardContent className="p-0 overflow-x-auto"&gt;
            &lt;div className="min-w-[800px]"&gt;
              &lt;div className="flex border-b border-border"&gt;
                &lt;div className="w-[220px] flex-shrink-0 p-3 text-sm font-medium text-muted-foreground border-r border-border"&gt;
                  {t("admin.projects")}
                &lt;/div&gt;
                &lt;div className="flex-1 flex"&gt;
                  {days.map((day, i) =&gt; {
                    const isToday = isSameDay(day, new Date());
                    return (
                      &lt;div key={i} className={`flex-1 text-center py-2 text-xs border-r border-border last:border-r-0 ${isToday ? "bg-primar
                        &lt;div&gt;{format(day, viewMode === "week" ? "EEE" : "dd")}&lt;/div&gt;
                        &lt;div className="font-medium"&gt;{format(day, viewMode === "week" ? "dd" : "MMM")}&lt;/div&gt;
                      &lt;/div&gt;
                    );
                  })}
                &lt;/div&gt;
              &lt;/div&gt;

              {visibleProjects.length === 0 ? (
                &lt;div className="p-8 text-center text-muted-foreground"&gt;{t("admin.noProjects")}&lt;/div&gt;
              ) : (
                visibleProjects.map(p =&gt; {
                  const barStyle = getBarStyle(p);
                  const stageColor = PROJECT_STAGE_COLORS[p.stage] || "bg-muted-foreground";
                  return (
                    &lt;div key={p.id} className="flex border-b border-border last:border-b-0 hover:bg-muted/30"&gt;
                      &lt;div className="w-[220px] flex-shrink-0 p-3 text-sm font-medium text-foreground border-r border-border truncate flex i
                        &lt;span className={`w-2 h-2 rounded-full flex-shrink-0 ${stageColor}`} /&gt;
                        {projectName(p)}
                      &lt;/div&gt;
                      &lt;div className="flex-1 relative h-12"&gt;
                        {todayInRange &amp;&amp; (
                          &lt;div className="absolute top-0 bottom-0 w-0.5 bg-destructive z-10" style={{ left: `${((todayOffset + 0.5) / daysCo
                        )}
                        &lt;div className={`absolute top-2 h-8 rounded-md ${stageColor} opacity-80 hover:opacity-100 transition-opacity flex it
                          &lt;span className="text-xs font-medium text-primary-foreground truncate"&gt;{p.companies?.name || ""}&lt;/span&gt;
                        &lt;/div&gt;
                      &lt;/div&gt;
                    &lt;/div&gt;
                  );
                })
              )}
            &lt;/div&gt;
          &lt;/CardContent&gt;
```

```
  &lt;/Card&gt;

  &lt;div className="flex flex-wrap gap-3"&gt;
    {Object.entries(PROJECT_STAGE_COLORS).map(([stage, color]) =&gt; (
      &lt;div key={stage} className="flex items-center gap-1.5 text-xs text-muted-foreground"&gt;
        &lt;span className={`w-3 h-3 rounded-sm ${color}`} /&gt;
        {getStageLabel(stage)}
      &lt;/div&gt;
    ))}
  &lt;/div&gt;
&lt;/TabsContent&gt;

{/* CALENDAR TAB */}
&lt;TabsContent value="calendar" className="space-y-4"&gt;
  &lt;div className="flex items-center justify-between"&gt;
    &lt;h2 className="text-lg font-semibold text-foreground"&gt;{format(calendarMonth, "MMMM yyyy")}&lt;/h2&gt;
    &lt;div className="flex items-center gap-2"&gt;
      &lt;Button variant="outline" size="icon" onClick={() =&gt; navigateCalendar(-1)}&gt;&lt;ChevronLeft className="h-4 w-4" /&gt;&lt;/B
      &lt;Button variant="outline" size="sm" onClick={() =&gt; { setCalendarMonth(new Date()); setSelectedDate(new Date()); }}&gt;Today&l
      &lt;Button variant="outline" size="icon" onClick={() =&gt; navigateCalendar(1)}&gt;&lt;ChevronRight className="h-4 w-4" /&gt;&lt;/B
    &lt;/div&gt;
  &lt;/div&gt;

  &lt;Card&gt;
    &lt;CardContent className="p-4"&gt;
      {/* Day headers */}
      &lt;div className="grid grid-cols-7 gap-1 mb-1"&gt;
        {["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"].map(d =&gt; (
          &lt;div key={d} className="text-center text-xs font-medium text-muted-foreground py-2"&gt;{d}&lt;/div&gt;
        ))}
      &lt;/div&gt;
      {/* Days grid */}
      &lt;div className="grid grid-cols-7 gap-1"&gt;
        {calendarDays.map((day, i) =&gt; {
          const inMonth = isSameMonth(day, calendarMonth);
          const isToday = isSameDay(day, new Date());
          const isSelected = selectedDate &amp;&amp; isSameDay(day, selectedDate);
          const dayProjects = getProjectsOnDate(day);
          const hasProjects = dayProjects.length &gt; 0;

          return (
            &lt;button
              key={i}
              onClick={() =&gt; setSelectedDate(day)}
              className={`relative p-2 h-16 rounded-md text-sm transition-colors text-left
                ${!inMonth ? "text-muted-foreground/40" : "text-foreground"}
                ${isToday ? "ring-2 ring-primary" : ""}
                ${isSelected ? "bg-primary/10" : "hover:bg-muted/50"}
              `}
            &gt;
              &lt;span className={`text-xs ${isToday ? "font-bold text-primary" : ""}`}&gt;{format(day, "d")}&lt;/span&gt;
              {hasProjects &amp;&amp; inMonth &amp;&amp; (
                &lt;div className="flex gap-0.5 mt-1 flex-wrap"&gt;
                  {dayProjects.slice(0, 3).map((p, j) =&gt; (
                    &lt;span key={j} className={`w-2 h-2 rounded-full ${PROJECT_STAGE_COLORS[p.stage] || "bg-muted-foreground"}`} /&gt;
                  ))}
                  {dayProjects.length &gt; 3 &amp;&amp; &lt;span className="text-[10px] text-muted-foreground"&gt;+{dayProjects.length -
                &lt;/div&gt;
              )}
            &lt;/button&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;

  {/* Selected date details */}
  {selectedDate &amp;&amp; (
    &lt;Card&gt;
      &lt;CardContent className="p-4"&gt;
        &lt;h3 className="font-semibold text-foreground mb-3"&gt;{format(selectedDate, "EEEE, MMMM d, yyyy")}&lt;/h3&gt;
        {selectedDateProjects.length === 0 ? (
          &lt;p className="text-sm text-muted-foreground"&gt;No active projects on this date&lt;/p&gt;
        ) : (
          &lt;div className="space-y-2"&gt;
            {selectedDateProjects.map(p =&gt; (
              &lt;div key={p.id} className="flex items-center justify-between p-3 rounded-lg bg-muted/50"&gt;
                &lt;div className="flex items-center gap-2"&gt;
                  &lt;span className={`w-2.5 h-2.5 rounded-full ${PROJECT_STAGE_COLORS[p.stage] || "bg-muted-foreground"}`} /&gt;
                  &lt;span className="font-medium text-sm text-foreground"&gt;{projectName(p)}&lt;/span&gt;
                &lt;/div&gt;
                &lt;div className="flex items-center gap-3 text-xs text-muted-foreground"&gt;
                  &lt;span&gt;{getStageLabel(p.stage || "—")}&lt;/span&gt;
                  &lt;span&gt;{p.companies?.name}&lt;/span&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            ))}
```

```
                    &lt;p className="text-xs text-muted-foreground mt-2"&gt;{selectedDateProjects.length} project(s) active&lt;/p&gt;
                  &lt;/div&gt;
                )}
              &lt;/CardContent&gt;
            &lt;/Card&gt;
          )}
        &lt;/TabsContent&gt;
      &lt;/Tabs&gt;
    &lt;/div&gt;
  );
};

export default AdminGantt;
```

src/pages/admin/AdminLeads.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuTrigger } from "@/components/ui/dropdown-menu";
import { Search, MoreHorizontal, UserPlus, Send, Phone, Inbox, CheckCircle } from "lucide-react";
import { formatDistanceToNow } from "date-fns";
import { toast } from "sonner";

const AdminLeads = () =&gt; {
  const [leads, setLeads] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");

  const fetchLeads = async () =&gt; {
    const { data } = await supabase.from("quote_requests").select("*").order("created_at", { ascending: false });
    setLeads(data || []);
  };

  useEffect(() =&gt; { fetchLeads(); }, []);

  const handleConvert = async (lead: any) =&gt; {
    const { data: company, error: compErr } = await supabase
      .from("companies").insert({ name: lead.company_name || lead.full_name, email: lead.email, phone: lead.phone }).select().single();
    if (compErr || !company) { toast.error("Failed to create client"); return; }
    const { error: projErr } = await supabase.from("projects").insert({
      company_id: company.id, name: `${lead.full_name} - ${lead.service_type}`,
      service_type: lead.service_type, stage: "Lead", description: lead.description, address: lead.project_address,
    });
    if (!projErr) {
      await supabase.from("quote_requests").update({ processed: true }).eq("id", lead.id);
      toast.success("Lead converted to project"); fetchLeads();
    } else toast.error("Failed to create project");
  };

  const serviceLabel = (s: string) =&gt; s?.replace(/_/g, " ").replace(/\b\w/g, (c) =&gt; c.toUpperCase()) || "—";

  const filtered = leads.filter(l =&gt;
    (l.full_name || "").toLowerCase().includes(search.toLowerCase()) ||
    (l.email || "").toLowerCase().includes(search.toLowerCase())
  );

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Leads&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Incoming leads from the intake form&lt;/p&gt;
        &lt;/div&gt;
        &lt;Badge variant="secondary"&gt;{leads.filter(l =&gt; !l.processed).length} new&lt;/Badge&gt;
      &lt;/div&gt;
      &lt;div className="relative max-w-xs"&gt;
        &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
        &lt;Input placeholder="Search leads..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Name&lt;/TableHead&gt;
                &lt;TableHead&gt;Email&lt;/TableHead&gt;
                &lt;TableHead&gt;Phone&lt;/TableHead&gt;
                &lt;TableHead&gt;Service Type&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Received&lt;/TableHead&gt;
                &lt;TableHead className="w-[50px]"&gt;&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {filtered.length === 0 &amp;amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={7} className="text-center text-muted-foreground py-12"&gt;&lt;Inbox className="h-8 w-8 mx-
              )}
              {filtered.map(lead =&gt; (
                &lt;TableRow key={lead.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{lead.full_name}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{lead.email}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{lead.phone || "—"}&lt;/TableCell&gt;
                  &lt;TableCell&gt;{serviceLabel(lead.service_type)}&lt;/TableCell&gt;
                  &lt;TableCell&gt;
                    {lead.processed
                      ? &lt;Badge variant="secondary"&gt;&lt;CheckCircle className="h-3 w-3 mr-1" /&gt;Processed&lt;/Badge&gt;
```

```
                    : &lt;Badge className="bg-blue-500/10 text-blue-600 border-blue-500/20" variant="outline"&gt;New&lt;/Badge&gt;}
                &lt;/TableCell&gt;
                &lt;TableCell className="text-xs text-muted-foreground"&gt;{formatDistanceToNow(new Date(lead.created_at), { addSuffix: true })
                &lt;TableCell&gt;
                  &lt;DropdownMenu&gt;
                    &lt;DropdownMenuTrigger asChild&gt;&lt;Button variant="ghost" size="icon" className="h-7 w-7"&gt;&lt;MoreHorizontal classNa
                    &lt;DropdownMenuContent align="end"&gt;
                      &lt;DropdownMenuItem onClick={() =&gt; handleConvert(lead)} className="gap-2"&gt;&lt;UserPlus className="h-3.5 w-3.5" /&g
                      &lt;DropdownMenuItem onClick={() =&gt; window.open(`mailto:${lead.email}?subject=Proposal`, "_blank")} className="gap-2"&
                      &lt;DropdownMenuItem onClick={() =&gt; lead.phone &amp;&amp; window.open(`tel:${lead.phone}`)} className="gap-2"&gt;&lt;P
                    &lt;/DropdownMenuContent&gt;
                  &lt;/DropdownMenu&gt;
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
  );
};

export default AdminLeads;
```

src/pages/admin/AdminLogin.tsx

```
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { z } from "zod";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { toast } from "sonner";
import { useAuth } from "@/hooks/useAuth";
import { useAdminRole } from "@/hooks/useAdminRole";
import { lovable } from "@/integrations/lovable/index";
import daraLogo from "@/assets/dara-logo.jpg";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Card, CardContent } from "@/components/ui/card";
import { Separator } from "@/components/ui/separator";
import { Loader2, Mail, Lock } from "lucide-react";

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string().min(6),
});

type LoginFormData = z.infer&lt;typeof loginSchema&gt;;

const AdminLogin = () =&gt; {
  const [isLoading, setIsLoading] = useState(false);
  const { user, signIn } = useAuth();
  const { isAdmin, loading: roleLoading } = useAdminRole();
  const navigate = useNavigate();

  useEffect(() =&gt; {
    if (user &amp;&amp; !roleLoading &amp;&amp; isAdmin) {
      navigate("/adm");
    }
  }, [user, isAdmin, roleLoading, navigate]);

  const form = useForm&lt;LoginFormData&gt;({
    resolver: zodResolver(loginSchema),
    defaultValues: { email: "", password: "" },
  });

  const handleLogin = async (data: LoginFormData) =&gt; {
    setIsLoading(true);
    try {
      const { error } = await signIn(data.email, data.password);
      if (error) {
        toast.error("Credenciais inválidas.");
      }
    } catch {
      toast.error("Erro inesperado.");
    } finally {
      setIsLoading(false);
    }
  };

  const handleGoogleLogin = async () =&gt; {
    setIsLoading(true);
    const { error } = await lovable.auth.signInWithOAuth("google", {
      redirect_uri: window.location.origin,
    });
    if (error) { toast.error("Erro ao entrar com Google"); setIsLoading(false); }
  };

  return (
    &lt;div className="min-h-screen flex items-center justify-center bg-muted/30"&gt;
      &lt;div className="w-full max-w-sm mx-auto px-4"&gt;
        {/* Logo &amp; Brand */}
        &lt;div className="text-center mb-8"&gt;
          &lt;img src={daraLogo} alt="DARA Studio" className="h-14 w-14 rounded-lg mx-auto mb-4" /&gt;
          &lt;h1 className="font-serif text-lg font-semibold text-primary"&gt;
            &lt;span className="font-bold"&gt;DARA Studio&lt;/span&gt; | Drafting &amp; 3D Support
          &lt;/h1&gt;
          &lt;p className="text-xs text-muted-foreground mt-1"&gt;Painel Administrativo&lt;/p&gt;
        &lt;/div&gt;

        &lt;Card&gt;
          &lt;CardContent className="p-6 space-y-4"&gt;
            &lt;Button variant="outline" className="w-full" onClick={handleGoogleLogin} disabled={isLoading}&gt;
              &lt;svg className="w-5 h-5 mr-2" viewBox="0 0 24 24"&gt;&lt;path d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92a5.06 5.06 0 01
              Entrar com Google
            &lt;/Button&gt;

            &lt;div className="relative"&gt;
              &lt;div className="absolute inset-0 flex items-center"&gt;&lt;Separator /&gt;&lt;/div&gt;
```

```
              &lt;div className="relative flex justify-center text-xs uppercase"&gt;&lt;span className="bg-card px-2 text-muted-foreground"&gt;ou
            &lt;/div&gt;

            &lt;form onSubmit={form.handleSubmit(handleLogin)} className="space-y-4"&gt;
              &lt;div className="space-y-2"&gt;
                &lt;Label className="text-xs"&gt;Email&lt;/Label&gt;
                &lt;div className="relative"&gt;
                  &lt;Mail className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
                  &lt;Input type="email" className="pl-10" placeholder="admin@darastudio.com" {...form.register("email")} /&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;div className="space-y-2"&gt;
                &lt;Label className="text-xs"&gt;Senha&lt;/Label&gt;
                &lt;div className="relative"&gt;
                  &lt;Lock className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
                  &lt;Input type="password" className="pl-10" placeholder="••••••••" {...form.register("password")} /&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;Button type="submit" className="w-full" disabled={isLoading}&gt;
                {isLoading ? &lt;&gt;&lt;Loader2 className="mr-2 h-4 w-4 animate-spin" /&gt;Entrando...&lt;/&gt; : "Entrar"}
              &lt;/Button&gt;
            &lt;/form&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;

        &lt;p className="text-center text-xs text-muted-foreground mt-6"&gt;
          Acesso restrito a administradores
        &lt;/p&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminLogin;
```

src/pages/admin/AdminMap.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Badge } from "@/components/ui/badge";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { MapPin } from "lucide-react";
import { useLanguage } from "@/hooks/useLanguage";

const STAGE_COLORS: Record&lt;string, string&gt; = {
  "Lead": "#6b7280",
  "Budget Sent": "#3b82f6",
  "Budget Approved": "#10b981",
  "Waiting Payment": "#f59e0b",
  "In Progress": "#8b5cf6",
  "Preview Sent": "#06b6d4",
  "Revision": "#f97316",
  "Final Review": "#14b8a6",
  "Completed": "#22c55e",
  "On Hold": "#9ca3af",
  "Cancelled": "#dc2626",
  "Delayed": "#ef4444",
};

let MapContainer: any, TileLayer: any, Marker: any, Popup: any, L: any;

const AdminMap = () =&gt; {
  const { t } = useLanguage();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [mapReady, setMapReady] = useState(false);
  const [selectedCity, setSelectedCity] = useState&lt;string | null&gt;(null);

  useEffect(() =&gt; {
    Promise.all([
      import("react-leaflet"),
      import("leaflet"),
      import("leaflet/dist/leaflet.css"),
    ]).then(([rl, leaflet]) =&gt; {
      MapContainer = rl.MapContainer;
      TileLayer = rl.TileLayer;
      Marker = rl.Marker;
      Popup = rl.Popup;
      L = leaflet.default;
      setMapReady(true);
    });
  }, []);

  useEffect(() =&gt; {
    const fetchProjects = async () =&gt; {
      const { data } = await supabase
        .from("projects")
        .select("*, companies(name)")
        .not("latitude", "is", null)
        .not("longitude", "is", null);
      setProjects(data || []);
    };
    fetchProjects();
  }, []);

  const createIcon = (color: string) =&gt;
    L.divIcon({
      className: "",
      html: `&lt;div style="width:24px;height:24px;border-radius:50%;background:${color};border:3px solid white;box-shadow:0 2px 6px rgba(0,0,0,0
      iconSize: [24, 24],
      iconAnchor: [12, 12],
    });

  // Massachusetts center
  const maCenter: [number, number] = [42.4072, -71.3824];

  // Get unique cities
  const cities = [...new Set(projects.map(p =&gt; p.city).filter(Boolean))].sort();
  const displayedProjects = selectedCity ? projects.filter(p =&gt; p.city === selectedCity) : projects;

  if (!mapReady) {
    return (
      &lt;div className="p-6 space-y-4"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.mapView")}&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;{t("admin.loadingMap")}&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    );
  }
```

```
  if (projects.length === 0) {
    return (
      &lt;div className="p-6 space-y-4"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.mapView")}&lt;/h1&gt;
        &lt;/div&gt;
        &lt;Card&gt;
          &lt;CardContent className="py-12 text-center"&gt;
            &lt;MapPin className="h-12 w-12 mx-auto text-muted-foreground mb-4" /&gt;
            &lt;p className="text-muted-foreground"&gt;{t("admin.noProjectsWithLocation")}&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
    );
  }

  return (
    &lt;div className="p-6 space-y-4"&gt;
      &lt;div&gt;
        &lt;h1 className="text-2xl font-bold text-foreground"&gt;{t("admin.mapView")}&lt;/h1&gt;
        &lt;p className="text-muted-foreground"&gt;{displayedProjects.length} {t("admin.projectsWithLocation")}&lt;/p&gt;
      &lt;/div&gt;

      {/* City filter buttons */}
      &lt;div className="flex flex-wrap gap-2"&gt;
        &lt;Button
          variant={selectedCity === null ? "default" : "outline"}
          size="sm"
          onClick={() =&gt; setSelectedCity(null)}
        &gt;
          {t("admin.allCities")} ({projects.length})
        &lt;/Button&gt;
        {cities.map(city =&gt; {
          const count = projects.filter(p =&gt; p.city === city).length;
          return (
            &lt;Button
              key={city}
              variant={selectedCity === city ? "default" : "outline"}
              size="sm"
              onClick={() =&gt; setSelectedCity(city)}
            &gt;
              {city} ({count})
            &lt;/Button&gt;
          );
        })}
      &lt;/div&gt;

      &lt;div className="flex flex-wrap gap-2 mb-2"&gt;
        {Object.entries(STAGE_COLORS).map(([stage, color]) =&gt; (
          &lt;div key={stage} className="flex items-center gap-1.5 text-xs text-muted-foreground"&gt;
            &lt;div className="w-3 h-3 rounded-full" style={{ background: color }} /&gt;
            {stage}
          &lt;/div&gt;
        ))}
      &lt;/div&gt;

      &lt;div className="rounded-lg border overflow-hidden" style={{ height: "calc(100vh - 300px)" }}&gt;
        &lt;MapContainer center={maCenter} zoom={9} style={{ height: "100%", width: "100%" }}&gt;
          &lt;TileLayer
            attribution='&amp;copy; &lt;a href="https://www.openstreetmap.org/copyright"&gt;OpenStreetMap&lt;/a&gt;'
            url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
          /&gt;
          {displayedProjects.map((p) =&gt; (
            &lt;Marker
              key={p.id}
              position={[p.latitude, p.longitude]}
              icon={createIcon(STAGE_COLORS[p.stage] || "#6b7280")}
            &gt;
              &lt;Popup&gt;
                &lt;div className="space-y-1"&gt;
                  &lt;p className="font-semibold"&gt;{[p.street_number, p.street_name].filter(Boolean).join(" ") || t("admin.noAddress")}&lt;/p&g
                  &lt;p class
                  {p.city &amp;&amp; &lt;p class
                  &lt;Badge variant="outline"&gt;{p.stage || "—"}&lt;/Badge&gt;
                  {p.total_value &gt; 0 &amp;&amp; (
                    &lt;p class
                  )}
                &lt;/div&gt;
              &lt;/Popup&gt;
            &lt;/Marker&gt;
          ))}
        &lt;/MapContainer&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};
```

```
export default AdminMap;
```

src/pages/admin/AdminMilestones.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Plus } from "lucide-react";
import { toast } from "sonner";

const MILESTONE_NAMES = ["Preview 01", "Preview 02", "Revision 01", "Revision 02", "Final Delivery"];
const STATUSES = ["open", "sent", "approved"];

const AdminMilestones = () =&gt; {
  const [milestones, setMilestones] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [form, setForm] = useState({ project_id: "", name: "Preview 01", status: "open", sent_date: "", approved_date: "", notes: "" });

  const fetchData = async () =&gt; {
    const [msRes, projRes] = await Promise.all([
      supabase.from("milestones").select("*, projects(name)").order("created_at", { ascending: false }),
      supabase.from("projects").select("id, name").order("name"),
    ]);
    setMilestones(msRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  const handleSave = async () =&gt; {
    if (!form.project_id) { toast.error("Selecione um projeto"); return; }
    const { error } = await supabase.from("milestones").insert({
      project_id: form.project_id, name: form.name, status: form.status,
      sent_date: form.sent_date || null, approved_date: form.approved_date || null,
      notes: form.notes || null,
    });
    if (error) { toast.error(error.message); return; }
    toast.success("Milestone criado");
    setDialogOpen(false); fetchData();
  };

  const updateStatus = async (id: string, status: string) =&gt; {
    const update: any = { status };
    if (status === "sent") update.sent_date = new Date().toISOString().split("T")[0];
    if (status === "approved") update.approved_date = new Date().toISOString().split("T")[0];
    await supabase.from("milestones").update(update).eq("id", id);
    fetchData();
  };

  const statusBadge = (s: string) =&gt; {
    if (s === "approved") return &lt;Badge className="bg-chart-2/10 text-chart-2 border-chart-2/20"&gt;Aprovado&lt;/Badge&gt;;
    if (s === "sent") return &lt;Badge variant="secondary"&gt;Enviado&lt;/Badge&gt;;
    return &lt;Badge variant="outline"&gt;Aberto&lt;/Badge&gt;;
  };

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Milestones&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Acompanhamento de entregas por projeto&lt;/p&gt;
        &lt;/div&gt;
        &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
          &lt;DialogTrigger asChild&gt;&lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;Novo Milestone&lt;/Button&gt;&lt;/DialogTrigger&gt;
          &lt;DialogContent&gt;
            &lt;DialogHeader&gt;&lt;DialogTitle&gt;Novo Milestone&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
            &lt;div className="space-y-4"&gt;
              &lt;div&gt;&lt;Label&gt;Projeto *&lt;/Label&gt;
                &lt;Select value={form.project_id} onValueChange={v =&gt; setForm({ ...form, project_id: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Selecione" /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{projects.map(p =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{p.name}&lt;/SelectItem&gt;)}&lt;/SelectCo
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;&lt;Label&gt;Milestone&lt;/Label&gt;
                &lt;Select value={form.name} onValueChange={v =&gt; setForm({ ...form, name: v })}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{MILESTONE_NAMES.map(n =&gt; &lt;SelectItem key={n} value={n}&gt;{n}&lt;/SelectItem&gt;)}&lt;/SelectConter
                &lt;/Select&gt;
              &lt;/div&gt;
```

```
            &lt;div&gt;&lt;Label&gt;Notas&lt;/Label&gt;&lt;Textarea value={form.notes} onChange={e =&gt; setForm({ ...form, notes: e.target.val
            &lt;Button className="w-full" onClick={handleSave}&gt;Criar&lt;/Button&gt;
          &lt;/div&gt;
        &lt;/DialogContent&gt;
      &lt;/Dialog&gt;
    &lt;/div&gt;

    &lt;Card&gt;
      &lt;CardContent className="p-0"&gt;
        &lt;Table&gt;
          &lt;TableHeader&gt;
            &lt;TableRow&gt;
              &lt;TableHead&gt;Projeto&lt;/TableHead&gt;
              &lt;TableHead&gt;Milestone&lt;/TableHead&gt;
              &lt;TableHead&gt;Status&lt;/TableHead&gt;
              &lt;TableHead&gt;Enviado&lt;/TableHead&gt;
              &lt;TableHead&gt;Aprovado&lt;/TableHead&gt;
              &lt;TableHead&gt;Ações&lt;/TableHead&gt;
            &lt;/TableRow&gt;
          &lt;/TableHeader&gt;
          &lt;TableBody&gt;
            {milestones.map(m =&gt; (
              &lt;TableRow key={m.id}&gt;
                &lt;TableCell className="font-medium"&gt;{m.projects?.name}&lt;/TableCell&gt;
                &lt;TableCell&gt;{m.name}&lt;/TableCell&gt;
                &lt;TableCell&gt;{statusBadge(m.status)}&lt;/TableCell&gt;
                &lt;TableCell className="text-muted-foreground"&gt;{m.sent_date || "—"}&lt;/TableCell&gt;
                &lt;TableCell className="text-muted-foreground"&gt;{m.approved_date || "—"}&lt;/TableCell&gt;
                &lt;TableCell&gt;
                  &lt;div className="flex gap-1"&gt;
                    {m.status === "open" &amp;&amp; &lt;Button variant="outline" size="sm" onClick={() =&gt; updateStatus(m.id, "sent")}&gt;Env
                    {m.status === "sent" &amp;&amp; &lt;Button variant="outline" size="sm" onClick={() =&gt; updateStatus(m.id, "approved")}&gt
                  &lt;/div&gt;
                &lt;/TableCell&gt;
              &lt;/TableRow&gt;
            ))}
            {milestones.length === 0 &amp;&amp; (
              &lt;TableRow&gt;&lt;TableCell colSpan={6} className="text-center text-muted-foreground py-8"&gt;Nenhum milestone&lt;/TableCell&gt
            )}
          &lt;/TableBody&gt;
        &lt;/Table&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;
  &lt;/div&gt;
  );
};

export default AdminMilestones;
```

src/pages/admin/AdminPayments.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Plus } from "lucide-react";
import { toast } from "sonner";
import { Separator } from "@/components/ui/separator";
import { useLanguage } from "@/hooks/useLanguage";

const INSTALLMENT_TYPES = ["Entry", "Final", "Extra"];
const PAYMENT_METHODS = ["Stripe", "Nomad", "Wise", "Transfer"];
const PRICE_TABLES = ["W/ Stripe (1.515)", "W/O Stripe (1.31)", "Full", "REL", "Partner"];
const STATUSES = ["pending", "partial", "paid"];

const AdminPayments = () =&gt; {
  const { t } = useLanguage();
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [form, setForm] = useState({
    project_id: "", installment_type: "Entry", amount_total: "", amount_paid: "",
    status: "pending", payment_method: "Stripe", stripe_fee_percent: "2.9", payment_date: "",
    value_brl: "", price_table: "", third_party_costs: "", payment_notes: "",
  });

  const fetchData = async () =&gt; {
    const [payRes, projRes] = await Promise.all([
      supabase.from("admin_payments").select("*, projects(name, display_address, total_value, entry_payment_percent, currency, street_name, stree
      supabase.from("projects").select("id, name, display_address, total_value, entry_payment_percent, currency, street_name, street_number, comp
    ]);
    setPayments(payRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  // Realtime subscription for payment changes (admin_payments + payment_milestones)
  useEffect(() =&gt; {
    const channel = supabase
      .channel('admin-payments-changes')
      .on('postgres_changes', { event: '*', schema: 'public', table: 'admin_payments' }, () =&gt; {
        fetchData();
      })
      .on('postgres_changes', { event: '*', schema: 'public', table: 'payment_milestones' }, () =&gt; {
        fetchData();
      })
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, []);

  const selectedProject = projects.find(p =&gt; p.id === form.project_id);
  const projectTotal = Number(selectedProject?.total_value || 0);
  const entryPct = Number(selectedProject?.entry_payment_percent || 50);
  const entryAmount = projectTotal * entryPct / 100;
  const finalAmount = projectTotal - entryAmount;

  const handleProjectSelect = (projectId: string) =&gt; {
    const proj = projects.find(p =&gt; p.id === projectId);
    const total = Number(proj?.total_value || 0);
    const ePct = Number(proj?.entry_payment_percent || 50);
    const suggestedAmount = form.installment_type === "Entry" ? (total * ePct / 100) : form.installment_type === "Final" ? (total - total * ePct
    setForm(f =&gt; ({ ...f, project_id: projectId, amount_total: suggestedAmount.toString() }));
  };

  const handleTypeChange = (type: string) =&gt; {
    const suggestedAmount = type === "Entry" ? entryAmount : type === "Final" ? finalAmount : 0;
    setForm(f =&gt; ({ ...f, installment_type: type, amount_total: suggestedAmount.toString() }));
  };

  const handleSave = async () =&gt; {
    if (!form.project_id) { toast.error(t("admin.selectProject")); return; }
    const { error } = await supabase.from("admin_payments").insert({
      project_id: form.project_id, installment_type: form.installment_type,
      amount_total: Number(form.amount_total) || 0, amount_paid: Number(form.amount_paid) || 0,
      status: form.status, payment_method: form.payment_method,
      stripe_fee_percent: Number(form.stripe_fee_percent) || 0,
      payment_date: form.payment_date || null,
      value_brl: form.value_brl ? Number(form.value_brl) : null,
```

```
    price_table: form.price_table || null,
    third_party_costs: form.third_party_costs ? Number(form.third_party_costs) : 0,
    payment_notes: form.payment_notes || null,
  });
  if (error) { toast.error(error.message); return; }
  await updateProjectPaymentStage(form.project_id);
  toast.success(t("admin.paymentRegistered"));
  setDialogOpen(false); fetchData();
};

const updateProjectPaymentStage = async (projectId: string) =&gt; {
  const proj = projects.find(p =&gt; p.id === projectId);
  if (!proj) return;
  const { data: allPayments } = await supabase.from("admin_payments").select("amount_paid, status").eq("project_id", projectId);
  const totalPaid = (allPayments || []).filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const total = Number(proj.total_value || 0);
  const ePct = Number(proj.entry_payment_percent || 50);
  const entry = total * ePct / 100;

  let paymentStage = "Not Sent";
  if (totalPaid &gt;= total &amp;&amp; total &gt; 0) paymentStage = "Paid";
  else if (totalPaid &gt;= entry &amp;&amp; entry &gt; 0) paymentStage = "Entry Paid";
  else if (totalPaid &gt; 0) paymentStage = "Waiting Final";

  await supabase.from("projects").update({ payment_stage: paymentStage }).eq("id", projectId);
};

// Compute per-project financial summaries from projects data
const projectFinancials = projects.map(proj =&gt; {
  const total = Number(proj.total_value || 0);
  const ePct = Number(proj.entry_payment_percent || 50);
  const entry = total * ePct / 100;
  const final_ = total - entry;
  const projPayments = payments.filter(p =&gt; p.project_id === proj.id);
  const totalPaid = projPayments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
  const balance = total - totalPaid;
  const sym = proj.currency === "BRL" ? "R$" : "$";
  const addr = [proj.street_number, proj.street_name].filter(Boolean).join(" ") || proj.display_address || proj.name || "—";
  const company = proj.companies?.name || "";
  return { ...proj, total, entry, final_, totalPaid, balance, sym, addr, company };
}).filter(p =&gt; p.total &gt; 0);

const totalReceived = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
const totalPending = projectFinancials.reduce((s, p) =&gt; s + p.balance, 0);

const statusBadge = (s: string) =&gt; {
  if (s === "paid") return &lt;Badge className="bg-chart-2/15 text-chart-2 border-chart-2/30"&gt;{t("admin.paidStatus")}&lt;/Badge&gt;;
  if (s === "partial") return &lt;Badge variant="secondary"&gt;{t("admin.partialStatus")}&lt;/Badge&gt;;
  return &lt;Badge variant="outline"&gt;{t("admin.pendingStatus")}&lt;/Badge&gt;;
};

const getProjectLabel = (p: any) =&gt; {
  const addr = [p.street_number, p.street_name].filter(Boolean).join(" ");
  const company = p.companies?.name || "";
  return addr ? `${addr} — ${company}` : p.display_address || p.name || company || "—";
};

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;{t("admin.payments")}&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;{t("admin.financialControl")}&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
        &lt;DialogTrigger asChild&gt;&lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;{t("admin.newPayment")}&lt;/Button&gt;&lt;/DialogTrig
        &lt;DialogContent className="max-w-lg"&gt;
          &lt;DialogHeader&gt;&lt;DialogTitle&gt;{t("admin.registerPayment")}&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            &lt;div&gt;&lt;Label&gt;{t("admin.project")} *&lt;/Label&gt;
              &lt;Select value={form.project_id || "none"} onValueChange={v =&gt; v !== "none" &amp;&amp; handleProjectSelect(v)}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue placeholder={t("admin.select")} /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;{projects.map(p =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{getProjectLabel(p)}&lt;/SelectItem&gt;)}&
              &lt;/Select&gt;
            &lt;/div&gt;

            {selectedProject &amp;&amp; (
              &lt;div className="grid grid-cols-3 gap-2 text-sm"&gt;
                &lt;div className="p-2 rounded bg-primary/10 border border-primary/20"&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;{t("admin.totalValue")}&lt;/p&gt;
                  &lt;p className="font-semibold text-primary"&gt;${projectTotal.toLocaleString()}&lt;/p&gt;
                &lt;/div&gt;
                &lt;div className="p-2 rounded bg-chart-3/10 border border-chart-3/20"&gt;
                  &lt;p className="text-xs text-muted-foreground"&gt;{t("admin.entryAmount")} ({entryPct}%)&lt;/p&gt;
                  &lt;p className="font-semibold text-chart-3"&gt;${entryAmount.toLocaleString()}&lt;/p&gt;
                &lt;/div&gt;
                &lt;div className="p-2 rounded bg-chart-4/10 border border-chart-4/20"&gt;
```

```
              &lt;p className="text-xs text-muted-foreground"&gt;{t("admin.finalAmount")}&lt;/p&gt;
              &lt;p className="font-semibold text-chart-4"&gt;${finalAmount.toLocaleString()}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}

        &lt;div className="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;{t("admin.type")}&lt;/Label&gt;
            &lt;Select value={form.installment_type} onValueChange={handleTypeChange}&gt;
              &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{INSTALLMENT_TYPES.map(t =&gt; &lt;SelectItem key={t} value={t}&gt;{t}&lt;/SelectItem&gt;)}&lt;/SelectCo
            &lt;/Select&gt;
          &lt;/div&gt;
          &lt;div&gt;&lt;Label&gt;{t("admin.method")}&lt;/Label&gt;
            &lt;Select value={form.payment_method} onValueChange={v =&gt; setForm({ ...form, payment_method: v })}&gt;
              &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{PAYMENT_METHODS.map(m =&gt; &lt;SelectItem key={m} value={m}&gt;{m}&lt;/SelectItem&gt;)}&lt;/SelectCont
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div className="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;{t("admin.amountTotal")} ($)&lt;/Label&gt;&lt;Input type="number" value={form.amount_total} onChange={e =
          &lt;div&gt;&lt;Label&gt;{t("admin.amountPaid")} ($)&lt;/Label&gt;&lt;Input type="number" value={form.amount_paid} onChange={e =&g
        &lt;/div&gt;
        &lt;div class="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;{t("admin.stripeFee")} (%)&lt;/Label&gt;&lt;Input type="number" step="0.1" value={form.stripe_fee_percent
          &lt;div&gt;&lt;Label&gt;{t("admin.status")}&lt;/Label&gt;
            &lt;Select value={form.status} onValueChange={v =&gt; setForm({ ...form, status: v })}&gt;
              &lt;SelectTrigger&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;{STATUSES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{s === "pending" ? t("admin.pendingStatus") :
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;{t("admin.paymentDate")}&lt;/Label&gt;&lt;Input type="date" value={form.payment_date} onChange={e =&gt; set

        &lt;Separator /&gt;
        &lt;p className="text-sm font-semibold text-muted-foreground uppercase tracking-wider"&gt;Detalhes Financeiros&lt;/p&gt;
        &lt;div class="grid grid-cols-2 gap-4"&gt;
          &lt;div&gt;&lt;Label&gt;Valor BRL (R$)&lt;/Label&gt;&lt;Input type="number" value={form.value_brl} onChange={e =&gt; setForm({ ..
          &lt;div&gt;&lt;Label&gt;Tabela de Preço&lt;/Label&gt;
            &lt;Select value={form.price_table || "none"} onValueChange={v =&gt; setForm({ ...form, price_table: v === "none" ? "" : v })}&
              &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Selecionar" /&gt;&lt;/SelectTrigger&gt;
              &lt;SelectContent&gt;
                &lt;SelectItem value="none"&gt;Nenhuma&lt;/SelectItem&gt;
                {PRICE_TABLES.map(pt =&gt; &lt;SelectItem key={pt} value={pt}&gt;{pt}&lt;/SelectItem&gt;)}
              &lt;/SelectContent&gt;
            &lt;/Select&gt;
          &lt;/div&gt;
        &lt;/div&gt;
        &lt;div&gt;&lt;Label&gt;Custos com Terceiros (Emily)&lt;/Label&gt;&lt;Input type="number" value={form.third_party_costs} onChange={
        &lt;div&gt;&lt;Label&gt;Observações de Pagamento&lt;/Label&gt;
          &lt;textarea className="w-full min-h-[80px] rounded-md border border-input bg-background px-3 py-2 text-sm" value={form.payment_n
        &lt;/div&gt;
        &lt;Button className="w-full" onClick={handleSave}&gt;{t("admin.register")}&lt;/Button&gt;
      &lt;/div&gt;
    &lt;/DialogContent&gt;
  &lt;/Dialog&gt;
&lt;/div&gt;

&lt;div className="grid grid-cols-1 sm:grid-cols-3 gap-4"&gt;
  &lt;Card className="bg-chart-2/10 border-chart-2/20"&gt;&lt;CardHeader className="pb-2"&gt;&lt;CardTitle className="text-sm text-muted-fo
    &lt;CardContent&gt;&lt;p class="text-2xl font-bold text-chart-2"&gt;${totalReceived.toLocaleString()}&lt;/p&gt;&lt;/CardContent&gt;
  &lt;Card class="bg-destructive/10 border-destructive/20"&gt;&lt;CardHeader class="pb-2"&gt;&lt;CardTitle class="text-sm text-
    &lt;CardContent&gt;&lt;p class="text-2xl font-bold text-destructive"&gt;${totalPending.toLocaleString()}&lt;/p&gt;&lt;/CardContent&
  &lt;Card&gt;&lt;CardHeader class="pb-2"&gt;&lt;CardTitle class="text-sm text-muted-foreground"&gt;{t("admin.records")}&lt;/CardTi
    &lt;CardContent&gt;&lt;p class="text-2xl font-bold text-foreground"&gt;{payments.length}&lt;/p&gt;&lt;/CardContent&gt;&lt;/Card&gt;
&lt;/div&gt;

{/* Project Financial Summary */}
&lt;Card&gt;
  &lt;CardHeader&gt;&lt;CardTitle class="text-sm"&gt;{t("admin.projectFinancials")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
  &lt;CardContent class="p-0"&gt;
    &lt;Table&gt;
      &lt;TableHeader&gt;
        &lt;TableRow&gt;
          &lt;TableHead&gt;{t("admin.project")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.totalValue")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.entryAmount")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.paid")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.balance")}&lt;/TableHead&gt;
        &lt;/TableRow&gt;
      &lt;/TableHeader&gt;
      &lt;TableBody&gt;
        {projectFinancials.map(p =&gt; (
          &lt;TableRow key={p.id}&gt;
            &lt;TableCell class="font-medium"&gt;
              &lt;div&gt;{p.addr}&lt;/div&gt;
```

```
              {p.company &amp;&amp; &lt;div className="text-xs text-muted-foreground"&gt;{p.company}&lt;/div&gt;}
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className="inline-flex items-center px-2 py-0.5 rounded text-xs font-medium bg-primary/10 text-primary border border
                {p.sym}{p.total.toLocaleString()}
              &lt;/span&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className="inline-flex items-center px-2 py-0.5 rounded text-xs font-medium bg-chart-3/10 text-chart-3 border border
                {p.sym}{p.entry.toLocaleString()} (50%)
              &lt;/span&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className="inline-flex items-center px-2 py-0.5 rounded text-xs font-medium bg-chart-2/15 text-chart-2 border border
                {p.sym}{p.totalPaid.toLocaleString()}
              &lt;/span&gt;
            &lt;/TableCell&gt;
            &lt;TableCell&gt;
              &lt;span className={`inline-flex items-center px-2 py-0.5 rounded text-xs font-medium border ${p.balance &gt; 0 ? "bg-destruc
                {p.sym}{p.balance.toLocaleString()}
              &lt;/span&gt;
            &lt;/TableCell&gt;
          &lt;/TableRow&gt;
        )}
        {projectFinancials.length === 0 &amp;&amp; (
          &lt;TableRow&gt;&lt;TableCell colSpan={5} className="text-center text-muted-foreground py-8"&gt;{t("admin.noProjects")}&lt;/Table
        )}
      &lt;/TableBody&gt;
    &lt;/Table&gt;
  &lt;/CardContent&gt;
&lt;/Card&gt;

{/* Payment Records */}
&lt;Card&gt;
  &lt;CardHeader&gt;&lt;CardTitle className="text-sm"&gt;{t("admin.paymentRecords")}&lt;/CardTitle&gt;&lt;/CardHeader&gt;
  &lt;CardContent className="p-0"&gt;
    &lt;Table&gt;
      &lt;TableHeader&gt;
        &lt;TableRow&gt;
          &lt;TableHead&gt;{t("admin.project")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.type")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.total")}&lt;/TableHead&gt;
          &lt;TableHead&gt;BRL&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.paid")}&lt;/TableHead&gt;
          &lt;TableHead&gt;Tabela&lt;/TableHead&gt;
          &lt;TableHead&gt;Terceiros&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.status")}&lt;/TableHead&gt;
          &lt;TableHead&gt;{t("admin.date")}&lt;/TableHead&gt;
          &lt;TableHead&gt;Notas&lt;/TableHead&gt;
        &lt;/TableRow&gt;
      &lt;/TableHeader&gt;
      &lt;TableBody&gt;
        {payments.map(p =&gt; (
          &lt;TableRow key={p.id}&gt;
            &lt;TableCell className="font-medium"&gt;
              {[p.projects?.street_number, p.projects?.street_name].filter(Boolean).join(" ") || p.projects?.display_address || p.projects?
            &lt;/TableCell&gt;
            &lt;TableCell&gt;{p.installment_type}&lt;/TableCell&gt;
            &lt;TableCell&gt;${Number(p.amount_total).toLocaleString()}&lt;/TableCell&gt;
            &lt;TableCell className="text-muted-foreground"&gt;{p.value_brl ? `R$${Number(p.value_brl).toLocaleString()}` : "—"}&lt;/TableC
            &lt;TableCell className="text-chart-2"&gt;${Number(p.amount_paid).toLocaleString()}&lt;/TableCell&gt;
            &lt;TableCell className="text-xs text-muted-foreground"&gt;{p.price_table || "—"}&lt;/TableCell&gt;
            &lt;TableCell className="text-muted-foreground"&gt;{p.third_party_costs ? `R$${Number(p.third_party_costs).toLocaleString()}` :
            &lt;TableCell&gt;
              &lt;Select
                value={p.status}
                onValueChange={async (newStatus) =&gt; {
                  const updatePayload: any = { status: newStatus };
                  if (newStatus === "paid" &amp;&amp; !p.payment_date) {
                    updatePayload.payment_date = new Date().toISOString().split("T")[0];
                    updatePayload.amount_paid = Number(p.amount_total);
                  }
                  if (newStatus === "pending") {
                    updatePayload.amount_paid = 0;
                  }
                  const { error } = await supabase.from("admin_payments").update(updatePayload).eq("id", p.id);
                  if (error) { toast.error(error.message); return; }
                  await updateProjectPaymentStage(p.project_id);
                  toast.success(`Payment marked as ${newStatus}`);
                  fetchData();
                }}
              &gt;
                &lt;SelectTrigger className="h-7 w-[110px] text-xs"&gt;
                  &lt;SelectValue /&gt;
                &lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;
                  &lt;SelectItem value="pending"&gt;{t("admin.pendingStatus")}&lt;/SelectItem&gt;
```

```
                        &lt;SelectItem value="paid"&gt;{t("admin.paidStatus")}&lt;/SelectItem&gt;
                        &lt;SelectItem value="cancelled"&gt;Cancelled&lt;/SelectItem&gt;
                      &lt;/SelectContent&gt;
                    &lt;/Select&gt;
                  &lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{p.payment_date || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-xs text-muted-foreground max-w-[200px] truncate"&gt;{p.payment_notes || "—"}&lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
              {payments.length === 0 &amp;amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={10} className="text-center text-muted-foreground py-8"&gt;{t("admin.noPayments")}&lt;/Tabl
              )}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminPayments;
```

src/pages/admin/AdminPermits.tsx

```
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { ShieldCheck } from "lucide-react";

const AdminPermits = () =&gt; {
  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Permits&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Track permit submissions and approvals&lt;/p&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="flex flex-col items-center justify-center py-16"&gt;
          &lt;ShieldCheck className="h-12 w-12 text-muted-foreground/30 mb-4" /&gt;
          &lt;h3 className="text-lg font-medium text-foreground mb-2"&gt;Permit Tracking&lt;/h3&gt;
          &lt;p className="text-sm text-muted-foreground text-center max-w-md"&gt;
            Track permit submissions, approvals, and inspection schedules for all projects. This module will be connected to project milestones.
          &lt;/p&gt;
          &lt;Badge variant="secondary" className="mt-4"&gt;Coming Soon&lt;/Badge&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminPermits;
```

src/pages/admin/AdminProjectDetail.tsx

```
import { useEffect, useState, useCallback, useRef } from "react";
import { useParams, useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Separator } from "@/components/ui/separator";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Textarea } from "@/components/ui/textarea";
import {
  ArrowLeft, Building2, Calendar, DollarSign, FileImage, FolderOpen,
  MessageSquare, Download, Star, Clock, MapPin, Ruler,
  CreditCard, Receipt, Ticket, Globe, Languages, Mail, Phone, User,
  Plus, Save, Trash2, Lock, Users, Eye, Upload, StickyNote, Monitor,
  CheckCircle2, AlertTriangle, XCircle, ExternalLink, Hash, Camera, HardHat, Pencil,
} from "lucide-react";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { format } from "date-fns";
import { getCountryFlag, getLanguageLabel } from "@/utils/countryFlags";
import { toast } from "sonner";
import ProjectKanbanBoard from "@/components/admin/ProjectKanbanBoard";
import { Progress } from "@/components/ui/progress";
import { PROJECT_STAGES, getStageLabel } from "@/constants/projectStages";
import ProjectTimeline from "@/components/admin/ProjectTimeline";
import ProjectDocuments from "@/components/admin/ProjectDocuments";

const VISIBILITY_OPTIONS = [
  { value: "private", label: "Private", icon: Lock, color: "text-muted-foreground" },
  { value: "freelancer", label: "Team", icon: Users, color: "text-chart-4" },
  { value: "client", label: "Client Visible", icon: Eye, color: "text-chart-2" },
];

interface DateNote {
  date: string;
  note: string;
  visibility: "private" | "freelancer" | "client";
}

// Inline Site Updates Tab component
const SiteUpdatesTab = ({ projectId }: { projectId: string }) =&gt; {
  const [updates, setUpdates] = useState&lt;any[]&gt;([]);
  const [uploading, setUploading] = useState(false);
  const [files, setFiles] = useState&lt;File[]&gt;([]);
  const [description, setDescription] = useState("");

  const fetchUpdates = useCallback(async () =&gt; {
    const { data } = await supabase
      .from("site_updates")
      .select("*")
      .eq("project_id", projectId)
      .order("created_at", { ascending: false });
    setUpdates(data || []);
  }, [projectId]);

  useEffect(() =&gt; { fetchUpdates(); }, [fetchUpdates]);

  useEffect(() =&gt; {
    const channel = supabase
      .channel(`site-updates-${projectId}`)
      .on("postgres_changes", { event: "*", schema: "public", table: "site_updates", filter: `project_id=eq.${projectId}` }, () =&gt; fetchUpdate
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, [projectId, fetchUpdates]);

  const handleUpload = async () =&gt; {
    if (files.length === 0) { toast.error("Select at least one file"); return; }
    setUploading(true);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      for (const file of files) {
        const ext = file.name.split(".").pop();
        const path = `${projectId}/${Date.now()}-${Math.random().toString(36).slice(2)}.${ext}`;
        const { error: uploadError } = await supabase.storage.from("site-updates").upload(path, file);
        if (uploadError) { toast.error(`Upload failed: ${file.name}`); continue; }
        const { data: urlData } = supabase.storage.from("site-updates").getPublicUrl(path);
        await supabase.from("site_updates").insert({
          project_id: projectId,
          uploaded_by: user?.id || null,
          media_url: urlData.publicUrl,
```

```
          description: description || null,
        });
      }
      toast.success(`${files.length} update(s) uploaded`);
      setFiles([]);
      setDescription("");
      fetchUpdates();
    } catch {
      toast.error("Upload failed");
    } finally {
      setUploading(false);
    }
  };

  return (
    &lt;div className="space-y-6"&gt;
      {/* Upload area */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;CardTitle className="text-base flex items-center gap-2"&gt;
            &lt;HardHat className="h-4 w-4" /&gt; Upload Site Update
          &lt;/CardTitle&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-4"&gt;
          &lt;label className="flex flex-col items-center justify-center w-full h-32 border-2 border-dashed rounded-lg cursor-pointer hover:bg-mu
            &lt;div className="flex flex-col items-center"&gt;
              &lt;Camera className="h-8 w-8 text-muted-foreground mb-2" /&gt;
              &lt;p className="text-sm text-muted-foreground"&gt;
                {files.length &gt; 0 ? `${files.length} file(s) selected` : "Tap to take photo or select files"}
              &lt;/p&gt;
            &lt;/div&gt;
            &lt;input type="file" className="hidden" multiple accept="image/*,video/*" capture="environment" onChange={e =&gt; setFiles(Array.fro
          &lt;/label&gt;
          &lt;Textarea value={description} onChange={e =&gt; setDescription(e.target.value)} placeholder="Describe what's happening on site..." /
          &lt;Button onClick={handleUpload} disabled={uploading || files.length === 0}&gt;
            {uploading ? "Uploading..." : "Upload"}
          &lt;/Button&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;

      {/* Timeline */}
      {updates.length === 0 ? (
        &lt;Card className="border-none shadow-sm"&gt;
          &lt;CardContent className="text-center py-12"&gt;
            &lt;HardHat className="h-10 w-10 mx-auto mb-3 text-muted-foreground/40" /&gt;
            &lt;p className="text-sm text-muted-foreground"&gt;No site updates yet&lt;/p&gt;
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      ) : (
        &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4"&gt;
          {updates.map(update =&gt; {
            const isVideo = update.media_url?.match(/\.(mp4|mov|webm|avi)$/i);
            return (
              &lt;Card key={update.id} className="border-none shadow-sm overflow-hidden"&gt;
                &lt;div className="aspect-video bg-muted"&gt;
                  {isVideo ? (
                    &lt;video src={update.media_url} controls className="w-full h-full object-cover" /&gt;
                  ) : (
                    &lt;img src={update.media_url} alt="Site update" className="w-full h-full object-cover" /&gt;
                  )}
                &lt;/div&gt;
                &lt;CardContent className="p-3 space-y-1"&gt;
                  &lt;span className="text-xs text-muted-foreground"&gt;
                    {format(new Date(update.created_at), "MMM d, yyyy · h:mm a")}
                  &lt;/span&gt;
                  {update.description &amp;&amp; &lt;p className="text-sm"&gt;{update.description}&lt;/p&gt;}
                &lt;/CardContent&gt;
              &lt;/Card&gt;
            );
          })}
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  );
};

const AdminProjectDetail = () =&gt; {
  const { id } = useParams&lt;{ id: string }&gt;();
  const navigate = useNavigate();
  const coverInputRef = useRef&lt;HTMLInputElement&gt;(null);
  const [project, setProject] = useState&lt;any&gt;(null);
  const [files, setFiles] = useState&lt;any[]&gt;([]);
  const [drawings, setDrawings] = useState&lt;any[]&gt;([]);
  const [payments, setPayments] = useState&lt;any[]&gt;([]);
  const [messages, setMessages] = useState&lt;any[]&gt;([]);
  const [milestones, setMilestones] = useState&lt;any[]&gt;([]);
  const [tickets, setTickets] = useState&lt;any[]&gt;([]);
```

```
const [services, setServices] = useState&lt;any[]&gt;([]);
const [allServices, setAllServices] = useState&lt;any[]&gt;([]);
const [selectedServiceIds, setSelectedServiceIds] = useState&lt;string[]&gt;([]);
const [activityLog, setActivityLog] = useState&lt;any[]&gt;([]);
const [paymentMilestones, setPaymentMilestones] = useState&lt;any[]&gt;([]);
const [homeowners, setHomeowners] = useState&lt;any[]&gt;([]);
const [loading, setLoading] = useState(true);
const [hasChanges, setHasChanges] = useState(false);
const [saving, setSaving] = useState(false);

// Editable project fields
const [form, setForm] = useState&lt;any&gt;({});
const [dateNotes, setDateNotes] = useState&lt;DateNote[]&gt;([]);
const [internalNotes, setInternalNotes] = useState("");

const loadData = useCallback(async () =&gt; {
  if (!id) return;
  const [proj, f, d, pay, msg, ms, tk, svc, allSvc, psvc, logs, pmRes, hoRes] = await Promise.all([
    supabase.from("projects").select("*, companies(*)").eq("id", id).single(),
    supabase.from("project_files").select("*").eq("project_id", id).order("created_at", { ascending: false }),
    supabase.from("drawing_versions").select("*").eq("project_id", id).order("drawing_name").order("version_number", { ascending: false }),
    supabase.from("admin_payments").select("*").eq("project_id", id).order("created_at", { ascending: false }),
    supabase.from("messages").select("*").eq("project_id", id).order("created_at", { ascending: true }),
    supabase.from("milestones").select("*").eq("project_id", id).order("created_at"),
    supabase.from("client_tickets").select("*, companies(name)").eq("project_id", id).order("updated_at", { ascending: false }),
    supabase.from("project_services").select("service_id, services(name, category)").eq("project_id", id),
    supabase.from("services").select("*").order("category").order("name"),
    supabase.from("project_services").select("service_id").eq("project_id", id),
    supabase.from("activity_log").select("*").eq("project_id", id).order("log_date", { ascending: false }),
    supabase.from("payment_milestones").select("*").eq("project_id", id).order("created_at"),
    supabase.from("project_homeowners").select("*").eq("project_id", id).order("created_at"),
  ]);

  const p = proj.data;
  setProject(p);
  setFiles(f.data || []);
  setDrawings(d.data || []);
  setPayments(pay.data || []);
  setMessages(msg.data || []);
  setMilestones(ms.data || []);
  setTickets(tk.data || []);
  setServices((svc.data || []).map((s: any) =&gt; s.services).filter(Boolean));
  setAllServices(allSvc.data || []);
  setSelectedServiceIds((psvc.data || []).map((s: any) =&gt; s.service_id));
  setActivityLog(logs.data || []);
  setPaymentMilestones(pmRes.data || []);
  setHomeowners(hoRes.data || []);
  if (p) {
    setForm({
      stage: p.stage || "Lead",
      service_type: p.service_type || "",
      square_feet: p.square_feet?.toString() || "",
      start_date: p.start_date || "",
      delivery_date: p.delivery_date || "",
      preview_delivery_date: p.preview_delivery_date || "",
      revision_return_date: p.revision_return_date || "",
      final_delivery_date: p.final_delivery_date || "",
      total_value: p.total_value?.toString() || "",
      currency: p.currency || "USD",
      entry_payment_percent: p.entry_payment_percent?.toString() || "50",
      payment_stage: p.payment_stage || "Not Sent",
      street_number: p.street_number || "",
      street_name: p.street_name || "",
      city: p.city || "",
      state: p.state || "",
      zip_code: p.zip_code || "",
      country: p.country || "USA",
      homeowner_name: (p as any).homeowner_name || "",
      homeowner_phone: (p as any).homeowner_phone || "",
      homeowner_email: (p as any).homeowner_email || "",
      homeowner_address: (p as any).homeowner_address || "",
    });
    setDateNotes(Array.isArray(p.date_notes) ? (p.date_notes as unknown as DateNote[]) : []);
    setInternalNotes(p.internal_notes || "");
  }
  setLoading(false);
  setHasChanges(false);
}, [id]);

useEffect(() =&gt; { loadData(); }, [loadData]);

// Realtime: refresh when payments change
useEffect(() =&gt; {
  if (!id) return;
  const channel = supabase
    .channel(`project-payments-${id}`)
    .on('postgres_changes', { event: '*', schema: 'public', table: 'admin_payments', filter: `project_id=eq.${id}` }, () =&gt; loadData())
```

```
    .on('postgres_changes', { event: '*', schema: 'public', table: 'payment_milestones', filter: `project_id=eq.${id}` }, () =&gt; loadData())
    .subscribe();
  return () =&gt; { supabase.removeChannel(channel); };
}, [id, loadData]);

const updateForm = (field: string, value: string) =&gt; {
  setForm((f: any) =&gt; ({ ...f, [field]: value }));
  setHasChanges(true);
};

// Auto-save stage changes immediately to DB
const updateStage = async (newStage: string) =&gt; {
  setForm((f: any) =&gt; ({ ...f, stage: newStage }));
  if (!id) return;
  const { error } = await supabase.from("projects").update({ stage: newStage }).eq("id", id);
  if (error) { toast.error(error.message); return; }
  // Auto-add timeline event for key stage transitions
  if (newStage === "approved") {
    const today = new Date().toISOString().split("T")[0];
    await supabase.from("project_timeline").insert({
      project_id: id,
      revision_number: -1,
      event_type: "project_approved",
      event_date: today,
      notes: "Project approved (auto-tracked from stage change)",
    });
  }
  toast.success(`Stage → ${getStageLabel(newStage)}`);
  loadData();
};

// Auto-sync stage from timeline (derived, not manual)
const handleTimelineStageSync = useCallback(async (derivedStage: string) =&gt; {
  if (!id || form.stage === derivedStage) return;
  // Only auto-sync if the derived stage is "further" in the pipeline
  const stageIndex = (s: string) =&gt; PROJECT_STAGES.indexOf(s as any);
  if (stageIndex(derivedStage) &gt; stageIndex(form.stage)) {
    setForm((f: any) =&gt; ({ ...f, stage: derivedStage }));
    await supabase.from("projects").update({ stage: derivedStage }).eq("id", id);
  }
}, [id, form.stage]);

const handleSave = async () =&gt; {
  if (!id) return;
  setSaving(true);
  const payload: any = {
    stage: form.stage,
    service_type: form.service_type || null,
    square_feet: form.square_feet ? Number(form.square_feet) : null,
    start_date: form.start_date || null,
    delivery_date: form.delivery_date || null,
    preview_delivery_date: form.preview_delivery_date || null,
    revision_return_date: form.revision_return_date || null,
    final_delivery_date: form.final_delivery_date || null,
    total_value: form.total_value ? Number(form.total_value) : 0,
    currency: form.currency,
    entry_payment_percent: Number(form.entry_payment_percent) || 50,
    payment_stage: form.payment_stage,
    internal_notes: internalNotes || null,
    date_notes: dateNotes,
    street_number: form.street_number || null,
    street_name: form.street_name || null,
    city: form.city || null,
    state: form.state || null,
    zip_code: form.zip_code || null,
    country: form.country || null,
    homeowner_name: form.homeowner_name || null,
    homeowner_phone: form.homeowner_phone || null,
    homeowner_email: form.homeowner_email || null,
    homeowner_address: form.homeowner_address || null,
  };

  const { error } = await supabase.from("projects").update(payload).eq("id", id);
  if (error) { toast.error(error.message); setSaving(false); return; }

  // Sync services
  await supabase.from("project_services").delete().eq("project_id", id);
  if (selectedServiceIds.length &gt; 0) {
    await supabase.from("project_services").insert(
      selectedServiceIds.map(sid =&gt; ({ project_id: id, service_id: sid }))
    );
  }

  toast.success("Project saved!");
  setSaving(false);
  loadData();
};
```

```
if (loading) return &lt;div className="p-6 text-muted-foreground"&gt;Loading...&lt;/div&gt;;
if (!project) return &lt;div className="p-6 text-muted-foreground"&gt;Project not found&lt;/div&gt;;

const totalPaidPayments = payments.filter(p =&gt; p.status === "paid").reduce((s, p) =&gt; s + Number(p.amount_paid || 0), 0);
const totalPaidMilestones = paymentMilestones.filter(m =&gt; m.status === "paid").reduce((s: number, m: any) =&gt; s + Number(m.amount || 0), 0
const totalPaid = totalPaidPayments + totalPaidMilestones;
const totalValue = Number(form.total_value || 0);
const isBRL = (form.currency || "USD") === "BRL";
const fmtVal = (v: number) =&gt; isBRL
  ? `R$${v.toLocaleString("pt-BR", { minimumFractionDigits: 2, maximumFractionDigits: 2 })}`
  : `$${v.toLocaleString("en-US", { minimumFractionDigits: 2, maximumFractionDigits: 2 })}`;

const entryPct = Number(form.entry_payment_percent || 50);
const entryAmt = totalValue * entryPct / 100;
const finalAmt = totalValue - entryAmt;
const isOverdue = project.delivery_date &amp;&amp; new Date(project.delivery_date) &lt; new Date() &amp;&amp; project.stage !== "Completed";
const areaUnit = project.country === "Brazil" ? "m²" : "ft²";

const remaining = Math.max(0, totalValue - totalPaid);
const paidPct = totalValue &gt; 0 ? Math.round((totalPaid / totalValue) * 100) : 0;

// Use form values (editable) for address display
const addrStreet = [form.street_number, form.street_name].filter(Boolean).join(" ");
const fullAddress = [addrStreet, form.city, form.state, form.country].filter(Boolean).join(", ");
const projectTitle = addrStreet || project.name || "Untitled Project";
const projectNumber = project.project_number || "";

// Google Maps link from form (live updated)
const mapsQuery = [form.street_number, form.street_name, form.city, form.state, form.zip_code].filter(Boolean).join("+");
const mapsUrl = mapsQuery ? `https://www.google.com/maps/search/?api=1&amp;query=${encodeURIComponent(mapsQuery)}` : null;

// Health status
const getHealthStatus = () =&gt; {
  if (project.stage === "Completed") return { label: "Completed", color: "text-chart-2", icon: CheckCircle2, bg: "bg-chart-2/15" };
  if (project.stage === "Cancelled") return { label: "Cancelled", color: "text-muted-foreground", icon: XCircle, bg: "bg-muted" };
  if (isOverdue) return { label: "Delayed", color: "text-destructive", icon: XCircle, bg: "bg-destructive/15" };
  const deadline = project.delivery_date ? new Date(project.delivery_date) : null;
  if (deadline) {
    const daysLeft = (deadline.getTime() - Date.now()) / (1000 * 60 * 60 * 24);
    if (daysLeft &lt;= 7 &amp;&amp; daysLeft &gt; 0) return { label: "Attention Needed", color: "text-chart-3", icon: AlertTriangle, bg: "bg-ch
  }
  return { label: "On Track", color: "text-chart-2", icon: CheckCircle2, bg: "bg-chart-2/15" };
};
const health = getHealthStatus();

// Group drawings by name
const drawingGroups: Record&lt;string, any[]&gt; = {};
drawings.forEach(d =&gt; {
  if (!drawingGroups[d.drawing_name]) drawingGroups[d.drawing_name] = [];
  drawingGroups[d.drawing_name].push(d);
});

const paymentStatusColor = (status: string) =&gt; {
  if (status === "paid") return "bg-chart-2/15 text-chart-2 border-chart-2/30";
  if (status === "pending") return "bg-chart-3/15 text-chart-3 border-chart-3/30";
  return "bg-destructive/15 text-destructive border-destructive/30";
};

const toggleService = (serviceId: string) =&gt; {
  setSelectedServiceIds(prev =&gt;
    prev.includes(serviceId) ? prev.filter(id =&gt; id !== serviceId) : [...prev, serviceId]
  );
  setHasChanges(true);
};

const addDateNote = () =&gt; {
  setDateNotes(prev =&gt; [...prev, { date: new Date().toISOString().split("T")[0], note: "", visibility: "private" }]);
  setHasChanges(true);
};

const updateDateNote = (index: number, field: keyof DateNote, value: string) =&gt; {
  setDateNotes(prev =&gt; prev.map((n, i) =&gt; i === index ? { ...n, [field]: value } : n));
  setHasChanges(true);
};

const removeDateNote = (index: number) =&gt; {
  setDateNotes(prev =&gt; prev.filter((_, i) =&gt; i !== index));
  setHasChanges(true);
};

// Group services by category
const servicesByCategory: Record&lt;string, any[]&gt; = {};
allServices.forEach(s =&gt; {
  const cat = s.category || "Other";
  if (!servicesByCategory[cat]) servicesByCategory[cat] = [];
  servicesByCategory[cat].push(s);
});
```

```
return (
  &lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
    {/* Back + Save */}
    &lt;div className="flex items-center justify-between"&gt;
      &lt;Button variant="ghost" className="gap-2 -ml-2" onClick={() =&gt; navigate("/adm/projects")}&gt;
        &lt;ArrowLeft className="h-4 w-4" /&gt; Back to Projects
      &lt;/Button&gt;
      {hasChanges &amp;&amp; (
        &lt;Button onClick={handleSave} disabled={saving} className="gap-2"&gt;
          &lt;Save className="h-4 w-4" /&gt; {saving ? "Saving..." : "Save Changes"}
        &lt;/Button&gt;
      )}
    &lt;/div&gt;

    {/* Header */}
    &lt;div className="flex items-start gap-4"&gt;
      &lt;div className="relative group/avatar"&gt;
        &lt;Avatar className="h-16 w-16 border border-border"&gt;
          &lt;AvatarImage src={project.cover_image || undefined} className="object-cover" /&gt;
          &lt;AvatarFallback className="bg-muted text-muted-foreground text-lg"&gt;
            {projectTitle.substring(0, 2).toUpperCase()}
          &lt;/AvatarFallback&gt;
        &lt;/Avatar&gt;
        &lt;button
          onClick={() =&gt; coverInputRef.current?.click()}
          className="absolute inset-0 flex items-center justify-center rounded-full bg-foreground/50 opacity-0 group-hover/avatar:opacity-100 t
        &gt;
          &lt;Pencil className="h-4 w-4 text-background" /&gt;
        &lt;/button&gt;
        &lt;input
          ref={coverInputRef}
          type="file"
          accept="image/*"
          className="hidden"
          onChange={async (e) =&gt; {
            const file = e.target.files?.[0];
            if (!file || !id) return;
            const ext = file.name.split(".").pop();
            const path = `${id}.${ext}`;
            const { error: uploadError } = await supabase.storage.from("portfolio").upload(path, file, { upsert: true });
            if (uploadError) { toast.error(uploadError.message); return; }
            const { data: urlData } = supabase.storage.from("portfolio").getPublicUrl(path);
            const coverUrl = urlData.publicUrl + "?t=" + Date.now();
            await supabase.from("projects").update({ cover_image: coverUrl }).eq("id", id);
            setProject((prev: any) =&gt; prev ? { ...prev, cover_image: coverUrl } : prev);
            toast.success("Cover image updated");
          }}
        /&gt;
      &lt;/div&gt;
      &lt;div className="flex-1 min-w-0"&gt;
        &lt;div className="flex items-center gap-3 flex-wrap"&gt;
          {projectNumber &amp;&amp; (
            &lt;span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-md bg-muted text-xs font-mono font-semibold text-muted-foreg
              &lt;Hash className="h-3 w-3" /&gt;
              {projectNumber}
            &lt;/span&gt;
          )}
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;{projectTitle}&lt;/h1&gt;
          &lt;span className={`inline-flex items-center gap-1.5 px-2.5 py-1 rounded-full text-xs font-medium ${health.bg} ${health.color}`}&gt;
            &lt;health.icon className="h-3 w-3" /&gt;
            {health.label}
          &lt;/span&gt;
        &lt;/div&gt;
        &lt;div className="flex items-center gap-4 mt-1 text-sm text-muted-foreground flex-wrap"&gt;
          &lt;span
            className="flex items-center gap-1 hover:text-primary cursor-pointer"
            onClick={() =&gt; navigate(`/adm/clients/${project.company_id}`)}
          &gt;
            &lt;Building2 className="h-3.5 w-3.5" /&gt;
            {project.companies?.name} {getCountryFlag(project.companies?.country)}
          &lt;/span&gt;
          {fullAddress &amp;&amp; (
            &lt;span className="flex items-center gap-1"&gt;
              &lt;MapPin className="h-3.5 w-3.5" /&gt;
              {fullAddress}
            &lt;/span&gt;
          )}
          {mapsUrl &amp;&amp; (
            &lt;a
              href={mapsUrl}
              target="_blank"
              rel="noopener noreferrer"
              className="flex items-center gap-1 text-primary hover:underline text-xs"
            &gt;
              &lt;ExternalLink className="h-3 w-3" /&gt;
              Open in Maps
            &lt;/a&gt;
```

```
          )}
          &lt;span className="flex items-center gap-1"&gt;
            &lt;Badge variant="outline" className="text-xs"&gt;{getStageLabel(form.stage)}&lt;/Badge&gt;
          &lt;/span&gt;
          {project.delivery_date &amp;&amp; (
            &lt;span className={`flex items-center gap-1 ${isOverdue ? "text-destructive" : ""}`}&gt;
              &lt;Calendar className="h-3.5 w-3.5" /&gt;
              {format(new Date(project.delivery_date), "MMM d")}
              {isOverdue &amp;&amp; " ■"}
            &lt;/span&gt;
          )}
        &lt;/div&gt;
        {/* Financial summary in header */}
        &lt;div className="flex items-center gap-4 mt-2 flex-wrap"&gt;
          &lt;div className="flex items-center gap-6 text-sm"&gt;
            &lt;span className="text-muted-foreground"&gt;Budget: &lt;span className="font-semibold text-foreground"&gt;{fmtVal(totalValue)}&lt
            &lt;span className="text-muted-foreground"&gt;Paid: &lt;span className="font-semibold text-chart-2"&gt;{fmtVal(totalPaid)}&lt;/span
            &lt;span className="text-muted-foreground"&gt;Remaining: &lt;span className={`font-semibold ${remaining &gt; 0 ? "text-chart-3" : "
          &lt;/div&gt;
          {totalValue &gt; 0 &amp;&amp; (
            &lt;div className="flex items-center gap-2 min-w-[160px]"&gt;
              &lt;Progress value={paidPct} className="h-2 flex-1" /&gt;
              &lt;span className="text-xs text-muted-foreground font-medium"&gt;{paidPct}%&lt;/span&gt;
            &lt;/div&gt;
          )}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Tabs */}
    &lt;Tabs defaultValue="overview"&gt;
      &lt;TabsList className="flex-wrap h-auto gap-1"&gt;
        &lt;TabsTrigger value="overview"&gt;Overview&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="progress"&gt;Progress&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="services"&gt;Services&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="timeline"&gt;Timeline&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="drawings"&gt;Drawings&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="files"&gt;Files&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="site-updates"&gt;Site Updates&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="finance"&gt;Finance&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="notes"&gt;Notes&lt;/TabsTrigger&gt;
        &lt;TabsTrigger value="client-portal"&gt;Client Portal&lt;/TabsTrigger&gt;
      &lt;/TabsList&gt;

      {/* ============ OVERVIEW ============ */}
      &lt;TabsContent value="overview"&gt;
        &lt;div className="grid grid-cols-1 lg:grid-cols-2 gap-6"&gt;
          {/* Client Info */}
          &lt;Card className="border-none shadow-sm"&gt;
            &lt;CardHeader className="pb-3"&gt;
              &lt;CardTitle className="text-base flex items-center gap-2"&gt;
                &lt;Building2 className="h-4 w-4" /&gt; Client Information
              &lt;/CardTitle&gt;
            &lt;/CardHeader&gt;
            &lt;CardContent className="space-y-3 text-sm"&gt;
              &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;User classN
              &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;Building2 c
              &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;Mail classN
              &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;Phone class
              &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;Globe class
              &lt;div className="flex justify-between"&gt;&lt;span className="text-muted-foreground flex items-center gap-1"&gt;&lt;Languages c
            &lt;/CardContent&gt;
          &lt;/Card&gt;

          {/* Project Details (editable) */}
          &lt;Card className="border-none shadow-sm"&gt;
            &lt;CardHeader className="pb-3"&gt;
              &lt;CardTitle className="text-base flex items-center gap-2"&gt;
                &lt;FolderOpen className="h-4 w-4" /&gt; Project Details
              &lt;/CardTitle&gt;
            &lt;/CardHeader&gt;
            &lt;CardContent className="space-y-4 text-sm"&gt;
              &lt;div&gt;
                &lt;Label className="text-xs text-muted-foreground"&gt;Project Stage&lt;/Label&gt;
                &lt;Select value={form.stage} onValueChange={v =&gt; updateStage(v)}&gt;
                  &lt;SelectTrigger className="mt-1"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;{PROJECT_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}&
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;Label className="text-xs text-muted-foreground"&gt;Service Type&lt;/Label&gt;
                &lt;Input className="mt-1" value={form.service_type} onChange={e =&gt; updateForm("service_type", e.target.value)} placeholder=
              &lt;/div&gt;
              &lt;div&gt;
                &lt;

... [TRUNCATED - file too large] ...
```

src/pages/admin/AdminProjects.tsx

```
import { useEffect, useState, useRef } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Badge } from "@/components/ui/badge";
import { Checkbox } from "@/components/ui/checkbox";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Plus, Pencil, Search, Copy, Camera, FolderOpen, Trash2, Share2, Lock, Users, Eye, ArrowUpDown, ArrowUp, ArrowDown, ChevronDown, ChevronR
import { Collapsible, CollapsibleContent, CollapsibleTrigger } from "@/components/ui/collapsible";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { toast } from "sonner";
import { Separator } from "@/components/ui/separator";
import { useLanguage } from "@/hooks/useLanguage";

interface Company {
  id: string;
  name: string;
  street_name?: string | null;
  street_number?: string | null;
  unit?: string | null;
  neighborhood?: string | null;
  city?: string | null;
  state?: string | null;
  zip_code?: string | null;
  country?: string | null;
}

interface Service { id: string; name: string; slug: string; category?: string | null; }
interface Package { id: string; name: string; description: string | null; }

interface DateNote {
  date: string;
  note: string;
  visibility: "freelancer" | "private" | "client";
}

import { PROJECT_STAGES, PROJECT_STAGE_COLORS as SHARED_STAGE_COLORS, getStageLabel, COMPLETED_STAGES, isActiveStage } from "@/constants/projectS

const SERVICE_CATEGORIES_ORDER = ["Design Services", "Drafting &amp; Technical Documentation", "Visualization"];

const PAYMENT_STAGES = [
  "Not Sent", "Waiting Entry", "Entry Paid",
  "Waiting Final", "Paid", "Overdue", "Refunded",
];

const COUNTRIES = ["USA", "Brazil", "Other"];
const CURRENCIES = ["USD", "BRL"];

const emptyForm = {
  company_id: "", stage: "WAITING_CLIENT_INFO", payment_stage: "Not Sent",
  start_date: "", delivery_date: "", total_value: "",
  street_name: "", street_number: "", unit: "", neighborhood: "",
  city: "", state: "Massachusetts", zip_code: "", country: "USA",
  latitude: "", longitude: "",
  payment_effective_date: "", preview_delivery_date: "",
  preview_sent_date: "", revision_return_date: "", final_delivery_date: "",
  revision_count: "0", currency: "USD", entry_payment_percent: "50",
  package_id: "", square_feet: "", service_type: "",
};

const PROJECT_STAGE_COLORS = SHARED_STAGE_COLORS;

const PAYMENT_STAGE_COLORS: Record&lt;string, { dot: string; bg: string; text: string; border: string }&gt; = {
  "Not Sent": { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" },
  "Waiting Entry": { dot: "bg-chart-3", bg: "bg-chart-3/15", text: "text-chart-3", border: "border-chart-3/30" },
  "Entry Paid": { dot: "bg-chart-4", bg: "bg-chart-4/15", text: "text-chart-4", border: "border-chart-4/30" },
  "Waiting Final": { dot: "bg-chart-3", bg: "bg-chart-3/15", text: "text-chart-3", border: "border-chart-3/30" },
  "Paid": { dot: "bg-chart-2", bg: "bg-chart-2/15", text: "text-chart-2", border: "border-chart-2/30" },
  "Overdue": { dot: "bg-destructive", bg: "bg-destructive/15", text: "text-destructive", border: "border-destructive/30" },
  "Refunded": { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" },
};

const VISIBILITY_OPTIONS = [
  { value: "private", label: "Privado", icon: Lock, color: "text-muted-foreground" },
  { value: "freelancer", label: "Freelancer", icon: Users, color: "text-chart-4" },
  { value: "client", label: "Área do Cliente", icon: Eye, color: "text-chart-2" },
];
```

```
const AdminProjects = () =&gt; {
  const navigate = useNavigate();
  const { t } = useLanguage();
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [companies, setCompanies] = useState&lt;Company[]&gt;([]);
  const [services, setServices] = useState&lt;Service[]&gt;([]);
  const [packages, setPackages] = useState&lt;Package[]&gt;([]);
  const [projectServices, setProjectServices] = useState&lt;Record&lt;string, string[]&gt;&gt;({});
  const [search, setSearch] = useState("");
  const [stageFilter, setStageFilter] = useState("all");
  const [companyFilter, setCompanyFilter] = useState("all");
  const [dialogOpen, setDialogOpen] = useState(false);
  const [editing, setEditing] = useState&lt;any&gt;(null);
  const [form, setForm] = useState(emptyForm);
  const [selectedServices, setSelectedServices] = useState&lt;string[]&gt;([]);
  const [payments, setPayments] = useState&lt;Record&lt;string, number&gt;&gt;({});
  const [milestoneBudgets, setMilestoneBudgets] = useState&lt;Record&lt;string, number&gt;&gt;({});
  const [coverFile, setCoverFile] = useState&lt;File | null&gt;(null);
  const [coverPreview, setCoverPreview] = useState&lt;string | null&gt;(null);
  const coverInputRef = useRef&lt;HTMLInputElement&gt;(null);
  const [dateNotes, setDateNotes] = useState&lt;DateNote[]&gt;([]);
  const [inlineEditId, setInlineEditId] = useState&lt;string | null&gt;(null);
  const [inlineEditField, setInlineEditField] = useState&lt;"stage" | "payment_stage" | null&gt;(null);
  const [countryStep, setCountryStep] = useState&lt;boolean&gt;(false);
  const [sortColumn, setSortColumn] = useState&lt;string | null&gt;(null);
  const [sortDir, setSortDir] = useState&lt;"asc" | "desc"&gt;("asc");
  const [completedOpen, setCompletedOpen] = useState(false);

  const toggleSort = (col: string) =&gt; {
    if (sortColumn === col) {
      setSortDir(d =&gt; d === "asc" ? "desc" : "asc");
    } else {
      setSortColumn(col);
      setSortDir("asc");
    }
  };

  const SortIcon = ({ col }: { col: string }) =&gt; {
    if (sortColumn !== col) return &lt;ArrowUpDown className="h-3 w-3 text-muted-foreground/40" /&gt;;
    return sortDir === "asc"
      ? &lt;ArrowUp className="h-3 w-3 text-primary" /&gt;
      : &lt;ArrowDown className="h-3 w-3 text-primary" /&gt;;
  };

  const isBrazil = form.country === "Brazil";
  const areaLabel = isBrazil ? "Área (m²)" : "Metragem (sq ft)";
  const areaUnit = isBrazil ? "m²" : "ft²";
  const areaPlaceholder = isBrazil ? "Ex: 120" : "Ex: 720";

  const handleInlineSave = async (projectId: string, field: "stage" | "payment_stage", value: string) =&gt; {
    const { error } = await supabase.from("projects").update({ [field]: value }).eq("id", projectId);
    if (error) { toast.error(error.message); return; }
    toast.success("Atualizado!");
    setInlineEditId(null);
    setInlineEditField(null);
    fetchData();
  };

  const uploadCoverImage = async (projectId: string): Promise&lt;string | null&gt; =&gt; {
    if (!coverFile) return editing?.cover_image || null;
    const ext = coverFile.name.split(".").pop();
    const path = `${projectId}.${ext}`;
    const { error } = await supabase.storage.from("portfolio").upload(path, coverFile, { upsert: true });
    if (error) { toast.error("Erro ao enviar imagem"); return null; }
    const { data } = supabase.storage.from("portfolio").getPublicUrl(path);
    return `${data.publicUrl}?t=${Date.now()}`;
  };

  const handleCoverChange = (e: React.ChangeEvent&lt;HTMLInputElement&gt;) =&gt; {
    const file = e.target.files?.[0];
    if (!file) return;
    setCoverFile(file);
    setCoverPreview(URL.createObjectURL(file));
  };

  const fetchData = async () =&gt; {
    const [projRes, compRes, svcRes, pkgRes, psRes, payRes] = await Promise.all([
      supabase.from("projects").select("*, companies(name)").order("created_at", { ascending: false }),
      supabase.from("companies").select("id, name, street_name, street_number, unit, neighborhood, city, state, zip_code, country").order("name")
      supabase.from("services").select("*").order("name"),
      supabase.from("packages").select("*"),
      supabase.from("project_services").select("project_id, service_id"),
      supabase.from("payment_milestones").select("project_id, amount, status"),
    ]);
    setProjects(projRes.data || []);
    setCompanies(compRes.data || []);
    setServices(svcRes.data || []);
```

```
  setPackages(pkgRes.data || []);

  const psMap: Record&lt;string, string[]&gt; = {};
  (psRes.data || []).forEach((ps: any) =&gt; {
    if (!psMap[ps.project_id]) psMap[ps.project_id] = [];
    psMap[ps.project_id].push(ps.service_id);
  });
  setProjectServices(psMap);

  // Use payment_milestones as source of truth
  const payMap: Record&lt;string, number&gt; = {};
  const budgetMap: Record&lt;string, number&gt; = {};
  (payRes.data || []).forEach((m: any) =&gt; {
    budgetMap[m.project_id] = (budgetMap[m.project_id] || 0) + Number(m.amount || 0);
    if (m.status === "paid") {
      payMap[m.project_id] = (payMap[m.project_id] || 0) + Number(m.amount || 0);
    }
  });
  setPayments(payMap);
  setMilestoneBudgets(budgetMap);
};

useEffect(() =&gt; { fetchData(); }, []);

const copyFromCompany = () =&gt; {
  const comp = companies.find(c =&gt; c.id === form.company_id);
  if (!comp) { toast.error(t("admin.selectCompanyFirst")); return; }
  setForm(f =&gt; ({
    ...f,
    street_name: comp.street_name || "", street_number: comp.street_number || "",
    unit: comp.unit || "", neighborhood: comp.neighborhood || "",
    city: comp.city || "", state: comp.state || "Massachusetts",
    zip_code: comp.zip_code || "", country: comp.country || "USA",
  }));
  toast.success(t("admin.addressCopied"));
};

const handleSave = async () =&gt; {
  if (!form.company_id) {
    toast.error("Please select a company/client before saving.");
    return;
  }
  const payload: any = {
    company_id: form.company_id,
    service_type: form.service_type?.trim() || "Technical Drafting",
    square_feet: form.square_feet ? Number(form.square_feet) : null,
    stage: form.stage,
    payment_stage: form.payment_stage,
    start_date: form.start_date || null,
    delivery_date: form.delivery_date || null,
    total_value: form.total_value ? Number(form.total_value) : 0,
    street_name: form.street_name || null, street_number: form.street_number || null,
    unit: form.unit || null, neighborhood: form.neighborhood || null,
    city: form.city || null, state: form.state || null,
    zip_code: form.zip_code || null, country: form.country || null,
    latitude: form.latitude ? Number(form.latitude) : null,
    longitude: form.longitude ? Number(form.longitude) : null,
    payment_effective_date: form.payment_effective_date || null,
    preview_delivery_date: form.preview_delivery_date || null,
    preview_sent_date: form.preview_sent_date || null,
    revision_return_date: form.revision_return_date || null,
    final_delivery_date: form.final_delivery_date || null,
    revision_count: Number(form.revision_count) || 0,
    currency: form.currency,
    entry_payment_percent: Number(form.entry_payment_percent) || 50,
    package_id: form.package_id &amp;amp;&amp;amp; form.package_id !== "none" ? form.package_id : null,
    date_notes: dateNotes,
  };

  let projectId = editing?.id;
  if (editing) {
    const coverUrl = await uploadCoverImage(editing.id);
    if (coverUrl) payload.cover_image = coverUrl;
    const { error } = await supabase.from("projects").update(payload).eq("id", editing.id);
    if (error) { toast.error(error.message); return; }
  } else {
    const { data, error } = await supabase.from("projects").insert(payload).select("id").single();
    if (error) { toast.error(error.message); return; }
    projectId = data.id;
    if (coverFile) {
      const coverUrl = await uploadCoverImage(projectId);
      if (coverUrl) await supabase.from("projects").update({ cover_image: coverUrl }).eq("id", projectId);
    }
  }

  await supabase.from("project_services").delete().eq("project_id", projectId);
  if (selectedServices.length &gt; 0) {
```

```
    await supabase.from("project_services").insert(
      selectedServices.map(sid =&gt; ({ project_id: projectId, service_id: sid }))
    );
  }

  toast.success(editing ? t("admin.projectUpdated") : t("admin.projectCreated"));
  setDialogOpen(false); setEditing(null); setCoverFile(null); setCoverPreview(null); fetchData();
};

const handleDeleteProject = async (id: string) =&gt; {
  if (!confirm("Tem certeza que deseja excluir este projeto?")) return;
  // Delete related records first
  await supabase.from("project_services").delete().eq("project_id", id);
  await supabase.from("activity_log").delete().eq("project_id", id);
  await supabase.from("admin_payments").delete().eq("project_id", id);
  await supabase.from("milestones").delete().eq("project_id", id);
  await supabase.from("project_files").delete().eq("project_id", id);
  await supabase.from("messages").delete().eq("project_id", id);
  await supabase.from("freelancer_assignments").delete().eq("project_id", id);
  await supabase.from("freelancer_contracts").delete().eq("project_id", id);
  await supabase.from("freelancer_deliveries").delete().eq("project_id", id);
  await supabase.from("freelancer_payments").delete().eq("project_id", id);
  await supabase.from("quotes").delete().eq("project_id", id);
  const { error } = await supabase.from("projects").delete().eq("id", id);
  if (error) { toast.error(error.message); return; }
  toast.success("Projeto excluído");
  fetchData();
};

const handleDuplicateProject = async (p: any) =&gt; {
  const payload: any = {
    company_id: p.company_id,
    service_type: p.service_type || "Technical Drafting",
    square_feet: p.square_feet,
    stage: "Lead",
    payment_stage: "Not Sent",
    start_date: null,
    delivery_date: null,
    total_value: p.total_value || 0,
    street_name: p.street_name,
    street_number: p.street_number,
    unit: p.unit,
    neighborhood: p.neighborhood,
    city: p.city,
    state: p.state,
    zip_code: p.zip_code,
    country: p.country,
    currency: p.currency || "USD",
    entry_payment_percent: p.entry_payment_percent || 50,
    package_id: p.package_id || null,
    revision_count: 0,
  };
  const { data, error } = await supabase.from("projects").insert(payload).select("id").single();
  if (error) { toast.error(error.message); return; }
  // Copy project services
  const existingServices = projectServices[p.id] || [];
  if (existingServices.length &gt; 0) {
    await supabase.from("project_services").insert(
      existingServices.map(sid =&gt; ({ project_id: data.id, service_id: sid }))
    );
  }
  toast.success("Projeto duplicado!");
  fetchData();
};

const openEdit = (p: any) =&gt; {
  setEditing(p);
  setCoverPreview(p.cover_image || null);
  setCoverFile(null);
  setForm({
    company_id: p.company_id || "", stage: p.stage || "Lead",
    payment_stage: p.payment_stage || "Not Sent",
    start_date: p.start_date || "", delivery_date: p.delivery_date || "",
    total_value: p.total_value?.toString() || "",
    street_name: p.street_name || "", street_number: p.street_number || "",
    unit: p.unit || "", neighborhood: p.neighborhood || "",
    city: p.city || "", state: p.state || "Massachusetts",
    zip_code: p.zip_code || "", country: p.country || "USA",
    latitude: p.latitude?.toString() || "", longitude: p.longitude?.toString() || "",
    payment_effective_date: p.payment_effective_date || "",
    preview_delivery_date: p.preview_delivery_date || "",
    preview_sent_date: p.preview_sent_date || "",
    revision_return_date: p.revision_return_date || "",
    final_delivery_date: p.final_delivery_date || "",
    revision_count: p.revision_count?.toString() || "0",
    currency: p.currency || "USD",
    entry_payment_percent: p.entry_payment_percent?.toString() || "50",
```

```
    package_id: p.package_id || "",
    square_feet: p.square_feet?.toString() || "",
    service_type: p.service_type || "Technical Drafting",
  });
  setSelectedServices(projectServices[p.id] || []);
  // Load date_notes from project
  const notes = Array.isArray(p.date_notes) ? p.date_notes as DateNote[] : [];
  setDateNotes(notes);
  setCountryStep(false);
  setDialogOpen(true);
};

const openNew = () =&gt; {
  setEditing(null);
  setForm({ ...emptyForm, company_id: companies[0]?.id || "" });
  setSelectedServices([]);
  setCoverFile(null); setCoverPreview(null);
  setDateNotes([]);
  setCountryStep(true);
  setDialogOpen(true);
};

const selectCountryForNew = (country: "Brazil" | "USA") =&gt; {
  if (country === "Brazil") {
    setForm(f =&gt; ({ ...f, country: "Brazil", currency: "BRL", state: "" }));
  } else {
    setForm(f =&gt; ({ ...f, country: "USA", currency: "USD", state: "Massachusetts" }));
  }
  setCountryStep(false);
};

const addDateNote = () =&gt; {
  setDateNotes(prev =&gt; [...prev, { date: "", note: "", visibility: "private" }]);
};

const updateDateNote = (index: number, field: keyof DateNote, value: string) =&gt; {
  setDateNotes(prev =&gt; prev.map((n, i) =&gt; i === index ? { ...n, [field]: value } : n));
};

const removeDateNote = (index: number) =&gt; {
  setDateNotes(prev =&gt; prev.filter((_, i) =&gt; i !== index));
};

const totalValue = Number(form.total_value) || 0;
const entryPercent = Number(form.entry_payment_percent) || 50;
const entryAmount = totalValue * entryPercent / 100;
const finalAmount = totalValue - entryAmount;

const filtered = projects
  .filter(p =&gt; {
    const addr = [p.street_number, p.street_name].filter(Boolean).join(" ") || p.display_address || "";
    const name = p.name || "";
    const projectNum = p.project_number || "";
    const searchLower = search.toLowerCase();
    return name.toLowerCase().includes(searchLower)
      || addr.toLowerCase().includes(searchLower)
      || (p.companies?.name || "").toLowerCase().includes(searchLower)
      || projectNum.toLowerCase().includes(searchLower);
  })
  .filter(p =&gt; stageFilter === "all" || p.stage === stageFilter)
  .filter(p =&gt; companyFilter === "all" || p.company_id === companyFilter);

const sorted = [...filtered].sort((a, b) =&gt; {
  if (!sortColumn) return 0;
  const dir = sortDir === "asc" ? 1 : -1;
  switch (sortColumn) {
    case "project": return dir * getProjectDisplayName(a).localeCompare(getProjectDisplayName(b));
    case "client": return dir * ((a.companies?.name || "").localeCompare(b.companies?.name || ""));
    case "stage": return dir * ((a.stage || "").localeCompare(b.stage || ""));
    case "deadline": {
      const da = a.delivery_date ? new Date(a.delivery_date).getTime() : Infinity;
      const db = b.delivery_date ? new Date(b.delivery_date).getTime() : Infinity;
      return dir * (da - db);
    }
    case "budget": return dir * (Number(a.total_value || 0) - Number(b.total_value || 0));
    default: return 0;
  }
});

const getProjectDisplayName = (p: any) =&gt; {
  const addr = [p.street_number, p.street_name].filter(Boolean).join(" ");
  return addr || p.name || p.display_address || "Untitled Project";
};

const getFinancialHealth = (p: any): { icon: string; label: string; color: string } =&gt; {
  const budget = milestoneBudgets[p.id] || Number(p.total_value || 0);
  if (budget === 0) return { icon: "■", label: "No Budget", color: "text-muted-foreground" };
```

```
  const paid = payments[p.id] || 0;
  const rem = budget - paid;
  if (rem &lt;= 0) return { icon: "■", label: "Paid", color: "text-chart-2" };
  if (paid &gt; 0) return { icon: "■", label: "Partial", color: "text-chart-3" };
  if (p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date()) return { icon: "■", label: "Overdue", color: "text-destructive" };
  return { icon: "■", label: "Unpaid", color: "text-destructive" };
};

const handleArchiveProject = async (id: string) =&gt; {
  const { error } = await supabase.from("projects").update({ stage: "CANCELED" }).eq("id", id);
  if (error) { toast.error(error.message); return; }
  toast.success("Project archived");
  fetchData();
};

// Split projects into 3 groups
const activeProjects = sorted.filter(p =&gt; isActiveStage(p.stage || ""));
const awaitingPayment = sorted.filter(p =&gt; {
  if (!COMPLETED_STAGES.includes(p.stage as any)) return false;
  const budget = milestoneBudgets[p.id] || Number(p.total_value || 0);
  const paid = payments[p.id] || 0;
  return budget &gt; 0 &amp;&amp; (budget - paid) &gt; 0;
});
const completedProjects = sorted.filter(p =&gt; {
  if (!COMPLETED_STAGES.includes(p.stage as any)) return false;
  const budget = milestoneBudgets[p.id] || Number(p.total_value || 0);
  const paid = payments[p.id] || 0;
  return budget === 0 || (budget - paid) &lt;= 0;
});

const currencySymbol = (c: string) =&gt; c === "BRL" ? "R$" : "$";
const getCountryFlag = (country: string | null | undefined) =&gt; {
  if (country === "Brazil") return "■■";
  if (country === "USA") return "■■";
  return "";
};

const renderStageBadge = (stage: string, colors: Record&lt;string, { dot: string; bg: string; text: string; border: string }&gt;) =&gt; {
  const c = colors[stage] || { dot: "bg-muted-foreground", bg: "bg-muted", text: "text-muted-foreground", border: "border-border" };
  return (
    &lt;span className={`inline-flex items-center gap-1.5 px-2 py-0.5 rounded-full text-xs font-medium border ${c.bg} ${c.text} ${c.border}`}&g
      &lt;span className={`w-2 h-2 rounded-full ${c.dot}`} /&gt;
      {getStageLabel(stage) || "—"}
    &lt;/span&gt;
  );
};

const renderProjectRow = (p: any, showArchive = false) =&gt; {
  const tv = Number(p.total_value || 0);
  const isBRL = (p.currency || "USD") === "BRL";
  const formattedBudget = isBRL
    ? `R$${tv.toLocaleString("pt-BR", { minimumFractionDigits: 0 })}`
    : `$${tv.toLocaleString("en-US", { minimumFractionDigits: 0 })}`;
  const isInlineStage = inlineEditId === p.id &amp;&amp; inlineEditField === "stage";
  const isOverdue = p.delivery_date &amp;&amp; new Date(p.delivery_date) &lt; new Date() &amp;&amp; p.stage !== "Completed";
  const health = getFinancialHealth(p);

  return (
    &lt;TableRow key={p.id} className="group"&gt;
      &lt;TableCell&gt;
        &lt;div className="flex items-center gap-3"&gt;
          &lt;span className="text-sm" title={health.label}&gt;{health.icon}&lt;/span&gt;
          &lt;div className="relative group/avatar"&gt;
            &lt;Avatar className="h-9 w-9 border border-border flex-shrink-0"&gt;
              &lt;AvatarImage src={p.cover_image || undefined} className="object-cover" /&gt;
              &lt;AvatarFallback className="bg-muted text-muted-foreground text-xs"&gt;
                {getProjectDisplayName(p).substring(0, 2).toUpperCase()}
              &lt;/AvatarFallback&gt;
            &lt;/Avatar&gt;
            &lt;button
              onClick={(e) =&gt; {
                e.stopPropagation();
                const input = document.createElement("input");
                input.type = "file";
                input.accept = "image/*";
                input.onchange = async (ev) =&gt; {
                  const file = (ev.target as HTMLInputElement).files?.[0];
                  if (!file) return;
                  const ext = file.name.split(".").pop();
                  const path = `${p.id}.${ext}`;
                  const { error: uploadError } = await supabase.storage.from("portfolio").upload(path, file, { upsert: true });
                  if (uploadError) { toast.error(uploadError.message); return; }
                  const { data: urlData } = supabase.storage.from("portfolio").getPublicUrl(path);
                  const coverUrl = urlData.publicUrl + "?t=" + Date.now();
                  await supabase.from("projects").update({ cover_image: coverUrl }).eq("id", p.id);
                  toast.success("Cover updated");
                  fetchData();
```

```
            };
            input.click();
          }}
          className="absolute inset-0 flex items-center justify-center rounded-full bg-foreground/50 opacity-0 group-hover/avatar:opacity-1
        &gt;
          &lt;Pencil className="h-3 w-3 text-background" /&gt;
        &lt;/button&gt;
      &lt;/div&gt;
      &lt;div className="min-w-0"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          {p.project_number &amp;&amp; (
            &lt;span className="text-[10px] font-mono font-semibold text-muted-foreground bg-muted px-1.5 py-0.5 rounded border border-bord
              {p.project_number}
            &lt;/span&gt;
          )}
          &lt;p className="text-sm font-medium text-foreground truncate hover:text-primary cursor-pointer" onClick={() =&gt; navigate(`/adm
            {getProjectDisplayName(p)}
          &lt;/p&gt;
        &lt;/div&gt;
        &lt;p className="text-xs text-muted-foreground truncate"&gt;
          {p.service_type || "—"} {p.city ? `· ${p.city}, ${p.state || ""}` : ""}
        &lt;/p&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/TableCell&gt;
  &lt;TableCell&gt;
    &lt;p className="text-sm text-foreground hover:text-primary cursor-pointer truncate" onClick={() =&gt; p.company_id &amp;&amp; navigate
      {p.companies?.name || "—"} {getCountryFlag(p.country)}
    &lt;/p&gt;
  &lt;/TableCell&gt;
  &lt;TableCell&gt;
    {isInlineStage ? (
      &lt;Select defaultValue={p.stage} onValueChange={v =&gt; handleInlineSave(p.id, "stage", v)}&gt;
        &lt;SelectTrigger className="h-8 text-xs w-[150px]"&gt;&lt;SelectValue /&gt;&lt;/SelectTrigger&gt;
        &lt;SelectContent&gt;{PROJECT_STAGES.map(s =&gt; &lt;SelectItem key={s} value={s}&gt;{getStageLabel(s)}&lt;/SelectItem&gt;)}&lt;/Se
      &lt;/Select&gt;
    ) : (
      &lt;div className="flex items-center gap-1"&gt;
        {renderStageBadge(p.stage || "—", PROJECT_STAGE_COLORS)}
        &lt;Button variant="ghost" size="icon" className="h-6 w-6 opacity-0 group-hover:opacity-100" onClick={() =&gt; { setInlineEditId(p.
          &lt;Pencil className="h-3 w-3 text-muted-foreground" /&gt;
        &lt;/Button&gt;
      &lt;/div&gt;
    )}
  &lt;/TableCell&gt;
  &lt;TableCell&gt;
    {p.delivery_date ? (
      &lt;div className="flex items-center gap-1.5"&gt;
        &lt;span className={`text-sm ${isOverdue ? "text-destructive font-medium" : "text-muted-foreground"}`}&gt;
          {new Date(p.delivery_date).toLocaleDateString("en-US", { month: "short", day: "numeric" })}
        &lt;/span&gt;
        {isOverdue &amp;&amp; (
          &lt;span className="inline-flex items-center gap-0.5 px-1.5 py-0.5 rounded text-[10px] font-medium bg-destructive/10 text-destruc
        )}
      &lt;/div&gt;
    ) : (
      &lt;span className="text-sm text-muted-foreground"&gt;—&lt;/span&gt;
    )}
  &lt;/TableCell&gt;
  &lt;TableCell&gt;
    &lt;div&gt;
      &lt;span className="text-sm font-medium text-foreground"&gt;{formattedBudget}&lt;/span&gt;
      {tv &gt; 0 &amp;&amp; (() =&gt; {
        const paid = payments[p.id] || 0;
        const rem = Math.max(0, tv - paid);
        return (
          &lt;div className="flex items-center gap-2 mt-0.5"&gt;
            &lt;span className="text-[10px] text-muted-foreground"&gt;
              Rem: {isBRL ? `R$${rem.toLocaleString("pt-BR")}` : `$${rem.toLocaleString("en-US")}`}
            &lt;/span&gt;
          &lt;/div&gt;
        );
      })()}
    &lt;/div&gt;
  &lt;/TableCell&gt;
  &lt;TableCell&gt;
    &lt;div className="flex gap-1 opacity-0 group-hover:opacity-100 transition-opacity"&gt;
      &lt;Button variant="ghost" size="icon" onClick={() =&gt; navigate(`/adm/projects/${p.id}`)} title="Edit"&gt;&lt;Pencil className="h-4
      &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleDuplicateProject(p)} title="Duplicate"&gt;&lt;Copy className="h-4 w-4"
      {showArchive &amp;&amp; (
        &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleArchiveProject(p.id)} title="Archive"&gt;&lt;Archive className="h-4
      )}
      &lt;Button variant="ghost" size="icon" onClick={() =&gt; handleDeleteProject(p.id)} title="Delete"&gt;&lt;Trash2 className="h-4 w-4 t
    &lt;/div&gt;
  &lt;/TableCell&gt;
&lt;/TableRow&gt;
);
```

```
  };

  const renderTableHeader = () =&gt; (
    &lt;TableHeader&gt;
      &lt;TableRow&gt;
        &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("project")}&gt;
          &lt;span className="flex items-center gap-1"&gt;Project &lt;SortIcon col="project" /&gt;&lt;/span&gt;
        &lt;/TableHead&gt;
        &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("client")}&gt;
          &lt;span className="flex items-center gap-1"&gt;Client &lt;SortIcon col="client" /&gt;&lt;/span&gt;
        &lt;/TableHead&gt;
        &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("stage")}&gt;
          &lt;span className="flex items-center gap-1"&gt;Stage &lt;SortIcon col="stage" /&gt;&lt;/span&gt;
        &lt;/TableHead&gt;
        &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("deadline")}&gt;
          &lt;span className="flex items-center gap-1"&gt;Deadline &lt;SortIcon col="deadline" /&gt;&lt;/span&gt;
        &lt;/TableHead&gt;
        &lt;TableHead className="cursor-pointer select-none" onClick={() =&gt; toggleSort("budget")}&gt;
          &lt;span className="flex items-center gap-1"&gt;Budget &lt;SortIcon col="budget" /&gt;&lt;/span&gt;
        &lt;/TableHead&gt;
        &lt;TableHead className="w-[100px]"&gt;&lt;/TableHead&gt;
      &lt;/TableRow&gt;
    &lt;/TableHeader&gt;
  );

  retur

... [TRUNCATED - file too large] ...
```

src/pages/admin/AdminProposals.tsx

```
import { useEffect, useState } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Input } from "@/components/ui/input";
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow } from "@/components/ui/table";
import { Search, FileText } from "lucide-react";
import { format } from "date-fns";

const AdminProposals = () =&gt; {
  const [quotes, setQuotes] = useState&lt;any[]&gt;([]);
  const [search, setSearch] = useState("");

  useEffect(() =&gt; {
    supabase.from("quotes").select("*, companies(name), projects(name)").order("created_at", { ascending: false })
      .then(({ data }) =&gt; setQuotes(data || []));
  }, []);

  const statusColor: Record&lt;string, string&gt; = {
    draft: "bg-muted text-muted-foreground", sent: "bg-blue-500/10 text-blue-600",
    approved: "bg-emerald-500/10 text-emerald-600", paid: "bg-green-500/10 text-green-700",
  };

  const filtered = quotes.filter(q =&gt;
    (q.title || "").toLowerCase().includes(search.toLowerCase()) ||
    (q.companies?.name || "").toLowerCase().includes(search.toLowerCase())
  );

  return (
    &lt;div className="p-6 space-y-6"&gt;
      &lt;div className="flex items-center justify-between"&gt;
        &lt;div&gt;
          &lt;h1 className="text-xl font-semibold text-foreground"&gt;Proposals&lt;/h1&gt;
          &lt;p className="text-sm text-muted-foreground"&gt;Manage client proposals and quotes&lt;/p&gt;
        &lt;/div&gt;
        &lt;Badge variant="secondary"&gt;{quotes.length} total&lt;/Badge&gt;
      &lt;/div&gt;
      &lt;div className="relative max-w-xs"&gt;
        &lt;Search className="absolute left-3 top-1/2 -translate-y-1/2 h-4 w-4 text-muted-foreground" /&gt;
        &lt;Input placeholder="Search proposals..." className="pl-10" value={search} onChange={e =&gt; setSearch(e.target.value)} /&gt;
      &lt;/div&gt;
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardContent className="p-0"&gt;
          &lt;Table&gt;
            &lt;TableHeader&gt;
              &lt;TableRow&gt;
                &lt;TableHead&gt;Title&lt;/TableHead&gt;
                &lt;TableHead&gt;Client&lt;/TableHead&gt;
                &lt;TableHead&gt;Project&lt;/TableHead&gt;
                &lt;TableHead&gt;Amount&lt;/TableHead&gt;
                &lt;TableHead&gt;Status&lt;/TableHead&gt;
                &lt;TableHead&gt;Date&lt;/TableHead&gt;
              &lt;/TableRow&gt;
            &lt;/TableHeader&gt;
            &lt;TableBody&gt;
              {filtered.length === 0 &amp;amp;&amp; (
                &lt;TableRow&gt;&lt;TableCell colSpan={6} className="text-center text-muted-foreground py-12"&gt;&lt;FileText className="h-8 w-8
              )}
              {filtered.map(q =&gt; (
                &lt;TableRow key={q.id}&gt;
                  &lt;TableCell className="font-medium"&gt;{q.title}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{q.companies?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="text-muted-foreground"&gt;{q.projects?.name || "—"}&lt;/TableCell&gt;
                  &lt;TableCell className="font-medium"&gt;${Number(q.amount).toLocaleString()}&lt;/TableCell&gt;
                  &lt;TableCell&gt;&lt;Badge variant="outline" className={statusColor[q.status] || ""}&gt;{q.status}&lt;/Badge&gt;&lt;/TableCell&
                  &lt;TableCell className="text-xs text-muted-foreground"&gt;{format(new Date(q.created_at), "MMM d, yyyy")}&lt;/TableCell&gt;
                &lt;/TableRow&gt;
              ))}
            &lt;/TableBody&gt;
          &lt;/Table&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    &lt;/div&gt;
  );
};

export default AdminProposals;
```

src/pages/admin/AdminSettings.tsx

```
import { useState, useEffect } from "react";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Skeleton } from "@/components/ui/skeleton";
import { toast } from "sonner";
import { Settings, Palette, Globe, Bell, Shield, Save, Loader2 } from "lucide-react";

const AdminSettings = () =&gt; {
  const [settings, setSettings] = useState&lt;any&gt;(null);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);

  useEffect(() =&gt; {
    const fetch = async () =&gt; {
      const { data } = await supabase.from("studio_settings").select("*").limit(1).single();
      if (data) setSettings(data);
      setLoading(false);
    };
    fetch();
  }, []);

  const handleSave = async () =&gt; {
    if (!settings) return;
    setSaving(true);
    const { error } = await supabase.from("studio_settings").update({
      studio_name: settings.studio_name,
      tagline: settings.tagline,
      logo_url: settings.logo_url,
      primary_color: settings.primary_color,
      accent_color: settings.accent_color,
      contact_email: settings.contact_email,
      contact_phone: settings.contact_phone,
      custom_domain: settings.custom_domain,
    }).eq("id", settings.id);
    if (error) toast.error("Failed to save settings");
    else toast.success("Settings saved successfully");
    setSaving(false);
  };

  if (loading) return &lt;div className="p-6 space-y-4"&gt;&lt;Skeleton className="h-8 w-48" /&gt;&lt;Skeleton className="h-64" /&gt;&lt;/div&gt;

  return (
    &lt;div className="p-6 space-y-6 max-w-[800px]"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Settings&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;System configuration and white-label branding&lt;/p&gt;
      &lt;/div&gt;

      {/* Branding / White Label */}
      &lt;Card className="border-none shadow-sm"&gt;
        &lt;CardHeader className="pb-3"&gt;
          &lt;div className="flex items-center gap-2"&gt;
            &lt;Palette className="h-4 w-4 text-muted-foreground" /&gt;
            &lt;CardTitle className="text-base"&gt;White Label Branding&lt;/CardTitle&gt;
          &lt;/div&gt;
        &lt;/CardHeader&gt;
        &lt;CardContent className="space-y-4"&gt;
          &lt;div className="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
            &lt;div&gt;
              &lt;Label&gt;Studio Name&lt;/Label&gt;
              &lt;Input value={settings?.studio_name || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, studio_name: e.target.value }))
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Tagline&lt;/Label&gt;
              &lt;Input value={settings?.tagline || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, tagline: e.target.value }))} placeh
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;div&gt;
            &lt;Label&gt;Logo URL&lt;/Label&gt;
            &lt;Input value={settings?.logo_url || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, logo_url: e.target.value }))} placeh
          &lt;/div&gt;
          &lt;div class="grid grid-cols-1 md:grid-cols-2 gap-4"&gt;
            &lt;div&gt;
              &lt;Label&gt;Primary Color&lt;/Label&gt;
              &lt;div class="flex gap-2"&gt;
                &lt;Input type="color" value={settings?.primary_color || "#b8860b"} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, primary
                &lt;Input value={settings?.primary_color || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, primary_color: e.target.val
              &lt;/div&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Accent Color&lt;/Label&gt;
```

```
            &lt;div className="flex gap-2"&gt;
              &lt;Input type="color" value={settings?.accent_color || "#f5a623"} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, accent_c
              &lt;Input value={settings?.accent_color || ""} onChange={e =&gt; setSettings((s: any) =&gt; ({ ...s, accent_color: e.target.value
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* General */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Settings className="h-4 w-4 text-muted-foreground" /&gt;
          &lt;CardTitle className="text-base"&gt;General&lt;/CardTitle&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent className="space-y-4"&gt;
        &lt;div&gt;&lt;Label&gt;Contact Email&lt;/Label&gt;&lt;Input value={settings?.contact_email || ""} onChange={e =&gt; setSettings((s: an
        &lt;div&gt;&lt;Label&gt;Contact Phone&lt;/Label&gt;&lt;Input value={settings?.contact_phone || ""} onChange={e =&gt; setSettings((s: an
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    {/* Domain */}
    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;
          &lt;Globe className="h-4 w-4 text-muted-foreground" /&gt;
          &lt;CardTitle className="text-base"&gt;Custom Domain&lt;/CardTitle&gt;
        &lt;/div&gt;
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;
        &lt;div&gt;&lt;Label&gt;Domain&lt;/Label&gt;&lt;Input value={settings?.custom_domain || ""} onChange={e =&gt; setSettings((s: any) =&gt
      &lt;/CardContent&gt;
    &lt;/Card&gt;

    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;&lt;Bell className="h-4 w-4 text-muted-foreground" /&gt;&lt;CardTitle className="text-ba
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;&lt;p className="text-sm text-muted-foreground"&gt;Email notification settings will be available here.&lt;/p&gt;&lt;/C
    &lt;/Card&gt;

    &lt;Card className="border-none shadow-sm"&gt;
      &lt;CardHeader className="pb-3"&gt;
        &lt;div className="flex items-center gap-2"&gt;&lt;Shield className="h-4 w-4 text-muted-foreground" /&gt;&lt;CardTitle className="text-
      &lt;/CardHeader&gt;
      &lt;CardContent&gt;&lt;p className="text-sm text-muted-foreground"&gt;Authentication and access control settings.&lt;/p&gt;&lt;/CardConte
    &lt;/Card&gt;

    &lt;Button onClick={handleSave} disabled={saving}&gt;
      {saving ? &lt;&gt;&lt;Loader2 className="mr-2 h-4 w-4 animate-spin" /&gt;Saving...&lt;/&gt; : &lt;&gt;&lt;Save className="mr-2 h-4 w-4" /
    &lt;/Button&gt;
  &lt;/div&gt;
  );
};

export default AdminSettings;
```

src/pages/admin/AdminSiteUpdates.tsx

```
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Textarea } from "@/components/ui/textarea";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
import { Badge } from "@/components/ui/badge";
import { Plus, Upload, Camera, HardHat, ExternalLink, Calendar } from "lucide-react";
import { toast } from "sonner";
import { format } from "date-fns";

const AdminSiteUpdates = () =&gt; {
  const navigate = useNavigate();
  const [updates, setUpdates] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [dialogOpen, setDialogOpen] = useState(false);
  const [uploading, setUploading] = useState(false);
  const [selectedProject, setSelectedProject] = useState("");
  const [description, setDescription] = useState("");
  const [files, setFiles] = useState&lt;File[]&gt;([]);
  const [filterProject, setFilterProject] = useState("all");

  const fetchData = async () =&gt; {
    const [updRes, projRes] = await Promise.all([
      supabase.from("site_updates").select("*, projects(name, street_name, street_number, companies(name))").order("created_at", { ascending: fal
      supabase.from("projects").select("id, name, street_name, street_number, companies(name)").order("created_at", { ascending: false }),
    ]);
    setUpdates(updRes.data || []);
    setProjects(projRes.data || []);
  };

  useEffect(() =&gt; { fetchData(); }, []);

  useEffect(() =&gt; {
    const channel = supabase
      .channel("site-updates-realtime")
      .on("postgres_changes", { event: "*", schema: "public", table: "site_updates" }, () =&gt; fetchData())
      .subscribe();
    return () =&gt; { supabase.removeChannel(channel); };
  }, []);

  const getProjectLabel = (p: any) =&gt; {
    const addr = [p.street_number, p.street_name].filter(Boolean).join(" ");
    return addr || p.name || p.companies?.name || "—";
  };

  const handleUpload = async () =&gt; {
    if (!selectedProject || files.length === 0) {
      toast.error("Select a project and at least one file");
      return;
    }
    setUploading(true);
    try {
      const { data: { user } } = await supabase.auth.getUser();
      for (const file of files) {
        const ext = file.name.split(".").pop();
        const path = `${selectedProject}/${Date.now()}-${Math.random().toString(36).slice(2)}.${ext}`;
        const { error: uploadError } = await supabase.storage.from("site-updates").upload(path, file);
        if (uploadError) { toast.error(`Upload failed: ${file.name}`); continue; }
        const { data: urlData } = supabase.storage.from("site-updates").getPublicUrl(path);

        await supabase.from("site_updates").insert({
          project_id: selectedProject,
          uploaded_by: user?.id || null,
          media_url: urlData.publicUrl,
          description: description || null,
        });
      }
      toast.success(`${files.length} update(s) uploaded`);
      setDialogOpen(false);
      setFiles([]);
      setDescription("");
      fetchData();
    } catch {
      toast.error("Upload failed");
    } finally {
      setUploading(false);
    }
  };
```

```
const filtered = filterProject === "all"
  ? updates
  : updates.filter(u =&gt; u.project_id === filterProject);

// Group by date
const grouped: Record&lt;string, any[]&gt; = {};
filtered.forEach(u =&gt; {
  const day = format(new Date(u.created_at), "yyyy-MM-dd");
  if (!grouped[day]) grouped[day] = [];
  grouped[day].push(u);
});

return (
  &lt;div className="p-6 space-y-6 max-w-[1400px] mx-auto"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Site Updates&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Construction progress photos and updates&lt;/p&gt;
      &lt;/div&gt;
      &lt;div className="flex items-center gap-3"&gt;
        &lt;Select value={filterProject} onValueChange={setFilterProject}&gt;
          &lt;SelectTrigger className="w-[220px]"&gt;
            &lt;SelectValue placeholder="Filter by project" /&gt;
          &lt;/SelectTrigger&gt;
          &lt;SelectContent&gt;
            &lt;SelectItem value="all"&gt;All Projects&lt;/SelectItem&gt;
            {projects.map(p =&gt; (
              &lt;SelectItem key={p.id} value={p.id}&gt;{getProjectLabel(p)}&lt;/SelectItem&gt;
            ))}
          &lt;/SelectContent&gt;
        &lt;/Select&gt;
        &lt;Dialog open={dialogOpen} onOpenChange={setDialogOpen}&gt;
          &lt;DialogTrigger asChild&gt;
            &lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt; New Update&lt;/Button&gt;
          &lt;/DialogTrigger&gt;
          &lt;DialogContent&gt;
            &lt;DialogHeader&gt;&lt;DialogTitle&gt;Upload Site Update&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
            &lt;div className="space-y-4"&gt;
              &lt;div&gt;
                &lt;Label&gt;Project *&lt;/Label&gt;
                &lt;Select value={selectedProject || "none"} onValueChange={v =&gt; v !== "none" &amp;&amp; setSelectedProject(v)}&gt;
                  &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Select project" /&gt;&lt;/SelectTrigger&gt;
                  &lt;SelectContent&gt;
                    {projects.map(p =&gt; (
                      &lt;SelectItem key={p.id} value={p.id}&gt;{getProjectLabel(p)}&lt;/SelectItem&gt;
                    ))}
                  &lt;/SelectContent&gt;
                &lt;/Select&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;Label&gt;Photos / Videos *&lt;/Label&gt;
                &lt;div className="mt-1"&gt;
                  &lt;label className="flex flex-col items-center justify-center w-full h-32 border-2 border-dashed rounded-lg cursor-pointer h
                    &lt;div className="flex flex-col items-center"&gt;
                      &lt;Camera className="h-8 w-8 text-muted-foreground mb-2" /&gt;
                      &lt;p className="text-sm text-muted-foreground"&gt;
                        {files.length &gt; 0 ? `${files.length} file(s) selected` : "Tap to take photo or select files"}
                      &lt;/p&gt;
                    &lt;/div&gt;
                    &lt;input
                      type="file"
                      className="hidden"
                      multiple
                      accept="image/*,video/*"
                      capture="environment"
                      onChange={e =&gt; setFiles(Array.from(e.target.files || []))}
                    /&gt;
                  &lt;/label&gt;
                &lt;/div&gt;
              &lt;/div&gt;
              &lt;div&gt;
                &lt;Label&gt;Description&lt;/Label&gt;
                &lt;Textarea
                  value={description}
                  onChange={e =&gt; setDescription(e.target.value)}
                  placeholder="Describe what's happening on site..."
                  className="mt-1"
                /&gt;
              &lt;/div&gt;
              &lt;Button className="w-full" onClick={handleUpload} disabled={uploading}&gt;
                {uploading ? (
                  &lt;&gt;
                    &lt;Upload className="h-4 w-4 mr-2 animate-spin" /&gt; Uploading...
                  &lt;/&gt;
                ) : (
                  &lt;&gt;
                    &lt;Upload className="h-4 w-4 mr-2" /&gt; Upload Update
```

```
              &lt;/&gt;
            )}
          &lt;/Button&gt;
        &lt;/div&gt;
      &lt;/DialogContent&gt;
    &lt;/Dialog&gt;
  &lt;/div&gt;
&lt;/div&gt;

{/* Stats */}
&lt;div className="grid grid-cols-1 sm:grid-cols-3 gap-4"&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-4 flex items-center gap-3"&gt;
      &lt;div className="p-2.5 rounded-lg bg-primary/15"&gt;
        &lt;HardHat className="h-5 w-5 text-primary" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;{updates.length}&lt;/p&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;Total Updates&lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-4 flex items-center gap-3"&gt;
      &lt;div className="p-2.5 rounded-lg bg-chart-2/15"&gt;
        &lt;Camera className="h-5 w-5 text-chart-2" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;
          {new Set(updates.map(u =&gt; u.project_id)).size}
        &lt;/p&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;Projects with Updates&lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="p-4 flex items-center gap-3"&gt;
      &lt;div className="p-2.5 rounded-lg bg-chart-4/15"&gt;
        &lt;Calendar className="h-5 w-5 text-chart-4" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="text-2xl font-bold text-foreground"&gt;
          {updates.filter(u =&gt; {
            const d = new Date(u.created_at);
            const now = new Date();
            return d.toDateString() === now.toDateString();
          }).length}
        &lt;/p&gt;
        &lt;p className="text-xs text-muted-foreground"&gt;Today's Updates&lt;/p&gt;
      &lt;/div&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
&lt;/div&gt;

{/* Timeline */}
{Object.keys(grouped).length === 0 ? (
  &lt;Card className="border-none shadow-sm"&gt;
    &lt;CardContent className="text-center py-16"&gt;
      &lt;HardHat className="h-12 w-12 mx-auto mb-3 text-muted-foreground/40" /&gt;
      &lt;p className="text-muted-foreground"&gt;No site updates yet&lt;/p&gt;
      &lt;p className="text-xs text-muted-foreground/70 mt-1"&gt;Upload progress photos from the construction site&lt;/p&gt;
    &lt;/CardContent&gt;
  &lt;/Card&gt;
) : (
  &lt;div className="space-y-6"&gt;
    {Object.entries(grouped).map(([date, items]) =&gt; (
      &lt;div key={date}&gt;
        &lt;div className="flex items-center gap-2 mb-3"&gt;
          &lt;Calendar className="h-4 w-4 text-muted-foreground" /&gt;
          &lt;span className="text-sm font-medium text-muted-foreground"&gt;
            {format(new Date(date), "MMMM d, yyyy")}
          &lt;/span&gt;
          &lt;Badge variant="secondary" className="text-xs"&gt;{items.length}&lt;/Badge&gt;
        &lt;/div&gt;
        &lt;div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4"&gt;
          {items.map(update =&gt; {
            const projectLabel = [update.projects?.street_number, update.projects?.street_name].filter(Boolean).join(" ") || update.project
            const isVideo = update.media_url?.match(/\.(mp4|mov|webm|avi)$/i);
            return (
              &lt;Card key={update.id} className="border-none shadow-sm overflow-hidden"&gt;
                &lt;div className="aspect-video bg-muted relative"&gt;
                  {isVideo ? (
                    &lt;video src={update.media_url} controls className="w-full h-full object-cover" /&gt;
                  ) : (
                    &lt;img src={update.media_url} alt="Site update" className="w-full h-full object-cover" /&gt;
                  )}
                &lt;/div&gt;
```

```
                &lt;CardContent className="p-3 space-y-1.5"&gt;
                  &lt;div className="flex items-center justify-between"&gt;
                    &lt;span
                      className="text-sm font-medium text-primary hover:underline cursor-pointer"
                      onClick={() =&gt; {
                        const proj = projects.find(p =&gt; p.id === update.project_id);
                        if (proj) navigate(`/adm/projects/${proj.id}`);
                      }}
                    &gt;
                      {projectLabel}
                    &lt;/span&gt;
                    &lt;span className="text-xs text-muted-foreground"&gt;
                      {format(new Date(update.created_at), "h:mm a")}
                    &lt;/span&gt;
                  &lt;/div&gt;
                  {update.projects?.companies?.name &amp;&amp; (
                    &lt;p className="text-xs text-muted-foreground"&gt;{update.projects.companies.name}&lt;/p&gt;
                  )}
                  {update.description &amp;&amp; (
                    &lt;p className="text-sm text-foreground"&gt;{update.description}&lt;/p&gt;
                  )}
                &lt;/CardContent&gt;
              &lt;/Card&gt;
            );
          })}
        &lt;/div&gt;
      &lt;/div&gt;
    ))}
  &lt;/div&gt;
 )}
&lt;/div&gt;
);
};

export default AdminSiteUpdates;
```

src/pages/admin/AdminTickets.tsx

```
import { useEffect, useState, useRef } from "react";
import { useSearchParams } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { useAuth } from "@/hooks/useAuth";
import { Card, CardContent } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Label } from "@/components/ui/label";
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue } from "@/components/ui/select";
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogTrigger } from "@/components/ui/dialog";
import { ScrollArea } from "@/components/ui/scroll-area";
import { Avatar, AvatarFallback } from "@/components/ui/avatar";
import { Plus, MessageSquare, Send, Ticket } from "lucide-react";
import { toast } from "sonner";
import { formatDistanceToNow, format } from "date-fns";

const STATUS_STYLES: Record&lt;string, { label: string; variant: "default" | "secondary" | "outline" | "destructive" }&gt; = {
  open: { label: "Open", variant: "destructive" },
  waiting: { label: "Waiting", variant: "secondary" },
  resolved: { label: "Resolved", variant: "outline" },
};

const AdminTickets = () =&gt; {
  const { user } = useAuth();
  const [searchParams, setSearchParams] = useSearchParams();
  const [tickets, setTickets] = useState&lt;any[]&gt;([]);
  const [projects, setProjects] = useState&lt;any[]&gt;([]);
  const [selectedTicket, setSelectedTicket] = useState&lt;any&gt;(null);
  const [messages, setMessages] = useState&lt;any[]&gt;([]);
  const [newMsg, setNewMsg] = useState("");
  const [createOpen, setCreateOpen] = useState(false);
  const [form, setForm] = useState({ project_id: "", subject: "", message: "" });
  const scrollRef = useRef&lt;HTMLDivElement&gt;(null);

  const fetchTickets = async () =&gt; {
    const { data } = await supabase
      .from("client_tickets")
      .select("*, projects(name, company_id), companies(name)")
      .order("updated_at", { ascending: false });
    setTickets(data || []);
    return data || [];
  };

  const fetchProjects = async () =&gt; {
    const { data } = await supabase.from("projects").select("id, name, company_id").order("name");
    setProjects(data || []);
  };

  const fetchMessages = async (ticketId: string) =&gt; {
    const { data } = await supabase
      .from("client_ticket_messages")
      .select("*")
      .eq("ticket_id", ticketId)
      .order("created_at", { ascending: true });
    setMessages(data || []);
    setTimeout(() =&gt; scrollRef.current?.scrollTo({ top: scrollRef.current.scrollHeight }), 100);
  };

  useEffect(() =&gt; {
    fetchTickets().then((tks) =&gt; {
      const urlId = searchParams.get("id");
      if (urlId) {
        const found = tks.find((t: any) =&gt; t.id === urlId);
        if (found) openTicket(found);
      }
    });
    fetchProjects();
  }, []);

  const openTicket = (ticket: any) =&gt; {
    setSelectedTicket(ticket);
    fetchMessages(ticket.id);
  };

  const handleCreate = async () =&gt; {
    if (!form.project_id || !form.subject.trim()) {
      toast.error("Project and subject are required");
      return;
    }
    const proj = projects.find((p) =&gt; p.id === form.project_id);
    const { data: ticket, error } = await supabase
      .from("client_tickets")
```

```
    .insert({
      project_id: form.project_id,
      company_id: proj?.company_id,
      subject: form.subject,
      created_by: user?.id,
    })
    .select()
    .single();
  if (error) { toast.error(error.message); return; }
  if (form.message.trim() &amp;&amp; ticket) {
    await supabase.from("client_ticket_messages").insert({
      ticket_id: ticket.id,
      sender_id: user?.id,
      is_from_admin: true,
      content: form.message,
    });
  }
  toast.success("Ticket created");
  setCreateOpen(false);
  setForm({ project_id: "", subject: "", message: "" });
  fetchTickets();
};

const handleSendMsg = async () =&gt; {
  if (!newMsg.trim() || !selectedTicket) return;
  const { error } = await supabase.from("client_ticket_messages").insert({
    ticket_id: selectedTicket.id,
    sender_id: user?.id,
    is_from_admin: true,
    content: newMsg,
  });
  if (error) { toast.error(error.message); return; }
  await supabase.from("client_tickets").update({ updated_at: new Date().toISOString() }).eq("id", selectedTicket.id);
  setNewMsg("");
  fetchMessages(selectedTicket.id);
};

const handleStatusChange = async (ticketId: string, status: string) =&gt; {
  await supabase.from("client_tickets").update({ status }).eq("id", ticketId);
  fetchTickets();
  if (selectedTicket?.id === ticketId) setSelectedTicket({ ...selectedTicket, status });
};

return (
  &lt;div className="p-6 space-y-6"&gt;
    &lt;div className="flex items-center justify-between"&gt;
      &lt;div&gt;
        &lt;h1 className="text-xl font-semibold text-foreground"&gt;Client Tickets&lt;/h1&gt;
        &lt;p className="text-sm text-muted-foreground"&gt;Manage client support conversations&lt;/p&gt;
      &lt;/div&gt;
      &lt;Dialog open={createOpen} onOpenChange={setCreateOpen}&gt;
        &lt;DialogTrigger asChild&gt;
          &lt;Button&gt;&lt;Plus className="h-4 w-4 mr-2" /&gt;New Ticket&lt;/Button&gt;
        &lt;/DialogTrigger&gt;
        &lt;DialogContent&gt;
          &lt;DialogHeader&gt;&lt;DialogTitle&gt;Create Ticket&lt;/DialogTitle&gt;&lt;/DialogHeader&gt;
          &lt;div className="space-y-4"&gt;
            &lt;div&gt;
              &lt;Label&gt;Project *&lt;/Label&gt;
              &lt;Select value={form.project_id} onValueChange={(v) =&gt; setForm({ ...form, project_id: v })}&gt;
                &lt;SelectTrigger&gt;&lt;SelectValue placeholder="Select project" /&gt;&lt;/SelectTrigger&gt;
                &lt;SelectContent&gt;{projects.map((p) =&gt; &lt;SelectItem key={p.id} value={p.id}&gt;{p.name || "Untitled"}&lt;/SelectItem&gt
              &lt;/Select&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Subject *&lt;/Label&gt;
              &lt;Input value={form.subject} onChange={(e) =&gt; setForm({ ...form, subject: e.target.value })} placeholder="e.g. Concept Desig
            &lt;/div&gt;
            &lt;div&gt;
              &lt;Label&gt;Initial Message&lt;/Label&gt;
              &lt;Textarea value={form.message} onChange={(e) =&gt; setForm({ ...form, message: e.target.value })} placeholder="Optional first
            &lt;/div&gt;
            &lt;Button className="w-full" onClick={handleCreate}&gt;Create Ticket&lt;/Button&gt;
          &lt;/div&gt;
        &lt;/DialogContent&gt;
      &lt;/Dialog&gt;
    &lt;/div&gt;

    &lt;div className="grid grid-cols-1 lg:grid-cols-[1fr_1.2fr] gap-6"&gt;
      {/* Ticket list */}
      &lt;div className="space-y-2"&gt;
        {tickets.length === 0 &amp;&amp; (
          &lt;div className="text-center text-muted-foreground py-12"&gt;
            &lt;Ticket className="h-10 w-10 mx-auto mb-2 text-muted-foreground/40" /&gt;
            No tickets yet
          &lt;/div&gt;
        )}
```

```
  {tickets.map((t) =&gt; {
    const style = STATUS_STYLES[t.status] || STATUS_STYLES.open;
    const isActive = selectedTicket?.id === t.id;
    return (
      &lt;Card
        key={t.id}
        className={`cursor-pointer transition-colors ${isActive ? "ring-2 ring-primary" : "hover:bg-muted/30"}`}
        onClick={() =&gt; openTicket(t)}
      &gt;
        &lt;CardContent className="flex items-start gap-3 py-3"&gt;
          &lt;div className="p-1.5 rounded-md bg-accent text-accent-foreground mt-0.5"&gt;
            &lt;MessageSquare className="h-4 w-4" /&gt;
          &lt;/div&gt;
          &lt;div className="flex-1 min-w-0"&gt;
            &lt;p className="text-sm font-medium text-foreground line-clamp-1"&gt;{t.subject}&lt;/p&gt;
            &lt;p className="text-xs text-muted-foreground truncate"&gt;
              {t.companies?.name} · {t.projects?.name}
            &lt;/p&gt;
            &lt;p className="text-[10px] text-muted-foreground mt-1"&gt;
              {formatDistanceToNow(new Date(t.updated_at), { addSuffix: true })}
            &lt;/p&gt;
          &lt;/div&gt;
          &lt;Select value={t.status} onValueChange={(v) =&gt; handleStatusChange(t.id, v)}&gt;
            &lt;SelectTrigger className="w-auto h-6 text-[10px] border-none p-0" onClick={(e) =&gt; e.stopPropagation()}&gt;
              &lt;Badge variant={style.variant} className="text-[10px] cursor-pointer"&gt;{style.label}&lt;/Badge&gt;
            &lt;/SelectTrigger&gt;
            &lt;SelectContent&gt;
              &lt;SelectItem value="open"&gt;Open&lt;/SelectItem&gt;
              &lt;SelectItem value="waiting"&gt;Waiting&lt;/SelectItem&gt;
              &lt;SelectItem value="resolved"&gt;Resolved&lt;/SelectItem&gt;
            &lt;/SelectContent&gt;
          &lt;/Select&gt;
        &lt;/CardContent&gt;
      &lt;/Card&gt;
    );
  })}
&lt;/div&gt;

{/* Conversation thread */}
&lt;Card className="border-none shadow-sm"&gt;
  &lt;CardContent className="p-0"&gt;
    {!selectedTicket ? (
      &lt;div className="flex flex-col items-center justify-center py-20 text-muted-foreground"&gt;
        &lt;MessageSquare className="h-10 w-10 mb-2 text-muted-foreground/30" /&gt;
        &lt;p className="text-sm"&gt;Select a ticket to view the conversation&lt;/p&gt;
      &lt;/div&gt;
    ) : (
      &lt;div className="flex flex-col h-[500px]"&gt;
        {/* Thread header */}
        &lt;div className="px-4 py-3 border-b"&gt;
          &lt;h3 className="text-sm font-semibold text-foreground"&gt;{selectedTicket.subject}&lt;/h3&gt;
          &lt;p className="text-xs text-muted-foreground"&gt;
            {selectedTicket.companies?.name} · {selectedTicket.projects?.name}
          &lt;/p&gt;
        &lt;/div&gt;

        {/* Messages */}
        &lt;ScrollArea className="flex-1 px-4 py-3" ref={scrollRef}&gt;
          {messages.length === 0 &amp;amp;&amp; (
            &lt;p className="text-sm text-muted-foreground text-center py-8"&gt;No messages yet&lt;/p&gt;
          )}
          &lt;div className="space-y-3"&gt;
            {messages.map((m) =&gt; (
              &lt;div key={m.id} className={`flex gap-2 ${m.is_from_admin ? "justify-end" : "justify-start"}`}&gt;
                {!m.is_from_admin &amp;amp;&amp; (
                  &lt;Avatar className="h-7 w-7 shrink-0"&gt;
                    &lt;AvatarFallback className="text-[10px] bg-primary/10 text-primary"&gt;CL&lt;/AvatarFallback&gt;
                  &lt;/Avatar&gt;
                )}
                &lt;div className={`max-w-[75%] rounded-lg px-3 py-2 text-sm ${
                  m.is_from_admin
                    ? "bg-primary text-primary-foreground"
                    : "bg-muted text-foreground"
                }`}&gt;
                  &lt;p&gt;{m.content}&lt;/p&gt;
                  &lt;p className={`text-[10px] mt-1 ${m.is_from_admin ? "text-primary-foreground/70" : "text-muted-foreground"}`}&gt;
                    {format(new Date(m.created_at), "MMM d, h:mm a")}
                  &lt;/p&gt;
                &lt;/div&gt;
                {m.is_from_admin &amp;amp;&amp; (
                  &lt;Avatar className="h-7 w-7 shrink-0"&gt;
                    &lt;AvatarFallback className="text-[10px] bg-accent text-accent-foreground"&gt;AD&lt;/AvatarFallback&gt;
                  &lt;/Avatar&gt;
                )}
              &lt;/div&gt;
            ))}
          &lt;/div&gt;
```

```
                &lt;/ScrollArea&gt;

                {/* Input */}
                &lt;div className="px-4 py-3 border-t flex gap-2"&gt;
                  &lt;Input
                    placeholder="Type a reply..."
                    value={newMsg}
                    onChange={(e) =&gt; setNewMsg(e.target.value)}
                    onKeyDown={(e) =&gt; e.key === "Enter" &amp;&amp; !e.shiftKey &amp;&amp; handleSendMsg()}
                    className="flex-1"
                  /&gt;
                  &lt;Button size="icon" onClick={handleSendMsg} disabled={!newMsg.trim()}&gt;
                    &lt;Send className="h-4 w-4" /&gt;
                  &lt;/Button&gt;
                &lt;/div&gt;
              &lt;/div&gt;
            )}
          &lt;/CardContent&gt;
        &lt;/Card&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
};

export default AdminTickets;
```

src/pages/admin/Dashboard.tsx

```
// Re-export existing AdminDashboard as Dashboard
export { default } from "@/pages/admin/AdminDashboard";
```

src/services/estimateService.ts

```
// src/services/estimateService.ts

export const calculateEstimate = (
  area: number,
  rate: number
) =&gt; {
  return area * rate;
};
```

src/services/index.ts

```
// Centralized Supabase services
export { supabase } from "./supabase/client";
export * as authService from "./supabase/authService";
export * as projectService from "./supabase/projectService";
export * as estimateService from "./supabase/estimateService";
export * as paymentService from "./supabase/paymentService";
export * as freelancerService from "./supabase/freelancerService";
```

src/services/paymentService.ts

```
// src/services/paymentService.ts

export const calculateRemaining = (
  total: number,
  paid: number
) =&gt; {
  return total - paid;
};
```

src/services/projectService.ts

```
// src/services/projectService.ts

export const getProjectStatus = (stage: string) =&gt; {
  return stage;
};
```

src/services/supabase/authService.ts

```
import { supabase } from "./client";

export const login = async (email: string, password: string) =&gt; {
  return supabase.auth.signInWithPassword({ email, password });
};

export const logout = async () =&gt; {
  return supabase.auth.signOut();
};

export const getUser = async () =&gt; {
  return supabase.auth.getUser();
};

export const onAuthStateChange = (callback: (event: string, session: any) =&gt; void) =&gt; {
  return supabase.auth.onAuthStateChange(callback);
};
```

src/services/supabase/client.ts

```
// Re-export the auto-generated Supabase client
// IMPORTANT: Do NOT create a new client — use the one managed by Lovable Cloud
export { supabase } from "@/integrations/supabase/client";
```

src/services/supabase/estimateService.ts

```
import { supabase } from "./client";

// Uses the quote_requests table (public estimate form submissions)
export const createEstimate = async (data: any) =&gt; {
  const { data: result, error } = await supabase
    .from("quote_requests")
    .insert([data])
    .select();
  return { result, error };
};

export const getEstimates = async () =&gt; {
  const { data, error } = await supabase
    .from("quote_requests")
    .select("*")
    .order("created_at", { ascending: false });
  return { data, error };
};

// Quotes (sent to clients)
export const getQuotes = async () =&gt; {
  const { data, error } = await supabase
    .from("quotes")
    .select("*, companies(name), projects(name)")
    .order("created_at", { ascending: false });
  return { data, error };
};

export const createQuote = async (quote: any) =&gt; {
  const { data, error } = await supabase
    .from("quotes")
    .insert([quote])
    .select();
  return { data, error };
};
```

src/services/supabase/freelancerService.ts

```
import { supabase } from "./client";

// Freelancer assignments
export const getFreelancerAssignments = async (freelancerId?: string) =&gt; {
  let query = supabase
    .from("freelancer_assignments")
    .select("*, projects(name, companies(name))");

  if (freelancerId) {
    query = query.eq("freelancer_id", freelancerId);
  }

  const { data, error } = await query;
  return { data, error };
};

// Freelancer contracts
export const getFreelancerContracts = async (freelancerId: string) =&gt; {
  const { data, error } = await supabase
    .from("freelancer_contracts")
    .select("*, projects(name)")
    .eq("freelancer_id", freelancerId)
    .order("created_at", { ascending: false });
  return { data, error };
};

// Freelancer deliveries
export const createDelivery = async (delivery: any) =&gt; {
  const { data, error } = await supabase
    .from("freelancer_deliveries")
    .insert([delivery])
    .select();
  return { data, error };
};

export const getDeliveries = async (freelancerId: string) =&gt; {
  const { data, error } = await supabase
    .from("freelancer_deliveries")
    .select("*, projects(name)")
    .eq("freelancer_id", freelancerId)
    .order("created_at", { ascending: false });
  return { data, error };
};
```

src/services/supabase/paymentService.ts

```
import { supabase } from "./client";

// Admin payments (studio income)
export const getAdminPayments = async () =&gt; {
  const { data, error } = await supabase
    .from("admin_payments")
    .select("*, projects(name, companies(name))")
    .order("created_at", { ascending: false });
  return { data, error };
};

// Freelancer payments (outgoing)
export const getFreelancerPayments = async (freelancerId?: string) =&gt; {
  let query = supabase
    .from("freelancer_payments")
    .select("*, projects(name)")
    .order("created_at", { ascending: false });

  if (freelancerId) {
    query = query.eq("freelancer_id", freelancerId);
  }

  const { data, error } = await query;
  return { data, error };
};

export const createAdminPayment = async (payment: any) =&gt; {
  const { data, error } = await supabase
    .from("admin_payments")
    .insert([payment])
    .select();
  return { data, error };
};
```

src/services/supabase/projectService.ts

```
import { supabase } from "./client";

export const getProjects = async () =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .select("*, companies(name)");
  return { data, error };
};

export const getProjectById = async (id: string) =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .select("*, companies(name)")
    .eq("id", id)
    .single();
  return { data, error };
};

export const createProject = async (project: any) =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .insert([project])
    .select();
  return { data, error };
};

export const updateProject = async (id: string, updates: any) =&gt; {
  const { data, error } = await supabase
    .from("projects")
    .update(updates)
    .eq("id", id)
    .select();
  return { data, error };
};
```

tailwind.config.ts

```
import type { Config } from "tailwindcss";

export default {
  darkMode: ["class"],
  content: ["./pages/**/*.{ts,tsx}", "./components/**/*.{ts,tsx}", "./app/**/*.{ts,tsx}", "./src/**/*.{ts,tsx}"],
  prefix: "",
  theme: {
    container: {
      center: true,
      padding: '2rem',
      screens: {
        '2xl': '1400px'
      }
    },
    extend: {
      colors: {
        border: 'hsl(var(--border))',
        input: 'hsl(var(--input))',
        ring: 'hsl(var(--ring))',
        background: 'hsl(var(--background))',
        foreground: 'hsl(var(--foreground))',
        primary: {
          DEFAULT: 'hsl(var(--primary))',
          foreground: 'hsl(var(--primary-foreground))'
        },
        secondary: {
          DEFAULT: 'hsl(var(--secondary))',
          foreground: 'hsl(var(--secondary-foreground))'
        },
        destructive: {
          DEFAULT: 'hsl(var(--destructive))',
          foreground: 'hsl(var(--destructive-foreground))'
        },
        muted: {
          DEFAULT: 'hsl(var(--muted))',
          foreground: 'hsl(var(--muted-foreground))'
        },
        accent: {
          DEFAULT: 'hsl(var(--accent))',
          foreground: 'hsl(var(--accent-foreground))'
        },
        popover: {
          DEFAULT: 'hsl(var(--popover))',
          foreground: 'hsl(var(--popover-foreground))'
        },
        card: {
          DEFAULT: 'hsl(var(--card))',
          foreground: 'hsl(var(--card-foreground))'
        },
        sidebar: {
          DEFAULT: 'hsl(var(--sidebar-background))',
          foreground: 'hsl(var(--sidebar-foreground))',
          primary: 'hsl(var(--sidebar-primary))',
          'primary-foreground': 'hsl(var(--sidebar-primary-foreground))',
          accent: 'hsl(var(--sidebar-accent))',
          'accent-foreground': 'hsl(var(--sidebar-accent-foreground))',
          border: 'hsl(var(--sidebar-border))',
          ring: 'hsl(var(--sidebar-ring))'
        }
      },
      borderRadius: {
        lg: 'var(--radius)',
        md: 'calc(var(--radius) - 2px)',
        sm: 'calc(var(--radius) - 4px)'
      },
      keyframes: {
        'accordion-down': {
          from: {
            height: '0'
          },
          to: {
            height: 'var(--radix-accordion-content-height)'
          }
        },
        'accordion-up': {
          from: {
            height: 'var(--radix-accordion-content-height)'
          },
          to: {
            height: '0'
          }
        }
      },
      animation: {
```

```
  ■■■'accordion-down': 'accordion-down 0.2s ease-out',
  ■■■'accordion-up': 'accordion-up 0.2s ease-out'
  ■■},
  ■■fontFamily: {
  ■■■sans: [
  ■■■■'Roboto',
  ■■■■'ui-sans-serif',
  ■■■■'system-ui',
  ■■■■'sans-serif',
  ■■■■'Apple Color Emoji',
  ■■■■'Segoe UI Emoji',
  ■■■■'Segoe UI Symbol',
  ■■■■'Noto Color Emoji'
  ■■■],
  ■■■serif: [
  ■■■■'Libre Caslon Text',
  ■■■■'ui-serif',
  ■■■■'Georgia',
  ■■■■'Cambria',
  ■■■■'Times New Roman',
  ■■■■'Times',
  ■■■■'serif'
  ■■■],
  ■■■mono: [
  ■■■■'Roboto Mono',
  ■■■■'ui-monospace',
  ■■■■'SFMono-Regular',
  ■■■■'Menlo',
  ■■■■'Monaco',
  ■■■■'Consolas',
  ■■■■'Liberation Mono',
  ■■■■'Courier New',
  ■■■■'monospace'
  ■■■]
  ■■}
  ■}
  },
  plugins: [require("tailwindcss-animate")],
} satisfies Config;
```

DATABASE DATA (CSV)

activity_log.csv

```
id,project_id,log_type,description,log_date,created_at
22a0b41c-1137-42ae-acc3-803ed8c65e64,aa077d41-d227-4008-a5cb-0416934c8e57,timeline_event,Estimate Created – Mar 19,2026-03-19 12:47:14.817141+00,2026-03-19 12:47:14.817141+
662cf69e-a573-4f52-a64c-178b269364ad,aa077d41-d227-4008-a5cb-0416934c8e57,timeline_event,Estimate Created – Mar 19,2026-03-19 12:47:14.927232+00,2026-03-19 12:47:14.927232+
223b5e92-a957-45e7-b901-ca3cdc069448,a09d3a89-3060-44e6-ad24-cca340f74925,timeline_event,Estimate Created – Mar 19,2026-03-19 12:47:47.762808+00,2026-03-19 12:47:47.762808+
8195bfd3-8573-47e5-a489-8c5a358e39c0,a09d3a89-3060-44e6-ad24-cca340f74925,timeline_event,Initial Invoice Sent – Mar 19,2026-03-19 15:53:54.094434+00,2026-03-19 15:53:54.094
3d27758a-132a-4d38-8b00-be0ef7f4fa39,a09d3a89-3060-44e6-ad24-cca340f74925,timeline_event,Initial Payment Received – Mar 23,2026-03-23 19:26:15.356956+00,2026-03-23 19:26:15
f67fac21-62b5-4f7c-baae-374261089f45,a09d3a89-3060-44e6-ad24-cca340f74925,timeline_event,REV 00 Started – Mar 24,2026-03-24 15:13:18.880348+00,2026-03-24 15:13:18.880348+00
e9e45e7b-1929-4b4e-8290-fc1455216779,a09d3a89-3060-44e6-ad24-cca340f74925,timeline_event,REV 00 Preview Sent – Mar 24,2026-03-24 15:13:26.347563+00,2026-03-24 15:13:26.3475
```

admin_payments.csv

```
id,project_id,installment_type,amount_total,amount_paid,status,payment_method,stripe_fee_percent,net_received,payment_date,created_at,updated_at,value_brl,price_table,third
de8f297c-c587-412e-b726-09ef55ad692a,158afd94-48b5-464b-85d3-7817a70cd787,Entry,0,0,paid,Stripe,0,0.00000000000000000000,2026-03-16,2026-03-16 12:14:43.284315+00,2026-03-16
b0454b8b-fe6a-4652-80be-b12bc5a149e5,158afd94-48b5-464b-85d3-7817a70cd787,Entry,0,0,paid,Stripe,0,0.00000000000000000000,2026-03-16,2026-03-16 12:14:43.457574+00,2026-03-16
d0fe82b4-9b34-41b0-94f5-079868585018,a080b904-a29b-46d6-8ace-59cc9f79f2a2,Entry Payment,125,125,paid,Bank Transfer,0,125.00000000000000000000,2026-03-18,2026-03-18 19:02:11
1f0dc9f4-3158-4599-b262-ca2f43ae5823,a080b904-a29b-46d6-8ace-59cc9f79f2a2,Final Payment,125,125,paid,Bank Transfer,0,125.00000000000000000000,2026-03-18,2026-03-18 19:02:17
92360de7-478d-4502-a864-e41c612f6e77,a09d3a89-3060-44e6-ad24-cca340f74925,Entry Payment,140,140,paid,Bank Transfer,0,140.00000000000000000000,2026-03-23,2026-03-23 19:26:49
```

admin_tasks.csv

(empty table)

client_ticket_messages.csv

(empty table)

client_tickets.csv

(empty table)

companies.csv

```
id,name,email,phone,address,created_at,updated_at,company_type,country,status,owner_name,website,instagram,linkedin,street_name,street_number,unit,neighborhood,city,state,z
7de907aa-333e-4fab-ab4a-54bfc008d93c,Square &amp; Plumb Builders,Info@squareandplumb.com,(978) 424-1354,"PO Box 6077 - Gloucester, MA 01930",2026-03-02 13:41:54.156809+00,2
12060449-ea10-40e9-8bc4-036d18b45e64,Thrive Contracting LLC,contact@upscale-contracting.com,+1 (508) 656-4260,,2026-03-10 13:03:53.056766+00,2026-03-10 14:34:24.232208+00,E
753027ba-2ca5-435a-99e0-5d126c3c5d1d,Carlos - Builder Construction,,+1 (609) 410-0386,,2026-03-12 15:06:55.539713+00,2026-03-12 15:06:55.539713+00,Builder,USA,inactive,Carl
78c5afb0-f164-4481-b74d-1c9cb0132086,Dacosta Builder GC INC,dacostacontractor@hotmail.com,+1 (978) 618-2447,,2026-03-12 14:58:51.871247+00,2026-03-12 14:59:03.525121+00,Bui
1c3d28fa-5cae-497d-b88d-33c2f924641f,Aland Realty ,mgiannone@alandrealty.com,+1 (978) 806-6446,,2026-03-10 13:47:45.394214+00,2026-03-10 14:33:06.89609+00,Real Estate,USA,a
fe3966e7-fa12-498a-9f02-90d72ea07179,Sem nome,,+1 (781) 504-2074,,2026-03-12 15:00:01.192603+00,2026-03-12 15:00:03.307586+00,Builder,USA,inactive,Rodney ,,,,,,,,,,https:/
202748c2-4df5-4c46-b4e9-0a7ef565ae17,CJ Associates LLC,cjassociatesma@gmail.com,+1 (617) 869-2273,,2026-03-10 13:07:10.534983+00,2026-03-10 14:33:15.792709+00,Engineering,U
b5695fc1-3a30-4718-8a58-969a08278853,Jack General Services Inc.,jackgeneraloffice@gmail.com,+1 (617) 775-0179,,2026-03-10 13:53:00.423478+00,2026-03-10 14:33:25.07938+00,Co
a8e3bdfb-b594-47e4-a173-8a06d03a1d2d,SixFortyFive LLC,sixfortyfivellcma@gmail.com,+1 (972) 415-5733,,2026-03-10 12:44:01.340039+00,2026-03-10 14:33:51.178595+00,Real Estate
5b80069b-7c05-446c-92c1-390b9219d095,Sem nome,,+1 (617) 650-7533,,2026-03-12 15:04:10.470111+00,2026-03-12 15:07:16.503979+00,Builder,USA,inactive,Carlos #1,,,,,,,,,,,,,,er
e7b160c8-2ac5-400a-a66b-bd4343e8554c,Sem nome,,+1 (781) 475-3328,,2026-03-12 15:07:33.388941+00,2026-03-12 15:07:33.388941+00,Builder,USA,inactive,Carlos #2,,,,,,,,,,,,,,er
ff98689a-ab3b-42d6-aeaa-7790456dbde7,Armondes Construction,,+1 (774) 606-3752,,2026-03-12 15:00:57.68086+00,2026-03-12 15:01:08.270252+00,Builder,USA,inactive,Guilherme Arm
0ab58b8c-da29-4079-9d6c-0c10534bd867,GRS Construction Service Corp,,+1 (617) 458-1597,,2026-03-12 15:03:04.233455+00,2026-03-12 15:03:04.233455+00,Builder,USA,inactive,Giov
24d197b1-0ba4-456a-9153-dae4bfbbebbb,JFC PROS Construction,sales@jfcpros.com,(617) 777-1114,"292 Main Street - Northborough, MA 01532 US",2026-03-02 13:39:21.433614+00,2026
8f44b9b0-0745-4e3e-954b-0d1cc2b68e6b,DS Empire Construction,contact-us@dsempiregc.com,+1 (857) 318-2450,,2026-03-12 15:06:26.395978+00,2026-03-12 15:06:27.956053+00,Contrac
6492ed42-ff16-4594-a6a7-0dc4685df9a8,Sem nome,,+1 (508) 967-4831,,2026-03-12 15:07:59.715245+00,2026-03-12 15:08:01.744665+00,Builder,USA,inactive,Sérgio ,,,,,,,,,,,https:/
61f0269e-d3aa-4b3d-8463-73070a869cb1,Sem nome,,+1 (978) 908-8149,,2026-03-12 15:08:17.738995+00,2026-03-12 15:08:17.738995+00,Builder,USA,inactive,Thales ,,,,,,,,,,,,,,en
0377f188-10ec-4eec-a14e-d68f4d0849b2,Home Improvement,,+1 (978) 333-2828,,2026-03-12 15:09:00.251949+00,2026-03-12 15:09:02.937374+00,Contractor,USA,inactive,Jordane ,,,,,,
f526b63b-8a94-4359-a6aa-a04d59447500,Eagle Remodeling,,+1 (781) 883-8297,,2026-03-12 15:09:22.275698+00,2026-03-12 15:09:22.275698+00,Contractor,USA,inactive,Igor ,,,,,,,,
db65c748-a994-4fe2-8956-a0c7f5477c5e,Sem nome,,+1 (508) 360-9849,,2026-03-12 15:09:40.163721+00,2026-03-12 15:09:40.163721+00,Builder,USA,inactive,Flaviano ,,,,,,,,,,,,,,er
9d86ff46-5908-4068-a226-eb3c5e7b707b,Sem nome,,+1 (781) 628-8702,,2026-03-12 15:10:06.911787+00,2026-03-12 15:10:06.911787+00,Builder,USA,inactive,Douglas ,,,,,,,,,,,,,,en
11bb51c0-40a6-4e2e-8197-0a4e55fd2cd2,Sem nome,,+1 (401) 528-9322,,2026-03-12 15:10:16.61424+00,2026-03-12 15:10:16.61424+00,Builder,USA,inactive,Deivid ,,,,,,,,,,,,,,en
0da3e76c-14e9-46d3-abdc-fadc01bafdda,Sem nome,,+1 (508) 714-1370,,2026-03-12 15:10:30.78486+00,2026-03-12 15:10:30.78486+00,Builder,USA,inactive,Alberto,,,,,,,,,,,,,en
01610003-b613-49e0-87ba-d2aefc8022ba,Sem nome,,+1 (978) 259-3715,,2026-03-12 15:10:44.009783+00,2026-03-12 15:10:44.009783+00,Builder,USA,inactive,Adilson ,,,,,,,,,,,,,,en
0b7ac4a1-b70d-4691-af25-c053328496c9,Sem nome,,+1 (781) 333-7276,,2026-03-12 15:10:59.845958+00,2026-03-12 15:10:59.845958+00,Builder,USA,inactive,Adelson ,,,,,,,,,,,,,,en
825abebf-ca2d-414d-b40b-10d6c89f261b,Sem nome,,+1 (617) 943-6136,,2026-03-12 15:11:21.100699+00,2026-03-12 15:11:21.100699+00,Architect,USA,inactive,Joseph ,,,,,,,,,,,,,,er
04d5b04c-9e5b-4130-af15-34c5d74b502f,Sem nome,,+1 (857) 888-1044,,2026-03-12 15:11:36.393153+00,2026-03-12 15:11:36.393153+00,Builder,USA,inactive,Marcos ,,,,,,,,,,,,,,en
e2ce5c57-c0fb-452f-b39b-3a78da23f056,Sem nome,,+1 (781) 389-7633,,2026-03-12 15:11:54.390793+00,2026-03-12 15:11:54.390793+00,Builder,USA,inactive,Jair Marchete,,,,,,,,,,,,
892f7623-6e2c-450c-a5bd-a15cd6eedc5a,Sem nome,,+1 (617) 913-0267,,2026-03-12 15:12:09.205673+00,2026-03-12 15:12:09.205673+00,Builder,USA,inactive,Amauri ,,,,,,,,,,,,,,en
80445ef4-5939-4620-9663-9be9355da19b,Sem nome,,+1 (857) 888-1336,,2026-03-12 15:12:50.375477+00,2026-03-12 15:12:50.375477+00,Builder,USA,inactive,Emerson ,,,,,,,,,,,,,,en
77295be3-f32f-4d04-88fa-27d744c62926,Sem nome,,+1 (508) 202-3752,,2026-03-12 15:13:08.443547+00,2026-03-12 15:13:08.443547+00,Builder,USA,inactive,Flaviano ,,,,,,,,,,,,,,er
608a358e-6cd1-41b1-988d-5e70f0dd44b6,Sem nome,,+1 (857) 417-9146,,2026-03-12 15:14:44.439263+00,2026-03-12 15:14:44.439263+00,Builder,USA,inactive,Leones ,,,,,,,,,,,,,,en
a015b6ac-cf2b-490c-b614-95bbd59adf48,Sem nome,,+1 (407) 693-2098,,2026-03-12 15:15:13.797023+00,2026-03-12 15:15:16.399466+00,Contractor,USA,inactive,Marcelo Alves,,,,,,,,
6b75218c-c898-4871-bec4-51b71d8deb6e,Sem nome,,+1 (617) 501-8025,,2026-03-12 15:15:29.844475+00,2026-03-12 15:15:29.844475+00,Builder,USA,inactive,Pedro ,,,,,,,,,,,,,,en
8060c280-52a7-44b6-a5d4-cb7de4d365a4,HmBrothersInsulation.com,contact@hmbrothersinstulation.com,+1 (978) 876-1199,,2026-03-12 15:17:29.130886+00,2026-03-12 15:17:31.51899+0
9490bf59-5239-4636-b76c-10015f3c1bdd,Sem nome,,+1 (857) 919-9790,,2026-03-12 15:17:49.774214+00,2026-03-12 15:17:49.774214+00,Builder,USA,inactive,Sandro ,,,,,,,,,,,,,,en
5220ccbd-9f3f-49bc-b734-75316b6e2962,Sem nome,,+1 (781) 484-7209,,2026-03-12 15:18:07.461423+00,2026-03-12 15:18:07.461423+00,Builder,USA,inactive,Penilton ,,,,,,,,,,,,,,er
bc7c0fb5-6561-47c6-a625-ca87138a3720,Sem nome,,+1 (617) 412-9514,,2026-03-12 15:18:21.623425+00,2026-03-12 15:18:21.623425+00,Builder,USA,inactive,Sanderson ,,,,,,,,,,,,,,e
b63584fa-5da5-4b2e-8e61-95722e7e86d4,Sem nome,,+1 (978) 595-4949,,2026-03-12 15:18:36.008996+00,2026-03-12 15:18:36.008996+00,Builder,USA,inactive,Leo ,,,,,,,,,,,,,,en
01f3a931-3900-499b-ab48-089595e93b2a,Sem nome,,+1 (508) 202-3721,,2026-03-12 15:18:52.552864+00,2026-03-12 15:18:52.552864+00,Builder,USA,inactive,Eder ,,,,,,,,,,,,,,en
bab24c11-5ca6-406c-bc81-90721e9c91da,Sem nome,,+1 (508) 596-8159,,2026-03-12 15:19:07.240197+00,2026-03-12 15:19:07.240197+00,Builder,USA,inactive,Wemerson,,,,,,,,,,,,,en
7b7f50ab-03f1-46f7-a750-ea48cc443832,Sem nome,,+1 (781) 760-2767,,2026-03-12 15:19:20.586904+00,2026-03-12 15:19:20.586904+00,Builder,USA,inactive,Ramone ,,,,,,,,,,,,,,en
c198874b-7852-4349-a7e8-2a8bdf14fd26,Sem nome,,+1 (508) 308-9765,,2026-03-12 15:19:36.333589+00,2026-03-12 15:19:36.333589+00,Builder,USA,inactive,Wasslley ,,,,,,,,,,,,,,en
8c899b2f-00ca-4735-9f48-20cbb42dfc31,Sem nome,,+1 (978) 882-1705,,2026-03-12 15:03:26.240685+00,2026-03-12 15:19:59.907535+00,Builder,USA,inactive,Hernany ,,,,,,,,,,https:
bbd79841-7e75-4ef7-bbce-7a1fc23c1a58,Sem nome,,+1 (781) 820-2360,,2026-03-12 15:20:20.356373+00,2026-03-12 15:20:20.356373+00,Builder,USA,inactive,Joe ,,,,,,,,,,,,,,en
ea13e58f-cc2d-4929-b195-b56028c94753,Sem nome,,+1 (508) 341-3848,,2026-03-12 15:21:31.542456+00,2026-03-12 15:21:31.542456+00,Builder,USA,inactive,Matt ,,,,,,,,,,,,,,en
74ee1042-75f5-4965-a660-62c5f55664cc,Sem nome,,+1 (774) 444-2814,,2026-03-12 15:21:46.096623+00,2026-03-12 15:21:46.096623+00,Builder,USA,inactive,Luiz ,,,,,,,,,,,,,,en
fd7fdb24-124d-4468-9e83-2d60c70a2861,Rall Smart Business - Negócios em Estratégias,contato@rallsmartbusiness.com,+1 (774) 707-7611,,2026-03-12 15:23:08.218212+00,2026-03-12
739c079d-8546-4c57-b3d2-87e8a52287a1,Sem nome,,+1 (781) 980-8061,,2026-03-12 15:23:33.026932+00,2026-03-12 15:23:33.026932+00,Builder,USA,inactive,Luciano ,,,,,,,,,,,,,,en
6e1bfe14-7ef4-4eda-9555-6bfd8143ffc4,Sem nome,,+1 (857) 526-1044,,2026-03-12 15:23:44.094167+00,2026-03-12 15:23:44.094167+00,Builder,USA,inactive,Nilton ,,,,,,,,,,,,,,en
20989ad6-f3b1-459d-b87c-9e2024014420,American Building LLC,,+1 (970) 258-4040,,2026-03-12 15:24:05.088824+00,2026-03-12 15:24:05.088824+00,Contractor,USA,inactive,Shelley ,
4711f3e1-fd7e-4861-86c6-e0ea59a4d82a,Sem nome,,+1 (904) 386-0837,,2026-03-12 15:24:21.85105+00,2026-03-12 15:24:21.85105+00,Builder,USA,inactive,Fernando Lima,,,,,,,,,,,,,,
1cc08220-47c0-4522-b71d-b5b35e81d36e,Sem nome,,+1 (508) 215-4809,,2026-03-12 15:24:43.008219+00,2026-03-12 15:24:43.008219+00,Builder,USA,inactive,Saulo ,,,,,,,,,,,,,,en
99d36e0e-a55c-4436-9be9-b13b69ab4c02,Sem nome,,+1 (857) 333-0816,,2026-03-12 15:24:56.633836+00,2026-03-12 15:24:56.633836+00,Builder,USA,inactive,Tony ,,,,,,,,,,,,,,en
8c725bb7-b27e-4e64-92fe-a5076e95ef9e,Sem nome,,+1 (617) 212-1358,,2026-03-12 15:25:10.528987+00,2026-03-12 15:25:10.528987+00,Builder,USA,inactive,Thiago ,,,,,,,,,,,,,,en
33797cb0-0077-4daf-8f09-4bf4fb851d33,Sem nome,,+1 (561) 710-8577,,2026-03-12 15:25:26.079466+00,2026-03-12 15:25:26.079466+00,Builder,USA,inactive,Rafael Morais,,,,,,,,,,,,
1ac0382b-551b-4de2-94bf-a4e0a00a2815,Sem nome,,+1 (978) 701-3007,,2026-03-12 15:25:42.669789+00,2026-03-12 15:25:42.669789+00,Builder,USA,inactive,Rodrigo ,,,,,,,,,,,,,,en
8d42e61b-b26e-40ed-a430-b5dc8708c74d,Sem nome,,+1 (508) 509-1129,,2026-03-12 15:25:56.469439+00,2026-03-12 15:25:56.469439+00,Builder,USA,inactive,Roy ,,,,,,,,,,,,,,en
f486e641-5ab1-48f8-a62d-61c71092548e,Sem nome,,+1 (617) 594-2352,,2026-03-12 15:26:11.572977+00,2026-03-12 15:26:11.572977+00,Builder,USA,inactive,Gustavo ,,,,,,,,,,,,,,en
1a7a498c-fd2a-4a75-aa30-4a8524b81c4e,Sem nome,,+1 (978) 419-3358,,2026-03-12 15:26:25.872203+00,2026-03-12 15:26:25.872203+00,Builder,USA,inactive,Julio ,,,,,,,,,,,,,,en
0ba8af83-d2c8-44f7-894c-86ca2ddab101,Sem nome,,+1 (857) 241-0108,,2026-03-12 15:26:43.520683+00,2026-03-12 15:26:43.520683+00,Builder,USA,inactive,Jonatas Leal,,,,,,,,,,,,,,
2fa47421-81e9-4626-abdf-08c6fef157a7,Sem nome,,+1 (857) 287-0042,,2026-03-12 15:26:57.368281+00,2026-03-12 15:26:57.368281+00,Builder,USA,inactive,Renan ,,,,,,,,,,,,,,en
5908ba8d-73d6-40fc-b919-89fd0974660b,Sem nome,,+1 (857) 266-8633,,2026-03-12 15:27:11.487173+00,

... [TRUNCATED] ...
```

company_employees.csv

```
id,company_id,name,phone,email,instagram,linkedin,created_at
b4947853-ff09-41b3-ab59-817b91187ba0,24d197b1-0ba4-456a-9153-dae4bfbbebbb,Fran - Office,+1 (617) 777-1114,,,,2026-03-10 15:12:45.188693+00
e6338da7-a484-4287-8461-a554b0c56961,b5695fc1-3a30-4718-8a58-969a08278853,Miriã - Office,+1 (617) 980-6320,,,,2026-03-10 15:13:20.696765+00
50c4710d-b53c-4317-9e53-467179362960,b5695fc1-3a30-4718-8a58-969a08278853,Jack - Owner,+1 (774) 486-2623,,,,2026-03-10 15:14:14.267228+00
fbdf7d40-e279-41e0-87a1-81af2c45ed17,7de907aa-333e-4fab-ab4a-54bfc008d93c,Rita - Office [Esposa],+1 (978) 855-8208,,,,2026-03-12 14:20:09.677131+00
```

drawing_versions.csv

(empty table)

freelancer_assignments.csv

(empty table)

freelancer_payments.csv

(empty table)

messages.csv

(empty table)

milestones.csv

(empty table)

packages.csv

```
id,name,slug,description,created_at
096fa5c0-ba5c-4a58-8621-944e718ad623,Basic Plan,basic,Starter Package,2026-03-03 12:42:18.037754+00
8ac006b3-043e-412a-8bdd-5189eb0142f1,Pro Plan,pro,Full Set,2026-03-03 12:42:18.037754+00
66bdf3c1-818e-4e25-89b0-3978c9d88fd4,3D Visualization,3d_viz,Bring It To Life,2026-03-03 12:42:18.037754+00
```

payment_milestones.csv

```
id,project_id,stage,percentage,amount,status,payment_date,created_at,updated_at
1bb59b39-4682-49f3-b539-55bae68a889f,b687fe0a-acc8-4848-a18b-c66a6a6661a2,Final Payment,50,255,pending,,2026-03-13 16:00:51.951203+00,2026-03-13 16:00:51.951203+00
22cd19e3-b238-43f9-8de7-04216d395dd0,b687fe0a-acc8-4848-a18b-c66a6a6661a2,Entry Payment,50,255,paid,2026-03-13,2026-03-13 16:00:51.951203+00,2026-03-13 16:00:51.951203+00
062044db-ca41-41e6-a76f-7089eb9248c9,36f4191a-d970-463a-8eec-57221c18c335,Entry Payment,50,432.72,paid,2026-03-13,2026-03-13 16:01:19.776418+00,2026-03-13 16:01:19.776418+0
10dcac59-c470-49bb-b6db-346c9e38b88c,36f4191a-d970-463a-8eec-57221c18c335,Final Payment,50,432.72,paid,2026-03-13,2026-03-13 16:01:19.776418+00,2026-03-13 16:01:19.776418+0
b447bfec-4e3f-42c9-966e-a8794ecea4d1,672bd2d1-a8af-4d98-a96c-49590d6d22b9,Final Payment,50,226.95,pending,,2026-03-13 16:01:33.713284+00,2026-03-13 16:01:33.713284+00
cd833b50-37f7-4a34-9c9f-2113b538398c,672bd2d1-a8af-4d98-a96c-49590d6d22b9,Entry Payment,50,226.95,paid,2026-03-13,2026-03-13 16:01:33.713284+00,2026-03-13 16:01:33.713284+0
4902cadf-5b87-47f6-9a39-5ba63bc26843,619253c4-acf9-4244-82e2-2e57ccb4faf5,Entry Payment,50,970,paid,2026-03-13,2026-03-13 16:01:45.897971+00,2026-03-13 16:01:45.897971+00
d29691a6-6ae7-4268-ab50-4eba4788f17e,619253c4-acf9-4244-82e2-2e57ccb4faf5,Final Payment,50,970,paid,2026-03-13,2026-03-13 16:01:45.897971+00,2026-03-13 16:01:45.897971+00
84086962-2afa-4f3a-b278-4d6b9cde826c,1e4674f9-f4f9-4836-b81e-6c82ef23e2bc,Entry Payment,50,450,paid,2026-03-13,2026-03-13 16:01:59.703355+00,2026-03-13 16:01:59.703355+00
c0881ee6-2540-42e7-9ed5-e13d725f9c76,1e4674f9-f4f9-4836-b81e-6c82ef23e2bc,Final Payment,50,450,paid,2026-03-13,2026-03-13 16:01:59.703355+00,2026-03-13 16:01:59.703355+00
f472e0bf-3640-41c5-a058-3b215b8f3dad,158afd94-48b5-464b-85d3-7817a70cd787,Entry Payment,50,582,paid,2026-03-13,2026-03-13 16:02:19.663617+00,2026-03-13 16:02:19.663617+00
18e828b5-e8f2-4075-8667-1413657cf1f4,158afd94-48b5-464b-85d3-7817a70cd787,Final Payment,50,582,paid,2026-03-13,2026-03-13 16:02:19.663617+00,2026-03-13 16:02:19.663617+00
a3d4b4e8-c880-4e4d-9532-4858d9f853be,a080b904-a29b-46d6-8ace-59cc9f79f2a2,Entry Payment,50,125,paid,2026-03-18,2026-03-16 18:05:57.873429+00,2026-03-16 18:05:57.873429+00
c464e1a8-24fd-4c29-80c1-493b6407f7fd,a080b904-a29b-46d6-8ace-59cc9f79f2a2,Final Payment,50,125,paid,2026-03-18,2026-03-16 18:05:57.873429+00,2026-03-16 18:05:57.873429+00
b3f859dc-5d41-4c8a-a5ba-8a9091fd9934,a09d3a89-3060-44e6-ad24-cca340f74925,Final Payment,50,140,pending,,2026-03-19 15:53:36.825762+00,2026-03-19 15:53:36.825762+00
d1ac5cea-4db1-4df4-bb56-286e6f600ab6,a09d3a89-3060-44e6-ad24-cca340f74925,Entry Payment,50,140,paid,2026-03-23,2026-03-19 15:53:36.825762+00,2026-03-19 15:53:36.825762+00
dbdade0f-4d17-4dab-b865-f174cb2b843a,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,Concept Design,40,142.08,pending,,2026-03-23 19:32:49.784386+00,2026-03-23 19:32:49.784386+00
4e6ddba0-61c2-4376-b587-22833c76615c,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,Construction Documents,30,106.56,pending,,2026-03-23 19:32:49.784386+00,2026-03-23 19:32:49.784386
9371bb7a-9b94-462a-8808-a96b23cf3a24,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,Permit Submission,30,106.56,pending,,2026-03-23 19:32:49.784386+00,2026-03-23 19:32:49.784386+00
```

profiles.csv

```
id,company_id,full_name,email,phone,avatar_url,created_at,updated_at,instagram
f9ef117e-4cb3-4726-a71d-5bd1410807e0,,Teste,teste@teste.com,,,2026-02-09 11:15:35.563364+00,2026-02-09 11:15:35.563364+00,
25c5f524-9923-45c0-8a48-f077f96898ac,,Administrador,adm@adm.com,,,2026-03-02 13:23:39.612674+00,2026-03-02 13:23:39.794097+00,
4124a173-3a25-48bf-a6fb-eba747a2633e,,Freelancer Teste,freelancer@freelancer.com,,,2026-03-02 13:23:40.076906+00,2026-03-02 13:23:40.143046+00,
24579a8e-81fb-4bc1-a079-4b6a9b54c410,24d197b1-0ba4-456a-9153-dae4bfbbebbb,Joe Chaves,sales@jfcpros.com,,,2026-03-13 16:38:51.145181+00,2026-03-13 16:38:51.909037+00,
8c4d707c-8fca-499a-a87f-ae8ee6a7ad19,,Jackson,jackgeneraloffice@gmail.com,,,2026-03-19 12:20:51.037588+00,2026-03-19 12:20:51.037588+00,
```

project_files.csv

(empty table)

project_homeowners.csv

```
id,project_id,name,phone,email,address,created_at
fe316dfa-dace-4076-bb0b-4bfa6e17fcaa,672bd2d1-a8af-4d98-a96c-49590d6d22b9,Rafael Lacerda,+1 (857) 266-7885,,,2026-03-17 17:52:45.193079+00
e218d41f-1dc2-41f5-86b7-cee41285b831,672bd2d1-a8af-4d98-a96c-49590d6d22b9,Geicy Lacerda,■+1 (857) 266■5906■,,,2026-03-17 17:53:27.510835+00
```

project_services.csv

```
id,project_id,service_id
1639e840-b5a6-473e-8268-68d463aa88a3,36f4191a-d970-463a-8eec-57221c18c335,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
7b7d1df5-1334-4373-acf5-b8021aeb01e8,36f4191a-d970-463a-8eec-57221c18c335,3fee38af-630f-4ad3-be4b-037d671214dc
89954c54-913c-40fd-a70b-750296a1dbcf,36f4191a-d970-463a-8eec-57221c18c335,e7034de1-b18f-4eca-bbef-f31ff7477da1
86ea391a-fc90-48a0-82ed-4a730fcc7714,36f4191a-d970-463a-8eec-57221c18c335,f1722963-e78c-4280-8908-5baf820fc8d1
0ee5fa9c-b4c8-47b9-ac40-d57275136e9d,1e4674f9-f4f9-4836-b81e-6c82ef23e2bc,1258cc56-42f3-4f8f-a94d-33706e603d27
9bc766fb-bf2c-4f96-8da5-f7c5af6cdb08,158afd94-48b5-464b-85d3-7817a70cd787,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
0f419c2e-d9cc-4803-9b26-869a98dfc4fe,158afd94-48b5-464b-85d3-7817a70cd787,d1b5cf23-b478-451b-94ae-02c1a62cc3ee
118eca08-7428-4c8f-9d4d-0988d237451a,158afd94-48b5-464b-85d3-7817a70cd787,f1722963-e78c-4280-8908-5baf820fc8d1
60c00858-22ea-4c08-aa23-bd77a389e534,619253c4-acf9-4244-82e2-2e57ccb4faf5,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
ad01b891-a5bf-4dca-8189-79061bcbb255,619253c4-acf9-4244-82e2-2e57ccb4faf5,d1b5cf23-b478-451b-94ae-02c1a62cc3ee
441146b9-b33f-47df-91cf-592d4c79301a,619253c4-acf9-4244-82e2-2e57ccb4faf5,f1722963-e78c-4280-8908-5baf820fc8d1
69d23b6e-5cd8-42b6-84c4-9e96ce827998,b687fe0a-acc8-4848-a18b-c66a6a6661a2,1258cc56-42f3-4f8f-a94d-33706e603d27
e8dda784-7964-4ed7-a8d1-b481301b2cd8,672bd2d1-a8af-4d98-a96c-49590d6d22b9,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
af3f8e74-7584-4e5d-aaf5-cbea8337f714,672bd2d1-a8af-4d98-a96c-49590d6d22b9,3fee38af-630f-4ad3-be4b-037d671214dc
3dc1a5f2-2235-4400-aa42-566ff5d9c4be,672bd2d1-a8af-4d98-a96c-49590d6d22b9,b651909c-1a0d-4593-86cd-2e8afae8a07e
5919677a-13e6-4cc4-8dfa-824b37dc2dc2,672bd2d1-a8af-4d98-a96c-49590d6d22b9,d1b5cf23-b478-451b-94ae-02c1a62cc3ee
976d2a53-6278-4123-aa68-8009ffa244ff,672bd2d1-a8af-4d98-a96c-49590d6d22b9,f1722963-e78c-4280-8908-5baf820fc8d1
7f19178b-0a1f-40bc-9290-66758db5d6fd,a080b904-a29b-46d6-8ace-59cc9f79f2a2,74282caf-8f92-4bda-b69a-058af100cbc7
42e43d36-c28b-4688-80bd-9641a4acf3da,aa077d41-d227-4008-a5cb-0416934c8e57,d1b5cf23-b478-451b-94ae-02c1a62cc3ee
d7ebcb48-704d-44bb-897b-614fe683577e,aa077d41-d227-4008-a5cb-0416934c8e57,f1722963-e78c-4280-8908-5baf820fc8d1
9a20b38b-fc84-4dcf-801c-b0f8fbcc95bc,aa077d41-d227-4008-a5cb-0416934c8e57,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
d5c24ec5-eea0-458f-9268-66f31b86f972,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
1bca8b6c-05bf-4eb8-9434-40f6a9297b2c,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,b651909c-1a0d-4593-86cd-2e8afae8a07e
714449da-b205-4137-b01f-cef212cb12a4,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,f1722963-e78c-4280-8908-5baf820fc8d1
09b4688c-4ba2-4f0c-beef-2fd28e5c78a1,a09d3a89-3060-44e6-ad24-cca340f74925,96f13f9c-5515-4139-a3ea-790f78fd51af
dc2d412c-d312-425a-970b-6d5cf8b295ca,13bbf8c0-6095-4189-aed4-1fbad4f7ccab,3fee38af-630f-4ad3-be4b-037d671214dc
1b124e9f-2484-43c3-b0c4-ed69e5e4c875,13bbf8c0-6095-4189-aed4-1fbad4f7ccab,b651909c-1a0d-4593-86cd-2e8afae8a07e
6eee7aff-7a48-4937-bdc2-c11c5873f3f1,13bbf8c0-6095-4189-aed4-1fbad4f7ccab,f1722963-e78c-4280-8908-5baf820fc8d1
2b7b2b98-0690-48ef-a7ef-f1bfba9fe77a,13bbf8c0-6095-4189-aed4-1fbad4f7ccab,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
91d34a20-ddce-44e3-8735-82c87ec67a00,758274ad-4645-4791-a18b-cbc2f3be2a8f,3fee38af-630f-4ad3-be4b-037d671214dc
a515d752-d3d3-49b4-88d5-3e1643dcdd8e,758274ad-4645-4791-a18b-cbc2f3be2a8f,b651909c-1a0d-4593-86cd-2e8afae8a07e
e8a3c81e-7439-4533-943d-22dae3b8baf7,758274ad-4645-4791-a18b-cbc2f3be2a8f,f1722963-e78c-4280-8908-5baf820fc8d1
fec9762e-1701-405f-b599-dd20c8b2d18b,758274ad-4645-4791-a18b-cbc2f3be2a8f,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
99e8d41b-a1a0-415e-8041-2344940b64ae,d4294c21-9f11-439f-baff-68a7371b8c98,3fee38af-630f-4ad3-be4b-037d671214dc
736610d1-fb66-443f-a000-d9947a27b66f,d4294c21-9f11-439f-baff-68a7371b8c98,b651909c-1a0d-4593-86cd-2e8afae8a07e
cdf4b2e0-bc70-4dbf-8ce7-18d2a57ea13c,d4294c21-9f11-439f-baff-68a7371b8c98,f1722963-e78c-4280-8908-5baf820fc8d1
ad9e5bf0-e088-4078-aa1d-ad813feaad6c,d4294c21-9f11-439f-baff-68a7371b8c98,211a2476-bd4f-4e8a-b3ad-74a58e8bf87d
```

project_tasks.csv

(empty table)

project_timeline.csv

```
id,project_id,revision_number,event_type,event_date,notes,created_at
26a2a468-ec93-4ce5-a198-84f6da815188,aa077d41-d227-4008-a5cb-0416934c8e57,0,estimate_created,2026-03-19,,2026-03-19 12:47:14.554048+00
f98e90fc-72e5-40c6-99cf-673a6b538049,aa077d41-d227-4008-a5cb-0416934c8e57,0,estimate_created,2026-03-19,,2026-03-19 12:47:14.693781+00
36ac8b98-5702-4865-9964-b67948326953,a09d3a89-3060-44e6-ad24-cca340f74925,0,estimate_created,2026-03-13,,2026-03-19 12:47:47.494505+00
672e407c-2ff6-463c-a50f-2a59139f9862,a09d3a89-3060-44e6-ad24-cca340f74925,-1,invoice_sent_auto,2026-03-19,"Invoices created: Entry Payment, Final Payment",2026-03-19 15:53:
e2c79a44-806d-4a04-b754-2ac77f51d296,a09d3a89-3060-44e6-ad24-cca340f74925,0,initial_invoice_sent,2026-03-19,,2026-03-19 15:53:53.767911+00
6c5d6249-f879-4d25-9b0f-71c6aa5615d9,aa077d41-d227-4008-a5cb-0416934c8e57,-1,invoice_sent_auto,2026-03-20,"Invoices created: Concept Design, Construction Documents, Permit
56fa40b1-4d60-4c46-9a2f-34cd10beb719,a09d3a89-3060-44e6-ad24-cca340f74925,0,initial_payment_received,2026-03-23,,2026-03-23 19:26:15.065333+00
8087f287-3aa2-4bc0-bb96-e6467de547b9,a09d3a89-3060-44e6-ad24-cca340f74925,-1,payment_received_auto,2026-03-23,Entry Payment — $140.00 paid,2026-03-23 19:26:50.242068+00
561a8b07-6a47-49e1-8c0b-84d644c2b764,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,-1,invoice_sent_auto,2026-03-23,"Invoices created: Concept Design, Construction Documents, Permit
076cdaa1-f8a2-441b-b042-07bdcdf36433,a09d3a89-3060-44e6-ad24-cca340f74925,0,rev_started,2026-03-23,,2026-03-24 15:13:18.637332+00
b31a1a0e-a5ad-466b-9692-3b8ce5d99286,a09d3a89-3060-44e6-ad24-cca340f74925,0,rev_preview_sent,2026-03-24,,2026-03-24 15:13:25.982586+00
```

projects.csv

```
id,company_id,name,description,address,service_type,status,cover_image,is_portfolio,created_at,updated_at,stage,start_date,delivery_date,total_value,internal_notes,street_n
1e4674f9-f4f9-4836-b81e-6c82ef23e2bc,7de907aa-333e-4fab-ab4a-54bfc008d93c,,,,3D Kitchen Design,pending,https://tcyfqcfqwjqmlkvalycy.supabase.co/storage/v1/object/public/por
a080b904-a29b-46d6-8ace-59cc9f79f2a2,7de907aa-333e-4fab-ab4a-54bfc008d93c,,,,3D Bath Design,pending,https://tcyfqcfqwjqmlkvalycy.supabase.co/storage/v1/object/public/portfo
158afd94-48b5-464b-85d3-7817a70cd787,7de907aa-333e-4fab-ab4a-54bfc008d93c,,,,New Construction [Aditivo],pending,https://tcyfqcfqwjqmlkvalycy.supabase.co/storage/v1/object/p
b687fe0a-acc8-4848-a18b-c66a6a6661a2,7de907aa-333e-4fab-ab4a-54bfc008d93c,,,,3D Kitchen Design,pending,https://tcyfqcfqwjqmlkvalycy.supabase.co/storage/v1/object/public/por
36f4191a-d970-463a-8eec-57221c18c335,24d197b1-0ba4-456a-9153-dae4bfbbebbb,Teste,,"88 Carter St- Berlin, MA 01503",Additions &amp; Extensions,pending,https://tcyfqcfqwjqmlkv
672bd2d1-a8af-4d98-a96c-49590d6d22b9,7de907aa-333e-4fab-ab4a-54bfc008d93c,Rafael &amp; Geicy,,"239 Brook St - Dracut, MA 01826, EUA",Additions &amp; Extensions,pending,http
a09d3a89-3060-44e6-ad24-cca340f74925,7de907aa-333e-4fab-ab4a-54bfc008d93c,,,,3D Laundry Design,pending,https://tcyfqcfqwjqmlkvalycy.supabase.co/storage/v1/object/public/por
619253c4-acf9-4244-82e2-2e57ccb4faf5,7de907aa-333e-4fab-ab4a-54bfc008d93c,,,,New Construction,pending,https://tcyfqcfqwjqmlkvalycy.supabase.co/storage/v1/object/public/port
aa077d41-d227-4008-a5cb-0416934c8e57,b5695fc1-3a30-4718-8a58-969a08278853,,,,New Construction – Single Family Residence (2-Story),pending,,f,2026-03-19 12:44:41.946927+00,2
822d197a-2c7a-42f2-b033-ef4d69c1ddc4,ae8bdf98-d5be-478a-ad4e-3c8af73d84ac,,,,Remodeling Deck,pending,,f,2026-03-23 19:30:54.717571+00,2026-03-23 19:33:19.436784+00,WAITING_
13bbf8c0-6095-4189-aed4-1fbad4f7ccab,a8e3bdfb-b594-47e4-a173-8a06d03a1d2d,,,,Residential Addition Design,pending,,f,2026-03-24 22:14:30.546561+00,2026-03-24 22:14:30.54656
758274ad-4645-4791-a18b-cbc2f3be2a8f,a8e3bdfb-b594-47e4-a173-8a06d03a1d2d,,,,Residential Addition Design,pending,,f,2026-03-24 22:16:21.94284+00,2026-03-24 22:16:21.94284+0
d4294c21-9f11-439f-baff-68a7371b8c98,a8e3bdfb-b594-47e4-a173-8a06d03a1d2d,,,,Residential Addition Design,pending,,f,2026-03-25 11:53:25.527928+00,2026-03-25 11:53:25.527928
```

quote_requests.csv

id,full_name,company_name,email,phone,project_address,service_type,description,file_url,processed,created_at
e933669f-ff2e-43cc-ab4b-7845990aa19e,Teste,,teste@teste.com,0000000000,"Teste, Teste, Teste 00000",technical_drafting,"CLIENT TYPE: cliente_final
PROJECT TYPE: Single-Family Residential
HAS EXISTING PLANS: No
HAS ENGINEER: Needs referral
BUILDING STATUS: new_construction

--- PROJECT SIZE & LEVELS ---
EXISTING AREA: 1000 sq ft
PROPOSED AREA: 2000 sq ft
PROJECT LEVELS: Second Floor

--- PROGRAM OF REQUIREMENTS ---
No spaces selected

--- REFERENCES ---

--- DARA SERVICES SELECTED ---
None selected

--- TIMELINE & BUDGET ---
DESIRED DEADLINE: Not specified
ESTIMATED BUDGET: Not specified
HOW FOUND US: Not specified

--- COMPANY INFORMATION ---
COMPANY NAME: N/A
COMPANY ADDRESS: N/A
COMPANY INSTAGRAM: N/A
COMPANY PHONE: N/A
COMPANY EMAIL: N/A

--- ADDITIONAL NOTES ---
No additional notes",,f,2026-02-09 11:28:04.186724+00
d9e6fbfc-7653-4b0e-aed4-71a45913490d,Teste,,teste@teste.com,0000000000,"Teste, Teste, Teste 000000",technical_drafting,"CLIENT TYPE: cliente_final
PROJECT TYPE: Renovação / Reforma
HAS EXISTING PLANS: No
HAS ENGINEER: Yes
BUILDING STATUS: occupied

--- PROJECT SIZE & LEVELS ---
EXISTING AREA: 1500 sq ft
PROPOSED AREA: 300 sq ft
PROJECT LEVELS: First Floor

--- PROGRAM OF REQUIREMENTS ---
REQUIRED SPACES: Primary Suite, Kitchen

--- REFERENCES ---

--- DARA SERVICES SELECTED ---
Residential architectural design layouts, Technical floor plans for design coordination and permitting support, Sections and elevations for visualization and documentation,

--- TIMELINE & BUDGET ---
DESIRED DEADLINE: flexible
ESTIMATED BUDGET: under_2500
HOW FOUND US: referral

--- COMPANY INFORMATION ---
COMPANY NAME: N/A
COMPANY ADDRESS: N/A
COMPANY INSTAGRAM: N/A
COMPANY PHONE: N/A
COMPANY EMAIL: N/A

--- ADDITIONAL NOTES ---
No additional notes",,f,2026-03-03 14:08:07.186438+00
846166c9-6a73-4805-84ed-9745b0790ebb,Teste,,teste@teste.com,0000000000,"Teste, Teste",architecture,"REGION: US
CLIENT TYPE: Builder / Contractor
PROJECT LOCATION: Teste, Teste

PROJECT TYPE: Addition / Extension
PROJECT SIZE: 1,500 – 3,000 sqft
FLOORS: Two Floors

SERVICES: 3D Bathroom Design

TIMELINE: Flexible

ESTIMATED PRICE: USD 3.910

CONTACT: Teste Teste",,f,2026-03-13 17:24:50.320746+00
1c073a15-cb8b-461a-940b-fd360e2e6b2c,Jackson,,jackgeneraloffice@gmail.com,,"41 Bowdoin Ave, Dorchester, MA",technical_drafting,"PROJECT TYPE: Single-Family Residential
ESTIMATED SIZE: 1800 sq ft
ADDRESS: 41 Bowdoin Ave, Dorchester, MA

ADDITIONAL NOTES:
This proposal includes basic drafting services for a proposed new construction residential project located at 41 Bowdoin Ave – Dorchester, MA.
The proposed project consists of a 2-story single-family residence with an approximate footprint of 30’ x 30’, totaling approximately 1,800 square feet.
All drawings produced under this proposal are intended for preliminary design visualization and basic permit submission purposes only, based on a simplified scope of work.

Submitted from Client Portal by: jackgeneraloffice@gmail.com",,t,2026-03-19 12:21:57.543181+00

quotes.csv

(empty table)

services.csv

```
id,name,slug,created_at,category
d1b5cf23-b478-451b-94ae-02c1a62cc3ee,Single-Family Homes,single_family,2026-03-03 12:42:18.037754+00,Design Services
78c13d0b-6ca8-4016-806d-4d8c8b89d14b,Multi-Family Residences,multi_family,2026-03-03 12:42:18.037754+00,Design Services
fc3d61e1-e8f5-4d20-8207-3cbcc9d19c04,ADU (Accessory Dwelling Units),adu,2026-03-03 12:42:18.037754+00,Design Services
3fee38af-630f-4ad3-be4b-037d671214dc,Additions &amp; Extensions,additions,2026-03-03 12:42:18.037754+00,Design Services
b651909c-1a0d-4593-86cd-2e8afae8a07e,Remodeling &amp; Renovations,remodeling,2026-03-03 12:42:18.037754+00,Design Services
a9f5506f-ece7-47de-8dbb-b983b5ed5425,Barndominium Design,barndominium,2026-03-03 12:42:18.037754+00,Design Services
0009cb23-4f88-4507-b2e0-150f6f04acb4,Commercial Projects,commercial,2026-03-03 12:42:18.037754+00,Design Services
e7034de1-b18f-4eca-bbef-f31ff7477da1,Garage,garage,2026-03-03 13:44:26.2882+00,Design Services
d75a7cd5-e68b-4690-8471-28c846818f77,Drafting Plan (PDF-CAD),drafting_plan,2026-03-03 12:42:18.037754+00,Drafting &amp; Technical Documentation
63b29d6d-b520-456d-8f83-84269c59def8,Technical Drafting,technical_drafting,2026-03-10 15:21:43.319882+00,Drafting &amp; Technical Documentation
f1722963-e78c-4280-8908-5baf820fc8d1,Executive Project,executive_project,2026-03-10 15:21:43.319882+00,Drafting &amp; Technical Documentation
388b4eb2-ab36-4105-873c-0475cf4275ad,Wood Frame Design,wood_frame_design,2026-03-10 15:21:43.319882+00,Drafting &amp; Technical Documentation
1258cc56-42f3-4f8f-a94d-33706e603d27,3D Kitchen Design,3d_kitchen_design,2026-03-10 15:21:43.319882+00,Visualization
74282caf-8f92-4bda-b69a-058af100cbc7,3D Bath Design,3d_bath_design,2026-03-10 15:21:43.319882+00,Visualization
211a2476-bd4f-4e8a-b3ad-74a58e8bf87d,3D Exterior Rendering,3d_visualization,2026-03-03 12:42:18.037754+00,Visualization
96f13f9c-5515-4139-a3ea-790f78fd51af,3D Laundry Design,3d_laundry_design,2026-03-19 11:16:31.45378+00,Visualization
```

site_updates.csv

```
id,project_id,uploaded_by,media_url,description,created_at
722659fc-403d-4cf3-95ee-6be985ac2492,822d197a-2c7a-42f2-b033-ef4d69c1ddc4,25c5f524-9923-45c0-8a48-f077f96898ac,https://tcyfqcfqwjqmlkvalycy.supabase.co/storage/v1/object/pu
```

studio_settings.csv

```
id,studio_name,tagline,logo_url,primary_color,accent_color,contact_email,contact_phone,custom_domain,updated_at
7d49063e-51a6-4f55-81be-235dd113db7f,DARA Studio,Drafting &amp; 3D Support,,#b8860b,#ee2fde,contact@darastudio.com,,,2026-03-13 16:31:42.231732+00
```

support_messages.csv

(empty table)

user_roles.csv

```csv
id,user_id,role
e50a00e5-2a7f-43b2-90ac-b40d93c158eb,25c5f524-9923-45c0-8a48-f077f96898ac,admin
ec706948-4bec-400b-9c6f-02543b6a0996,25c5f524-9923-45c0-8a48-f077f96898ac,freelancer
0a3f9d0f-ee1c-4c5f-bb6b-979a76758d57,4124a173-3a25-48bf-a6fb-eba747a2633e,freelancer
9c182bc1-6292-4721-bff3-a74aefc401c6,24579a8e-81fb-4bc1-a079-4b6a9b54c410,user
```